첫째마당 유니티 살펴보기

01장 유니티 엔진 이해하고 개발 환경 준비하기 18

01-1 게임 엔진과 유니티 엔진 이해하기 19

게임 제작의 도구 상자 — 게임 엔진 19

게임 엔진의 6가지 기능 20

유니티 엔진이란? 22

01-2 유니티 허브와 에디터 설치하기 25

Do it! 실습 유니티 허브 준비하기 25

Do it! 실습 유니티 에디터 설치하기 34

02장 유니티 에디터 인터페이스와 핵심 기능 알아보기 40

02-1 유니티 인터페이스 살펴보기 41

Do it! 실습 새 프로젝트 생성하기 41

Do it! 실습 기존 프로젝트 열기 44

유니티 에디터 화면 구성 살펴보기 45

기본으로 사용하는 뷰 48

02-2 게임오브젝트와 컴포넌트 알아보기 55

유니티 기본 용어 익히기 55

유니티 좌표 체계 이해하기 57

게임오브젝트 살펴보기 58

카메라 오브젝트 66

빛 오브젝트 73

둘째마당 게임 월드, 플레이어, 적 만들기

03장 게임 월드와 플레이어 오브젝트 78

03-1 2D 타일맵으로 게임 월드 제작하기 79

Do it! 실습 프로젝트 만들기 79

Do it! 실습 게임 화면 설정하기 80

Do it! 실습 게임 리소스 가져오기 82

Do it! 실습 플레이어와 적 이미지 설정하기 84

Do it! 실습 타일 팔레트와 타일 에셋 생성하고 설정하기 87

Do it! 실습 타일맵 오브젝트로 게임 월드 만들기 91

03-2 플레이어 오브젝트 만들고 물리 기반 이동 구현하기 103

Do it! 실습 플레이어 오브젝트 만들기 103

Do it! 실습 플레이어 움직이기 106

Do it! 실습 플레이어와 벽 타일의 Y 좌표 기준으로 렌더링 순서 설정하기 111

Do it! 실습 플레이어 애니메이션 만들기 112

Do it! 실습 플레이어 움직임에 효과 추가하기 119

Do it! 실습 플레이어를 추적하는 카메라와 조명 설정하기 124

04장 적 오브젝트와 기본 전투 시스템 130

04-1 적 오브젝트 만들고 행동 제어하기 131

Do it! 실습 적 캐릭터 만들기 131

Do it! 실습 적 캐릭터 대기 애니메이션 만들기 133

Do it! 실습 적 캐릭터 자동 생성하기 135

04-2 기본 스탯 설정과 체력 출력하기 139

플레이어와 적의 기본 능력치 구현하기 139

Do it! 실습 플레이어와 적의 기본 스탯 만들기 139

Do it! 실습 플레이어를 따라다니는 체력 UI 추가하기 144

04-3 적 기본 공격 구현하기 156

Do it! 실습 적 발사체 만들기 156

Do it! 실습 플레이어와 벽 충돌 설정하기 160

Do it! 실습 플레이어 위치로 직선 공격하기 161

04-4 플레이어 공격 목표 설정하기 167

Do it! 실습 공격 목표 설정하기 167

Do it! 실습 목표 방향으로 플레이어 회전하기 172

05장 플레이어 능력치와 기본 스킬 시스템 만들기 177

05-1 데이터를 유연하게 관리하는 스탯 클래스 만들기 178

스캣 클래스 정의하기 178

Do it! 실습 스탯 클래스 만들기 179

05-2 기본 공격 스킬 만들기 189

Do it! 실습 기본 공격 발사체 만들기 189

Do it! 실습 기본 공격 스킬 제어하기 193

Do it! 실습 기본 공격 피격 효과 표현하기 196

Do it! 실습 기본 공격 데미지 표시하기 203

05-3 버프 스킬 만들기 211

스킬 클래스 구현하고 버프 구현하기 211

Do it! 실습 스킬 에셋 만들기 214

Do it! 실습 디버깅용 메시지 출력하기 220

Do it! 실습 버프 스킬 등록하고 사용하기 221

Do it! 실습 공격하는 플레이어 구현하기 227

06장 다양한 주 공격 스킬과 보조 공격 스킬 구현하기 235

06-1 다양한 주 공격 발사체 구현하기 236

Do it! 실습 아이스 스피어 공격 구현하기 236

Do it! 실습 파이어 볼 공격 구현하기 249

Do it! 실습 윈드 블레이드 공격 구현하기 251

Do it! 실습 라이트닝 공격 구현하기 255

Do it! 실습 다중 발사체 공격 구현하기 259

06-2 보조 공격 스킬 구현하기 267

Do it! 실습 아이스 해머 공격 구현하기 267

Do it! 실습 파이어, 윈드, 라이트, 다크 해머 공격 추가하기 272

07장 근접 회전 공격과 광역 공격 스킬 구현하기 278

07-1 플레이어 주위를 회전하는 곡괭이 구현하기 279

Do it! 실습 아이스 곡괭이 공격 추가하기 279

Do it! 실습 파이어, 윈드, 라이트, 다크 곡괭이 공격 추가하기 286

07-2 전 범위 타격과 유도 공격 패턴 확장하기 291

Do it! 실습 블리자드 공격 더하기 291

Do it! 실습 파이어 드래곤 공격 더하기 299

Do it! 실습 토네이도 공격 더하기 303

Do it! 실습 라이트닝 스트라이크 공격 더하기 308

Do it! 실습 암흑 공격 더하기 312

08장 속성 보너스 스킬 구현하고 스킬 UI 제작하기 317

08-1 스킬 레벨 합계에 따른 자동 보너스 얻기 318

Do it! 실습 속성 보너스 스탯과 스킬 적용하기 318

Do it! 실습 각 스킬에 공격력 적용하기 324

08-2 습득 가능한 모든 스킬 목록 UI 구현하기 329

Do it! 실습 스킬 아이콘 오브젝트 만들기 329

Do it! 실습 스킬 목록 UI 만들기 330

Do it! 실습 게임을 일시 정지하고 스킬 목록 표시하기 341

08-3 무작위로 스킬 배우고 레벨업 하기 347

Do it! 실습 스킬 선택 아이콘 UI 추가하기 347

Do it! 실습 스킬 선택 UI 구현하기 349

셋째마당 게임 시스템과 적 AI 구현하기

09장 레벨업 시스템과 자원 최적화하기 357

09-1 경험치 테이블 설계하고 레벨업 하기 358

Do it! 실습 레벨과 경험치 스탯 추가하기 358

Do it! 실습 플레이어 레벨 관리하기 361

Do it! 실습 적을 물리쳤을 때 처리하기 366
Do it! 실습 플레이어 레벨과 경험치 출력하기 369

09-2 메모리 풀을 활용하여 보석 생성하고 최적화하기 374
Do it! 실습 메모리 풀과 페이드 효과 준비하기 374
Do it! 실습 보석 수집 효과 관리하고 움직임 제어하기 381
Do it! 실습 보석 효과와 UI 출력하기 385

10장 똑똑한 적 AI — 행동 트리와 스마트 경로 탐색하기 392

10-1 NavMesh2D를 이용한 장애물 회피와 길 찾기 393
NavMesh2D 에셋 가져오기 393
Do it! 실습 NavMesh2D로 경로 데이터 생성하기 394
Do it! 실습 경로 데이터로 적 이동 제어하기 397

10-2 행동 트리로 적 상태 유기적으로 제어하기 400
행동 트리 살펴보기 400
Do it! 실습 순찰하는 적 AI 구현하기 401
Do it! 실습 열거형으로 대기와 순찰 행동 관리하기 411
Do it! 실습 배회하는 적 구현하기 419
Do it! 실습 추적하는 적 구현하기 424
Do it! 실습 행동을 바꾸는 데 필요한 거리 계산하기 428
Do it! 실습 다양한 무기를 이용한 적 공격 구현하기 435

넷째마당 게임 완성! 씬과 스테이지 구성하기

11장 비동기 씬 로딩과 로비 시스템 갖추기 443

11-1 비동기 로딩과 씬 구성하기 444
Do it! 실습 로딩 시스템 구현하기 444
Do it! 실습 인트로 씬 만들기 455
Do it! 실습 로비 씬 만들기 461

11-2 스와이프 UI로 챕터 선택하기 464
Do it! 실습 챕터 데이터 설정하기 464
Do it! 실습 챕터 이동 스와이프 UI 만들기 476

12장 챕터와 스테이지 관리하고 JSON 게임 데이터 저장하기 483

12-1 스테이지 클리어 로직과 챕터 진행 시스템 만들기 484
Do it! 실습 적 등장 위치 알리기 484
Do it! 실습 모든 적을 물리치면 다음 스테이지 실행하기 490
Do it! 실습 선택 챕터 저장하고 불러오기 & 챕터 클리어 검사하기 494

12-2 JSON 직렬화로 게임 세이브와 로드 구현하기 497
Do it! 실습 게임 데이터 저장하고 불러오기 497
Do it! 실습 게임 오버, 챕터 클리어, 보상 구현하기 506
Do it! 실습 로비 UI와 게임 데이터 연동하기 514
Do it! 실습 하트 사용하고 자동 충전하기 519

부록 컨트롤러 추가하여 빌드하고 배포하기

A-1 가상 컨트롤러 추가하기 528
Do it! 실습 모바일용 가상 컨트롤러 추가하기 528

A-2 모바일용 빌드하기 536
Do it! 실습 안드로이드용으로 빌드하기 536

찾아보기 542

첫째마당

유니티 살펴보기

이번 마당에서는 게임 엔진과 유니티 엔진을 이해하고 허브와 에디터 등 게임 개발 환경을 준비합니다. 준비가 끝나면 설치한 에디터로 새로운 프로젝트를 만들고 인터페이스를 살펴보며 게임오브젝트와 이에 다양한 기능을 추가하는 컴포넌트를 알아봅니다.

01장 유니티 엔진 이해하고 개발 환경 준비하기

02장 유니티 에디터 인터페이스와 핵심 기능 알아보기

유니티 엔진 이해하고 개발 환경 준비하기

이번 장에서는 더 쉽게 게임을 개발할 수 있는 게임 엔진의 개념과 종류, 주요 기능을 살펴봅니다. 또한 이 책에서 다룰 유니티 엔진을 알아보고, 유니티 엔진을 사용할 때 필요한 허브와 에디터를 설치합니다.

학습 목표

1 게임 엔진의 개념과 유니티의 특징을 설명할 수 있다.
2 유니티 허브를 활용하여 LTS 에디터와 필수 모듈을 설치하고 프로젝트를 생성·관리할 수 있다.

01-1 게임 엔진과 유니티 엔진 이해하기
01-2 유니티 허브와 에디터 설치하기

01-1

게임 엔진과 유니티 엔진 이해하기

게임 개발은 그래픽, 물리, 입력 처리, 사운드 등 다양한 요소를 함께 다뤄 처음에는 복잡하고 어렵게 느껴질 수 있습니다. 하지만 게임 엔진을 활용하면 이러한 기능을 미리 제공받을 수 있어, 누구나 비교적 쉽게 개발을 시작할 수 있습니다.

이 절에서는 게임을 보다 쉽게 개발할 수 있도록 도와주는 게임 엔진이 무엇인지 알아봅시다. 또한 게임 엔진의 종류와 6가지 기능을 간단히 살펴보고, 앞으로 사용할 유니티 엔진을 소개합니다. 그리고 개발에 필요한 유니티 허브와 유니티 에디터를 설치해 봅시다.

게임 제작의 도구 상자 — 게임 엔진

게임 엔진Game Engine은 비디오 게임과 같은 실시간 상호작용 프로그램을 개발하기 위한 소프트웨어입니다. 조금 더 쉽게 말하면 게임 엔진은 게임을 만들기 위한 기본 도구 세트이자 조립 키트와 같습니다. 게임 엔진에는 그래픽을 화면에 그리는 기능, 물체의 움직임을 계산하는 물리 기능, 소리를 재생하는 오디오 기능 등이 마치 완성된 부품처럼 미리 준비되어 있습니다. 개발자는 이 부품들을 조립하듯 활용해, 처음부터 모든 것을 만들 필요 없이 빠르게 게임을 제작할 수 있습니다.

또한 게임 엔진은 하나의 게임만을 위해 만들어진 것이 아니라 여러 종류의 게임에 반복해 사용할 수 있는 '재사용 가능한 툴'로 설계되어 있습니다. 즉, 같은 도구 세트를 활용해 퍼즐 게임이나 액션 게임 등 다양한 게임을 만들 수 있습니다.

대표적인 게임 엔진으로는 우리가 배울 **유니티**Unity와 **언리얼**Unreal이 있습니다. 유니티는 가볍고 빠르게 제작할 수 있어 2D 및 모바일 게임 개발에 많이 사용되며, 언리얼은 고품질 그래픽 표현에 강점이 있어 3D 게임 개발에서 자주 활용됩니다. 하지만 두 엔진 모두 2D와 3D 게임을 모두 만들 수 있는 범용 제작 도구이므로, 게임 장르만으로 엔진을 구분하기보다는 프로젝트 목적과 개발 환경에 맞게 선택하는 것이 중요합니다. 다음은 게임 엔진의 종류입니다.

게임 엔진의 종류

게임 엔진	지원	언어
Unity	2D/3D	C#
Unreal	2D/3D	C++
CRYENGINE	2D/3D	C++, Lua, C#
Armory3D	3D	W/A, Java
GameMaker	2D/3D	Delphi
Godot Engine	2D/3D	C++/C#
Defold	2D	Lua
GameSalad	2D/3D	Visual Script
MonoGame	2D/3D	C#
Corona	2D	Lua

게임 엔진의 6가지 기능

게임 엔진은 종류에 따라 제공하는 기능에 차이가 있지만, 대표적으로 GUI, 그래픽, 물리, 다중 플랫폼, 오디오, 인공지능 등을 제공합니다. 이제 각 기능이 어떤 역할을 하는지 차례로 살펴봅시다.

게임 개발의 과정

GUI

게임을 개발할 게임 엔진을 더 쉽게 다룰 수 있는 GUI를 제공합니다. 게임 개발에 필요한 메뉴를 GUI로 제공하므로 개발자는 이 메뉴를 이용하여 카메라, 빛과 같은 **게임오브젝트** GameObject를 손쉽게 생성하거나 편집할 수 있습니다. 또한 게임이 오류 없이 잘 작동하는지 확인할 수 있도록 다양한 **뷰** View를 제공합니다.

뷰는 개발의 편의성과 효율을 높이고자 게임, 씬, 계층 등 역할에 따라 나누는 기준입니다. 뷰는 02-1절에서 자세하게 설명합니다.

그래픽스와 렌더링

그래픽스 Graphics와 렌더링 Rendering 기능을 활용하면 2D 이미지, 3D 모델(애니메이션)을 화면에 출력하는 과정을 쉽게 처리할 수 있습니다. 또한 게임 장면을 표현하는 텍스처 매핑, 조명/그림자, 포스트 프로세싱, 셰이더, 렌더 파이프라인과 같은 다양한 그래픽 요소를 제공합니다.

- **텍스처 매핑** Texture Mapping: 오브젝트 표면에 이미지(Albedo/Base Color)를 적용합니다.
- **조명** Lighting**/그림자** Shadow: 다양한 종류의 2D/3D 빛과 그림자를 제공합니다.
- **포스트 프로세싱** Post-Processing: 눈부심 Bloom, 색 보정, 피사계 심도 DoF, 모션 블러 Motion Blur 등의 후처리 작업을 제공합니다.
- **셰이더** Shader**/셰이더 그래프** Shader Graph: 다양한 재질을 표현하는 사용자 정의 재질 만들기 로직을 제공합니다.
- **렌더 파이프라인** Render Pipeline: 씬에 제작한 2D/3D 데이터를 '화면 픽셀'로 그리는 전 과정을 정의한 처리 흐름입니다.

물리

물리 Physics 기능을 활용하면 게임오브젝트가 중력의 영향으로 떨어지거나 다른 오브젝트와 충돌했을 때처럼 게임 내에서 물리적인 영향을 받았을 때 발생하는 움직임이나 상황을 구현할 수 있습니다.

다중 플랫폼

다중 플랫폼 Multi-platform 기능을 활용하면 코드를 조금만 수정하더라도 윈도우, macOS, 리눅스, 안드로이드, iOS 등을 비롯해 PS, Xbox, 닌텐도 등 게임 콘솔이나 WebGL 등 다양한 플랫폼에서 게임을 실행할 수 있습니다. 이처럼 플랫폼별로 빌드 대상과 해상도 설정, 입력, 퍼블리싱 옵션을 손쉽게 전환할 수 있습니다.

빌드란 소스 코드 파일을 실행할 수 있도록 변환하는 과정 또는 변환한 결과물을 말합니다.

오디오

오디오 Audio 기능을 활용하면 게임에서 사용되는 배경 음악 BGM과 효과음 SFX을 재생할 수 있을 뿐만 아니라 실시간 믹싱과 마스터링, 믹서 계층 구조와 같은 다양한 사운드 라이브러리도 사용할 수 있습니다. 즉, 게임에 필요한 오디오를 제작하거나 재생합니다.

인공지능

유한 상태 기계Finite State Machine, FSM, 행동 트리Behavior Tree와 같은 상태 기반 로직, 경로 탐색Pathfinding, NavMesh/Grid 기반 지형 분석Navigation/Area Cost, A* 알고리즘, 장애물 회피Steering/Obstacle Avoidance, 트리거 시스템Trigger System을 이용한 감지 범위/이벤트 기반 상호 작용 처리 등 게임 제작에 활용할 수 있는 다양한 인공지능Artificial Intelligence, AI 라이브러리를 제공합니다.

유니티 엔진이란?

유니티는 다양한 플랫폼에서 게임과 실시간 콘텐츠를 개발할 수 있는 대표적인 게임 엔진입니다. 게임뿐만 아니라 애니메이션, 영상, 자동차, 건축 등 실시간 3D 콘텐츠가 필요한 다양한 분야에서도 활용되고 있습니다.

유니티의 가장 큰 특징은 빠르게 결과물을 만들어 볼 수 있는 개발 환경입니다. 그래픽, 물리, UI 등 주요 기능이 기본 제공되기 때문에 복잡한 기능을 처음부터 구현하지 않아도 됩니다. 또한 하나의 프로젝트로 PC, 모바일, 콘솔 등 여러 플랫폼에 배포할 수 있어 활용 범위가 넓습니다. 스크립트는 비교적 배우기 쉬운 C# 언어를 사용해 구현합니다.

실무에서 유니티는 다음과 같은 3가지 이유로 널리 사용됩니다.

라이선스 비용이 저렴하다

유니티는 개인이나 학생 사용자에게 무료 라이선스를 제공하므로 비용 부담 없이 학습하고 프로젝트를 제작할 수 있습니다. 유료 라이선스로는 프로Pro, 엔터프라이즈Enterprise, 인더스트리Industry가 있으며 월 단위의 정액 요금 또는 게임 판매 수익의 일부를 유니티에 사용료(또는 저작권료)로 내야 합니다. 자세한 내용은 다음 표를 참고하세요.

● 플랜과 가격은 유니티 홈페이지 첫 화면 위 [가격 책정] 버튼을 클릭하면 자세히 확인할 수 있습니다(https://unity.com/kr/products).

유니티 라이선스 종류(2026년 4월 기준)

라이선스(플랜)	가격	설명
Personal	무료	• 무료 버전으로, 유니티를 시작하고 학습하는 개인 또는 팀을 위한 리소스 제공 • 단, 개인 또는 팀에서 제품을 출시한 후 12개월 매출 또는 자본금 합계가 20만 달러(USD) 미만일 때만 사용 가능
Pro	₩289,800/월	• 전문가 또는 전문 팀의 지난 12개월 매출 또는 자본금 합계가 20만 달러(USD) 이상일 때 필수 사용 • 여러 게임 기기와 플랫폼으로 쉽게 내보낼 수 있는 전문 도구 제공 • 유니티 클라우드로 협업과 개발을 지원하는 통합 환경 제공
Enterprise	문의	• 기업용 라이선스 • 증강 현실(AR), 가상 현실(VR) 등을 제작할 수 있고 캐드(CAD)나 3D 데이터로도 활용할 수 있음 • 고급 기술을 지원받거나 전문 교육 리소스를 제공받을 수 있음
Industry	문의	• 자동차 제조, 건축 설계, 에너지 산업 등에 속한 기업용 라이선스 • Enterprise 라이선스의 모든 기능 포함 • 제품을 시각화하거나 스마트 팩토리, 디지털 트윈, 시뮬레이션 등을 만들 수 있는 도구 제공

활용 분야가 다양하다

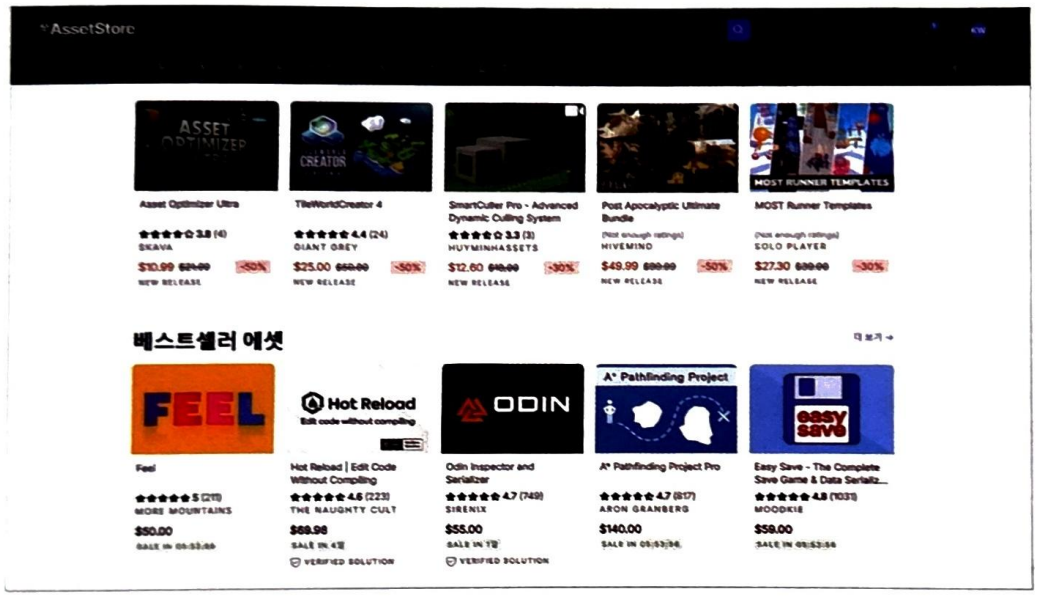
유니티 엔진은 처음에는 게임 개발을 위해 제작했지만, 발전을 거듭하면서 지금은 2D나 3D 게임, PC, 모바일, 콘솔 등 게임 개발뿐만 아니라 가상 현실과 증강 현실, 영화, 애니메이션, 자동차 제조, 건축 설계, 에너지 산업 등 다양한 분야에서 활용합니다. 다음은 유니티 엔진 활용 사례입니다.

- **애니메이션 《뽀로로》**: 제작사인 스튜디오게일이 유니티 코리아와 MOU를 체결하고 유니티를 애니메이션 제작 파이프라인에 도입했습니다.
- **애니메이션 《핑크퐁 아기상어》**: 더핑크퐁컴퍼니가 수십 편의 율동/애니메이션 영상 제작 공정 대부분을 유니티로 처리했습니다.
- **영화 《레디 플레이어 원》**: 가상 제작Virtual Production 단계에서 유니티를 실시간 그래픽/가상 카메라용 엔진으로 활용했습니다.
- **애니메이션 《베이맥스 드림즈》**: 디즈니 TV 단편을 유니티로 실시간 제작/렌더링했으며, 이 협업으로 기술 공학 에미상을 받았습니다.
- **자동차(차량 UI, 내비게이션)**: 메르세데스-벤츠의 MBUX 서라운드 내비게이션은 유니티로 만든 3D 그래픽을 차량에 실시간으로 출력합니다.
- **자동차(제조, 디지털 트윈)**: 현대자동차는 유니티와 협력해 실제 공장을 가상으로 복제하여 시뮬레이션하는 메타팩토리(디지털 트윈) 구상을 발표했습니다.

에셋 스토어로 수익을 창출할 수 있다

유니티 **에셋 스토어** Asset Store에서는 게임 개발에 필요한 콘텐츠와 리소스를 거래할 수 있습니다. 2D나 3D 그래픽 리소스, 사운드, 효과뿐만 아니라 완성된 게임 프로젝트도 판매하므로 이를 활용하면 게임을 더 쉽고 빠르게 만들 수 있습니다. 또한 사용자가 직접 판매자가 되어 게임 출시 외에도 여러 콘텐츠를 거래해서 수익을 창출할 수도 있습니다.

● 효과(Effect)란 게임에서 시각적/청각적 피드백을 강화하는 모든 연출 요소입니다. 유니티에서는 파티클(Particle System/VFX Graph), Shader/Material, Post-Processing, 카메라/빛 연출, Decal/Line/Trail 등을 예로 들 수 있습니다.



유니티 에셋 스토어(https://assetstore.unity.com)

알아 두면 좋아요! 유니티에서 말하는 에셋이란?

유니티에서는 사운드, 이미지, 3D 모델 등의 리소스를 에셋 Asset이라고 합니다. 그래서 리소스를 판매하는 곳의 이름도 에셋 스토어입니다. 또한 프로젝트에서 사용하는 리소스를 저장하는 폴더 이름 역시 Assets입니다.

유니티 2020.x.x 버전부터 에셋 검색과 구매는 웹에서 이루어집니다. 웹에서 구매하거나 추가한 에셋은 유니티 에디터 안에 있는 Package Manager View의 [My Assets] 탭에서 내려받아 사용할 수 있습니다.

● 에셋은 02-2절에서 자세히 알아봅니다.

01-2

유니티 허브와 에디터 설치하기

유니티 엔진은 허브와 에디터로 구성됩니다. **유니티 허브**Unity Hub는 사용할 버전의 에디터를 설치하고 실행할 수 있도록 돕는 버전 관리 소프트웨어입니다. 이와 달리 **유니티 에디터**Unity Editor는 게임 개발에 필요한 다양한 메뉴와 툴바, 뷰 등을 제공하는 게임 개발 소프트웨어입니다.

Do it! 실습 유니티 허브 준비하기

01 유니티 허브는 유니티 홈페이지에서 내려받아 설치할 수 있습니다. 먼저 다음 URL로 홈페이지에 접속합니다.

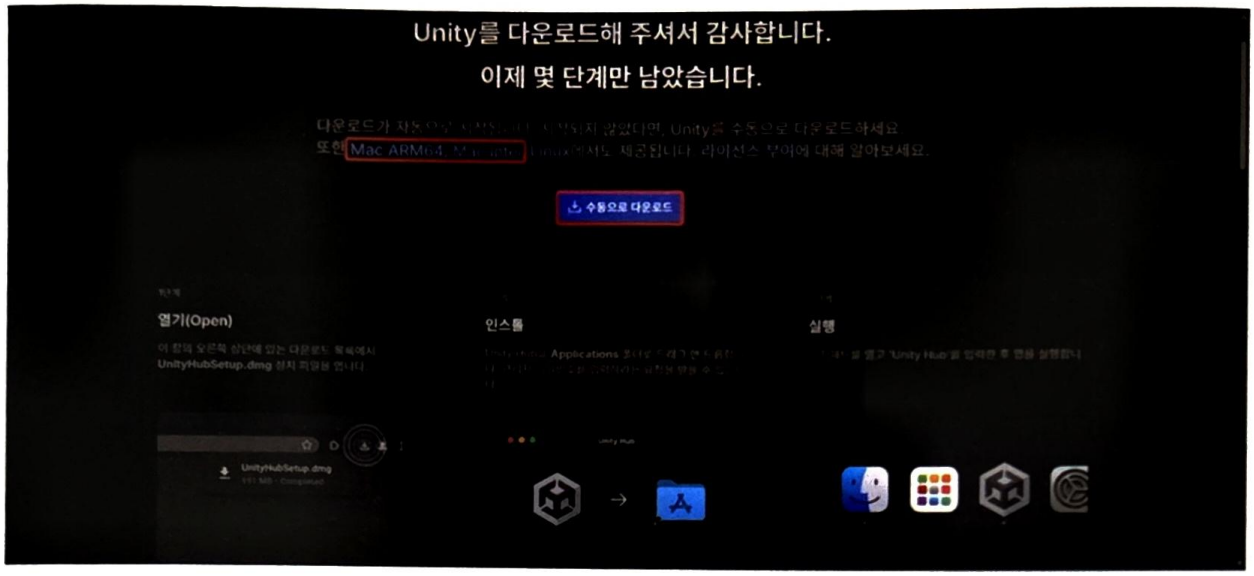
> 유니티 코리아: https://unity.com/kr

02 접속한 화면 왼쪽 아래 UNITY 다운로드 → 버튼을 클릭해서 유니티 허브 내려받기 페이지로 이동합니다.

유니티 홈페이지

03 내려받기 페이지가 열리면 수동으로 다운로드 버튼을 클릭해서 유니티 허브를 내려받습니다.

macOS를 사용한다면 [Mac ARM64] 또는 [Mac Intel] 링크를 클릭해서 내려받습니다.



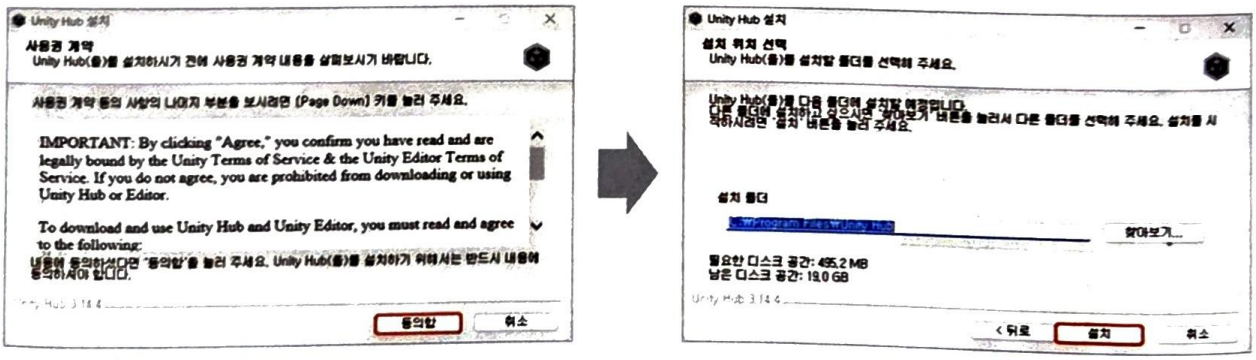
유니티 내려받기 페이지

유니티 홈페이지와 내려받기 페이지 등은 자주 바뀌므로 여러분이 보는 화면과 다를 수 있습니다.

04 내려받은 유니티 허브 설치 프로그램을 더블클릭해서 설치를 진행합니다.

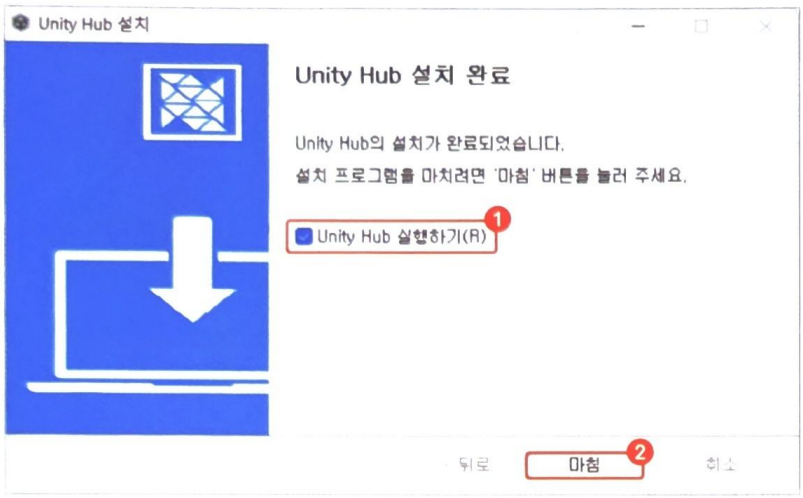
유니티 허브 설치 프로그램

05 다음과 같은 사용권 계약 창이 등장하면 〈동의함〉 버튼을 클릭하고 설치 폴더를 지정한 뒤, 〈설치〉 버튼을 클릭하여 유니티 허브를 설치합니다.



설치 시작하기

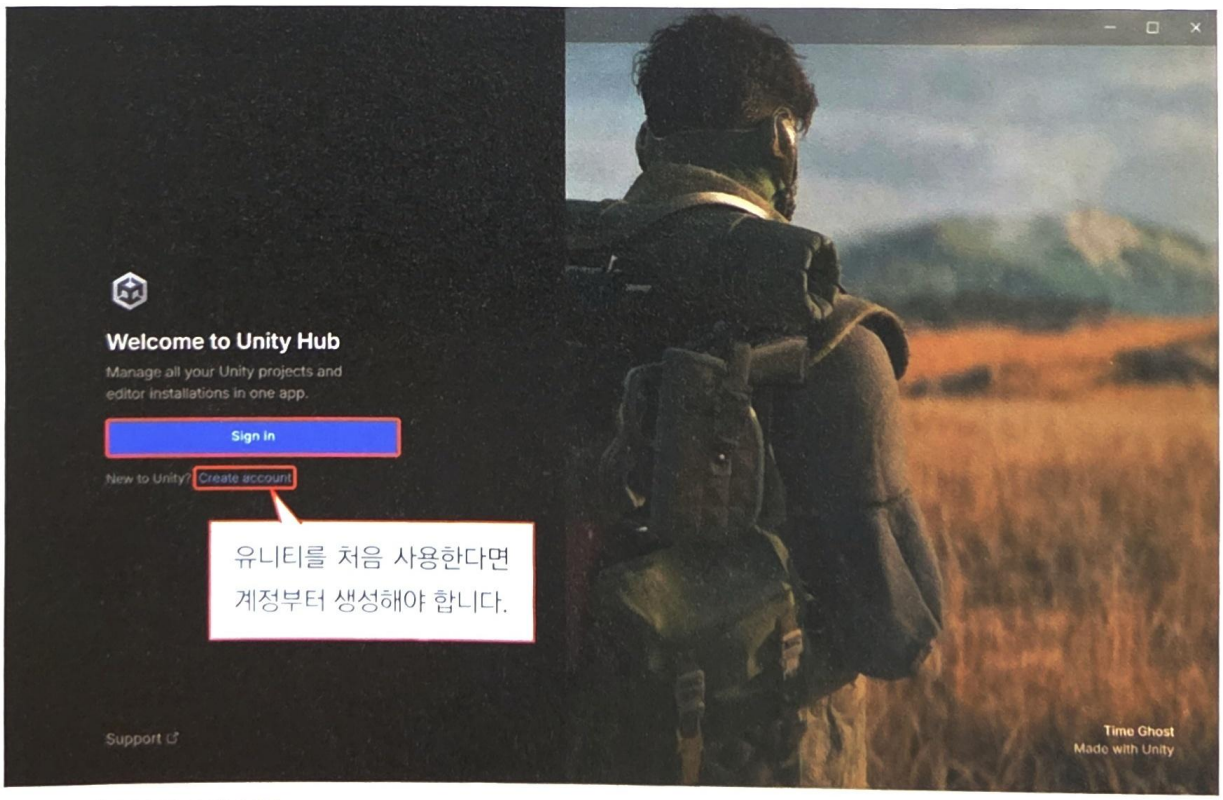
06 그러면 다음과 같이 유니티 허브 설치를 끝냈다고 알리는 화면이 등장합니다. ❶ [Unity Hub 실행하기] 앞에 체크 표시를 유지한 채 ❷ 〈마침〉 버튼을 클릭합니다.



설치 완료

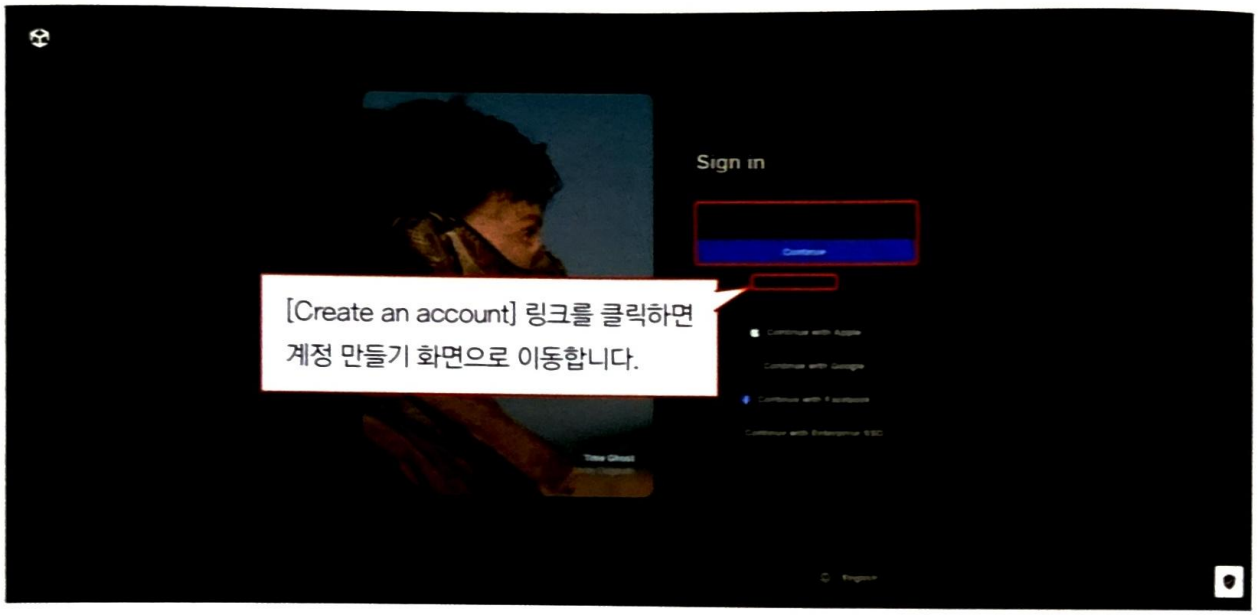
07 유니티 허브를 실행하면 다음과 같이 로그인 화면이 나타납니다. 로그인을 해야 유니티 허브 메인 화면으로 이동할 수 있습니다. 유니티를 처음 사용한다면 [Create account] 링크를 클릭해 계정을 생성해 봅시다.

PC 환경에 따라 실행 화면이 한글 또는 영문으로 나타날 수 있습니다.



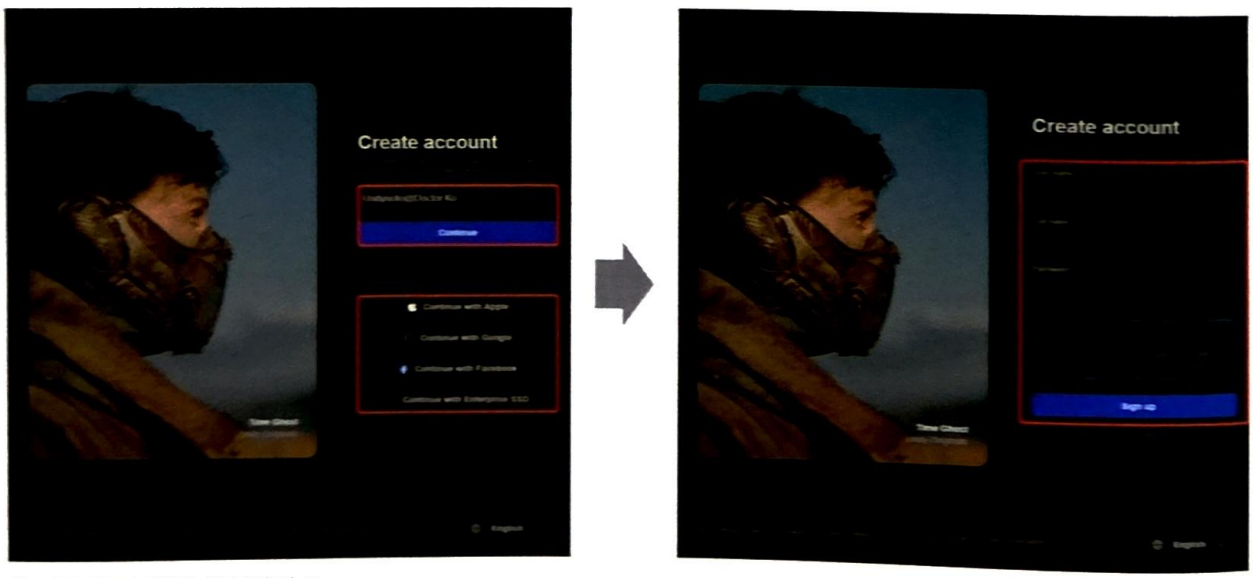
유니티 허브의 로그인 화면

08 [Create account] 링크를 클릭하면 로그인 화면으로 이동합니다. 계정이 없으면 [Create an account] 링크를 클릭해서 계정 생성 화면으로 이동하세요.



유니티 허브 계정 생성하기 1

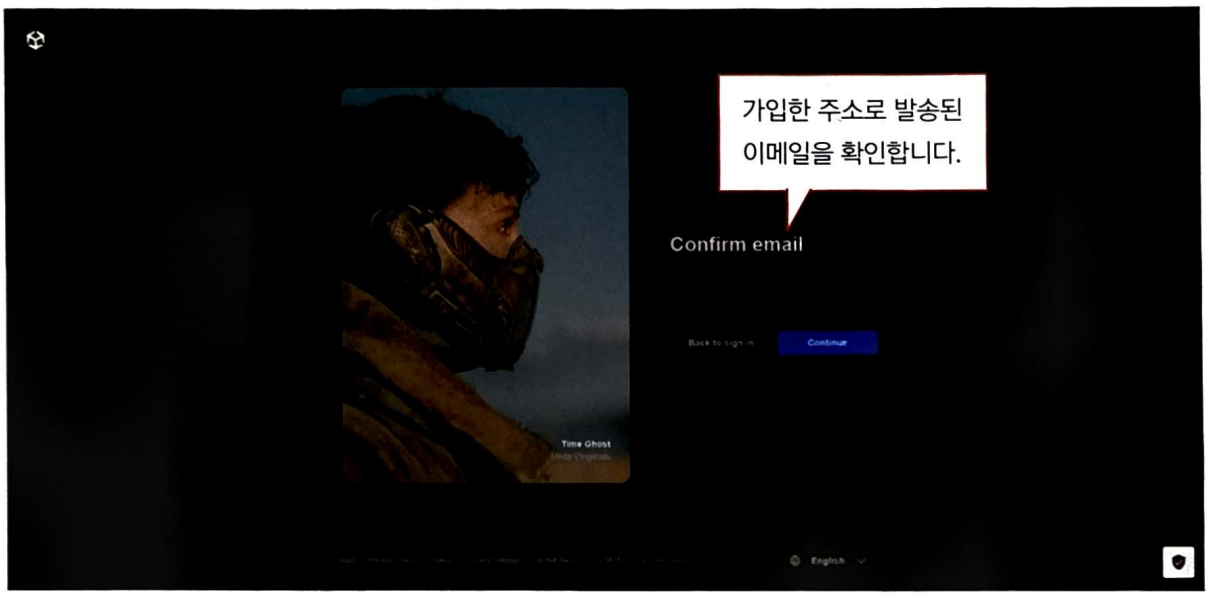
09 다음과 같은 유니티 계정 생성 화면에서 이메일, 사용자 이름, 암호 등을 입력하고 계정을 생성합니다. 또는 화면 아래에 있는 구글, 페이스북 등 외부 계정을 연동해서 유니티 계정으로 사용할 수도 있습니다.



유니티 허브 계정 생성하기 2

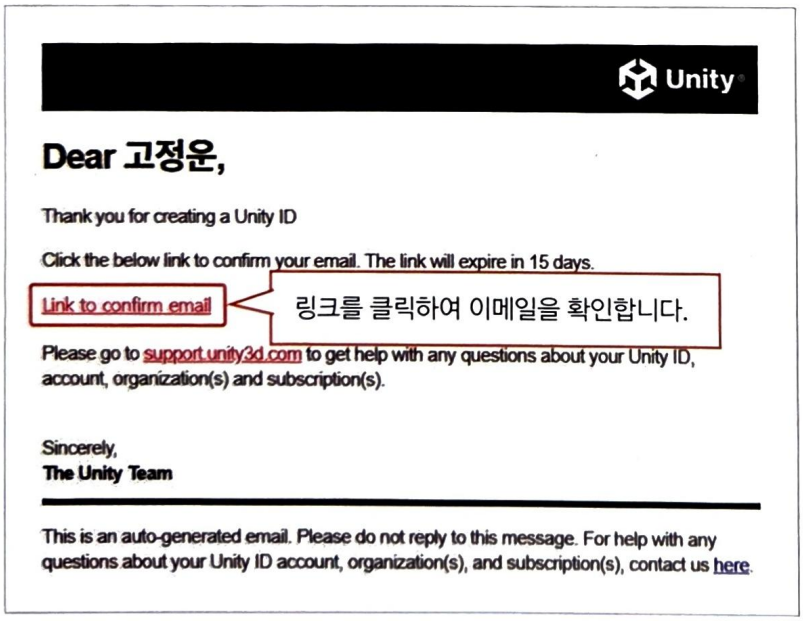
10 계정 생성이 끝나면 Confirm email 화면으로 이동하는데, 방금 가입한 이메일 주소로 메일을 발송했으니 확인하라는 안내 문구가 나옵니다.

외부 계정을 연동해서 가입했다면 이메일 검증 단계는 생략하고 서비스 약관 동의 단계를 진행합니다.



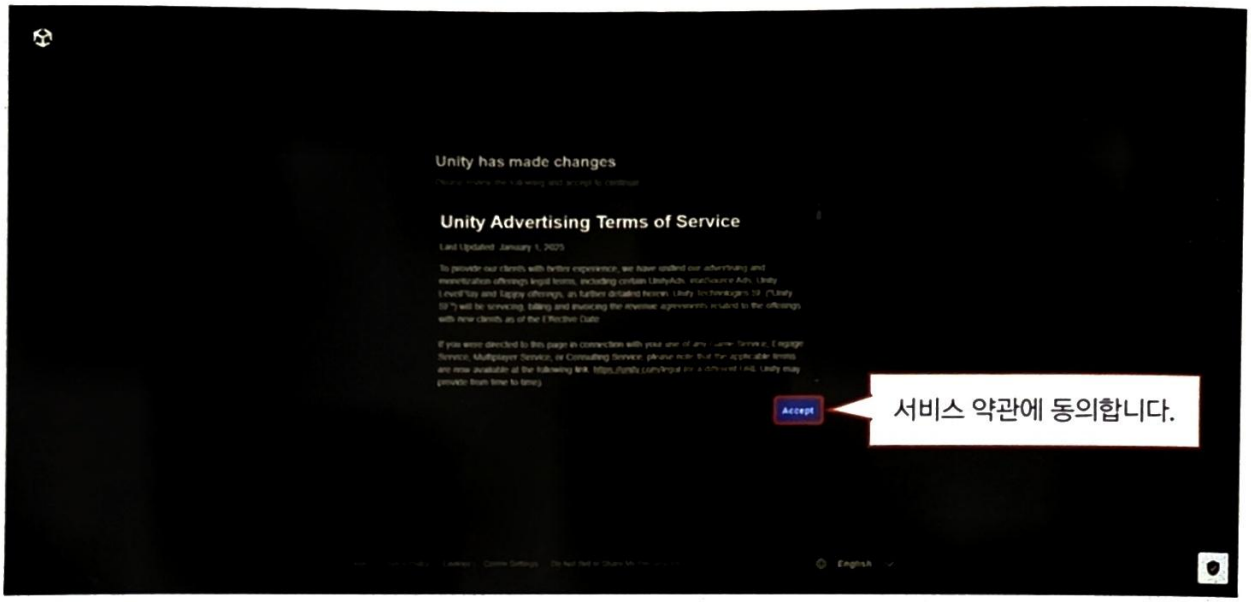
유니티 허브 계정 생성하기 3

11 유니티에서 발송한 이메일을 열고 [Link to confirm email] 링크를 클릭해 가입 여부를 확인합니다.



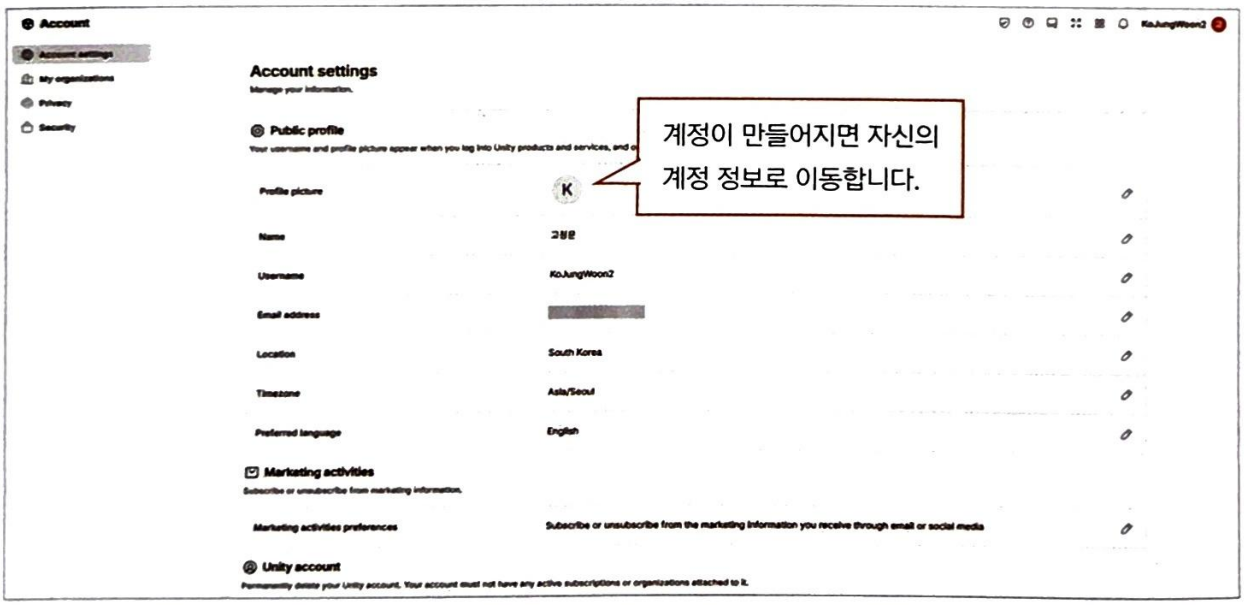
가입할 때 입력한 이메일 확인하기

12 [Link to confirm email] 링크를 클릭하면 유니티 서비스 약관 페이지로 이동하는데, 〈Agree〉 버튼을 클릭해서 서비스 약관에 동의합니다.



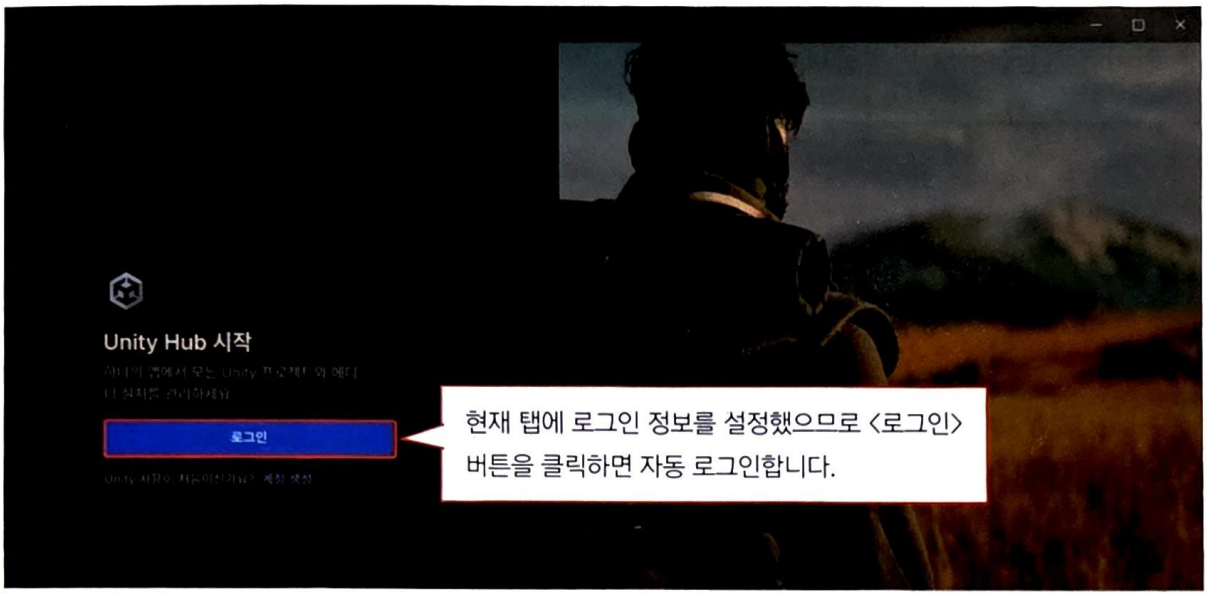
유니티 서비스 약관 페이지

13 유니티 계정 생성이 끝나면 다음처럼 계정 페이지가 나타납니다. 계정 정보를 확인했다면 창을 닫고 다시 유니티 허브로 이동합니다.



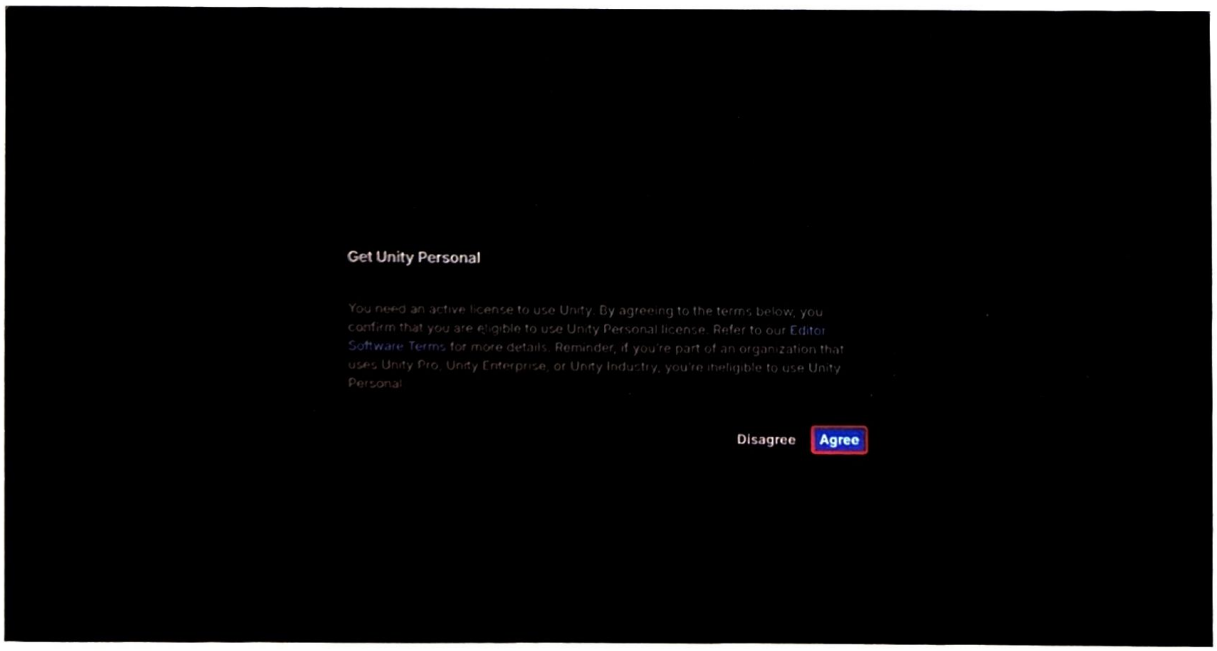
계정 페이지

14 〈로그인(Sign in)〉 버튼을 클릭하면 유니티 허브 메인 페이지로 이동합니다.



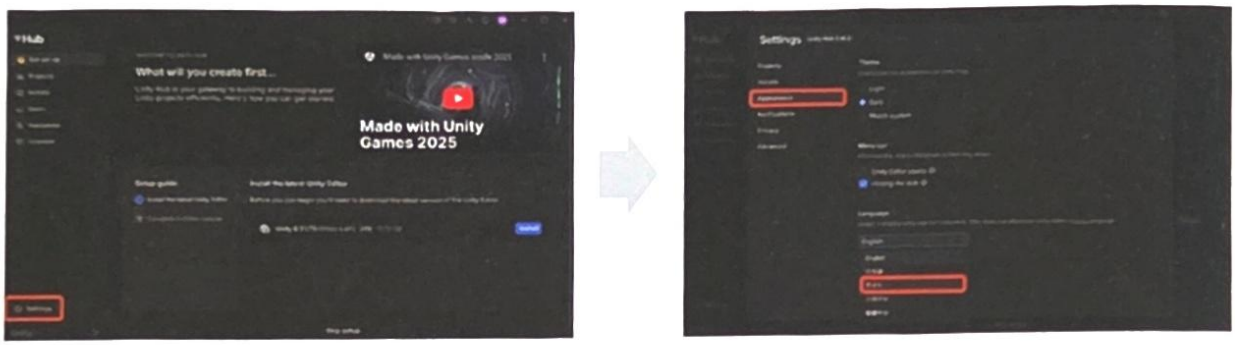
유니티 허브 로그인하기

15 유니티를 사용하려면 라이선스가 필요합니다. 유니티 허브 메인 페이지로 이동하면 무료로 사용할 수 있는 Get Unity Personal 팝업 창이 활성화됩니다. 이 창에서 〈Agree〉 버튼을 클릭해 Unity Personal 라이선스를 활성화합니다. Pro나 Enterprise 등의 라이선스를 사용하려면 유니티 허브 왼쪽 메뉴의 [Licenses] 탭에서 〈Add license〉 버튼을 클릭하고 해당하는 라이선스를 활성화합니다.



라이선스 동의하기

16 유니티 허브를 편리하게 사용하고자 언어를 '한국어'로 변경하겠습니다. 〈Settings〉 버튼을 클릭하면 나타나는 창에서 [Appearance] 탭을 클릭합니다. [Language] 옵션에서 사용 언어를 '한국어'로 변경합니다.

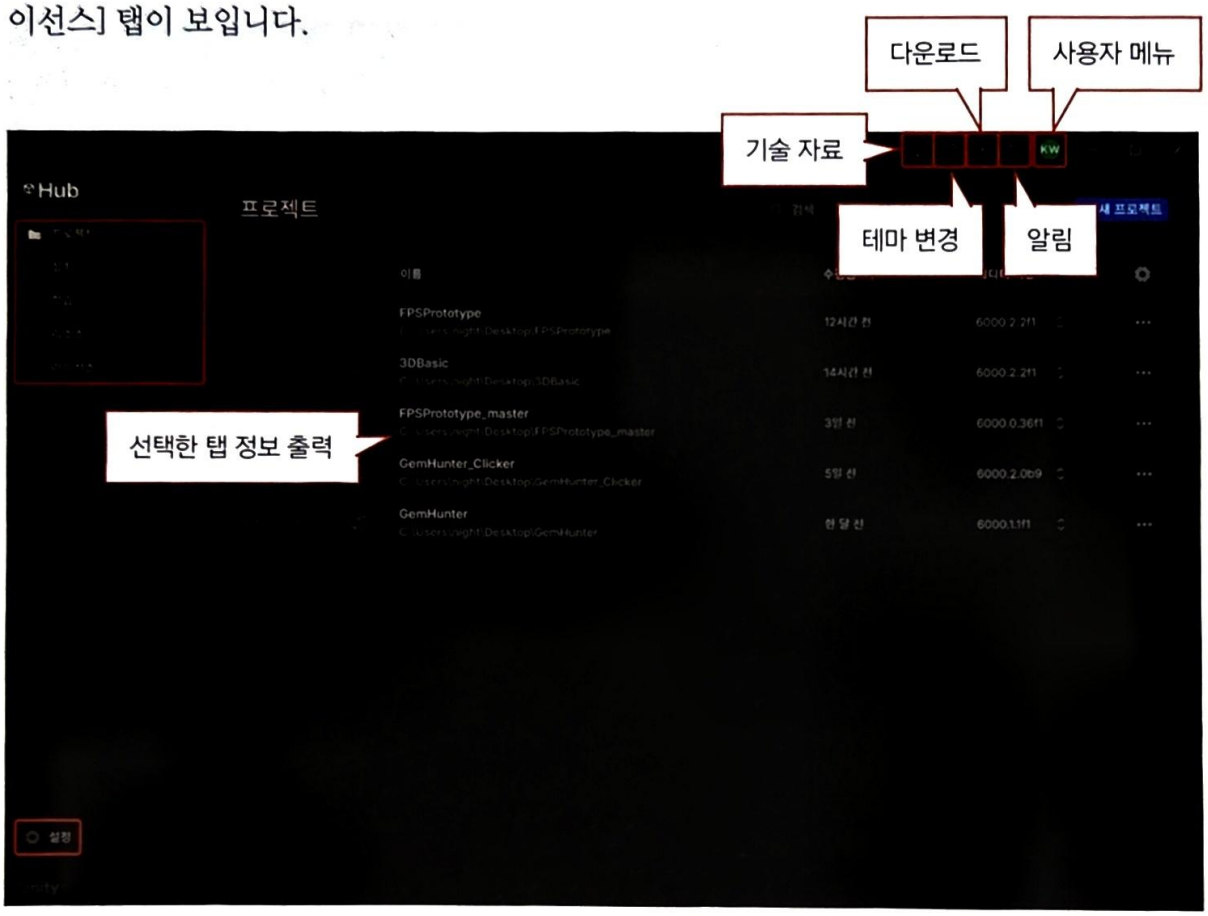


유니티 허브에서 사용할 언어 바꾸기

17 게임을 개발하려면 유니티 에디터가 필요합니다. 유니티 에디터를 설치하지 않았다면 최신 버전의 유니티 에디터 설치 가이드가 자동으로 활성화됩니다. 사용할 버전의 유니티 에디터를 설치하고자 화면 아래에서 〈설정 건너뛰기〉 버튼을 클릭합니다.

유니티 허브 시작하기

18 유니티 허브의 메인 화면 오른쪽 위에는 [기술 자료], [테마 변경], [다운로드], [알림], [사용자 메뉴] 아이콘이 있습니다. 그리고 왼쪽에는 [프로젝트], [설치], [학습], [리소스], [라이선스] 탭이 보입니다.

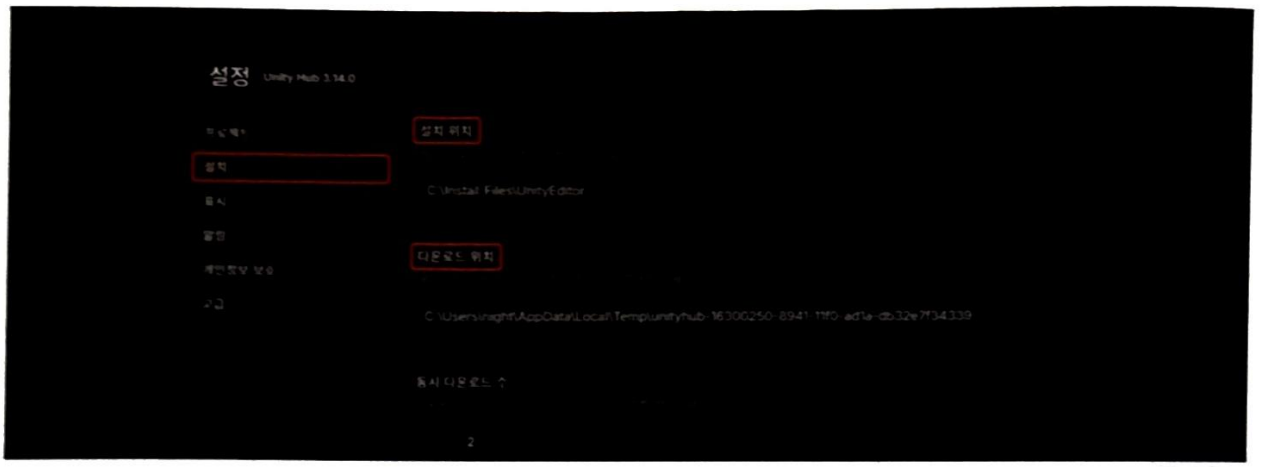


유니티 허브 메인 화면

메인 화면 왼쪽에 있는 5가지 탭의 기능을 하나씩 살펴봅시다.

- **프로젝트**: 이전에 작업한 유니티 프로젝트 정보를 볼 수 있고, 설치한 버전으로 유니티 프로젝트를 생성하거나 열 수 있습니다.
- **설치**: 사용할 버전의 유니티 에디터를 설치하거나 제거할 수 있습니다.
- **학습**: 프로젝트, 튜토리얼 등 유니티에서 제공하는 콘텐츠 학습 자료를 볼 수 있습니다.
- **리소스**: 유니티와 관련한 다양한 정보를 얻을 수 있는 웹 사이트 목록을 볼 수 있습니다.
- **라이선스**: Personal, Pro 등의 라이선스를 활성화하고 계정 연동, 시트 할당/반납, 오프라인 파일 처리 등을 관리합니다.

19 메인 화면의 왼쪽 아래 〈설정〉 버튼을 클릭하면 설정 창이 열리는데, 이 창에서 환경 설정을 변경할 수 있습니다.



유니티 허브 환경 설정하기([설치] 탭)

설정 창의 탭 기능을 하나씩 살펴봅시다.

- **프로젝트**: 새 프로젝트를 저장할 기본 위치를 설정할 수 있습니다.
- **설치**: 에디터를 설치할 위치와 내려받은 파일을 저장할 위치를 설정할 수 있습니다. 기본 설치 위치는 'C:/Program Files/Unity/Hub/Editor/' 입니다.
- **표시**: 유니티 허브의 테마, 메뉴 표시줄, 언어를 선택할 수 있습니다. 테마는 밝은 화면을 선호하면 '밝음'을, 이 책처럼 어두운 화면을 선호하면 '어둠'을 선택합니다.

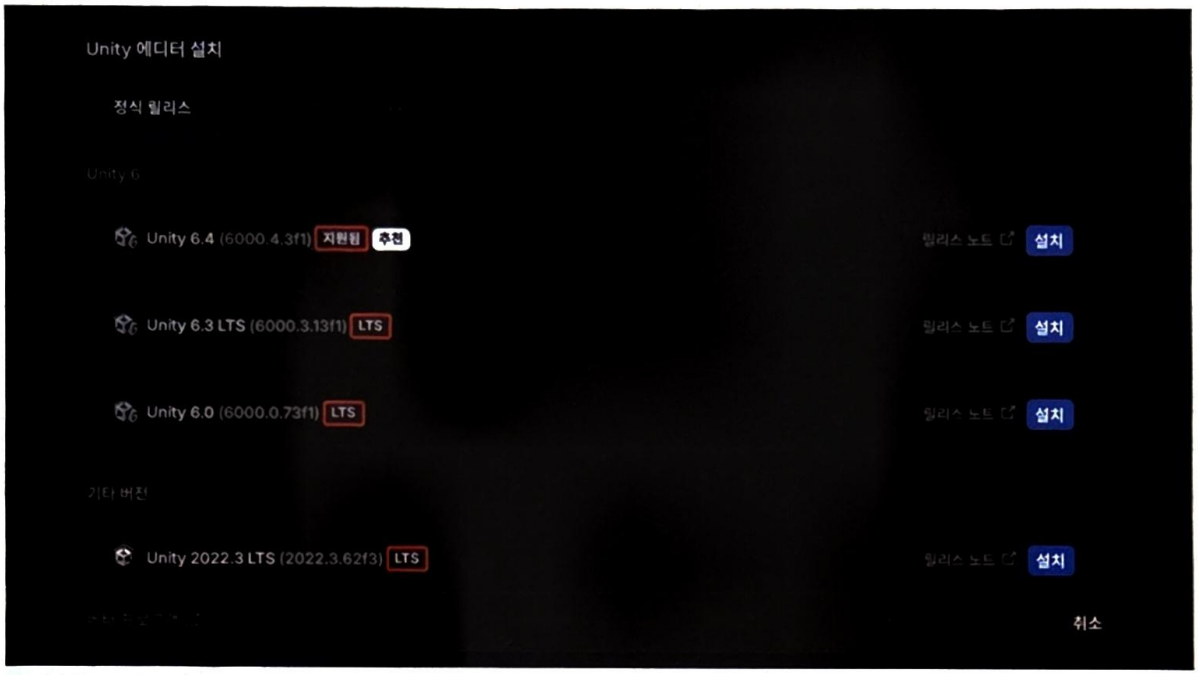
Do it! 실습 유니티 에디터 설치하기

유니티 허브를 준비했다면 이제 실제로 작업이 이루어지는 유니티 에디터를 설치합니다.

01 ❶ [설치] 탭 오른쪽 위에 있는 ❷ **에디터 설치** 버튼을 클릭해 봅시다.

에디터 설치 시작하기

02 그러면 다음과 같이 유니티 에디터 설치 창이 나타납니다.



설치할 버전 확인하기

[정식 릴리스] 탭에는 **지원됨** Supported과 LTS Long Term Support 버전이 있는데, 지원됨 버전은 정식 출시 버전f, final 중 최신 버전이고 LTS 버전은 오랫동안 지원하는 안정 버전입니다.

[사전 릴리스] 탭에서는 알파와 베타 버전 중에서 최신 버전을 설치할 수 있지만, 정식으로 출시된 버전보다 버그가 많으므로 프로젝트 개발보다 새로운 기능을 미리 경험하는 용도로만 사용하는 것이 좋습니다.

알아 두면 좋아요! 유니티 버전은 이렇게 해석하세요

유니티 에디터 설치 화면을 보면 버전을 6000.4.3f1처럼 표기합니다. 일반적으로 유니티 버전은 A.B.Cf1 형식으로 나타내는데, 유니티 5에서는 2017.x.xf1~2023.3.xf1과 같이 A 위치에 출시한 연도를 표기했습니다.

이와 달리 유니티 6은 연도 대신 6000으로 통일해서 표기합니다. 기존 유니티 5와 차별된 버전이라는 것을 나타내고자 6을 강조해 6.0.0f1처럼 나타내고 싶었지만, 기존 버전의 연도 표기 형식이 4자리 숫자여서 6000으로 나타냈습니다.

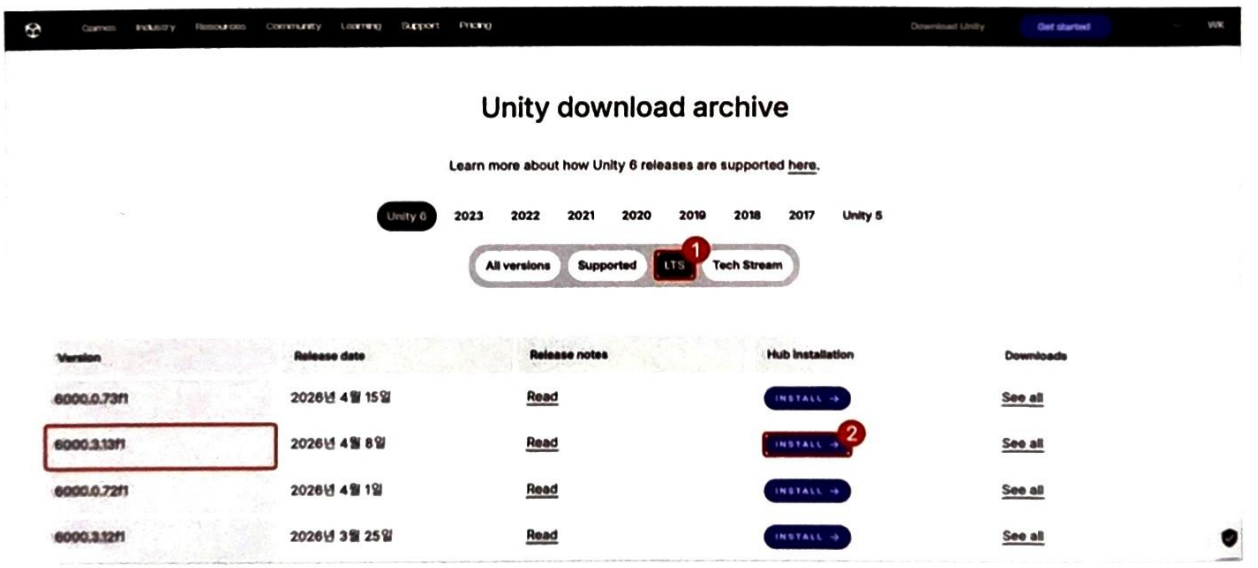
▶ 계속

B 위치에는 유니티 6.1, 6.2, 6.3처럼 기능을 대폭 업데이트할 때 유니티 6의 새로운 버전을 숫자로 나타냅니다. 그래서 B 위치의 숫자가 다르면 버전 간에 호환 문제가 발생할 수도 있습니다.
C 위치 숫자는 전체 내용이 바뀌지 않는 작은 기능을 추가하거나 버그처럼 사소한 수정을 했을 때 증가합니다. 따라서 C 위치 숫자가 달라도 호환성에는 거의 영향이 없으므로 예를 들어 6000.4.0f1 버전에서 6000.4.3f1 버전으로 업데이트해도 큰 문제가 생기지 않습니다.

03 유니티 허브는 각 버전의 최신 버전만 표시하므로 다른 버전을 내려받고 싶다면 ❶ [아카이브] 탭에서 ❷ [아카이브 다운로드] 링크를 클릭한 후 선택해야 합니다. 여기에서는 집필 시점 최신 버전인 6000.4.3f1을 선택합니다.

설치할 버전 선택하기 1

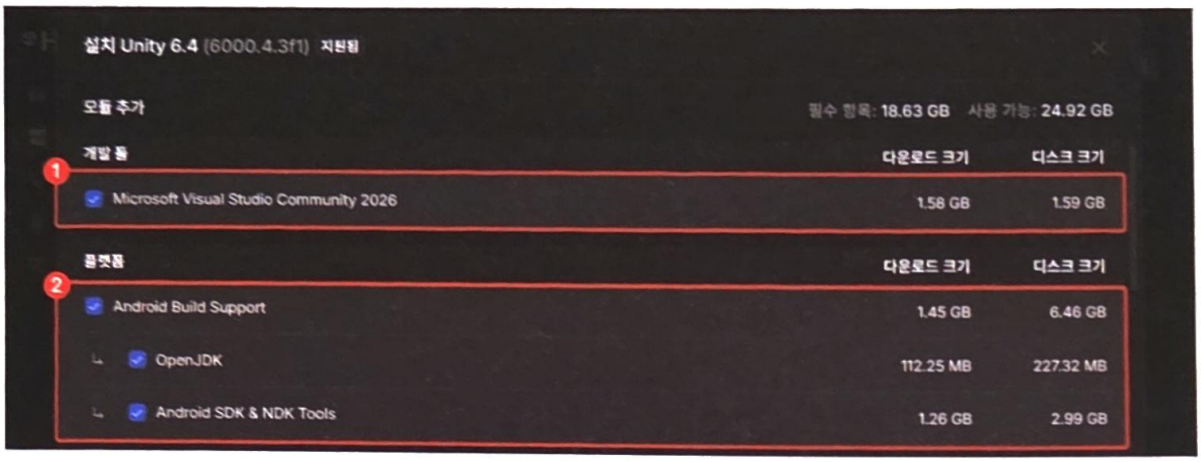
04 LTS 버전을 설치하려면 모든 버전이 있는 [Unity download archive] 페이지에서 ❶ LTS 버전 중 ❷ 6000.3.13.f1 버전을 선택하고 INSTALL → 버튼을 클릭합니다.



설치할 버전 선택하기 2

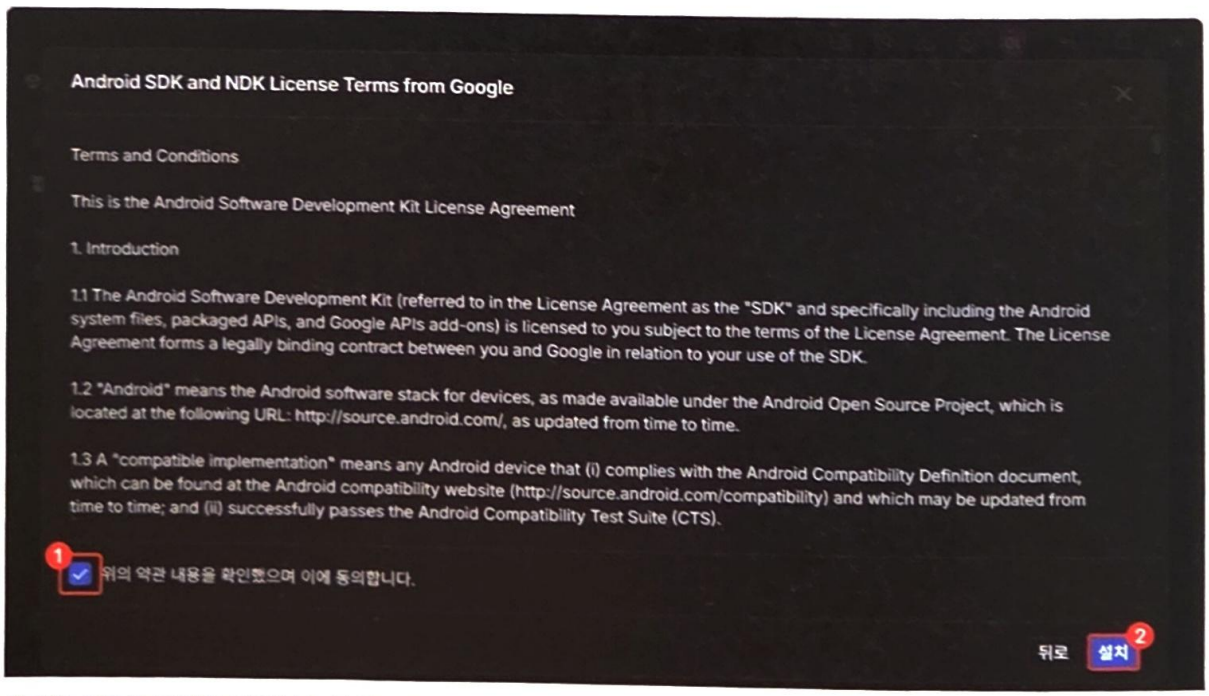
05 비주얼 스튜디오Visual Studio와 같은 코드 편집 도구를 설치하지 않았다면 ❶ [Microsoft Visual Studio Community 2026]에 체크합니다. 설치하는 버전의 유니티 에디터에 필요한 추가 모듈도 함께 선택합니다. 안드로이드에서 동작하는 게임을 개발하려면 ❷ [Android Build Support], [OpenJDK], [Android SDK & NDK Tools] 모듈에 체크합니다. 그리고 **계속** 버튼을 클릭합니다.

macOS나 iOS에서도 실행하려면 [iOS Build Support] 모듈에 체크합니다. 이처럼 사용자 운영체제에 맞는 모듈에 체크 표시하면 쉽게 추가할 수 있습니다.



함께 설치할 모듈 선택하기

06 각 모듈과 관련된 라이선스를 안내하고 동의를 구하는 창이 나타납니다. ❶ [위의 약관 내용을 확인했으며 이에 동의합니다.]에 체크하고 ❷ **설치** 버튼을 클릭해서 모듈을 설치합니다.

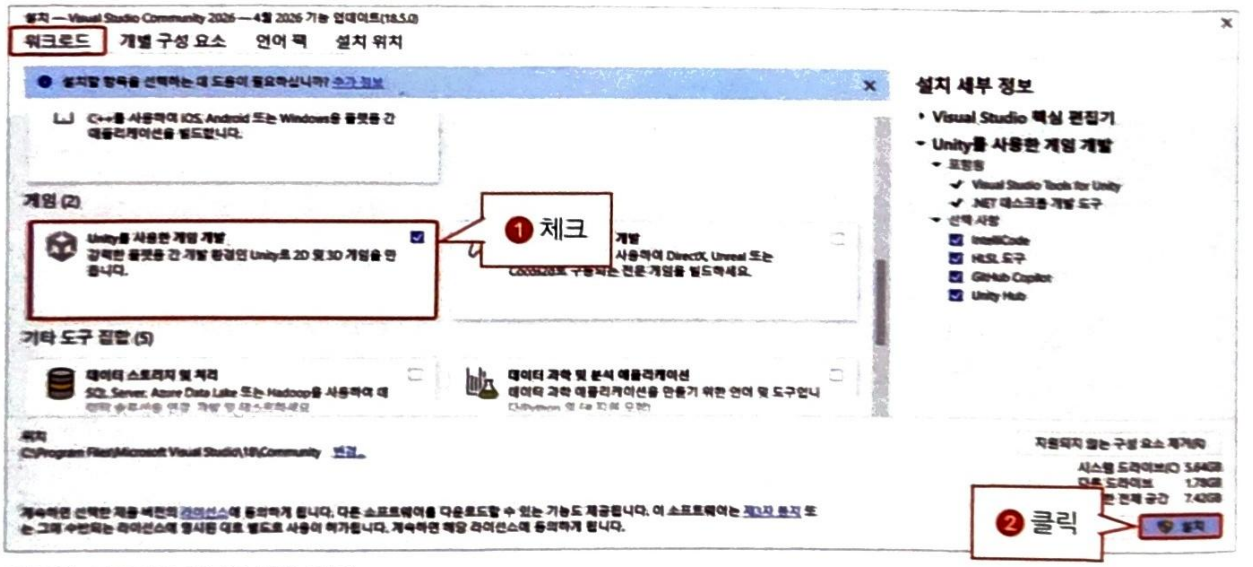


설치할 모듈과 관련한 라이선스 동의하기

07 다시 설치 창이 나타나면 오른쪽 위 내려받기 버튼 ❶을 클릭합니다. 다음처럼 현재 설치 중인 유니티 에디터 정보와 함께 설치 상태를 확인할 수 있습니다.

에디터와 모듈을 설치하는 모습

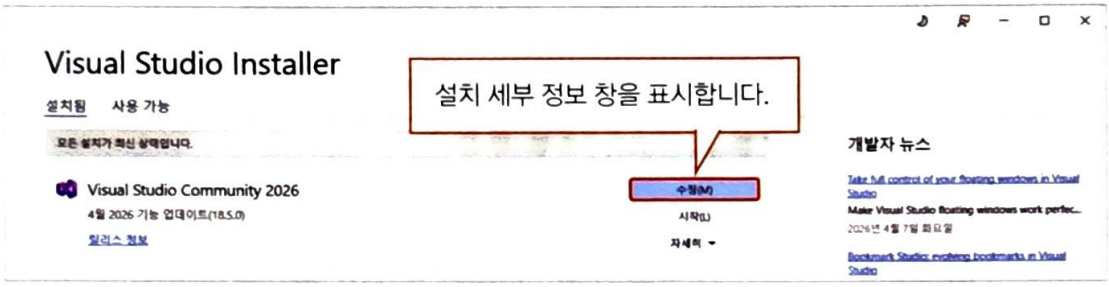
08 비주얼 스튜디오 2026도 설치한다면 유니티 6을 설치하는 도중에 설치 화면이 나타나고 설치 세부 정보 창이 활성화됩니다. 유니티 에디터와 비주얼 스튜디오를 연동해서 코드 자동 완성과 같은 기능을 사용하려면 [워크로드] 탭에서 아래로 스크롤해 ❶ [Unity를 사용한 게임 개발]에 체크한 뒤 ❷ 설치 버튼을 클릭합니다.



비주얼 스튜디오 2026 설치 화면

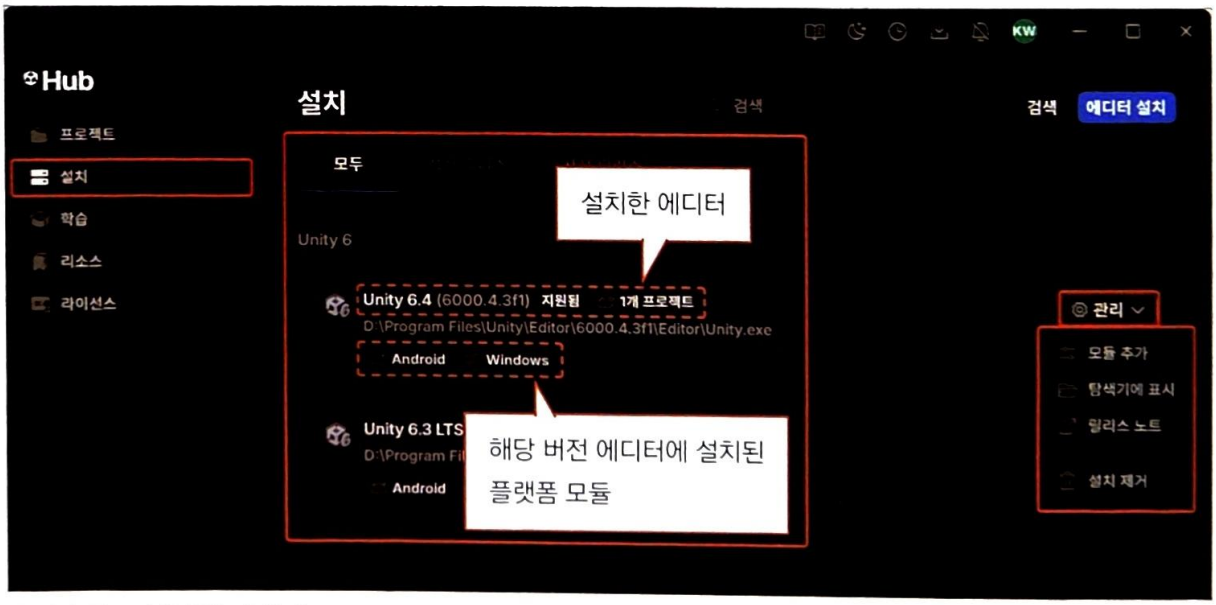
알아 두면 좋아요! 비주얼 스튜디오를 설치할 때 워크로드를 확인하지 못했다면?

비주얼 스튜디오를 이미 설치했거나 설치 세부 정보 창을 보지 못한 채 설치를 진행했다면 다시 Visual Studio Installer 프로그램을 실행합니다. 그리고 현재 설치한 버전인 Visual Studio Community 2026의 <수정(M)> 버튼을 클릭해 봅시다. 그러면 앞서 설명한 내용을 따라 하며 [Unity를 사용한 게임 개발]을 설치할 수 있습니다.



Visual Studio Community 2026 설치 수정하기

09 [설치] 탭을 보면 앞에서 설치한 유니티 에디터 버전, 설치 위치, 추가로 설치한 모듈 정보를 알 수 있습니다. 유니티 에디터 버전별로 오른쪽에 있는 관리 버튼을 클릭하면 모듈을 추가로 설치하거나 삭제할 수 있습니다.



유니티 허브의 [설치] 탭 화면

02장

유니티 에디터 인터페이스와 핵심 기능 알아보기

유니티 에디터를 설치했다면 유니티 허브로 새로운 프로젝트를 만들고 기존 프로젝트도 열어 봅시다. 그다음으로 유니티 에디터 구성 요소를 살펴보고, 이어서 게임을 개발할 때 기본으로 사용하는 각종 뷰의 역할을 알아봅니다.
마지막으로, 유니티의 기본 용어와 좌표 체계, 게임오브젝트와 컴포넌트를 이해하고 카메라와 빛 오브젝트 기능까지 자세히 살펴봅니다.

학습 목표

1 유니티 에디터의 주요 인터페이스(Scene, Game, Hierarchy, Inspector) 기능을 이해하고 조작할 수 있다.
2 게임오브젝트, 컴포넌트, 에셋, 프리팹의 관계를 이해하고 카메라와 조명 설정을 적용할 수 있다.

02-1 유니티 인터페이스 살펴보기
02-2 게임오브젝트와 컴포넌트 알아보기

02-1

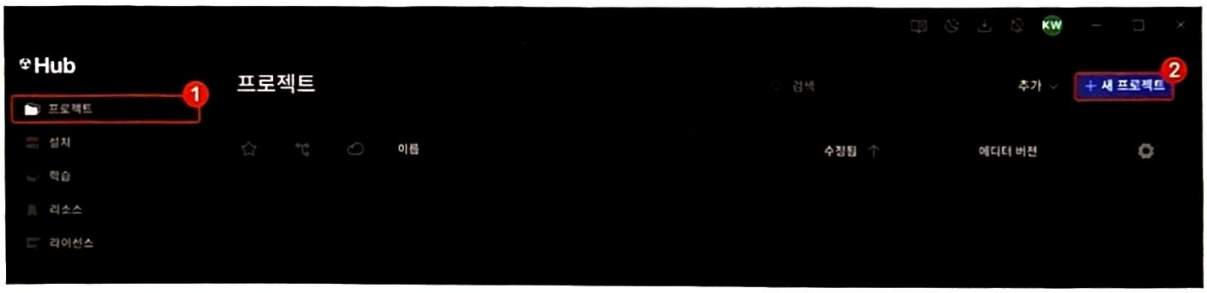
유니티 인터페이스 살펴보기

먼저 앞서 설치한 유니티 허브로 새로운 프로젝트를 생성합니다. 새 프로젝트가 생성되면 자동으로 유니티 에디터를 실행하는데, 여기에서는 주로 사용하는 메뉴, 툴바, 뷰 등을 살펴봅니다. 이와 함께 게임을 개발할 때 기본으로 사용하는 Scene View, Game View, Hierarchy View, Inspector View, Project View, Console View 등 각종 뷰View를 자세히 알아봅니다.

Do it! 실습 새 프로젝트 생성하기

유니티 에디터와 필요한 모듈을 모두 설치했다면 새 프로젝트를 생성해 보고 기존 프로젝트도 열어 봅시다.

01 유니티 허브의 메인 화면에서 ❶ [프로젝트] 탭을 클릭한 후 오른쪽 위 ❷ + 새 프로젝트 버튼을 클릭합니다.



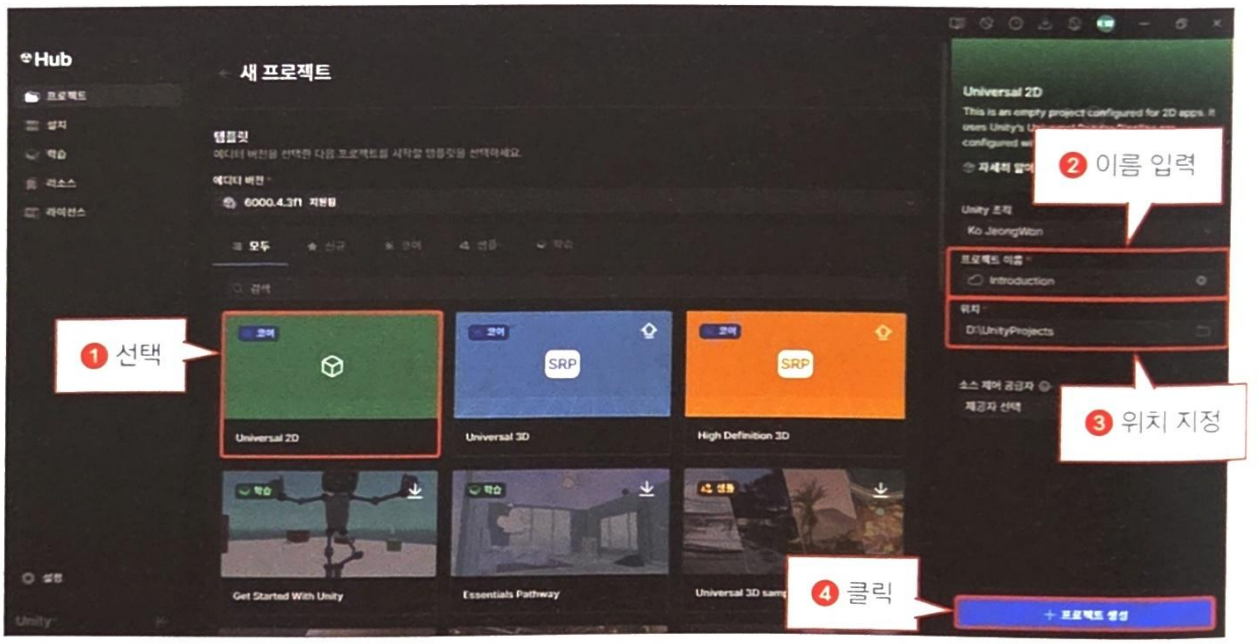
유니티 허브의 [프로젝트] 탭

02 에디터 버전 항목에서 사용할 버전을 선택합니다. 앞서 설치한 6000.4.3f1 버전을 선택합니다.

새 프로젝트 만들기

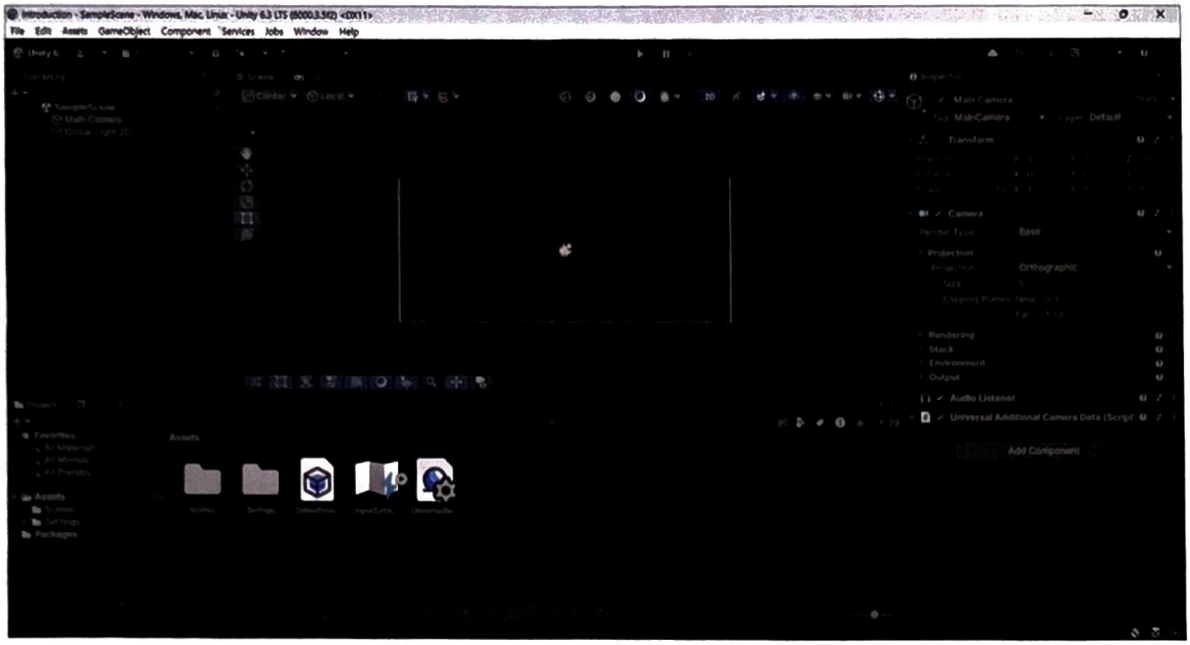
03 프로젝트 템플릿을 선택합니다. 일반적으로 유니티 6의 기본 템플릿인 Universal 2D 또는 Universal 3D를 사용합니다. ❶ 여기서는 [Universal 2D]를 선택합니다. 그다음으로 ❷ 프로젝트 이름과 ❸ 프로젝트 생성 위치를 지정하고 ❹ + 프로젝트 생성 버튼을 클릭합니다.

고해상도 3D 게임을 제작하려면 [High Definition 3D] 템플릿을 선택합니다.



템플릿 종류를 선택하고 프로젝트 이름과 위치 지정하기

04 프로젝트 생성이 끝나면 자동으로 에디터를 실행하고 다음처럼 프로젝트 화면을 엽니다.



새 프로젝트 화면

유니티 에디터 화면 구성은 화면 구성 살펴보기 항에서 자세히 살펴봅니다.

알아 두면 좋아요! 템플릿에서 Universal과 High Definition이란?

Universal은 Universal Render Pipeline (URP)을 나타내는 용어로, 유니티 6의 기본 렌더 파이프라인을 뜻합니다. 유니티에서 만들어 미리 빌드한 스크립터블 렌더 파이프라인Scriptable Render Pipeline, SRP으로, 렌더 파이프라인은 스크립트로 수정할 수 있습니다.

URP는 아티스트 친화적인 워크플로를 지원합니다. 즉, URP는 파티클, 셰이더 같은 요소를 코드 없이 노드 기반으로 구성할 수 있게 만들어졌습니다. 이를 통해 모바일, 고사양 콘솔, PC 등 다양한 플랫폼에서 최적화된 그래픽을 쉽고 빠르게 구현할 수 있습니다.

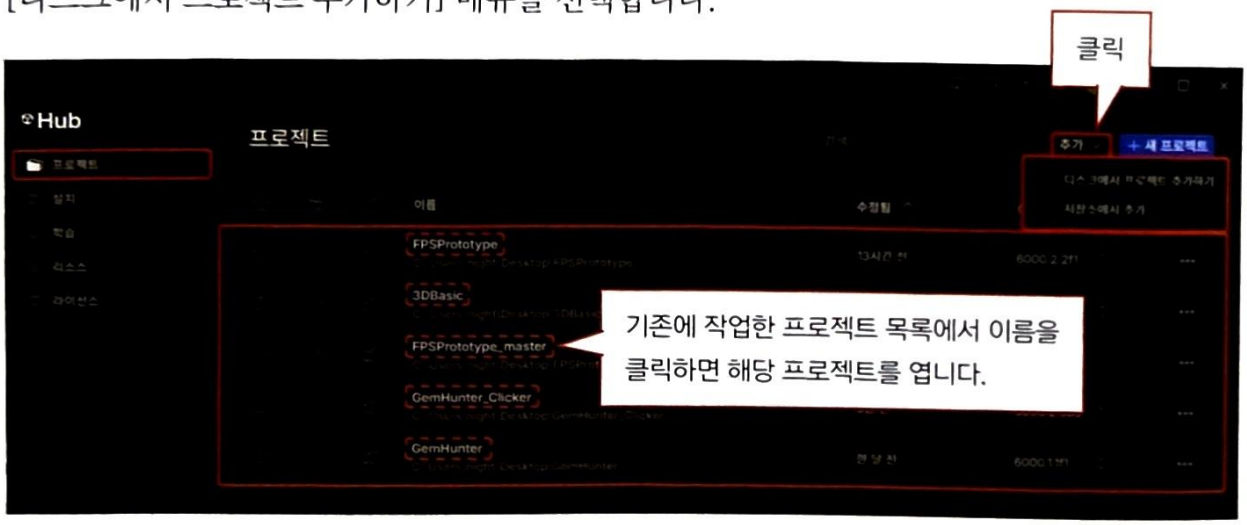
High Definition은 High Definition Render Pipeline (HDRP)을 나타내는 용어로, URP와 마찬가지로 유니티에서 만들어 빌드한 SRP입니다. HDRP를 사용하면 고사양 플랫폼을 위한 최신 고해상도 그래픽을 구현할 수 있습니다.

Do it! 실습 기존 프로젝트 열기

01 유니티 허브를 실행하면 다음과 같이 [프로젝트] 탭에서 기존에 작업한 프로젝트를 확인할 수 있으며 이름을 클릭하면 해당 프로젝트를 엽니다.

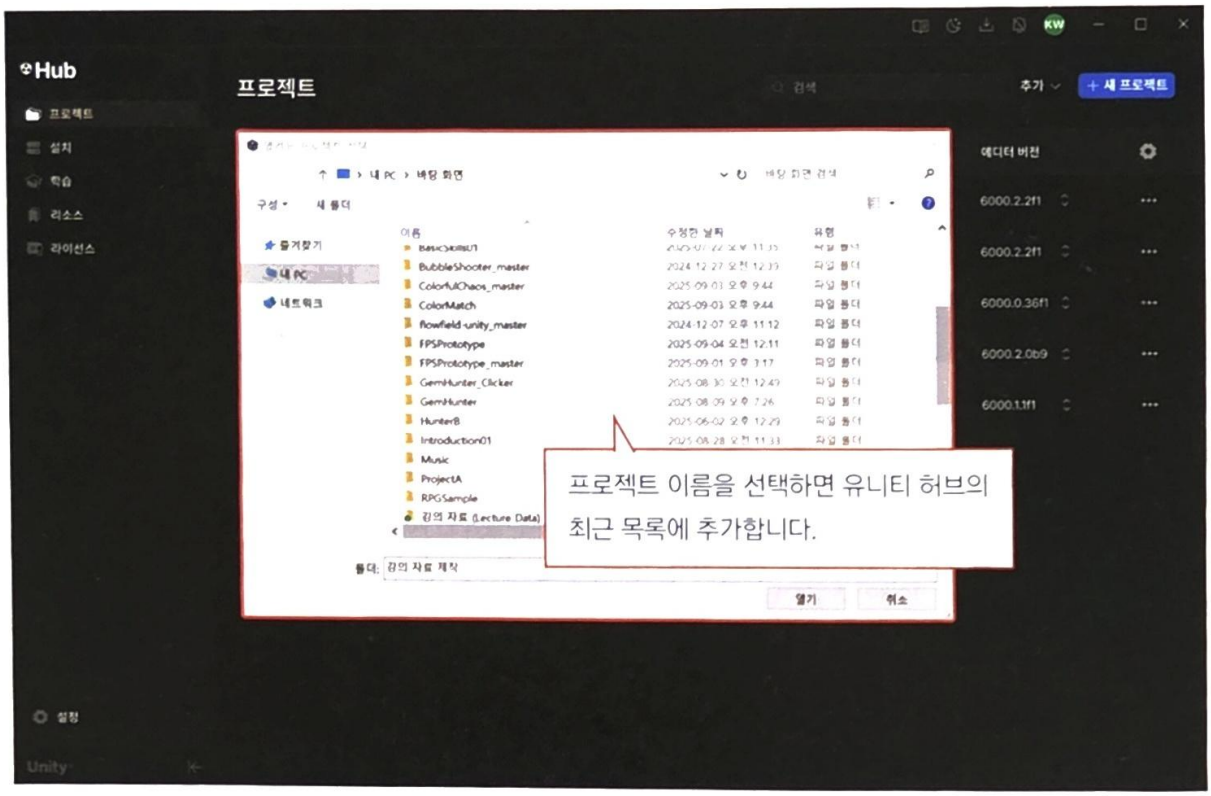
작업 중인 프로젝트

02 목록에 없는 다른 프로젝트를 추가하려면 화면 오른쪽 위에서 추가 버튼을 클릭하고 [디스크에서 프로젝트 추가하기] 메뉴를 선택합니다.



프로젝트 목록

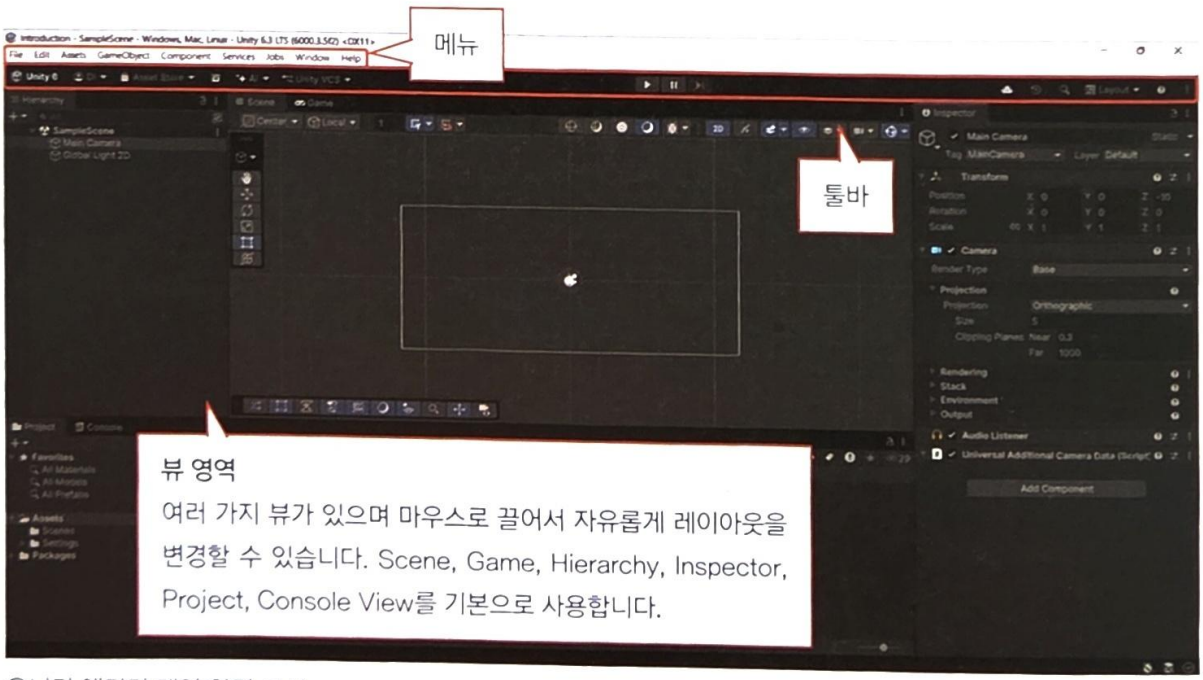
03 프로젝트를 저장한 폴더를 선택해서 해당 프로젝트를 가져옵니다.



프로젝트 가져오기

유니티 에디터 화면 구성 살펴보기

유니티 에디터는 화면 위 **메뉴**Menu와 **툴바**Toolbar 영역, 그리고 기능에 따라 분류한 **뷰**View를 배치한 영역으로 나눌 수 있습니다.

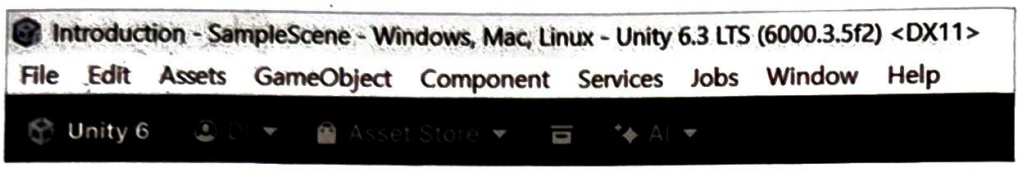


유니티 에디터 메인 화면 구성

유니티 에디터의 레이아웃은 자유롭게 변경할 수 있으므로 이 화면은 여러분이 사용하는 화면과 다를 수 있습니다.

메뉴

유니티 에디터의 메뉴 영역에는 [File], [Edit], [Assets], [GameObject], [Component] 등의 메뉴가 있습니다.

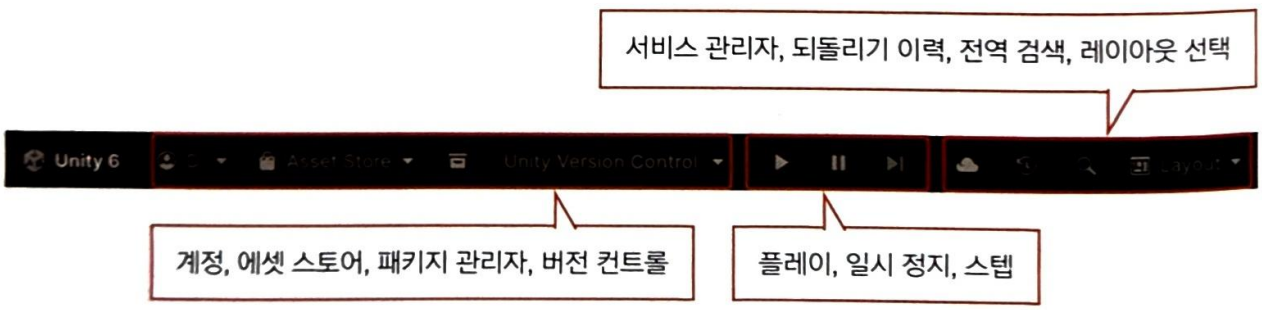


메뉴 영역

- **File 메뉴**: 프로젝트 또는 게임 장면이나 상태를 저장하는 단위인 씬scene을 생성하거나 저장하고, 프로젝트를 빌드build해서 *.exe, *.apk 등의 파일을 생성할 때 필요한 메뉴가 있습니다.
- **Edit 메뉴**: 프로젝트에서 사용하는 단축키 설정, 프로젝트 빌드 설정 등 현재 프로젝트나 에디터의 옵션을 설정할 때 사용하는 메뉴가 있습니다.
- **Assets 메뉴**: 프로젝트에 사용한 리소스인 에셋을 제어할 때 필요한 메뉴가 있습니다. 프로젝트에 포함된 에셋 목록을 나열하는 Project View(화면 왼쪽 아래)의 빈 곳을 마우스 오른쪽 버튼으로 눌러도 똑같은 메뉴가 나타납니다.
- **GameObject 메뉴**: 게임에서 사용하는 오브젝트를 생성할 때 사용하는 메뉴가 있습니다. 씬의 오브젝트 목록을 나열하는 Hierarchy View(화면 왼쪽)의 왼쪽 위에 있는 ＋▾ 버튼을 클릭하거나 Hierarchy View에서 빈 곳을 마우스 오른쪽 버튼으로 눌러도 똑같은 메뉴가 나타납니다.
- **Component 메뉴**: 물리, 충돌 범위, 오디오 등 게임오브젝트에 부착하는 속성을 추가할 때 사용하는 메뉴가 있습니다. Hierarchy View에서 게임오브젝트를 선택하고 Inspector View(화면 오른쪽) 아래에 있는 <Add Component> 버튼을 클릭해도 똑같은 메뉴가 나타납니다.
- **Window 메뉴**: 에셋 스토어, 패키지 매니저를 포함해 다양한 뷰View를 열 때 사용하는 메뉴가 있습니다.
- **Help 메뉴**: 유니티 정보, 라이선스, 도움말 등을 확인할 때 사용하는 메뉴가 있습니다.

툴바

툴바는 메뉴 영역 바로 아래에 있으며 왼쪽, 가운데, 오른쪽의 세 영역으로 이루어집니다.



툴바의 세 영역

툴바 왼쪽에는 유니티 계정 정보를 확인하거나 로그인/로그아웃을 할 수 있는 계정, 에셋 스토어, 프로젝트에서 사용하는 패키지 관리자, 유니티의 버전 관리 시스템인 버전 컨트롤 등이 있습니다.

툴바 가운데에는 게임 플레이, 게임 일시 정지, 일시 정지한 게임을 1단계씩 실행하는 스텝 버튼이 있습니다.

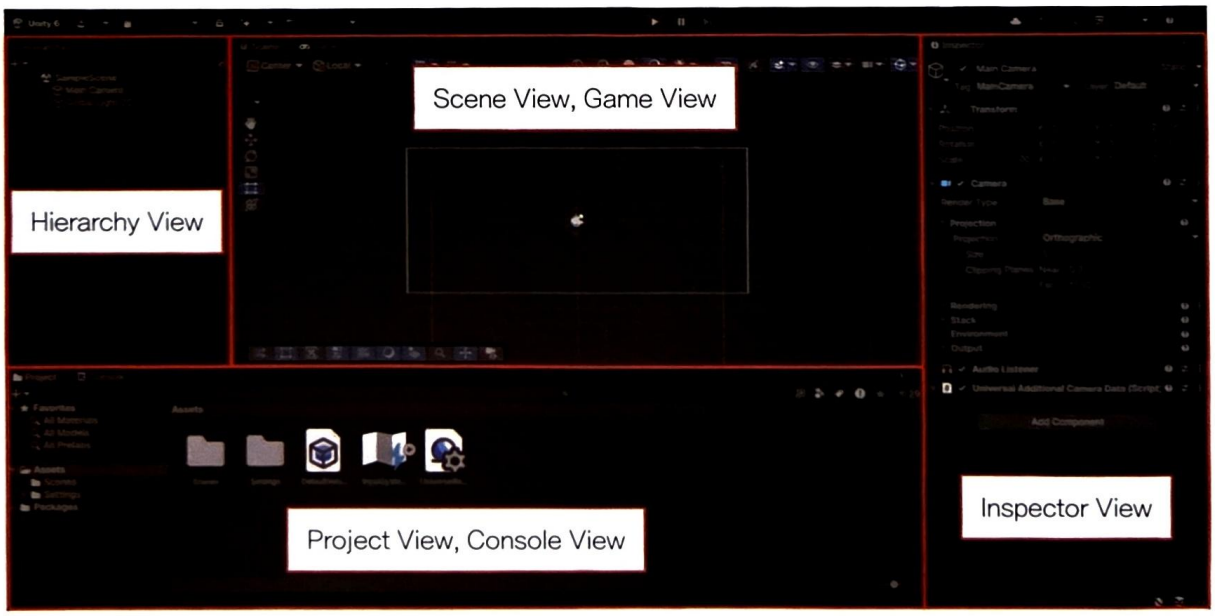
● 버튼 대신 단축키를 사용할 수도 있습니다. 플레이는 Ctrl+P, 일시 정지는 Ctrl+Shift+P, 스텝은 Ctrl+Alt+P를 눌러 실행합니다.

툴바 오른쪽에는 광고, 인앱 결제 등의 서비스를 설정하는 [Services] 탭을 여는 서비스 관리자, 이전 작업으로 되돌리거나 되돌릴 수 있는 분기 목록을 확인하는 되돌리기 이력, 현재 프로젝트의 모든 게임오브젝트와 에셋을 검색하는 전역 검색, 정해진 레이아웃을 선택하거나 나만의 레이아웃을 저장·삭제·선택하는 레이아웃 메뉴가 있습니다.

뷰

툴바 아래는 기능별 뷰 영역입니다. 왼쪽 위에 이름이 표시되는데, 이 부분을 마우스로 끌어서 놓으면 원하는 대로 화면을 구성할 수 있습니다.

유니티에서 제공하는 뷰는 매우 다양합니다. 기본인 Scene View, Game View, Inspector View, Hierarchy View, Project View, Console View에서 작업하며 필요에 따라 열고 닫을 수 있습니다.



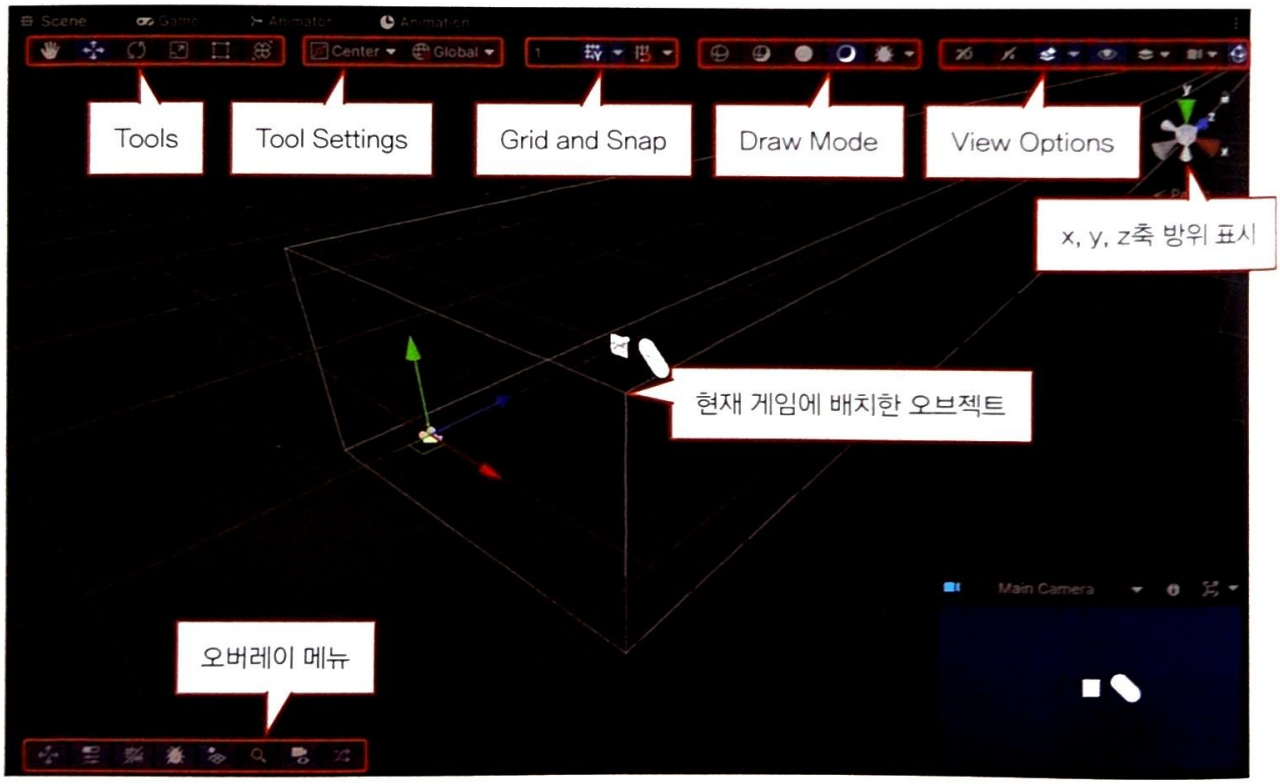
뷰 영역

기본으로 사용하는 뷰

뷰를 좀 더 자세히 살펴볼까요? 유니티는 용도에 따라 수많은 뷰를 지원합니다. 그중에 자주 사용하는 뷰에는 Scene View, Game View, Inspector View, Hierarchy View, Project View, Console View가 있습니다. 이들은 유니티로 게임을 개발할 때 필수입니다.

Scene View

Scene View는 현재 씬에 있는 모든 게임오브젝트를 그래픽으로 표현하고 오브젝트의 위치, 크기, 회전값을 조정하는 개발자 전용 편집 화면입니다. Scene View의 화면 위 툴바에는 도구Tools, 도구 설정Tool Settings, 그리드와 스냅Grid and Snap, 그리기 모드Draw Modes, 보기 옵션View Options 메뉴가 있으며 화면 왼쪽 아래에는 오버레이 메뉴Overlay Menu 등도 있습니다. 툴바 메뉴는 Scene View 내에서 마우스로 끌어서 이동할 수 있습니다.



Scene View

Scene View에는 게임에 배치한 게임오브젝트의 실제 모습을 출력하고 현재 방향을 알 수 있도록 x, y, z축 방위를 표시합니다.

자주 사용하는 Scene View의 툴바 메뉴를 살펴봅시다.

- **Tools**: Scene View에서 화면을 이동하거나 오브젝트를 이동·회전하고 크기를 편집하는 View, Move, Rotate, Scale, Rect, Transform 도구로 구성됩니다.
- **Tool Settings**: Pivot 또는 Center 등 오브젝트의 중심점을 설정하거나 오브젝트 회전 기준을 Local이나 Global로 설정합니다.
- **Grid and Snap**: Scene View에 보이는 격자를 활성화하거나 격자에 오브젝트 스냅 기능을 활성화하는 옵션입니다.
- **Draw Modes**: Scene View에 표시되는 게임오브젝트를 어떻게 표현할지 선택하는 옵션입니다.
- **View Options**: 2D 또는 3D를 선택하거나 빛 활성화 여부를 선택하는 보기 옵션입니다.

● 스냅이란 오브젝트의 위치·회전·크기를 미리 정한 간격이나 다른 오브젝트의 표면 또는 모서리에 정확히 맞춰 정렬하는 기능을 말합니다.

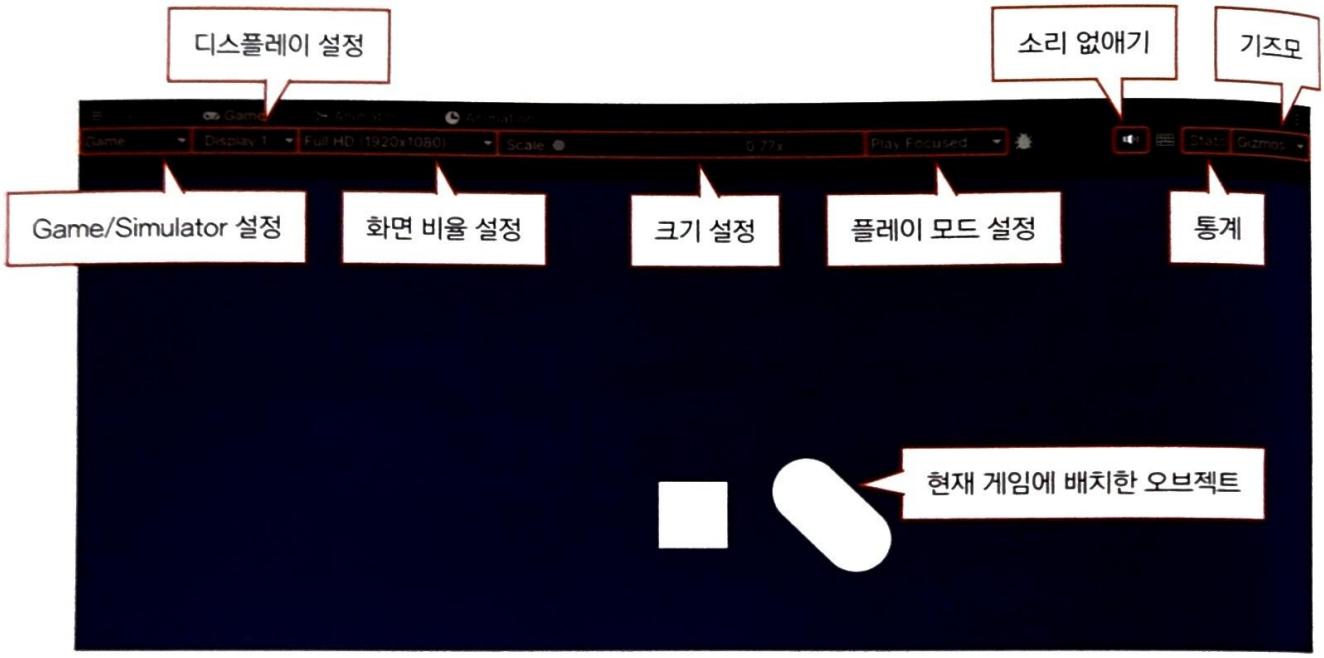
● 다른 뷰에서 Scene View로 이동하고 싶다면 단축키 Ctrl+1을 누릅니다.

Scene View 화면(카메라) 조작법

동작		명령 키
화면 이동		마우스 휠 버튼을 누른 채 드래그
화면 회전		Alt + 마우스 왼쪽 버튼을 누른 채 드래그 마우스 오른쪽 버튼을 누른 채 드래그
확대/축소		Alt + 마우스 오른쪽 버튼을 누른 채 드래그 마우스 휠 버튼을 앞/뒤로 드래그
비행 모드	전, 후, 좌, 우	마우스 오른쪽 버튼 + W, A, S, D(또는 방향키)
	상승/하강	마우스 오른쪽 버튼 + Q, E
	빠른 이동	Shift + 전, 후, 좌, 우 또는 상승/하강

Game View

Game View는 카메라 게임오브젝트가 촬영한 장면을 출력하며, 실제 사용자가 보는 게임 화면입니다. Game View의 툴바에서 [Game] 또는 [Simulator]를 선택할 수 있습니다. 모바일 게임을 제작할 때 [Simulator]로 설정하면 게임 실행 화면이 기기별로 어떻게 출력되는지를 확인할 수 있습니다.



Game View

Game View의 툴바 메뉴를 살펴봅시다.

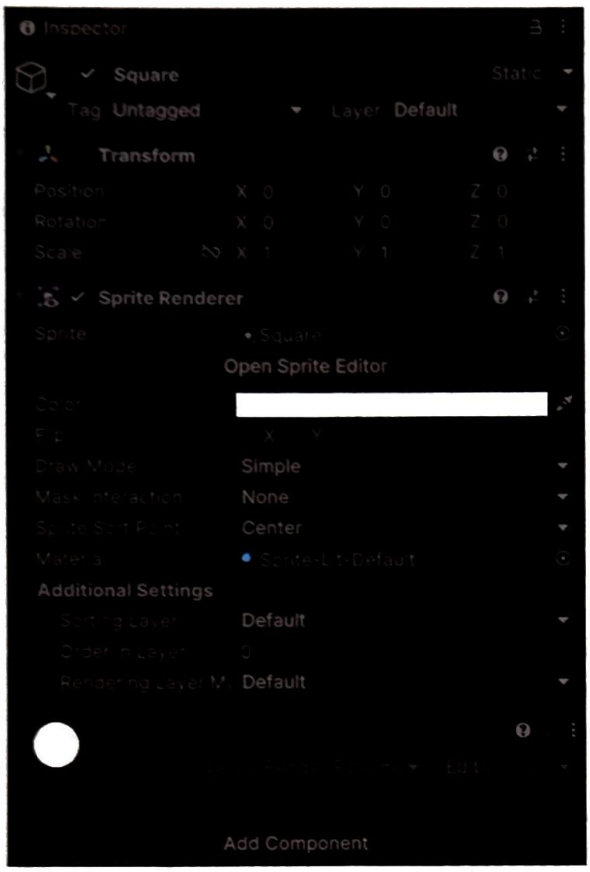
- **Game/Simulator**: 게임 실행 화면을 설정하고자 [Game] 또는 [Simulator]를 선택합니다. 기본값은 [Game]이며, 그림처럼 오브젝트가 보이는 게임 화면을 출력합니다. 이와 달리 [Simulator]를 선택하면 게임을 실행하는 기기에 따라 화면 크기를 다르게 표시합니다.
- **Display**: 출력 화면을 전환합니다. 게임 안에 여러 카메라 오브젝트를 사용하고 각 카메라가 촬영한 화면을 출력하는 디스플레이를 선택할 수 있습니다. 즉, 여러 모니터에 서로 다른 화면을 출력할 때 사용합니다.
- **Aspect**: Full HD, 4K UHD 등 Game View에서 보이는 해상도와 화면 비율을 설정합니다.
- **Scale**: 슬라이더를 움직여서 Game View를 확대 또는 축소합니다.
- **Enter Play Mode**: 에디터에서 게임을 실행했을 때 Game View가 어떻게 표시될지 플레이 모드를 선택합니다. 플레이 모드에는 게임을 실행했을 때 Game View로 포커스를 이동하는 [Play Focused], Game View를 최대 크기로 설정하는 [Play Maximized], Game View로 포커스를 이동하지 않는 [Play Unfocused]가 있습니다.
- **Mute Audio**: 오디오를 설정합니다. Mute Audio로 설정하면 효과음이나 BGM 등을 재생하지 않습니다.
- **Stats**: 현재 게임의 오디오나 그래픽 등의 정보를 출력합니다.
- **Gizmos**: 충돌 범위, 아이콘 등 기즈모(장치) 출력을 설정합니다.

[Play Unfocused]는 게임을 실행하는 동안 다른 뷰를 모니터링하거나 디버깅할 때 주로 사용합니다.

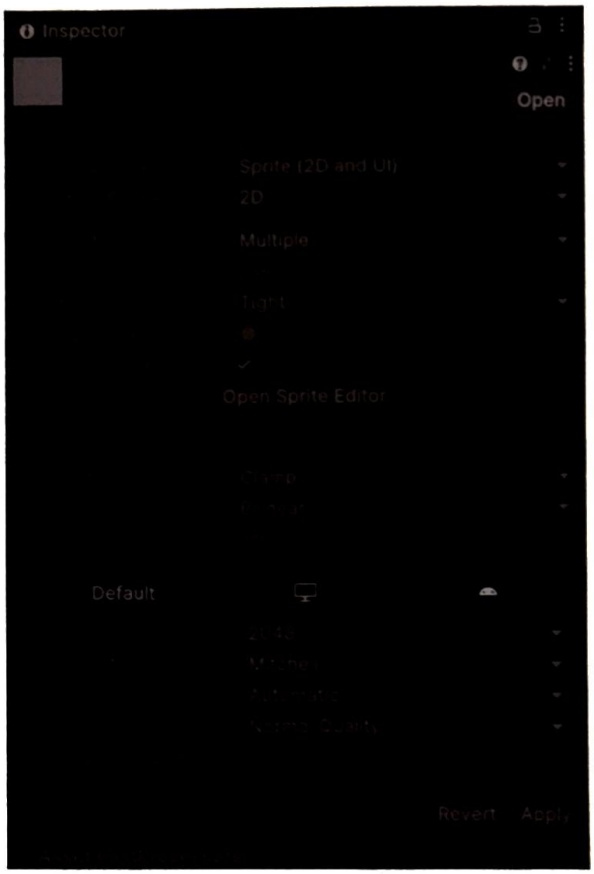
다른 뷰에서 Game View로 이동하고 싶다면 단축키 Ctrl+2를 누릅니다.

Inspector View

Inspector View는 Scene View, Hierarchy View에서 선택한 게임오브젝트나 Project View에서 선택한 에셋의 세부 정보를 출력합니다.



Hierarchy View에서 게임오브젝트를 선택했을 때

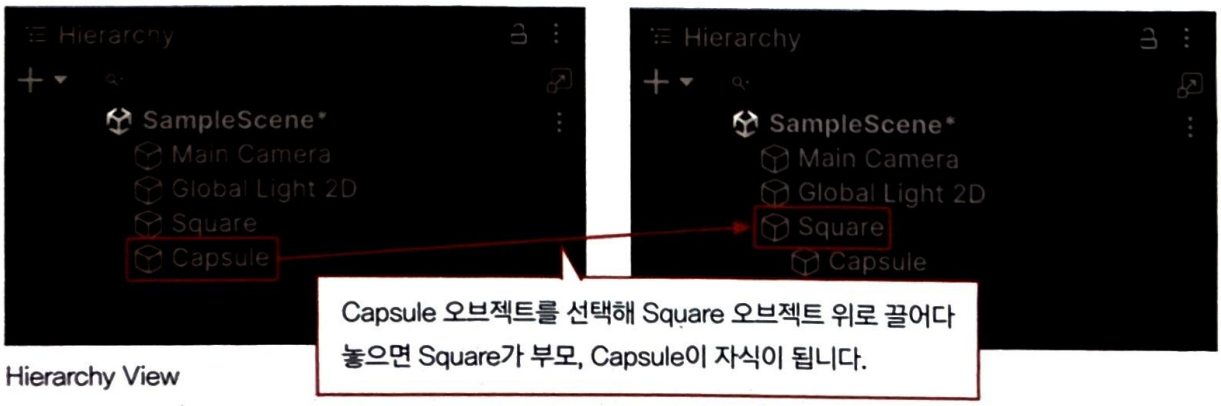


Project View에서 이미지 에셋을 선택했을 때

- 왼쪽 그림은 Project View에 있는 프리팹을 선택해도 표시합니다. 프리팹(Prefab)은 02-2절에서 설명합니다.
- 다른 뷰에서 Inspector View로 이동하고 싶다면 단축키 Ctrl+3를 누릅니다.

Hierarchy View

Hierarchy View는 현재 씬에 있는 모든 게임오브젝트를 목록 형태로 나타냅니다. 이름으로 나타내므로 Scene View보다 게임오브젝트를 관리하기 쉽고 게임오브젝트 간의 계층 구조(부모-자식 관계)를 설정하거나 확인할 수도 있습니다.

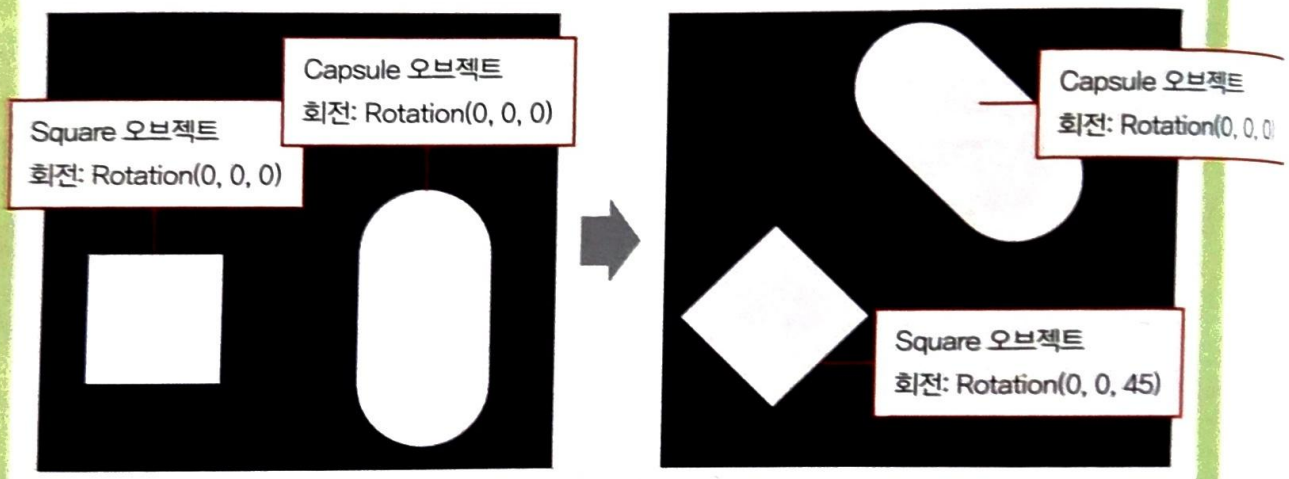


Hierarchy View

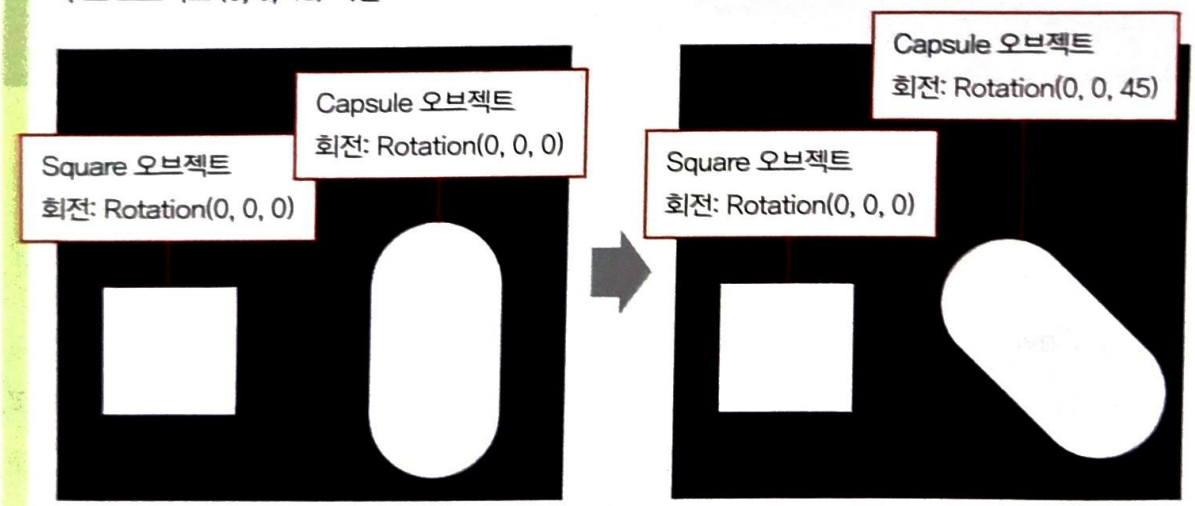
- 다른 뷰에서 Hierarchy View로 이동하고 싶다면 단축키 Ctrl+4를 누릅니다.

알아 두면 좋아요! 게임오브젝트의 부모-자식 관계란?

게임오브젝트가 부모-자식 관계일 때 부모의 변화는 자식에게 영향을 미치지만 자식의 변화는 부모에게 영향을 미치지 않습니다. 예를 들어 자식 오브젝트를 회전하더라도 부모 오브젝트에는 아무런 영향을 미치지 않지만, 부모 오브젝트를 회전하면 부모 위치를 축으로 자식 오브젝트도 함께 회전합니다.



부모 오브젝트 (0, 0, 45) 회전



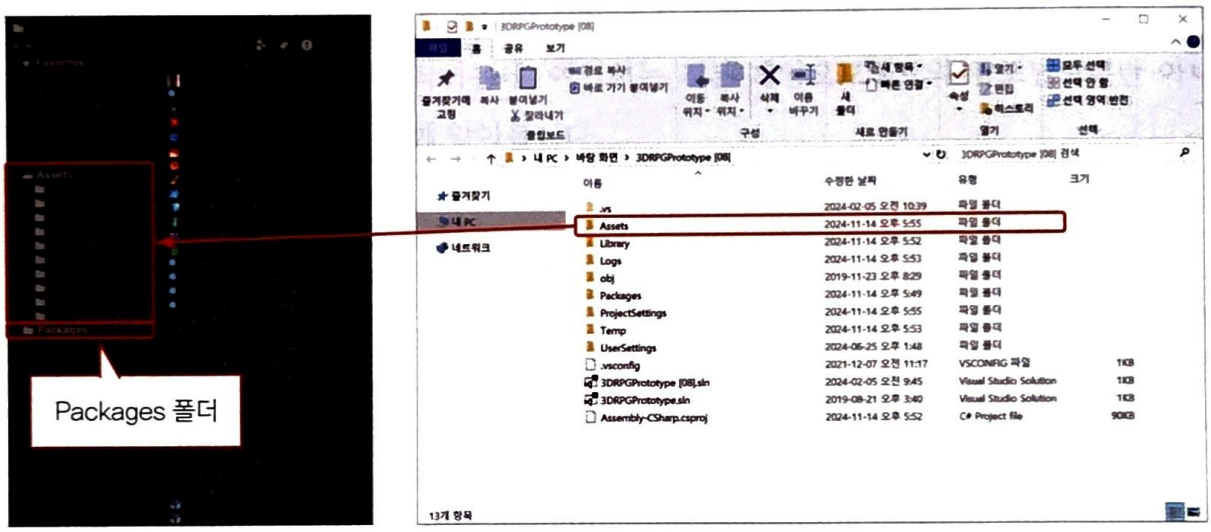
자식 오브젝트 (0, 0, 45) 회전

다시 정리하면, 부모-자식 관계일 때 부모의 변화는 자식에게 영향을 미치지만, 자식의 변화는 부모에게 영향을 미치지 않습니다.

Project View

Project View에서는 현재 프로젝트에서 사용하는 모든 리소스(에셋)를 관리합니다. 게임 제작에 필요한 외부 리소스를 현재 프로젝트의 Project View로 끌어서 놓으면 프로젝트에 추가할 수 있습니다.

다음 왼쪽 그림처럼 Project View에 **Assets 폴더**가 있습니다. 이 폴더 안에는 Project View에서 보는 모든 폴더와 파일을 저장합니다.



Project View

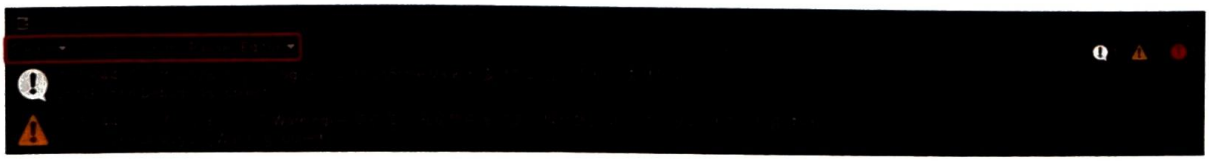
다른 뷰에서 Project View로 이동하고 싶다면 단축키 Ctrl+5를 누릅니다.

Project View에 있는 에셋은 게임에 사용할 수 있는 상태일 뿐 실제 게임에는 영향을 주지 않습니다. 게임에 영향을 주려면 에셋을 Hierarchy View에 배치해야 합니다. 또한 Project View에는 있지만 게임에서 사용하지 않는 에셋은 프로젝트를 빌드할 때 자동으로 제외합니다.

이와 더불어 Packages 폴더는 유니티에서 제공하는 각종 패키지 파일을 포함하며 수정할 수 없습니다.

Console View

Console View는 게임에 필요한 여러 정보와 작성한 스크립트에서 발생한 경고나 오류를 출력합니다.



Console View

Console View의 툴바 메뉴를 살펴봅시다.

- **Clear**: 이 메뉴를 클릭하면 Console View에 출력된 모든 데이터를 삭제합니다. [Clear] 드롭다운 메뉴에는 [Clear On Play](플레이 시 지우기), [Clear On Build](빌드 시 지우기), [Clear On Recompile](다시 컴파일 시 지우기) 옵션이 있습니다. 각 옵션은 체크해서 활성화 또는 비활성화할 수 있습니다.
- **Collapse**: 이 메뉴를 클릭해 활성화하면 같은 데이터를 묶어서 한 줄로 출력하고, 오른쪽 끝에 데이터를 호출한 횟수를 출력합니다.
- **Error Pause**: 이 메뉴를 클릭해 활성화하면 에디터에서 게임 실행 중 오류가 발생했을 때 프로그램을 중지합니다.

다른 뷰에서 Console View로 이동하고 싶다면 단축키 Ctrl+Shift+C를 누릅니다.

02-2

게임오브젝트와 컴포넌트 알아보기

이 절에서는 먼저 유니티의 기본 용어와 좌표 체계를 살펴봅니다. 유니티는 게임오브젝트와 컴포넌트 구성으로 게임을 제작하는데, 여기서는 기본 게임오브젝트와 컴포넌트뿐만 아니라 카메라와 빛 오브젝트도 함께 알아봅니다.

유니티 기본 용어 익히기

유니티에서 사용하는 용어는 다른 프로그래밍 용어와 조금 다릅니다. 본격적으로 게임을 개발하기에 앞서 용어부터 정리하고 익숙해집시다.

프로젝트

프로젝트Project는 하나의 게임을 뜻합니다. 유니티 에디터 안에서 해당 게임에 사용하는 에셋, 게임 설정 등 모든 것을 통틀어 하나의 프로젝트라고 합니다.

씬

씬Scene은 게임에서 한 장면이나 상태를 저장하는 단위입니다. 게임은 씬 단위로 관리하며 스크립트를 이용해 씬을 전환할 수 있습니다. 예를 들어 기존에 만든 게임에서 씬이 Main, Shop, Game 등 3개라고 할 때 Main 씬을 첫 번째 씬으로 설정했다면 게임을 시작할 때는 Main 씬을 먼저 출력합니다. 그리고 Main 씬에서 사용자의 선택에 따라 Shop 또는 Game 씬으로 이동해 게임을 진행하도록 설정합니다.

게임오브젝트

게임오브젝트GameObject란 씬에 배치하는 객체를 말합니다. 게임오브젝트는 여러 컴포넌트를 묶어서 관리하는데, 유니티에 있는 모든 게임오브젝트에는 위치·회전·크기를 제어하는 Transform 컴포넌트가 있습니다. 또한 컴포넌트를 추가하여 플레이어 캐릭터, 적 캐릭터, 아이템, 요소 등과 같은 다양한 오브젝트를 만들 수 있습니다.

컴포넌트

컴포넌트Component란 게임오브젝트에 부착할 수 있는 C# 스크립트 파일을 일컫습니다. 게임오브젝트에 컴포넌트를 추가하면 소리를 재생하거나 화면에 이미지를 출력하는 등 다양한 기능을 더할 수 있습니다.

에셋

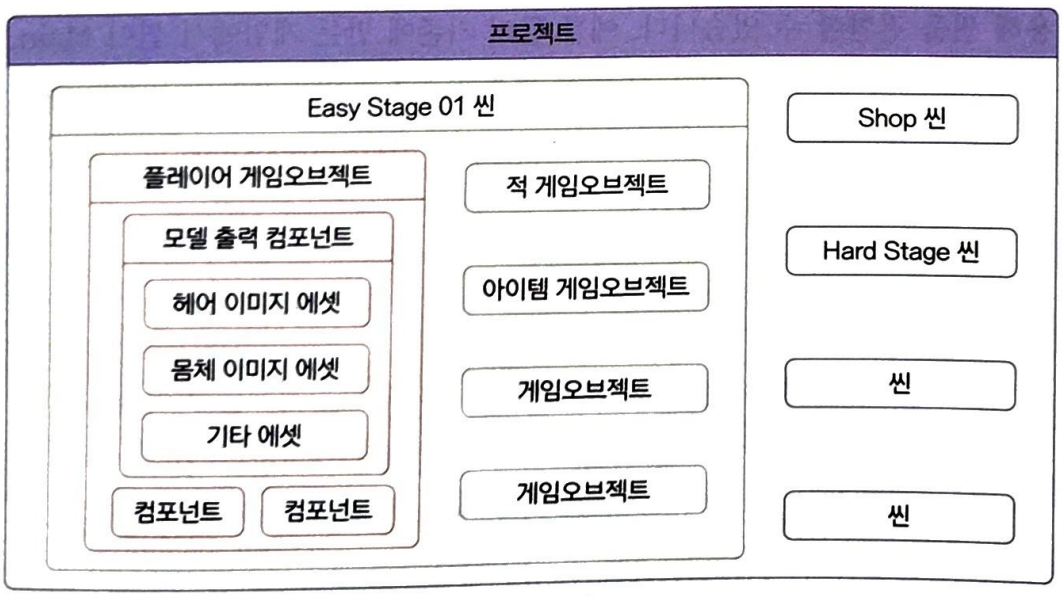
에셋Asset이란 사운드 파일, 3D 모델 파일, 이미지, 스크립트, 프리팹 등 프로젝트 내부에서 사용하는 모든 리소스를 말합니다.

프리팹

프리팹Prefab이란 Hierarchy View에 있는 게임오브젝트를 파일 형태로 저장한 것을 말합니다. 주로 동일한 게임오브젝트를 여러 개 배치하거나 게임하는 도중에 생성한 게임오브젝트를 관리할 때 프리팹으로 저장해 사용합니다.
게임을 개발할 때 기획 내용이 바뀌면 게임오브젝트에 새로운 기능이 추가되기도 합니다. 이때 원본 프리팹에 새로운 컴포넌트를 추가하거나 기존 컴포넌트를 삭제한다면 게임에 배치된 프리팹의 복제 오브젝트도 원본 프리팹과 똑같이 업데이트됩니다.

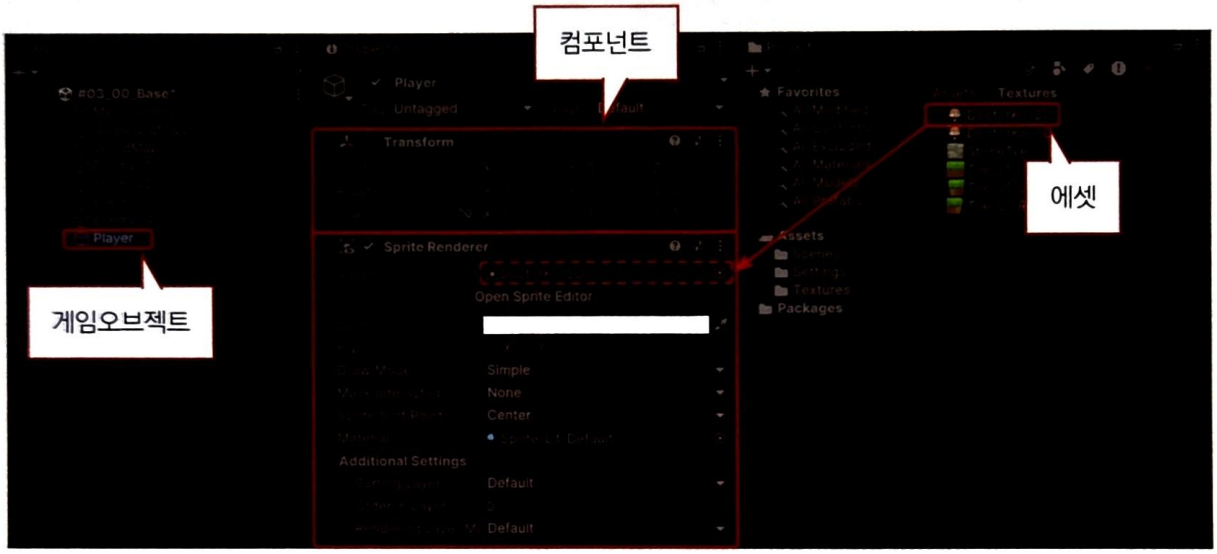
● 원본 프리팹의 컴포넌트 내부에서 위치, 회전과 같은 변수 데이터를 수정해도 복제 오브젝트의 해당 컴포넌트 데이터는 자동으로 변경되지 않습니다. 새로운 컴포넌트를 추가하거나 기존 컴포넌트를 삭제할 때만 그 내용이 복제 오브젝트에 반영됩니다.

지금까지 설명한 프로젝트, 씬, 게임오브젝트, 컴포넌트, 에셋 사이의 관계를 그림으로 설명하면 다음과 같습니다.



프로젝트, 씬, 게임오브젝트, 컴포넌트, 에셋의 관계도 예

어떤 게임 프로젝트 안에 Intro, Menu, Shop, Stage, GameOver 씬이 있다고 할 때 각 씬에는 여러 게임오브젝트가 배치됩니다. 그리고 이 게임오브젝트에는 화면에 이미지나 소리를 출력하고 오브젝트가 동작할 수 있도록 하는 컴포넌트가 있습니다. 예를 들어 화면에 이미지를 출력하는 컴포넌트는 이미지 에셋을 사용하고 소리를 출력하는 컴포넌트는 사운드 에셋을 사용합니다.



유니티 에디터로 보는 프로젝트, 씬, 게임오브젝트, 컴포넌트, 에셋의 관계

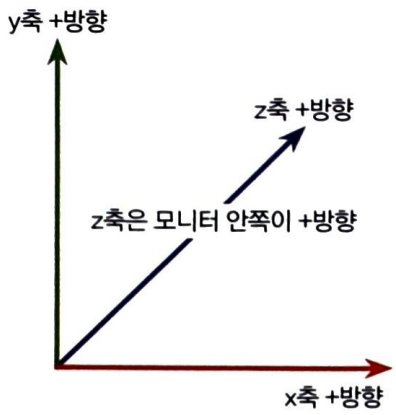
● 씬에 배치된 게임오브젝트와 컴포넌트 정보는 씬 파일 안에 저장합니다. 프로젝트를 만들 때 자동 생성되는 SampleScene 씬은 Assets > Scenes 폴더에 저장하며, 씬 파일을 더블클릭하면 Hierarchy View에 그 내용을 출력합니다.

유니티 좌표 체계 이해하기

유니티의 좌표 체계를 이해해야 게임 월드에 배치된 오브젝트가 어느 축 또는 방향으로 이동하는지, 오브젝트 아랫부분을 보려면 어느 축이 회전해야 하는지를 쉽게 파악하고 실제 데이터를 수정할 수 있습니다.

● 좌표 체계는 게임 엔진마다 다르므로 게임을 개발할 때 사용하는 게임 엔진의 좌표 체계부터 먼저 이해해야 합니다.

유니티에서는 왼손 좌표계를 기준으로 x, y, z축을 나타냅니다. 왼손을 앞으로 쭉 뻗은 상태에서 엄지는 오른쪽으로, 검지는 위쪽으로, 중지는 앞쪽으로 뻗으면 엄지가 x축, 검지가 y축, 중지가 z축 양(+)의 방향을 가리킵니다.



유니티 좌표 체계

유니티 화면은 xy축을 사용하는 2D와 xyz축을 사용하는 3D로 설정할 수 있습니다. 실제 게임 화면이 아닌 Scene View 화면은 2D 토글 버튼으로 설정할 수 있고, 실제 게임 화면인 Game View는 카메라 게임오브젝트의 Camera 컴포넌트에 있는 Projection 옵션으로 설정합니다.

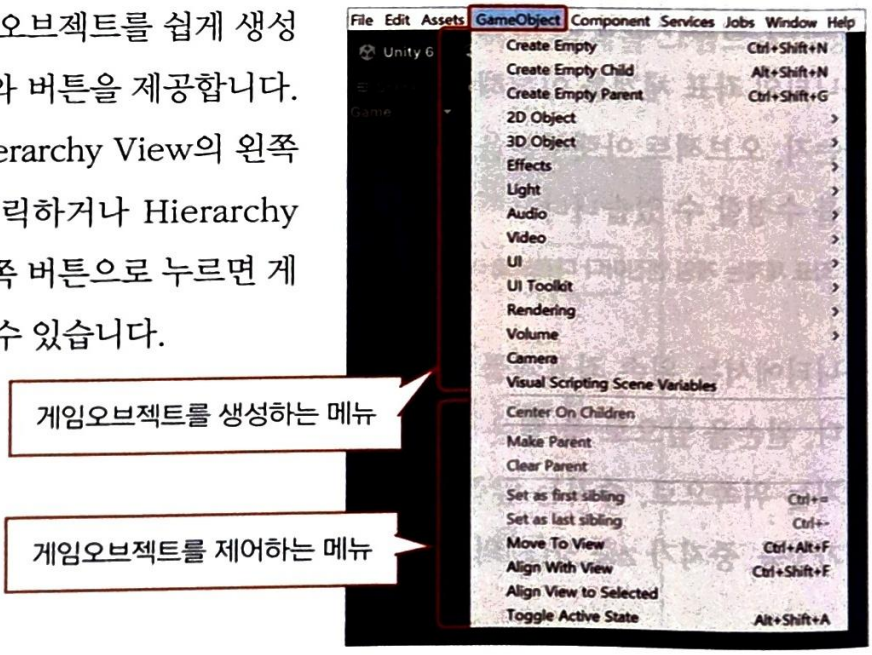
Scene View에서 2D와 3D의 화면 설정

게임오브젝트 살펴보기

게임오브젝트는 2D와 3D 오브젝트는 물론이고 오디오, 비디오 등 다양한 종류가 있습니다. 이들을 살펴보기 전에 유니티 에디터에서 게임오브젝트를 생성하는 방법부터 알아볼까요?

유니티 에디터의 게임오브젝트 메뉴

유니티는 많이 사용하는 게임오브젝트를 쉽게 생성할 수 있도록 여러 가지 메뉴와 버튼을 제공합니다. [GameObject] 메뉴 또는 Hierarchy View의 왼쪽 위에서 생성 버튼 ＋▾을 클릭하거나 Hierarchy View의 빈 곳을 마우스 오른쪽 버튼으로 누르면 게임오브젝트 메뉴를 활성화할 수 있습니다.

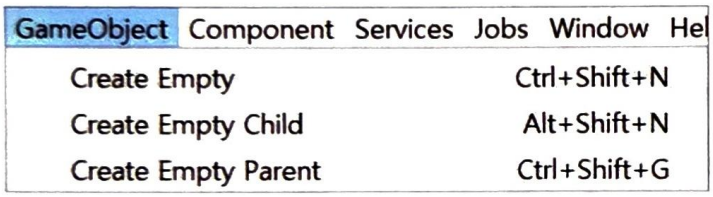


[GameObject] 메뉴

게임오브젝트에 다양한 컴포넌트를 부착해 개발자가 원하는 것을 만들 수 있습니다. 이처럼 유니티 에디터에서 기본으로 제공하는 게임오브젝트에 여러 컴포넌트를 추가하거나 삭제해서 사용합니다.

빈 오브젝트

빈 오브젝트Empty Object란 이동·회전·크기를 제어할 수 있는 Transform 컴포넌트 하나만 있는 게임오브젝트를 말합니다. 빈 오브젝트는 [GameObject → Create Empty] 메뉴로 생성합니다. [Create Empty Child] 메뉴는 현재 Hierarchy View에서 선택한 게임오브젝트의 자식 오브젝트로 빈 오브젝트를 생성할 때 사용합니다. [Create Empty Parent] 메뉴는 현재 Hierarchy View에서 선택한 게임오브젝트의 부모 오브젝트로 빈 오브젝트를 생성할 때 사용합니다.

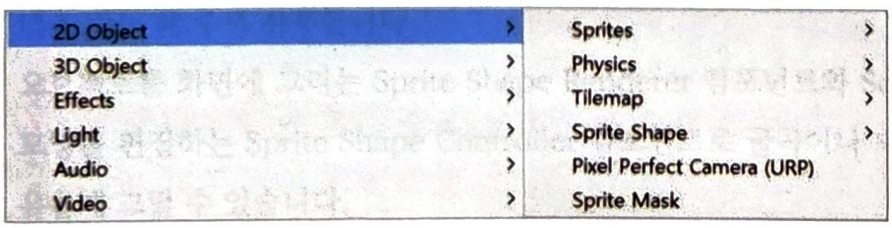


빈 오브젝트 관련 메뉴

여러 오브젝트를 한 그룹으로 묶어 관리할 때는 빈 오브젝트를 생성한 뒤, 그 아래 자식 오브젝트로 묶어서 사용합니다. 또한 게임에 보이지 않지만 게임의 여러 요소를 처리하는 오브젝트를 만들 때도 빈 오브젝트를 생성하고 게임의 여러 요소를 처리하는 컴포넌트를 추가합니다. 예를 들어 적을 주기적으로 생성하는 EnemySpawner, 스테이지를 관리하는 StageController와 같은 오브젝트가 이 책에서 만들 대표적인 빈 오브젝트입니다.

2D 오브젝트

게임 화면에 2D 이미지를 출력하거나 출력 효과를 제어하는 게임오브젝트입니다. [GameObject → 2D Object] 메뉴를 선택하고 필요한 2D 오브젝트를 클릭해서 생성합니다.



[2D Object] 메뉴

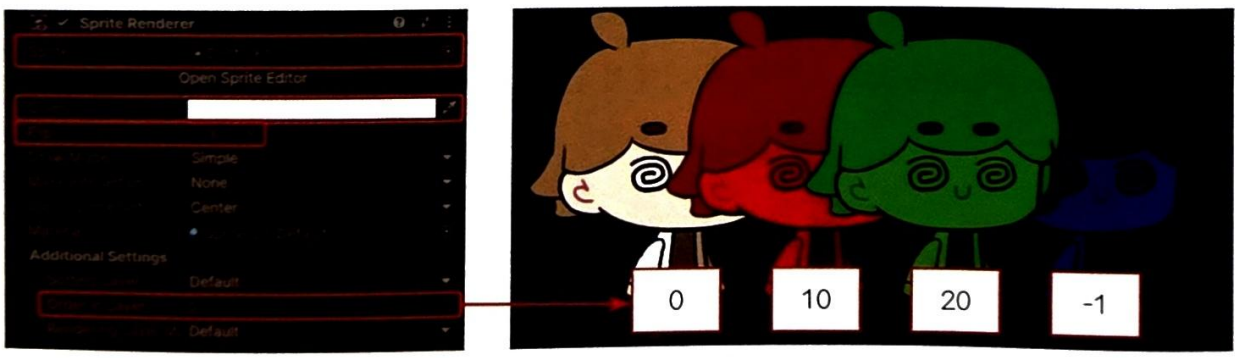
[2D Object] 메뉴를 간단히 설명한 후, [Sprites], [Sprite Mask], [Physics], [Sprite Shape]의 4가지 메뉴를 자세히 살펴봅니다.

- Sprites: 게임 화면에 2D 이미지를 출력하는 오브젝트를 생성합니다.
- Physics: [Sprites] 메뉴로 생성한 오브젝트에 물리를 담당하는 Rigidbody 2D와 충돌 범위 표현을 담당하는 Collider 2D 컴포넌트가 포함된 오브젝트를 생성합니다.
- Tilemap: 타일 형태의 2D 맵을 만들 때 사용하는 오브젝트를 생성합니다.
- Sprite Shape: 플랫폼 게임의 맵을 만들 때 주로 사용하는 오브젝트를 생성합니다. 굴곡이나 다각형 등 자유로운 모양으로 오브젝트를 만들 수 있습니다.
- Pixel Perfect Camera (URP): 다양한 해상도에서 픽셀 아트를 선명하게 유지하고 안정된 동작을 구현하도록 하는 카메라 오브젝트를 생성합니다.
- Sprite Mask: Sprite 오브젝트의 특정 부분을 보이거나 보이지 않게 할 때 사용합니다.

이어서 오브젝트와 모듈 4가지를 알아봅니다.

Sprite 오브젝트

[GameObject → 2D Object → Sprites] 메뉴로 Sprite 오브젝트를 생성합니다. 이 게임오브젝트는 화면에 2D 이미지를 출력합니다. Square, Circle, Capsule 등 8가지 모양의 게임오브젝트를 선택해서 생성할 수 있습니다. 8가지 모양 가운데 하나를 선택해서 Sprite 오브젝트를 생성하면 Inspector View에 Sprite Renderer 컴포넌트를 표시합니다. Sprite 오브젝트는 Sprite Renderer 컴포넌트의 Sprite 옵션에서 설정한 이미지 파일에 색상Color, 반전Flip 등의 옵션을 적용하여 화면에 출력합니다.



Sprite 오브젝트와 레이어 순서

Sprite Mask 오브젝트

[GameObject → 2D Object → Sprite Mask] 메뉴로 Sprite Mask 오브젝트를 생성합니다. 이 게임오브젝트는 화면에 출력하는 2D 이미지의 특정 부분만 보이거나 보이지 않도록 설정합니다.

Sprite Mask 오브젝트 위치에 Sprite 오브젝트를 배치하면 Sprite Mask 오브젝트의 모양과 크기에 따라 Sprite 오브젝트의 일부만 보이거나 특정 부분을 가립니다. 이때 Sprite Renderer 컴포넌트의 Mask Interaction 옵션 설정에 따라 보이는 영역이 달라집니다. 해당 옵션이 [Visible Inside Mask]이면 Sprite Mask 오브젝트 내부에 겹친 부분만 보이고, [Visible Outside Mask]이면 마스크 밖의 영역만 보입니다.

Sprite Mask 오브젝트 예시

Physics 물리 연산 모듈

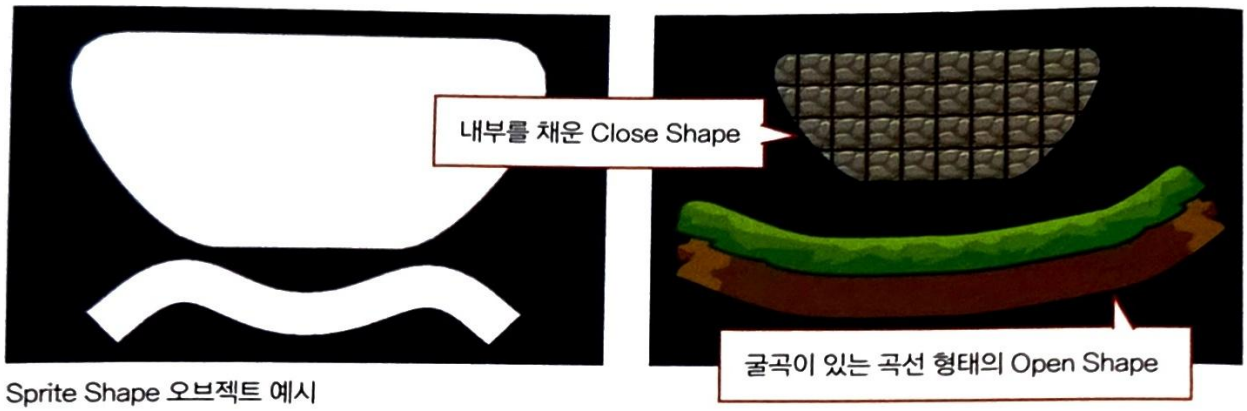
[GameObject → 2D Object → Physics] 메뉴에서는 [Static Sprite] 또는 [Dynamic Sprite] 메뉴를 선택할 수 있습니다. Static Sprite와 Dynamic Sprite는 기본 Sprite 오브젝트에 물리를 담당하는 Rigidbody 2D 컴포넌트와 충돌 범위 표현을 담당하는 Collider 2D 컴포넌트를 포함한 게임오브젝트입니다.

● 일반적으로는 [Physics] 메뉴를 사용하지 않고 [Sprites] 메뉴로 Sprite 오브젝트를 생성하고 나서 직접 컴포넌트를 추가해 물리를 적용합니다.

Sprite Shape 오브젝트

[GameObject → 2D Object → Sprite Shape] 메뉴에서 [Closed Shape] 또는 [Open Shape] 메뉴를 선택해 Sprite Shape 오브젝트를 생성합니다. 이 오브젝트는 플랫폼 게임의 맵을 만들 때 주로 사용합니다.

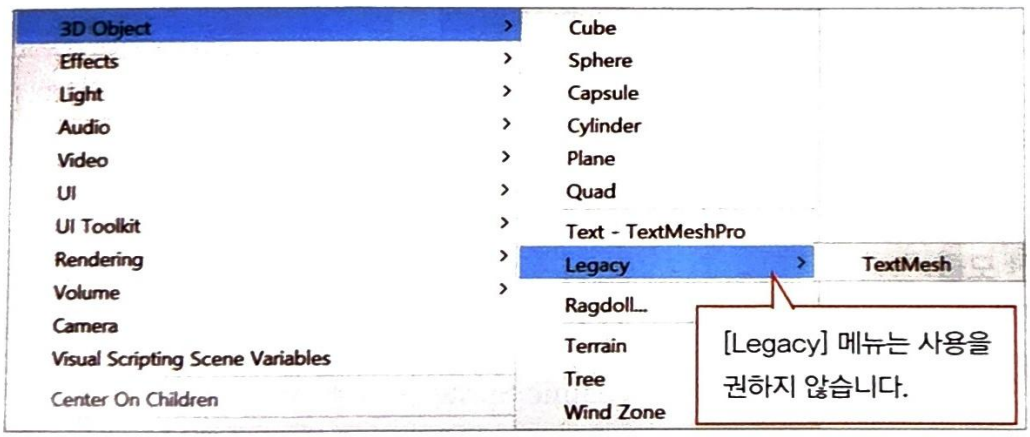
오브젝트를 화면에 그리는 Sprite Shape Renderer 컴포넌트와 Scene View에서 오브젝트 모양을 편집하는 Sprite Shape Controller 컴포넌트로 굴곡이나 다각형 등 2D 이미지를 자유롭게 그릴 수 있습니다.



Sprite Shape 오브젝트 예시

3D 오브젝트

게임 화면에 육면체, 구체와 같은 입체 도형이나 텍스트, 지형 등을 표현할 때 사용합니다. [GameObject → 3D Object] 메뉴로 원하는 모양의 오브젝트를 선택해 생성합니다.



[3D Object] 메뉴

[3D Object] 메뉴를 간단히 설명하면 다음과 같습니다.

- Cube, Sphere, Capsule, Cylinder, Plane, Quad: 3차원 도형 오브젝트를 생성합니다.
- Text - TextMeshPro: 화면에 3차원 텍스트를 출력합니다.
- Ragdoll...: 뼈대 정보가 있는 모델 오브젝트를 봉제 인형처럼 움직이게 만들 수 있습니다.
- Terrain, Tree, Wind Zone: 3차원 지형, 나무, 바람 구간을 제작할 때 사용합니다.

[Legacy] 메뉴는 이전 버전의 3D 오브젝트 생성 기능을 모은 메뉴로, 사용을 권하지 않습니다.

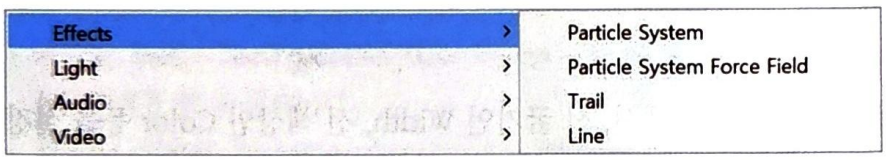
알아 두면 좋아요! 3D 게임의 오브젝트는 이렇게 사용합니다

유니티는 기본 도형 오브젝트만 제공하므로 3D 게임 제작에 필요한 실제 사람이나 건물 등 다양한 형태의 3D 모델은 보통 외부 프로그램에서 만들고 나서 유니티로 가져와서 사용합니다.

간단한 건축물 오브젝트라면 [Window → Package Management → Package Manager] 메뉴로 Package Manager View를 열고 Unity Registry에 있는 ProBuilder 패키지를 설치하면 유니티 안에서도 제작할 수 있습니다.

Effects 오브젝트

게임에 보이는 여러 가지 효과, 무기의 잔상, 선 그리기 등을 표현하는 게임오브젝트입니다. [GameObject → Effects] 메뉴에서 사용할 Effects 오브젝트를 선택하여 생성합니다.



[Effects] 메뉴

Particle System 오브젝트

게임에서 불, 번개와 같은 다양한 효과Effect를 낼 때 사용합니다. Particle System 컴포넌트의 기본 옵션이나 Emission 모듈, Shape 모듈 등의 옵션을 설정해서 여러 가지 효과를 표현할 수 있습니다.

Particle System 오브젝트 예

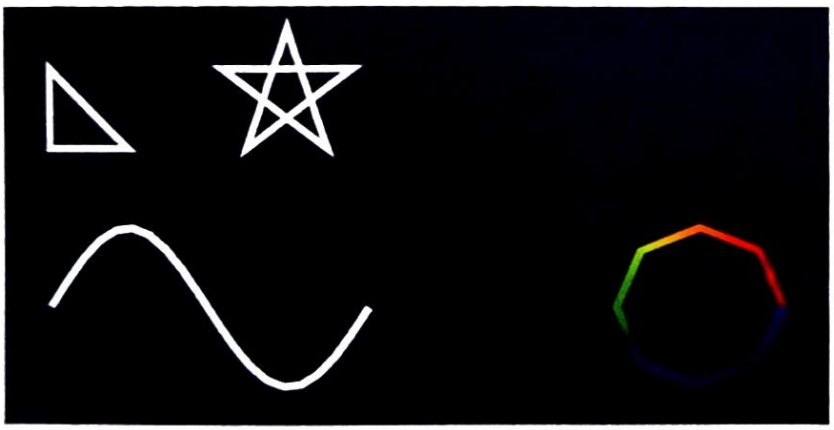
Trail 오브젝트

무기를 휘두르거나 게임오브젝트가 이동할 때 생기는 잔상의 굵기인 Width, 잔상이 남는 시간인 Time, 잔상 색상인 Color 등을 설정해서 게임 화면에 잔상 효과를 출력합니다.

Trail 오브젝트 예

Line 오브젝트

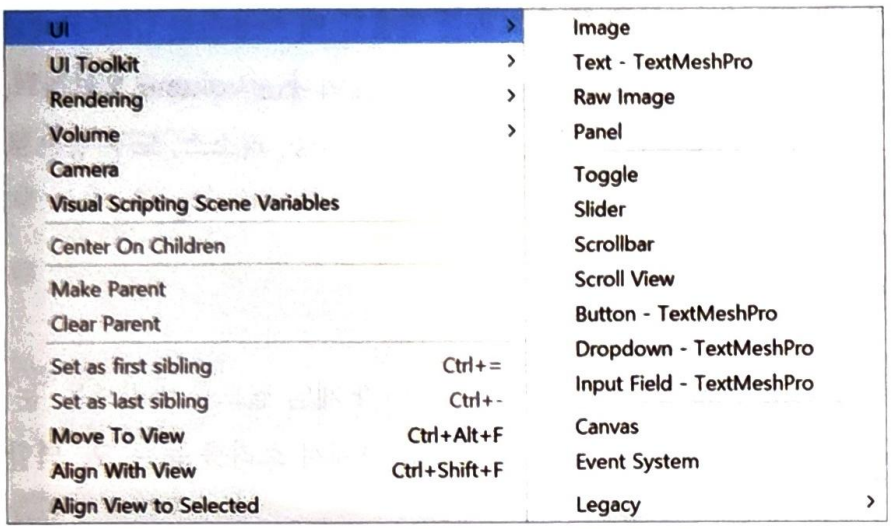
점의 개수와 위치를 지정하는 Positions 섹션, 선 굵기인 Width, 선 색상인 Color 등을 설정해서 게임 화면에 선을 출력합니다.



Line 오브젝트 예

UI 오브젝트

사용자가 게임 정보를 실시간으로 확인하거나 게임과 상호 작용할 때 사용하는 게임오브젝트입니다. [GameObject → UI] 메뉴에서 사용할 UI 오브젝트를 선택하면 생성할 수 있습니다.



[UI] 메뉴

[UI] 메뉴를 간단히 설명하면 다음과 같습니다.

- Image, Text - TextMeshPro, Raw Image, Panel: 텍스트, 이미지를 화면에 출력하는 오브젝트로, 상호 작용은 할 수 없습니다.
- Toggle, Slider, Scrollbar, Scroll View, Button - TextMeshPro, Dropdown - TextMeshPro, Input Field - TextMeshPro: 버튼, 슬라이더, 드롭다운처럼 상호 작용할 수 있는 오브젝트입니다.
- Canvas, Event System: UI를 화면에 그리고 이 UI와 상호 작용할 때 필요한 오브젝트입니다.

[Legacy] 메뉴에 있는 [Text], [Button], [Dropdown], [Input Field] 메뉴 대신 이름이 같은 'OOO - TextMeshPro' 오브젝트를 사용합니다.

그 밖의 게임오브젝트

Audio 오브젝트

게임 안에서 재생하는 소리(사운드)와 관련한 게임오브젝트입니다. [Audio] 메뉴로 선택한 소리를 재생하는 Audio Source 오브젝트와 동굴과 같은 실내에서 에코 효과를 내는 Audio Reverb Zone 오브젝트를 생성할 수 있습니다. 이러한 소리를 듣는 역할은 Camera 오브젝트의 Audio Listener 컴포넌트가 담당합니다.

Video 오브젝트

동영상을 재생하는 게임오브젝트입니다. Video 오브젝트는 [Video → Video Player] 메뉴로 만들 수 있습니다.

이 외에도 포스터, 표지판, 페인트 자국, 얼룩 등의 효과를 연출할 때 사용하는 URP Decal Projector 오브젝트, 후처리와 다양한 비주얼 효과를 적용하고 제어하는 Volume 오브젝트, 사용자 인터페이스와 에디터 확장 프로그램을 개발할 때 필요한 기능, 리소스, 도구 등으로 구성된 UI Document 오브젝트 등이 있습니다.

카메라 오브젝트

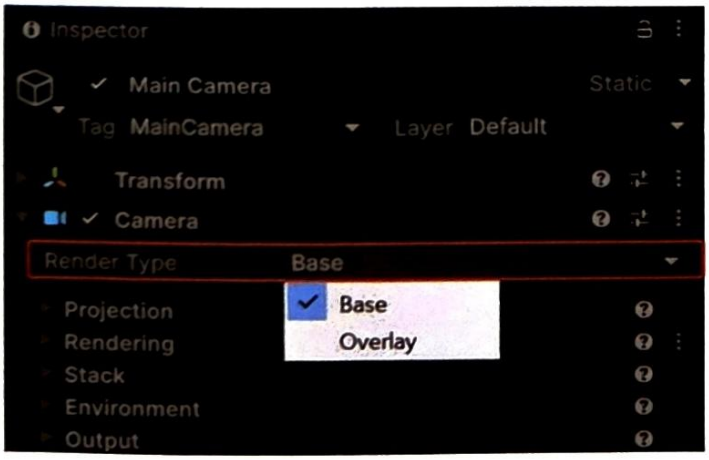
카메라Camera 오브젝트는 Camera 컴포넌트를 이용해 플레이어가 게임 월드를 바라보는 '눈' 역할을 합니다. 또한 Audio Listener 컴포넌트를 이용해 게임 안에서 소리를 듣는 '귀' 역할도 합니다.

유니티에서 말하는 '게임 월드'란 오브젝트가 상호 작용하는 가상 공간 전체입니다.

Camera 컴포넌트는 게임 개발에서 매우 중요한 역할을 하며 설정해야 할 옵션도 많으므로 관련 옵션 또한 잘 알아야 합니다.
그리고 카메라 오브젝트는 한 씬에 1개 이상 있어야 합니다. 따라서 교통 정보 센터처럼 여러 개의 카메라 오브젝트를 이용한 분할 화면으로 각 씬을 관찰할 수도 있습니다. 또한 카메라 오브젝트에 이동이나 회전과 같은 애니메이션을 적용하여 연출 효과를 낼 수도 있습니다.
Hierarchy View에서 Main Camera를 선택하고 Inspector View를 살펴봅시다.

Render Type 기본 옵션

Render Type은 카메라가 장면을 어떻게 렌더링할지 결정하는 옵션으로, [Base] 또는 [Overlay]를 선택할 수 있습니다.



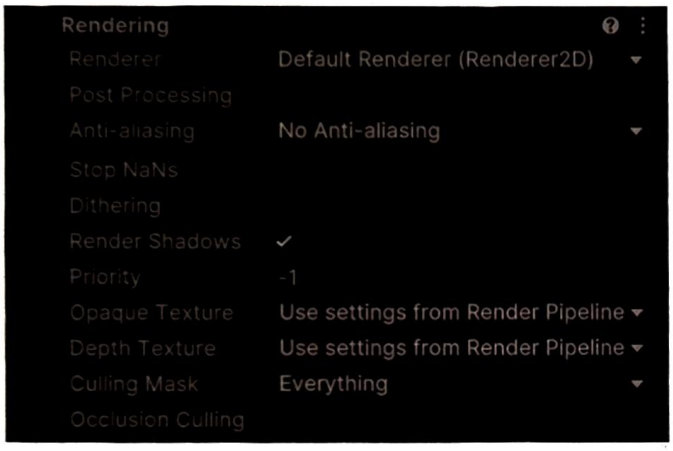
Camera 컴포넌트의 Render Type 드롭다운 메뉴

[Base]는 여러 카메라를 사용할 때 가장 먼저 렌더링하는 카메라로, 장면 전체를 렌더링합니다. [Overlay]는 Base 카메라로 렌더링한 이후 그 위에 추가 정보를 렌더링하는 카메라로, 보통 HUD Heads-Up Display나 UI 요소 등을 렌더링할 때 자주 사용합니다.

- HUD는 게임 화면 위에 고정으로 표시하는 UI로, 플레이어가 시선을 옮기지 않고도 상태를 즉시 파악하도록 도와줍니다. 일반적으로 체력, 스킬 쿨다운 등과 같은 정보를 출력합니다.
- 렌더링이란 2차원 또는 3차원 장면을 바탕으로 컴퓨터를 이용해 사진이나 영상을 만드는 과정을 말합니다.

Rendering 섹션

이 섹션은 카메라가 무엇을 어떤 품질로, 어떤 순서로 그릴지를 결정합니다.



Camera 컴포넌트의 Rendering 섹션

Rendering 섹션의 옵션 몇 가지를 살펴봅니다.

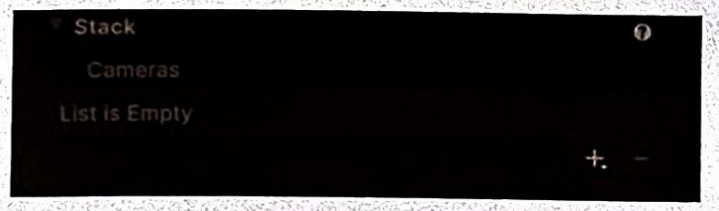
- **Renderer 옵션**: URP에서 사용할 렌더러를 설정할 때 사용합니다. 기본 렌더러 외에 커스텀 렌더러도 설정할 수 있습니다.
- **Post Processing 옵션**: 이 옵션을 활성화하면 카메라에 후처리 효과를 적용할 수 있습니다.
- **Anti-aliasing 옵션**: 해상도가 높은 그림을 낮은 해상도에서 나타낼 때 계단 현상이 발생하는 것을 방지합니다.
- **Priority 옵션**: 카메라의 장면을 모니터에 출력하는 순서를 말하며, 숫자가 클수록 앞에 그려집니다.
- **Culling Mask 옵션**: 카메라에 보이게 할 레이어를 설정할 때 사용합니다. 선택 해제된 레이어의 오브젝트는 해당 카메라에 비치지 않습니다.
- **Occlusion Culling 옵션**: 카메라에 보이지 않는 오브젝트를 렌더링에서 생략할 때 사용합니다. 불필요한 그래픽 연산을 줄여 렌더링 성능을 최적화합니다.

- 렌더러는 게임오브젝트의 모양, 색상, 재질, 그림자 등을 계산해서 화면에 그리는, 즉 렌더링하는 컴포넌트입니다.

Stack 섹션

이 섹션에서는 여러 개의 카메라를 겹쳐서 그리는 스태킹을 구성하고 관리합니다. Stack 섹션에 있는 Cameras는 Render Type이 [Overlay]인 카메라를 등록할 때 사용합니다.

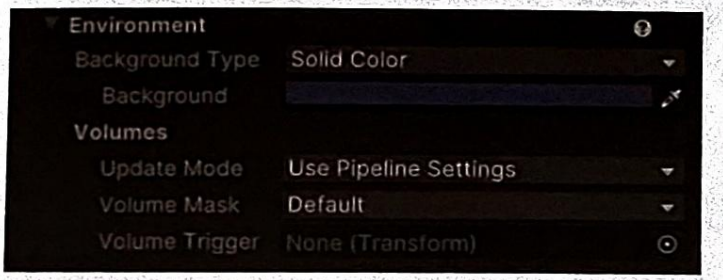
스태킹이란 여러 대의 카메라가 순서대로 화면을 렌더링하도록 겹쳐서 배치하는 방식입니다.



Camera 컴포넌트의 Stack 섹션

Environment 섹션

이 섹션은 카메라 프레임의 배경을 초기화하는 방식을 결정합니다.



Camera 컴포넌트의 Environment 섹션

Environment 섹션의 몇 가지 옵션을 자세히 살펴봅시다.

Background Type 옵션

오브젝트가 없는 빈 배경을 어떻게 채울지를 결정합니다. Background Type은 게임 월드를 하늘 배경으로 채우는 [Skybox], Background 옵션에 지정한 색상으로 배경을 채우는 [Solid Color], 카메라를 동시에 여러 대 사용할 때 특정 색상이나 그림으로 재설정하지 않는 [Uninitialized] 중에서 하나를 선택할 수 있습니다. 주로 2D 게임을 제작할 때는 [Solid Color]를, 3D 게임을 제작할 때는 [Skybox]를 사용합니다.

카메라를 동시에 여러 대 사용할 때 Render Type이 [Overlay]인 카메라는 Background Type 옵션이 보이지 않고 [Uninitialized]로 자동 설정됩니다.

Skybox(왼쪽)와 Solid Color(오른쪽)

Background 옵션

Background Type이 [Solid Color]일 때 배경 색상을 선택할 수 있습니다.

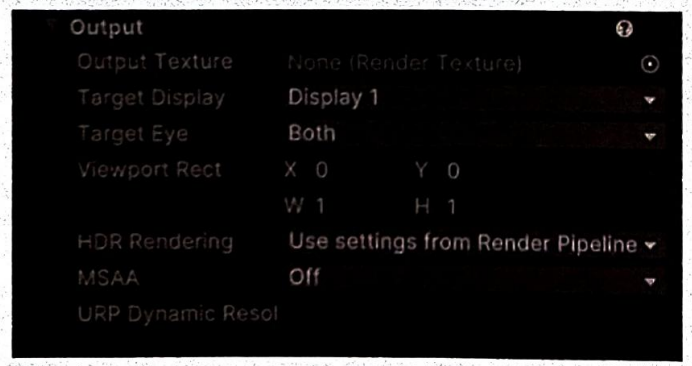
Volumes 옵션

눈부심 Bloom, 톤 매핑 Tone Mapping 등의 다양한 비주얼 효과를 적용(후처리)할 수 있습니다. 하위 메뉴인 Volume Mask는 Volume 효과(비주얼 효과)를 적용할 레이어를 선택합니다. Volume Trigger는 Volume 효과의 기준점을 선택합니다. 일반적으로 카메라 자체를 기준점으로 설정하지만 다른 Transform 컴포넌트를 사용할 때 등록해서 사용하기도 합니다.

- Bloom은 화면의 밝은 영역이 부드럽게 번져 보이도록 윤기를 더해 하이라이트를 강조하는 효과입니다.
- Tone Mapping은 HDR 밝기/색 범위를 디스플레이에 맞게 압축해 클리핑을 줄이고 자연스러운 대비를 만드는 효과입니다.

Output 섹션

이 섹션은 렌더링 결과의 출력 대상과 영역을 지정합니다.



Camera 컴포넌트의 Output 섹션

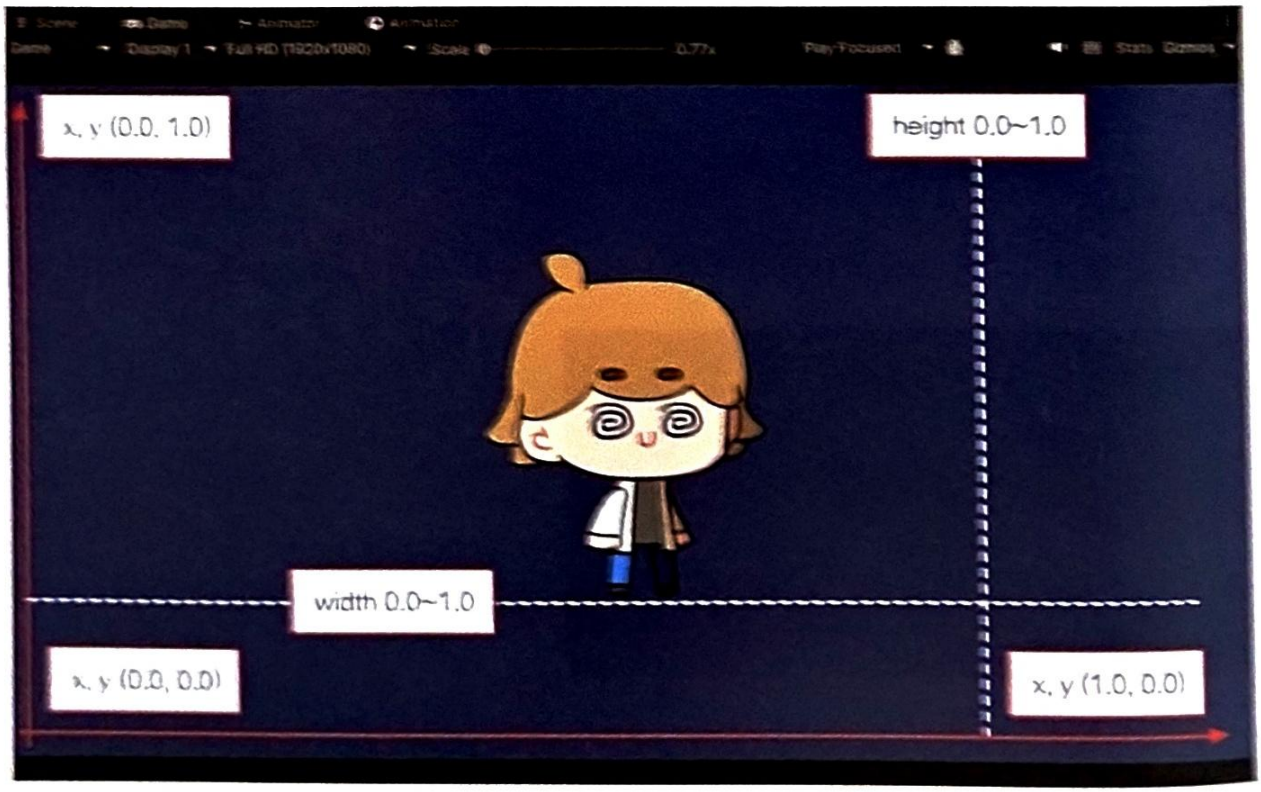
Output 섹션의 옵션은 다음과 같습니다.

- **Output Texture 옵션**: 카메라가 본 장면을 화면에 출력하지 않고 등록된 Render Texture 파일에 저장할 때 사용합니다.
- **Target Display 옵션**: 카메라가 보는 장면을 출력할 모니터를 설정합니다. 최대 8개까지 설정할 수 있습니다.
- **Viewport Rect 옵션**: 카메라가 본 장면을 게임 화면에 출력하는 영역을 설정할 때 사용합니다.
- **HDR Rendering 옵션**: 눈부심Bloom이나 후광Glow 등의 조명 효과를 표현할 때 사용합니다.
- **MSAA 옵션**: Multi Sample Anti-Aliasing의 줄임말로, 렌더링 품질을 향상시킬 때 주로 사용합니다.
- **URP Dynamic Resolution 옵션**: 장면이 무거울 땐 해상도를 일시적으로 낮춰 프레임을 유지하고 여유가 생기면 해상도를 다시 올려 선명하게 보이도록 하는 동적 해상도 지원 여부를 설정합니다.

- 미니맵을 출력할 때 카메라가 촬영한 장면을 Render Texture 파일에 저장하고 Render Texture를 UI에 출력합니다.
- X, Y값을 입력해서 위치를, W(너비)와 H(높이)를 입력해서 크기를 설정합니다. 0.0~1.0 사이의 값을 입력합니다.

Viewport Rect 옵션

앞서 살펴본 Output 섹션 옵션 중 Viewport Rect를 좀 더 알아봅니다. Viewport Rect는 카메라가 촬영한 내용을 게임 화면에 출력하는 영역입니다. 영역의 좌표는 왼쪽 아래가 (0, 0), 오른쪽 위가 (1, 1)입니다. X와 Y는 출력을 시작하는 위치 좌표를, W와 H는 출력하는 화면 크기를 뜻합니다.



Viewport Rect

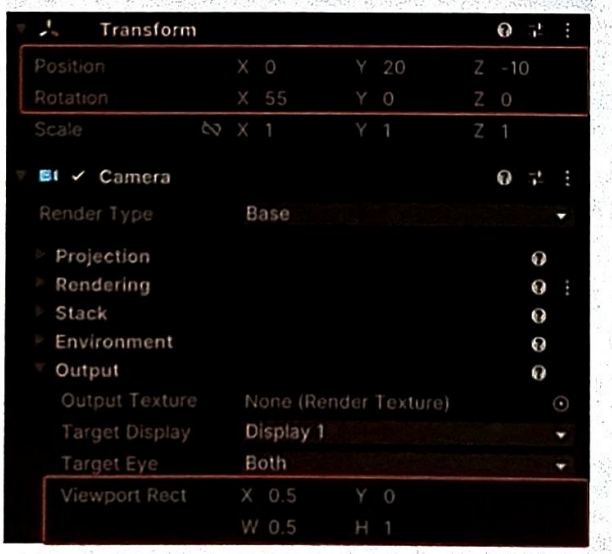
예를 들어 Viewport Rect를 X: 0, Y: 0, W: 0.5, H: 1로 설정하면 다음 그림처럼 Game View의 왼쪽 절반에만 Main Camera 오브젝트가 촬영한 장면을 출력하고 오른쪽 절반에는 검은 화면을 출력합니다.

Viewport Rect 출력 예 1

Main Camera 오브젝트를 복제한 후 이번에는 Viewport Rect를 X: 0.5, Y: 0, W: 0.5, H: 1로 설정해 봅시다.

오브젝트를 복제(복사+붙여 넣기)할 때는 단축키 Ctrl+D를 누릅니다.

복제한 오브젝트와 왼쪽 화면에 출력된 Main Camera를 다르게 설정하고자 Transform 컴포넌트의 Position(위치)은 (0, 20, -10)으로, Rotation(회전)은 (55, 0, 0)으로 설정합니다. 그러면 다음처럼 Main Camera 오브젝트가 촬영한 장면은 Game View의 왼쪽 절반에 출력하고, 복제한 Main Camera (1) 오브젝트가 촬영한 장면은 오른쪽 절반에 출력합니다.



Viewport Rect 출력 예 2

Projection 섹션

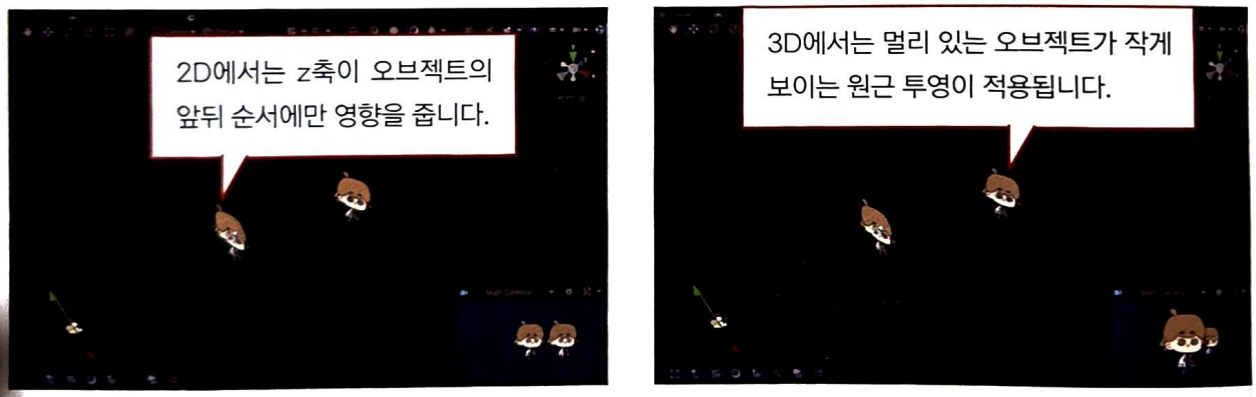
이 섹션은 카메라의 시점을 설정합니다.

Camera 컴포넌트의 Projection 섹션

Projection 섹션의 옵션을 좀 더 살펴봅시다.

Projection 옵션

2D 시점인 [Orthographic] 또는 3D 시점인 [Perspective]를 선택할 수 있습니다. 선택한 시점에 따라 아래에 활성화되는 변수가 달라집니다. [Orthographic]을 선택하면 카메라의 시야 범위를 설정하는 Size 변수가 활성화됩니다. [Perspective]를 선택하면 카메라의 시야 범위를 넓히는 방향인 Field of View Axis 변수와 카메라의 시야 범위를 나타내는 Field of View 변수, 그리고 시야와 투영을 제어해 더 현실적인 구도와 심도를 표현할 수 있는 Physical Camera 변수가 활성화됩니다.

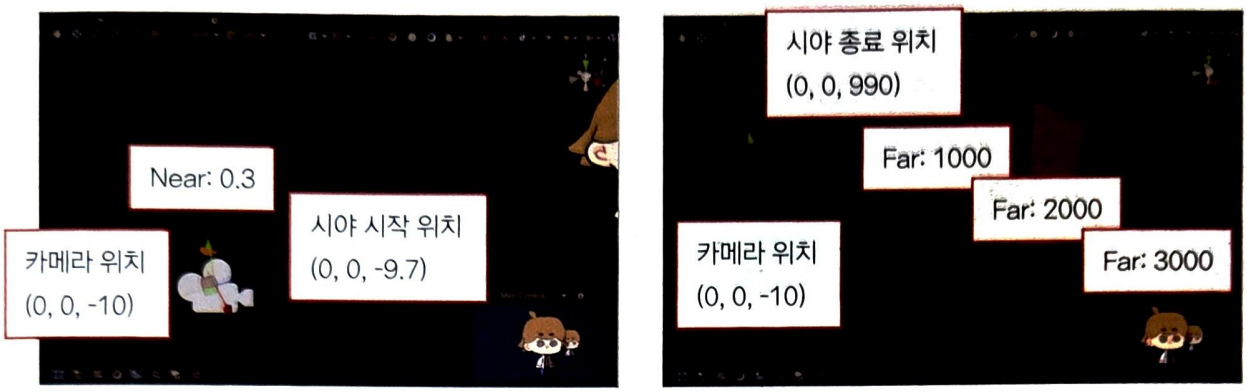


2D와 3D의 Projection 차이

Clipping Planes 옵션

카메라가 오브젝트를 볼 수 있는 시야 거리를 설정합니다. Near에는 시작 지점(기본값 0.3)을, Far에는 끝 지점(기본값 1000)을 입력합니다. 기본값을 수정하지 않은 상태에서 현재 카메라의 위치가 (0, 0, -10)이라고 하면 (0, 0, -9.7)부터 (0, 0, 990)까지 배치된 오브젝트를 볼 수 있습니다.

Far값을 늘리면 카메라가 볼 수 있는 시야 거리를 얼마든지 늘릴 수 있지만, 시야가 늘어나는 만큼 화면에 그려야 하는 오브젝트 수가 많아지므로 성능에 영향을 미칠 수 있습니다. 그러므로 Far값은 개발하는 게임에 따라 적절히 조절하는 것이 좋습니다.



시야 거리와 카메라 위치 설정하기

빛 오브젝트

빛 Light 오브젝트는 현실 세계의 빛과 같이 모델의 재질이나 색상을 다양한 형태로 보여 줄 때 사용합니다. 빛 오브젝트에는 2D 게임을 위한 Light 2D 컴포넌트와 3D 게임을 위한 Light 컴포넌트가 있습니다.

빛 오브젝트가 없을 때

빛 오브젝트가 있을 때

이 책에서는 주로 2D 게임 제작과 관련한 내용을 다루므로 2D 게임에서 빛 효과를 제공하는 Light 2D 컴포넌트를 살펴봅니다. Hierarchy View 왼쪽 위에서 ＋▾ 버튼을 클릭하고 [Light → Sprite Light 2D] 메뉴로 빛 오브젝트를 생성합니다. 그런 다음 Inspector View를 봅니다.

[GameObject → Light → Sprite Light 2D] 메뉴로도 오브젝트를 생성할 수 있습니다.

기본 옵션

먼저 Light 2D의 기본 옵션을 알아봅시다. Light Type에서는 [Global], [Sprite], [Spot], [Freeform] 등 4가지 빛 종류를 선택할 수 있습니다. 또한 빛 오브젝트는 빛 색상인 Color와 세기인 Intensity를 설정할 수 있습니다.

Light 2D 컴포넌트의 기본 옵션

Light Type에서 선택할 수 있는 4가지 빛 종류를 자세히 살펴볼까요?

- [GameObject → Light] 메뉴로 Light Type에 따라 오브젝트를 생성할 수도 있고, 이미 생성된 빛 오브젝트의 Light Type을 변경해서 사용할 수도 있습니다.

Global

빛 오브젝트의 Transform 컴포넌트 정보와 관계없이 게임 월드에 있는 모든 오브젝트에 동일한 빛을 제공합니다.

Intensity를 0으로 설정했을 때

Intensity를 1로 설정했을 때

Sprite

Sprite 변수에 등록된 이미지 에셋 모양으로 빛을 생성합니다. 사용하는 이미지 에셋의 밝은 부분만 빛으로 할당하고 어두운 부분은 제외합니다.

이미지 모양대로 출력되는 빛과 빛 모양으로 사용할 이미지 에셋(TargetMark.png)

Spot

반지름과 각도를 설정해 구 형태로 방사하거나 원뿔 형태로 뻗어 나가는 빛을 제공합니다. Radius의 Inner 변수는 원 중심에서 주변까지 감쇠가 시작되는 길이를 나타내고 Outer 변수는 원 중심에서 도달하는 광원의 크기를 나타냅니다.

Spot Angle의 Inner는 빛의 감쇠가 시작되는 반경(360°일 때 전체 원에 적용)으로, Inner 값에서 Outer값까지는 감쇠가 적용됩니다. Outer는 빛이 영향을 주는 반경으로, 이 값을 벗어나면 빛이 적용되지 않습니다.

구 형태로 방사하는 빛

원뿔 형태로 뻗어 나가는 빛

Freeform

사용자가 원하는 모양으로 편집해서 빛을 적용할 수 있습니다. Falloff 변수는 빛이 중심에서 가장자리로 사라지는(감쇠하는) 범위를 정하며, 값이 클수록 퍼지는 영역이 넓어집니다. Falloff Strength 변수는 가장자리의 부드러움(감쇠 곡선)을 조절하며 값이 클수록 가장자리가 더 부드럽게 퍼집니다.

Freeform 빛 — Scene View

Freeform 빛 — Game View

그 밖의 섹션

Blending 섹션에서는 빛 오브젝트가 여러 개 있을 때 이를 섞는 방법 또는 순서 등을 설정합니다.

Light 2D 컴포넌트의 Blending 섹션

Shadows 섹션에서는 그림자 생성 여부, 강도 등을 설정합니다.

Light 2D 컴포넌트의 Shadows 섹션

Volumetric 섹션에서는 2D 빛 주변에 빛 기둥, 후광 등 체적광Volumetric Lighting을 화면에 렌더링하는 속성을 설정합니다.

Light 2D 컴포넌트의 Volumetric 섹션

● 체적광이란 공기, 안개 같은 매질에서 빛이 산란하여 화면에 빛 기둥처럼 보이는 효과를 말합니다.

Normal Maps 섹션은 Sprite 오브젝트에 법선 텍스처(보조 텍스처)를 붙여 2D에서도 빛의 각도에 따라 입체감과 하이라이트를 표현하는 기능을 적용할지를 설정합니다. 옵션은 3가지로, 노멀 맵 조명을 계산하지 않아 평면적으로 보이지만 가장 가벼운 [Disabled](기본값), 근사 계산으로 성능을 우선하는 [Fast](정확도 낮음, 빠르게 입체감을 낼 때 사용), 빛의 위치나 각도에 따른 반응을 더 정밀하게 계산해서 품질은 높지만 비용이 큰 [Accurate] 가운데 선택할 수 있습니다.

Light 2D 컴포넌트의 Normal Maps 섹션

둘째마당

게임 월드, 플레이어, 적 만들기

이번 마당에서는 2D 타일맵으로 게임 배경이 될 게임 월드를 만들고 플레이어와 적 오브젝트를 배치합니다. 기본 공격 스킬, 버프 스킬 등을 구현하고 주 공격 2가지와 보조 공격으로 나누어 만들어 봅니다. 마지막으로, 속성 보너스와 스킬 UI를 구현하여 스킬을 선택할 수 있도록 합니다.

03장 게임 월드와 플레이어 오브젝트

04장 적 오브젝트와 기본 전투 시스템

05장 플레이어 능력치와 기본 스킬 시스템 만들기

06장 다양한 주 공격 스킬과 보조 공격 스킬 구현하기

07장 근접 회전 공격과 광역 공격 스킬 구현하기

08장 속성 보너스 스킬 구현하고 스킬 UI 제작하기

03장

게임 월드와 플레이어 오브젝트

이번 장에서는 프로젝트 제작에 필요한 기본 요소를 설정합니다. Universal 2D 프로젝트를 생성하여 게임 화면 해상도, 카메라, 조명을 설정하고 이미지 등 사용할 리소스를 가져와 2D 타일맵 게임 월드를 만들어 봅시다. 이어서 플레이어 오브젝트를 생성하고 이동을 제어합니다.

학습 목표

1 Universal 2D 프로젝트 환경을 구성하고 Tilemap을 활용하여 2D 배경을 제작할 수 있다.
2 Rigidbody2D를 활용하여 8방향 이동 시스템을 구현할 수 있다.

03-1 2D 타일맵으로 게임 월드 제작하기
03-2 플레이어 오브젝트 만들고 물리 기반 이동 구현하기

03-1

2D 타일맵으로 게임 월드 제작하기

이 절에서는 프로젝트를 만들 때 필요한 기본 요소를 설정합니다. Universal 2D 프로젝트를 생성하고 나서 게임 화면 해상도, 카메라, 조명을 설정하고 외부에서 가져온 리소스를 게임 환경에 알맞게 설정한 뒤 유니티에서 제공하는 2D 타일맵을 이용해 게임 월드를 제작해 봅시다.

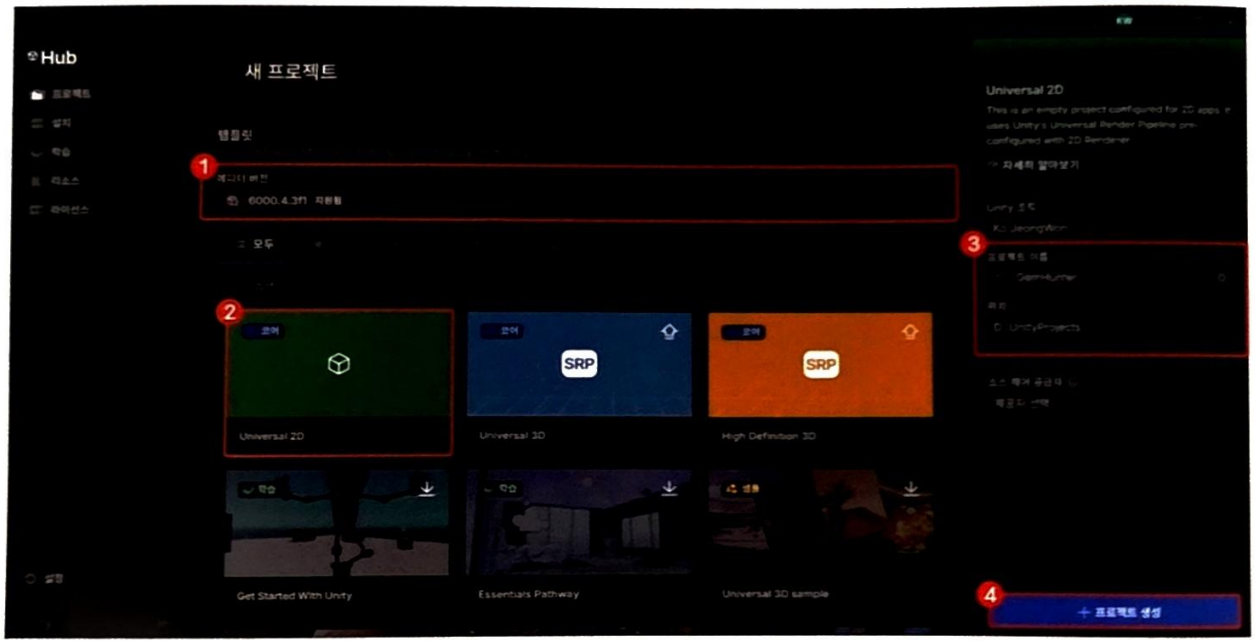
Do it! 실습 프로젝트 만들기

새로운 프로젝트를 생성하고 게임 화면을 설정하여 게임 월드 만들기를 준비합시다.

01 유니티 허브를 실행하고 ❶ [프로젝트] 탭에서 ❷ 화면 오른쪽 + 새 프로젝트 버튼을 클릭하여 새로운 프로젝트를 생성합니다.

유니티 허브의 [프로젝트] 탭

02 ❶ 에디터 버전에서 6000.4.3f1을 선택하고 ❷ Universal 2D 템플릿을 선택합니다. 이 책에서 만들 게임 이름은 〈젬 헌터 Gem Hunter〉로 정하겠습니다. ❸ 화면 오른쪽에서 프로젝트 이름에 'GemHunter'를 입력하고 저장할 위치를 지정한 후 ❹ + 프로젝트 생성 버튼을 클릭해 프로젝트를 생성합니다.



유니티 허브에서 새 프로젝트 만들기

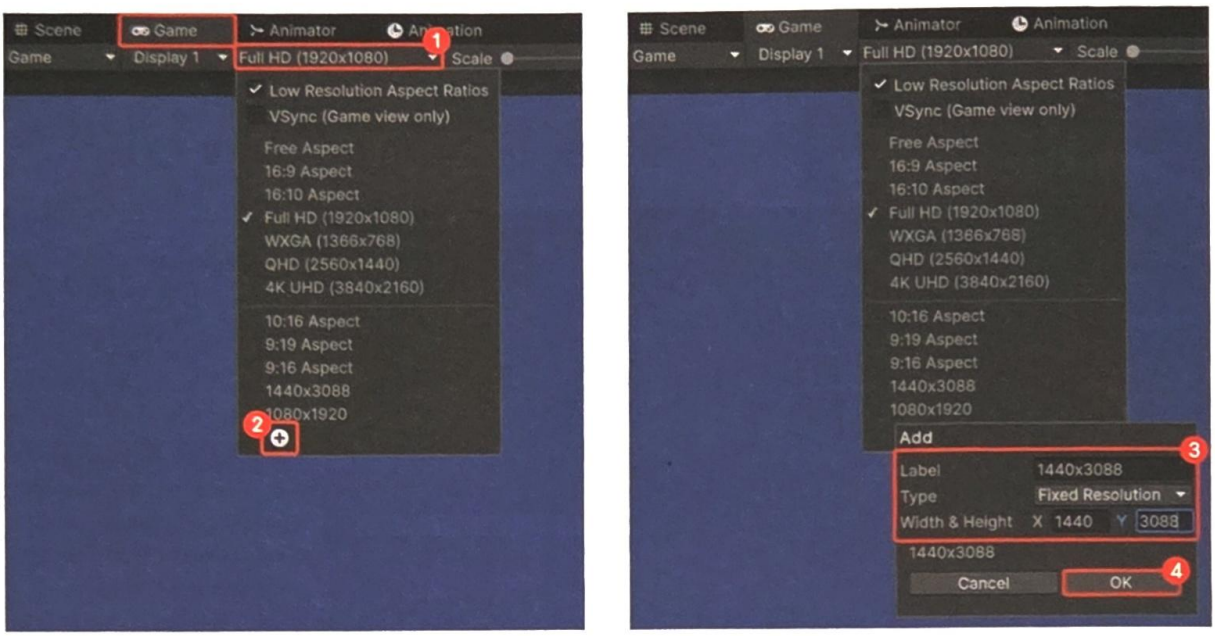
Do it! 실습 게임 화면 설정하기

프로젝트를 생성했다면 이번에는 카메라, 조명을 포함한 게임 화면을 설정합니다.

01 프로젝트를 생성하고 유니티 에디터를 실행합니다. 유니티 에디터 화면에서 ❶ Game View의 해상도를 설정하는 드롭다운 버튼 ▾을 클릭하고 ❷ 〈Add New Item〉 버튼 ➕으로 새로운 해상도를 생성합니다.

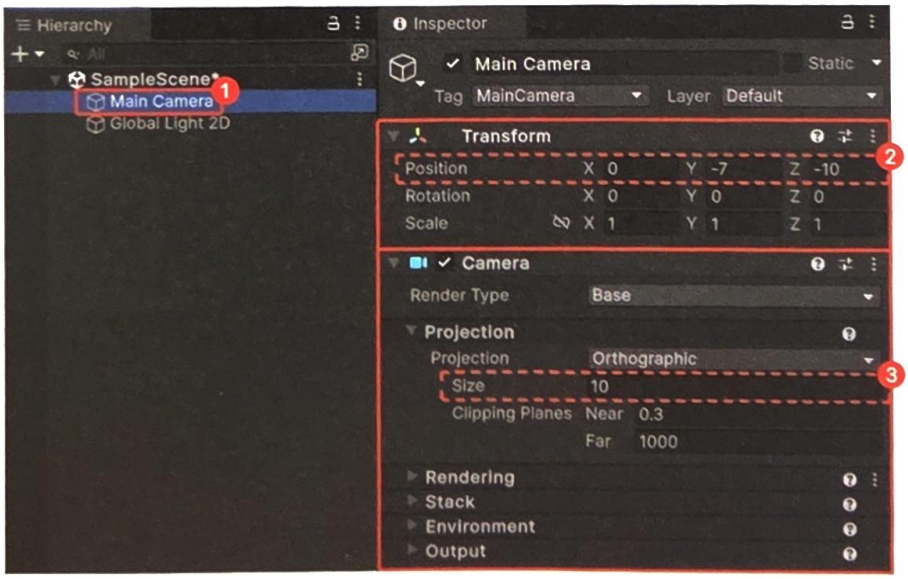
❸ Add 창이 뜨면 새로운 해상도의 Label은 '1440×3088', Type은 [Fixed Resolution], Width & Height는 X: '1440', Y: '3088'로 설정하고 ❹ 〈OK〉 버튼을 클릭합니다.

Game View의 해상도 설정에서 해상도란 현재 프로젝트에서 작업할 때 보는 화면 상태로, UI 배치 등을 고려해서 최종 출력할 해상도와 동일하게 맞추는 것이 좋습니다.



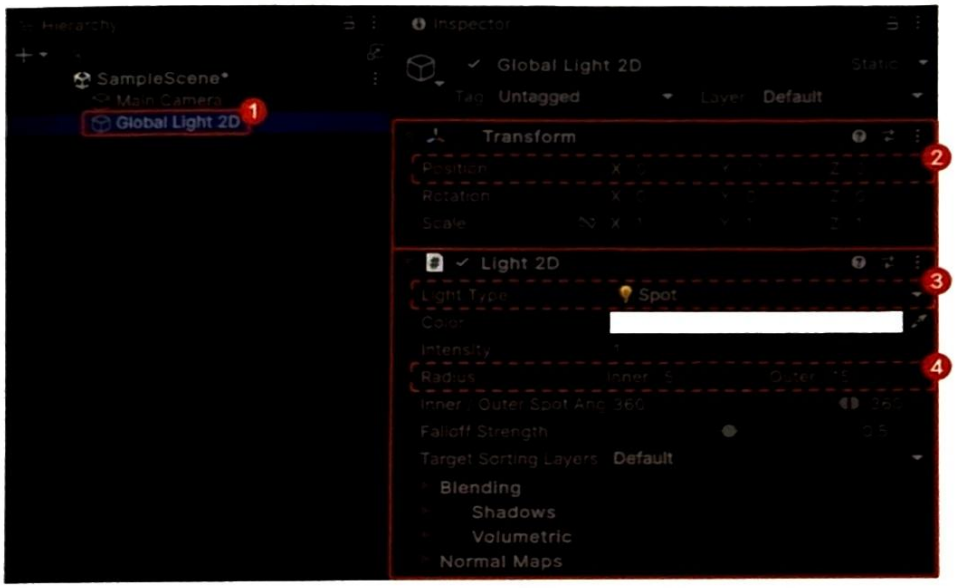
Game View 해상도 설정하기

02 ❶ 다음으로, 카메라와 조명을 설정합니다. ❷ 플레이어가 화면 가운데에 오도록 카메라의 위치는 (0, -7, -10)으로, ❸ Camera 컴포넌트에서 Projection 섹션의 Size 변수를 '10'으로 설정합니다.



Main Camera 오브젝트의 컴포넌트 설정하기

03 플레이어 위치가 가장 밝고 멀수록 어둡도록 플레이어 위치와 같게 ❶ 조명 위치를 ❷ (0, -7, 0)으로 설정하고 ❸ Light 2D 컴포넌트의 Light Type 옵션은 [Spot]으로 선택합니다. ❹ 조명 범위를 조절하는 Radius 변수를 Inner: '5', Outer: '15'로 설정합니다.



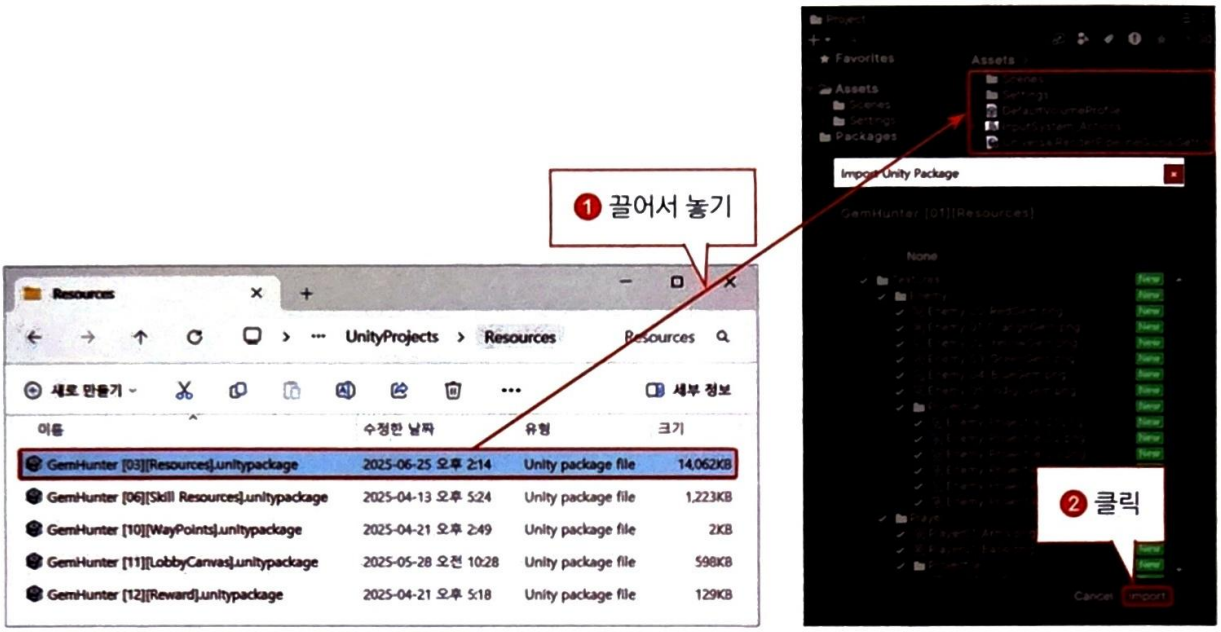
Global Light 2D 오브젝트의 컴포넌트 설정하기

Do it! 실습 게임 리소스 가져오기

이번에는 패키지 파일(*.unitypackage)에서 배경, 플레이어, 적 등에 사용할 이미지를 가져와서 설정합니다. 이들 이미지는 03-2절 플레이어 오브젝트와 04-1절 적 오브젝트를 비롯한 게임 전반에서 사용합니다.

01 게임에서 사용할 리소스를 가져옵니다. ❶ 자료실에서 내려받은 'GemHunter [03] [Resources].unitypackage' 파일을 Project View로 끌어서 놓은 후 패키지 불러오기 창에서 파일을 확인하고 나서 ❷ 〈Import〉 버튼을 클릭해 리소스 파일을 가져옵니다.

● 내려받은 자료 안 Resources 폴더에는 패키지 파일이, Scripts 폴더에는 스크립트 파일이 있습니다.

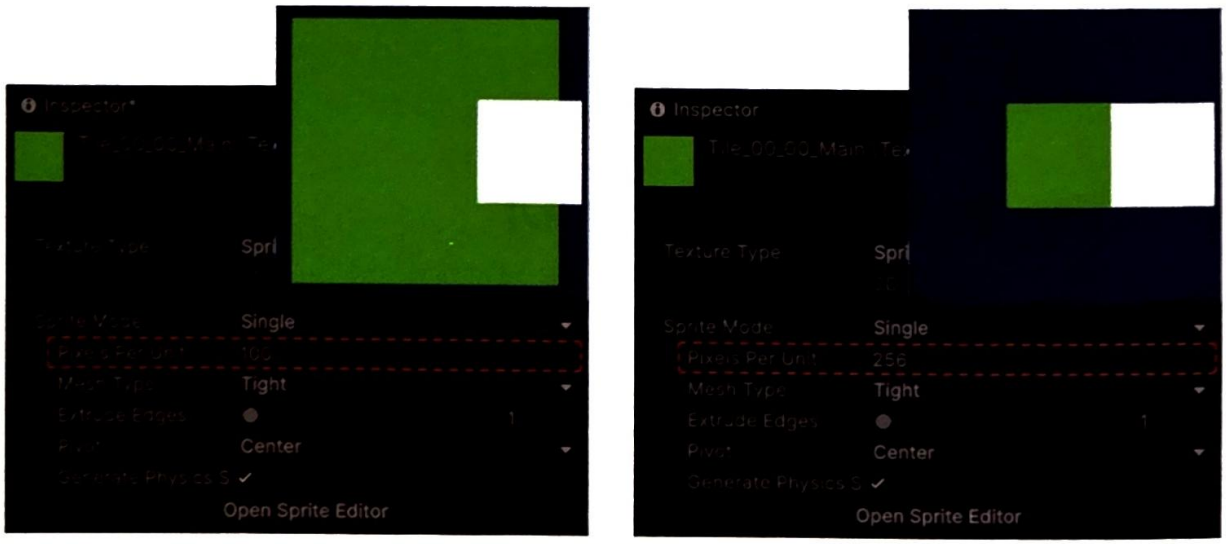


리소스 가져오기

02 Pixels Per Unit은 몇 픽셀을 Unity Scale 1로 설정할지 결정하는 변수입니다. Sprite Renderer에 적용하는 이미지는 Pixels Per Unit과 실제 이미지 크기에 따라 Transform의 Scale이 같은 1이어도 크기가 달라 보입니다. 일반적으로 크기를 맞추려면 원본 이미지의 크기와 Pixels Per Unit을 동일하게 설정하거나 원하는 배율에 따라 설정합니다.

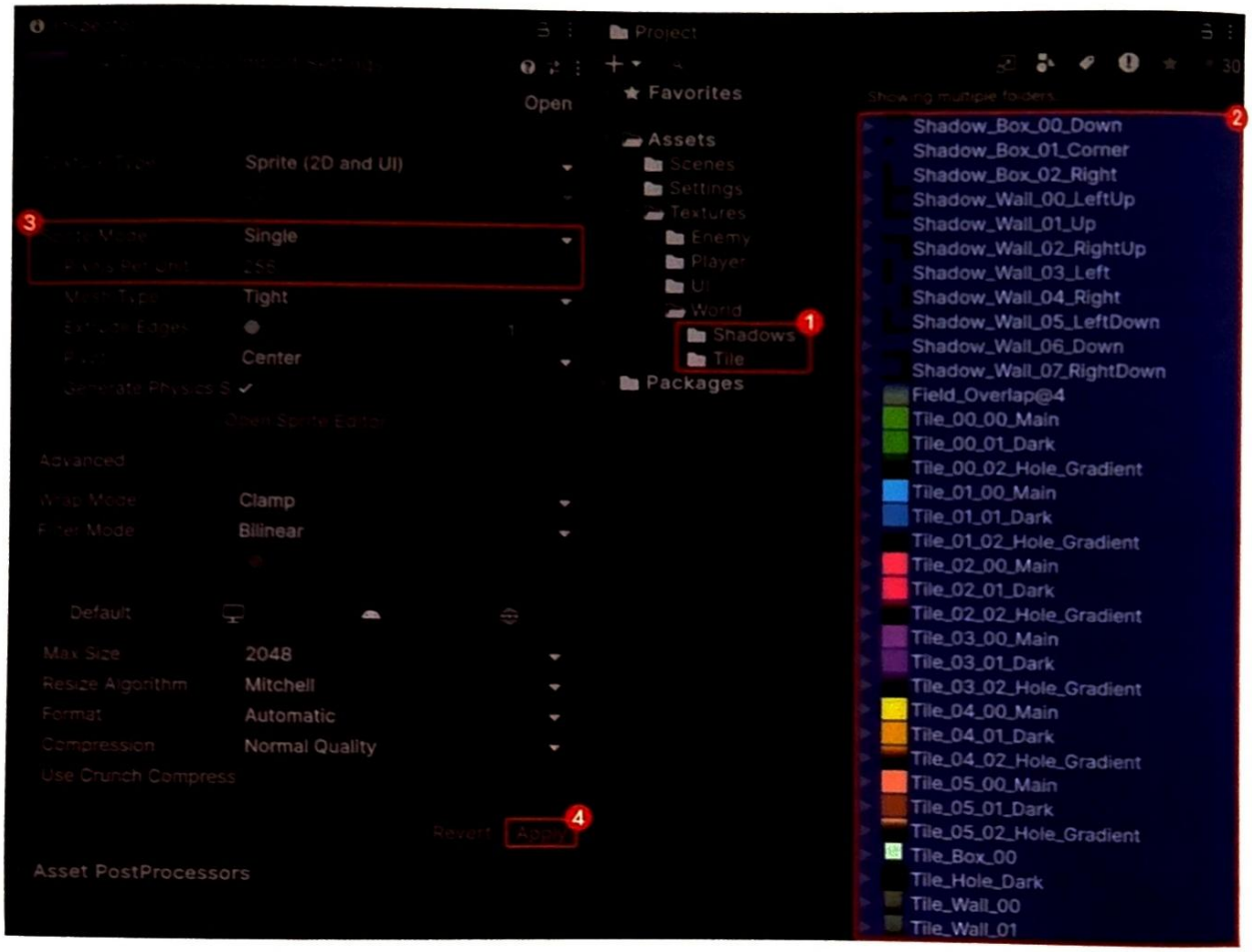
다음 그림에서 Tile_00_00_Main.png 이미지의 원본 크기가 256×256이므로 Pixels Per Unit을 '256'으로 설정해야 유니티에서 기본으로 제공하는 2D 오브젝트의 크기와 같아집니다.

타일 이미지는 Assets > Textures > World > Tile 폴더에 저장했습니다.



Pixels Per Unit이 100일 때(왼쪽)와 256일 때(오른쪽) 이미지 크기

03 게임 월드에 배치하는 타일 크기는 256×256이므로 게임 월드를 제작할 때 사용하는 바닥, 벽, 낭떠러지, 그림자 타일 역시 Pixels Per Unit을 '256'으로 설정합니다. ❶ 타일을 저장한 Shadows 폴더와 Tile 폴더를 함께 선택하고 ❷ 모든 타일을 선택한 후 ❸ Pixels Per Unit을 '256'으로 설정하고 ❹ 〈Apply〉 버튼을 누릅니다.



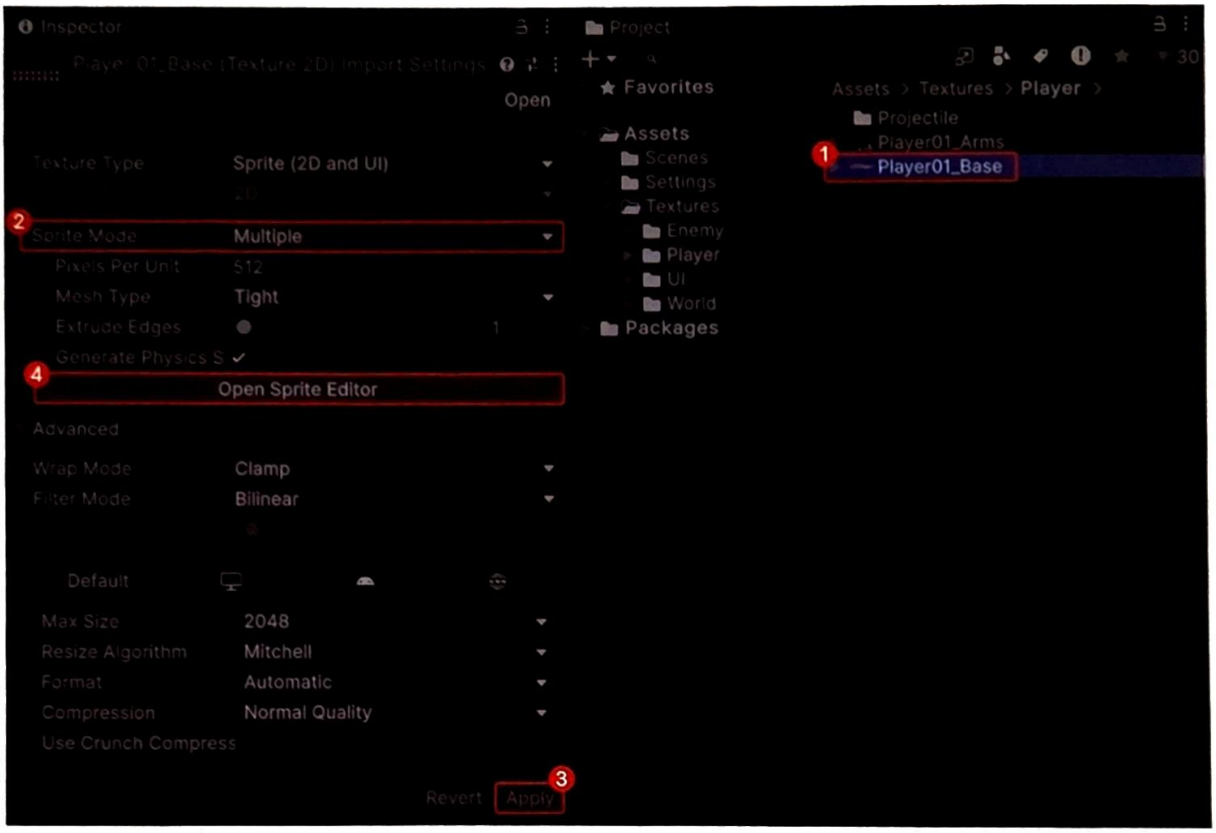
타일 이미지의 Pixels Per Unit 설정하기

Do it! 실습 플레이어와 적 이미지 설정하기

플레이어와 적 캐릭터는 512×512로 제작했으므로 발사체를 제외한 플레이어와 적 이미지의 Pixels Per Unit은 '512'로 설정합니다. 그런데 플레이어와 적 캐릭터를 설정하기 전에 유니티의 Sprite Editor 기능을 이용해서 이미지를 분할해야 합니다. 플레이어와 적 이미지를 보면 애니메이션을 재생할 때 필요한 이미지를 텍스처 아틀라스Texture Atlas 형태로 저장했기 때문입니다. 플레이어와 적 이미지를 분할하고 Pixels Per Unit을 설정해 봅시다.

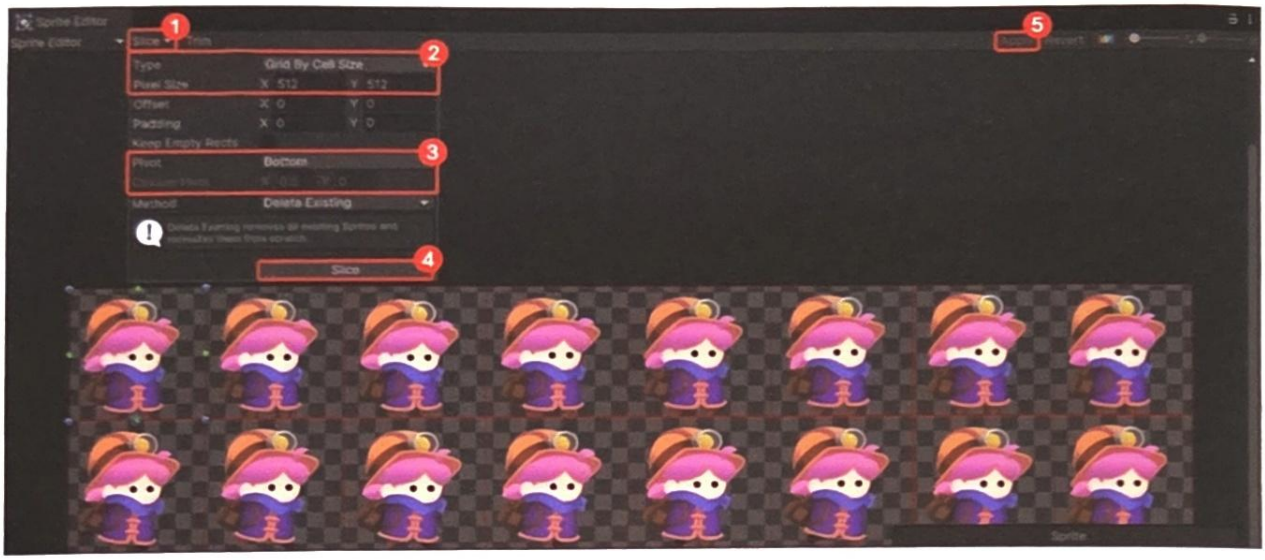
● 텍스처 아틀라스란 여러 스프라이트 이미지를 하나의 큰 이미지로 모은 형태를 말합니다.

01 ❶ Project View에서 Player01_Base 이미지를 선택한 후 ❷ Inspector View에서 Sprite Mode를 [Multiple]로 설정하고 ❸ 〈Apply〉 버튼을 클릭해 저장합니다. ❹ 〈Open Sprite Editor〉 버튼을 클릭해서 Sprite Editor View를 엽니다.



플레이어 이미지의 Sprite Mode 설정하기

02 캐릭터 하나를 512×512 크기로 만들었으므로 Sprite Editor View에서 ❶ Slice 드롭다운 메뉴를 열어 ❷ Type은 셀 크기 단위로 분할하는 [Grid By Cell Size]로, Pixel Size는 X: '512', Y: '512'로 설정합니다. ❸ Pivot을 [Bottom]으로 설정해서 발 부분이 중심축이 되도록 하고 ❹ 〈Slice〉 버튼을 클릭해 이미지를 분할합니다. ❺ 설정이 끝나면 〈Apply〉 버튼을 클릭해서 현재 설정한 내용을 저장합니다. 적 캐릭터 이미지 6개도 플레이어 이미지와 마찬가지로 분할합니다.



플레이어 이미지 분할과 피벗 설정하기

03 플레이어의 무기 이미지는 타깃 방향에 따라 회전합니다. 회전할 때 기준이 되는 Pivot 옵션을 설정하고자 Player01_Arms 이미지를 선택하고 〈Open Sprite Editor〉 버튼을 클릭해서 Sprite Editor View를 엽니다. ❶ Sprite Editor View에 보이는 무기 이미지를 선택하고 ❷ Pivot을 [Custom]으로, Custom Pivot을 X: '0.5', Y: '0.22'로 설정합니다. ❸ 마지막으로 〈Apply〉 버튼을 클릭하여 설정을 적용합니다.

플레이어 무기(손) 이미지의 Pivot 옵션 설정하기

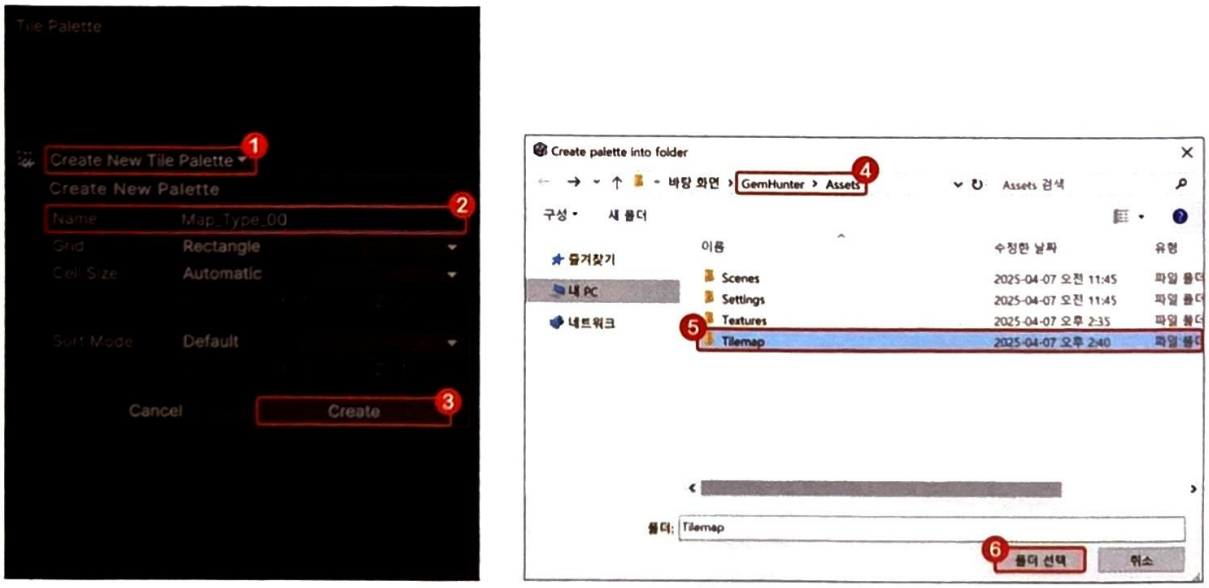
여기서는 그림으로 정확한 위치를 확인하고자 Sprite Editor View를 이용했지만, Inspector View에서 Pivot 옵션을 설정해도 결과는 마찬가지입니다.

Do it! 실습 타일 팔레트와 타일 에셋 생성하고 설정하기

타일맵 Tilemap이란 2D 게임을 개발할 때 타일 형태의 리소스를 배치해서 게임 월드를 표현하는 방법으로, 적은 리소스로도 다양한 맵을 제작할 수 있다는 점이 장점입니다. 유니티에서는 그래픽 디자이너가 아니더라도 타일맵 기능을 이용하면 타일 리소스로 맵을 만들 수 있습니다. **타일 팔레트** Tile Palette는 타일맵 오브젝트에 배치할 타일 에셋을 등록해 두는 저장소입니다. 팔레트에 있는 타일 에셋은 여러 속성에 따라 배치하거나 삭제할 수 있습니다. 그림을 그릴 때 물감을 짜서 사용하는 도구인 팔레트와 같은 역할입니다.

01 [Window → 2D → Tile Palette] 메뉴로 Tile Palette View를 열고 ❶ [Create New Tile Palette] 드롭다운 메뉴로 ❷ 새로운 타일 팔레트를 생성합니다. 타일 팔레트의 이름인 Name 변수를 'Map_Type_00'으로 설정하고 ❸ 〈Create〉 버튼을 클릭해서 팔레트를 생성합니다. 타일 팔레트 에셋은 현재 프로젝트 내부에 저장해야 하므로 ❹ Assets 폴더 아래에 ❺ Tilemap 폴더를 만들고 ❻ 〈폴더 선택〉 버튼을 클릭해서 저장합니다.

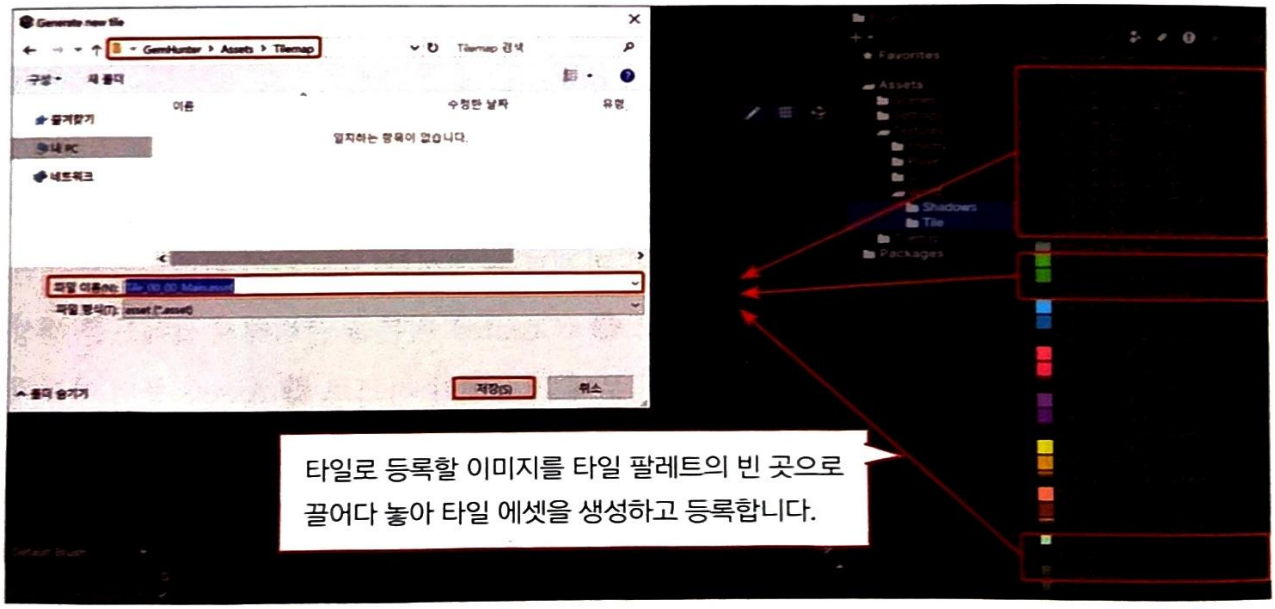
Project View에서 하위 폴더를 생성할 폴더를 선택하고 나서 + ▾ 버튼을 클릭하고 [Folder] 메뉴를 선택하면 새 폴더를 만들 수 있습니다.



새로운 타일 팔레트 생성하기

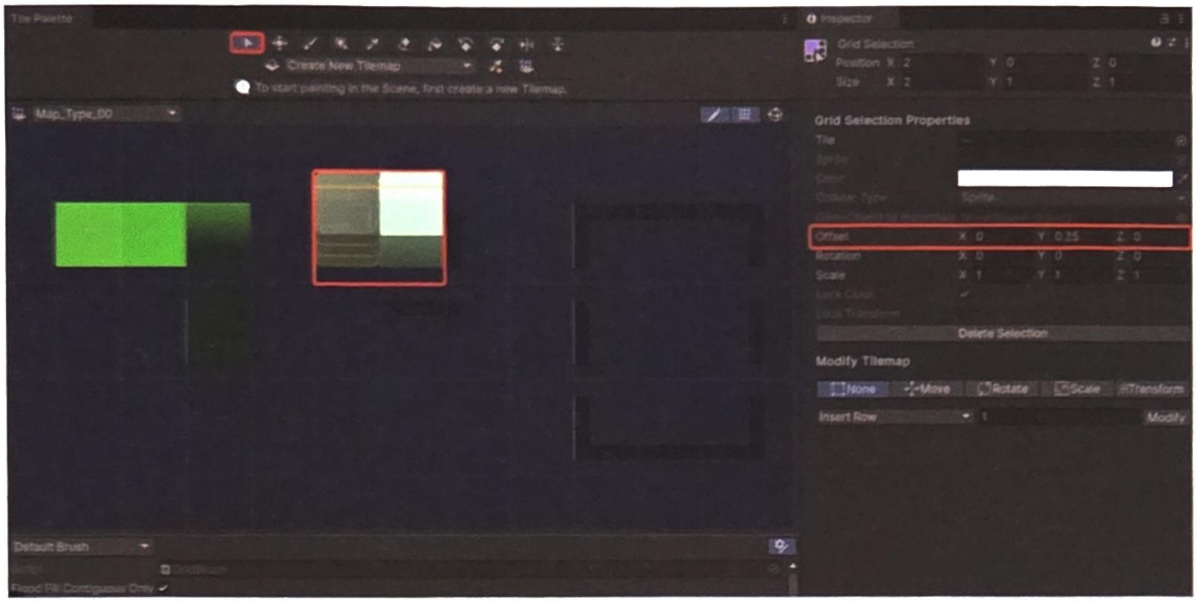
02 타일 에셋은 맵에 배치할 수 있는 하나의 타일로, Sprite, Color, Collider Type 등의 속성을 설정할 수 있습니다. 쉽게 말해 팔레트에 짜는 물감과 같은 개념입니다. 앞서 생성한 타일 팔레트에 이미지 에셋을 드래그하면 타일 에셋을 생성할 수 있습니다.

월드의 바닥에 배치하는 기본 타일(Tile_00_00_Main, Tile_00_01_Dark)과 낭떠러지 타일(Tile_00_02_Hole_Gradient, Tile_Hole_Dark), 벽 타일(Tile_Box_00), 외곽 벽 타일(Tile_Wall_00), 그림자 타일(Shadow_*)을 타일 팔레트의 빈 곳으로 끌어서 놓아 타일 에셋을 생성합니다. 타일 에셋도 현재 프로젝트 내부에 저장해야 하므로 Assets 〉 Tilemap 폴더에 저장합니다.



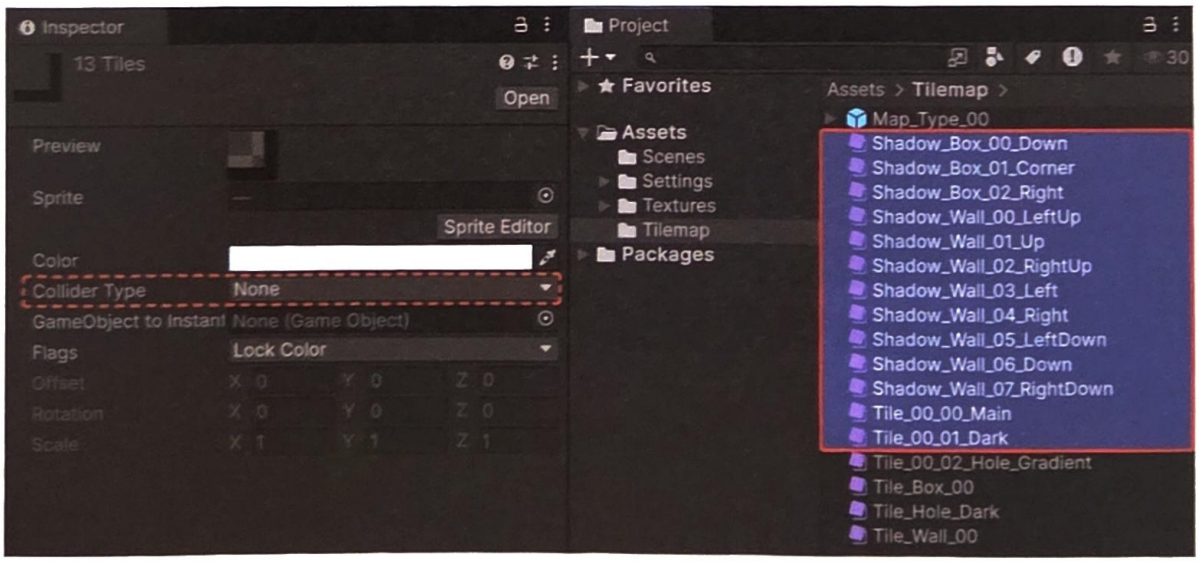
타일 에셋 생성 후 타일 팔레트에 등록하기

03 Tile_Wall_00과 Tile_Box_00 타일 에셋은 벽을 표현하고자 일반 타일보다 Y값을 더 크게 제작했습니다. 벽 뒤에 섰을 때는 벽이 플레이어나 적 오브젝트 일부를 가리도록 하고 다른 타일과 바닥 높이를 맞추도록 Y축 Offset을 '0.25'로 설정합니다.



Tile_Wall_00과 Tile_Box_00 타일의 Y축 Offset 설정하기

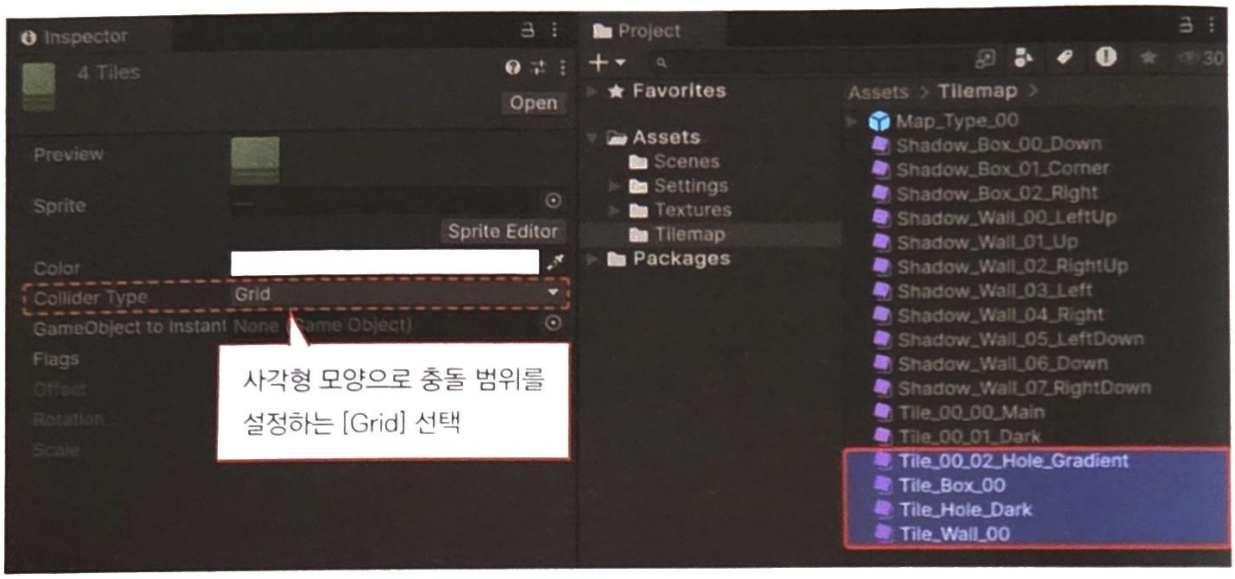
04 타일맵에 배치한 타일 에셋은 Collider Type을 설정해서 충돌하거나 충돌하지 못하도록 설정할 수 있습니다. 먼저 Tile_00_00_Main과 Tile_00_01_Dark 타일 에셋 2개는 맵 바닥을 표현하는 용도로, 그림자 타일 에셋은 벽의 그림자를 표현하는 용도로 사용하므로 플레이어가 이동할 때 충돌하지 않도록 Collider Type을 [None]으로 설정합니다.



타일 에셋의 충돌 속성 설정하기([None]이면 충돌 불가능)

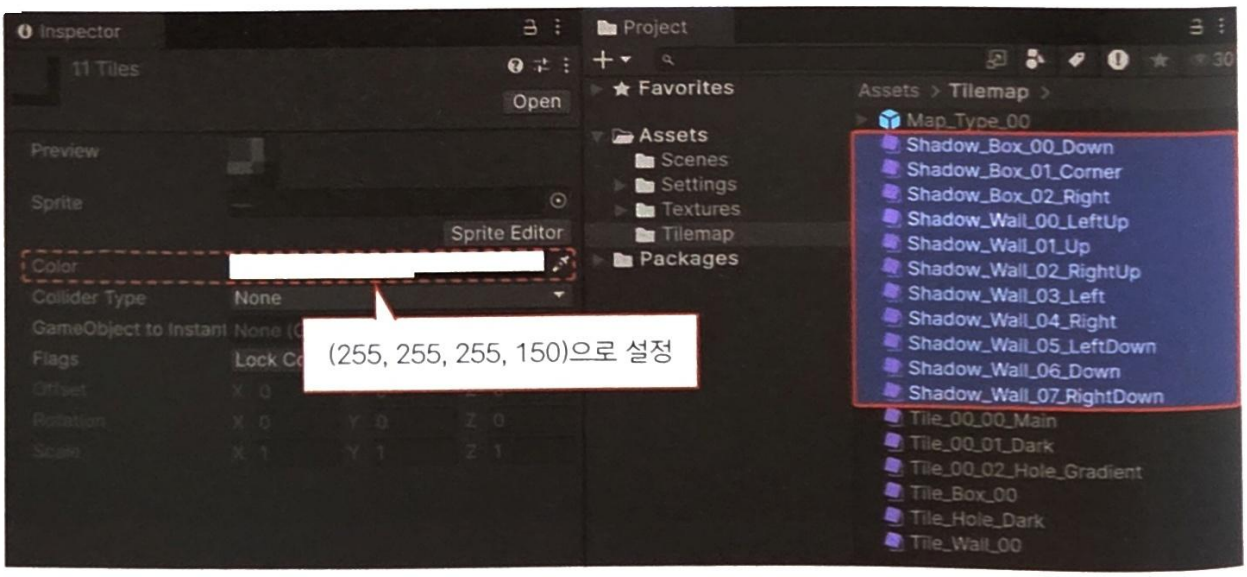
05 Tile_00_02_Hole_Gradient, Tile_Box_00, Tile_Hole_Dark, Tile_Wall_00 등 타일 에셋 4개는 플레이어와 충돌하도록 Collider Type을 [Grid]로 설정합니다.

[Grid]는 격자 단위, [Sprite]는 스프라이트 모양 단위로 충돌을 판정합니다.



타일 에셋의 충돌 속성 설정하기([Grid]이면 충돌 가능)

06 그림자(Shadow_) 타일 에셋은 Color를 (255, 255, 255, 150)으로 설정해서 반투명하게 표현합니다.



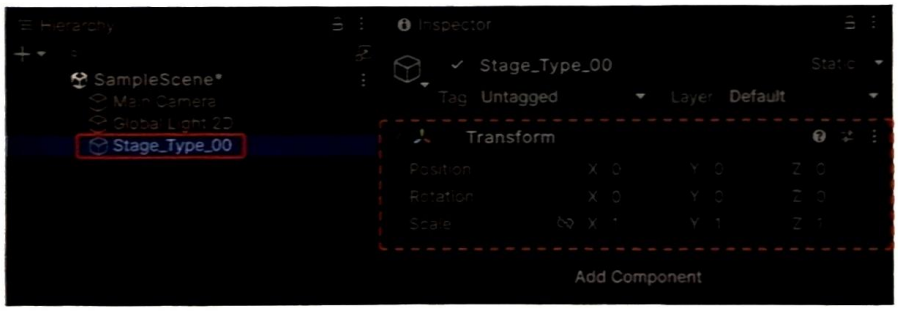
그림자 타일 에셋의 색상 설정하기

Do it! 실습 타일맵 오브젝트로 게임 월드 만들기

타일맵 오브젝트는 타일 에셋을 배치하는 공간으로, 실제 게임에서 보는 타일 형태의 게임 월드를 구성할 수 있습니다. 마치 그림을 그리는 도화지와 같습니다. 타일맵 오브젝트를 생성하면 Grid 오브젝트가 함께 생성되고 타일맵 오브젝트가 Grid 오브젝트의 자식으로 배치됩니다.

Grid 오브젝트는 자식으로 배치하는 타일맵 오브젝트를 관리하는 역할로, Cell Layout과 Cell Swizzle 정보를 이용해서 배치하는 타일맵 방식을 Rectangle, Hexagonal, Isometric, Isometric Z As Y 등으로 설정할 수 있습니다.

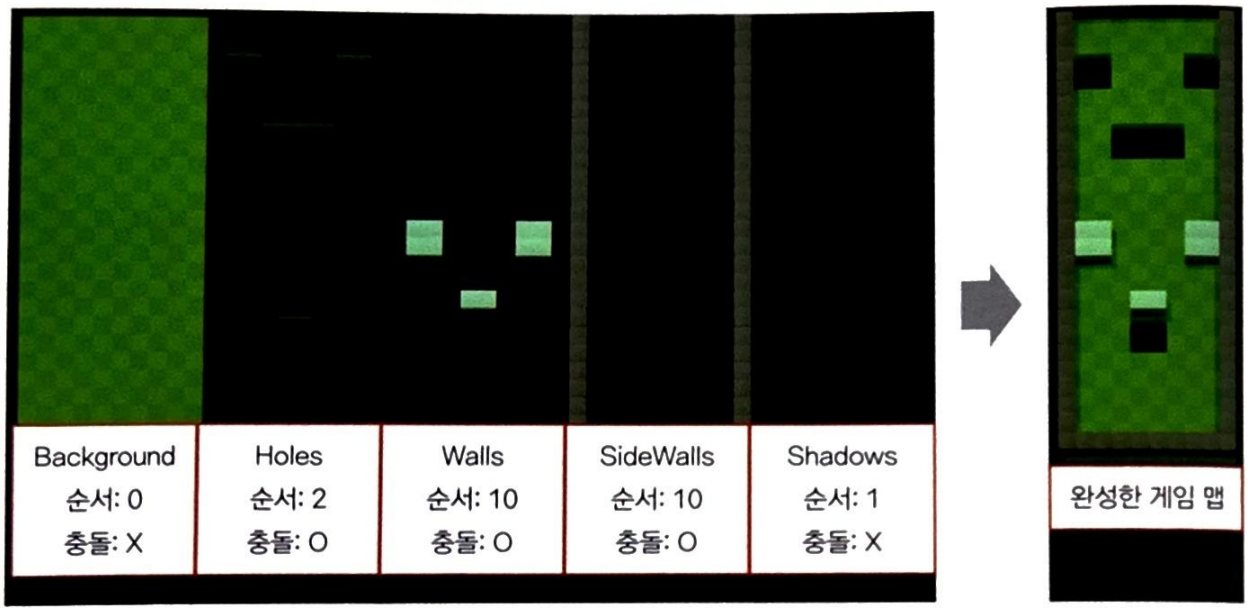
01 [GameObject → Create Empty] 메뉴로 게임 월드를 출력하는 타일맵과 현재 스테이지 숫자를 출력하는 텍스트와 같은 오브젝트를 관리할 빈 오브젝트를 생성하고 이름을 'Stage_Type_00'으로 변경합니다.



맵 오브젝트 관리용 빈 오브젝트 생성하고 설정하기

02 타일맵 오브젝트를 하나만 사용해 게임 월드를 표현할 수도 있지만, 이 책처럼 탑다운 형태의 게임을 제작할 때는 충돌하는 타일, 충돌하지 않는 타일, 더 앞에 그려야 하는 타일 등 용도에 따라 타일맵을 구분해서 표현합니다.

배경, 낭떠러지, 벽, 외곽 벽, 그림자 등 모두 5개의 타일맵을 제작해서 하나의 게임 월드를 완성합니다.



보이는 순서와 용도에 따라 구분한 타일맵 오브젝트(Order in Layer 순서대로)

03 [GameObject → 2D Object → Tilemap → Rectangular] 메뉴로 배경 타일을 배치할 사각형 타일맵 오브젝트를 생성하고 이름을 'Background'로 변경합니다.

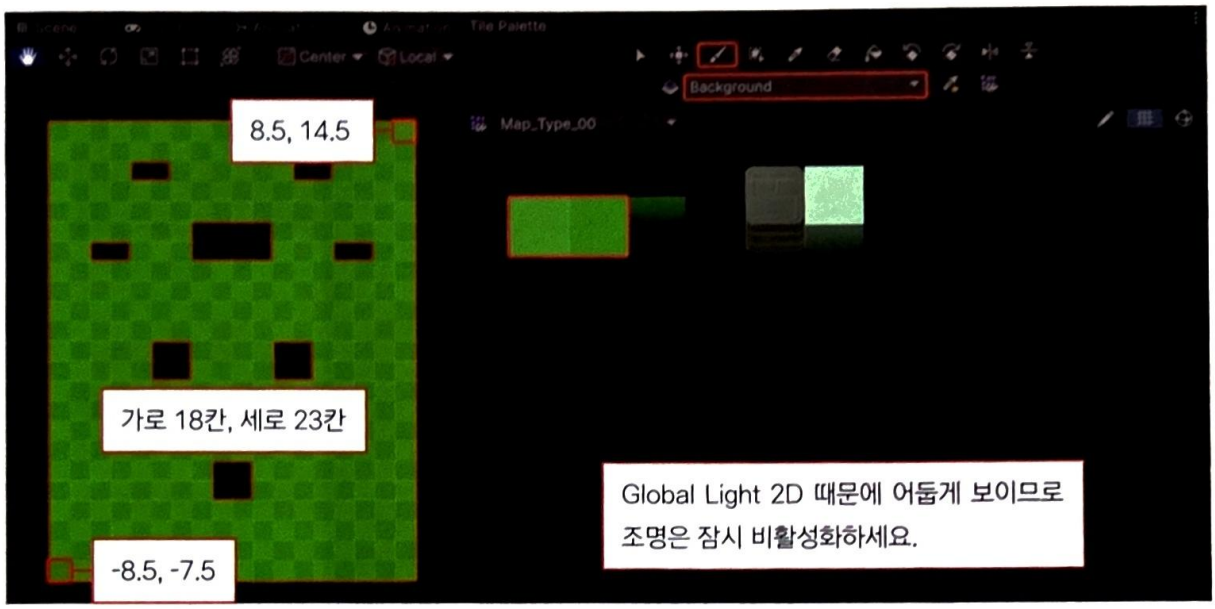
배경 타일을 배치하는 타일맵 오브젝트 생성하고 설정하기

04 타일 에셋을 배치한 타일 팔레트를 열고 Background 오브젝트를 선택합니다. Scene View를 보면 타일맵에 타일 에셋을 배치할 수 있는 격자 맵이 활성화됩니다. 현재 설정한 조명 Global Light 2D 때문에 어두워 보이므로 잠시 조명을 비활성화하고 작업합니다.

해당 오브젝트를 비활성화하려면 Inspector View에서 이름 앞 체크 박스를 해제하면 됩니다.

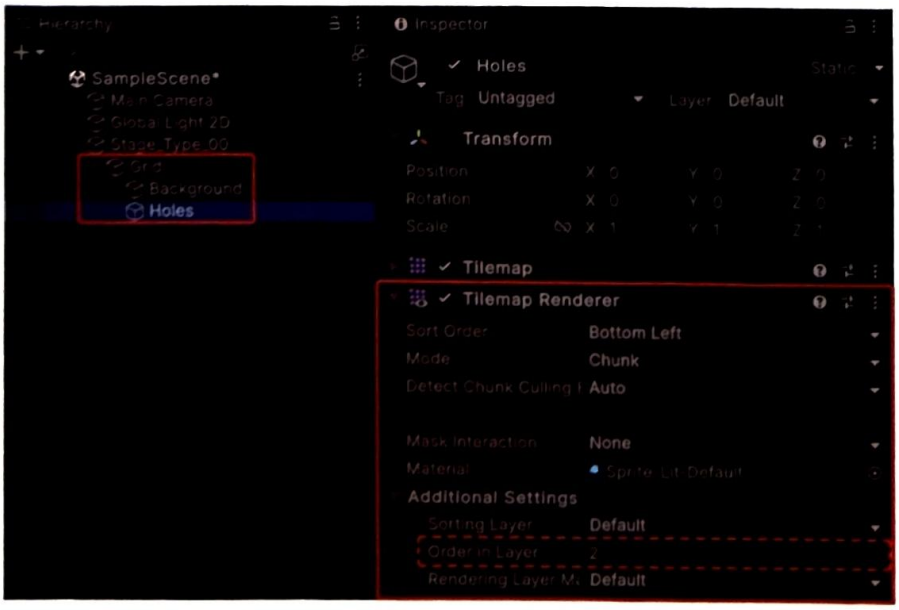
타일맵은 여러 개 생성해서 활용할 수 있으므로 타일 팔레트에서 타일맵을 선택하고 타일을 배치합니다. 클릭한 위치에 타일을 하나씩 배치하는 〈Paint with active brush〉 버튼 으로 Tile_00_00_Main과 Tile_00_01_Dark 타일을 18×23 격자 형태로 번갈아 배치합니다. 타일을 배치하는 왼쪽 아래 위치는 (-8.5, -7.5), 오른쪽 위 위치는 (8.5, 14.5)입니다.

- 2×2 격자 4칸만 만들고 <Pick or marquee select new brush> 버튼 으로 2×2 격자를 선택한 후, <Paint a filled with active brush> 버튼 으로 전체 맵을 배치하면 격자 맵을 쉽게 만들 수 있습니다.
- 외곽에 Square 오브젝트를 임시로 배치해 두면 위치를 쉽게 알 수 있습니다.



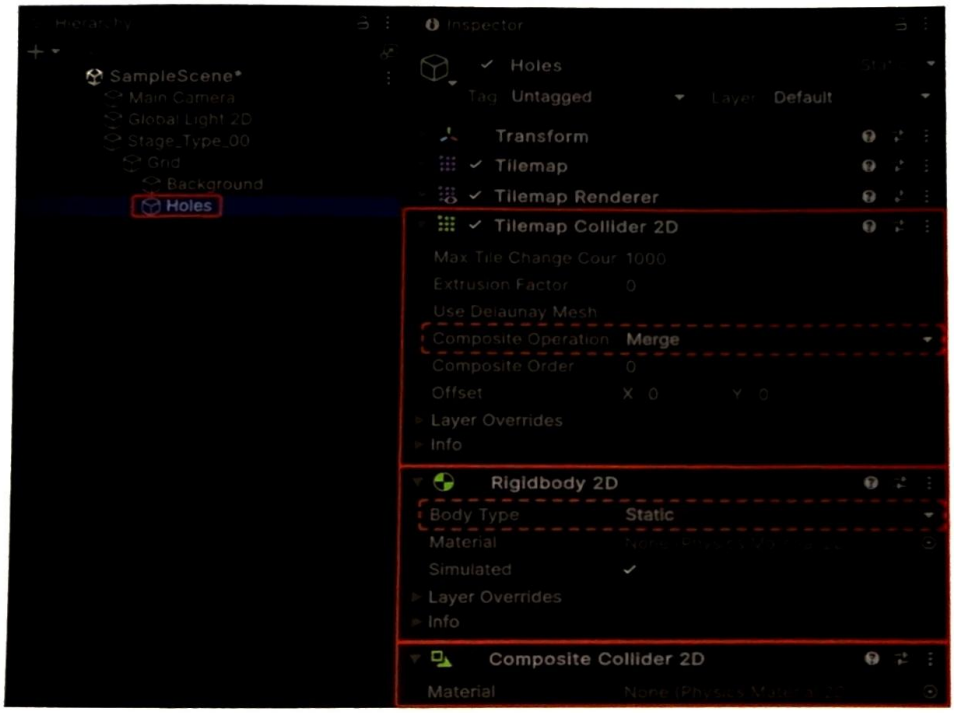
Background 타일맵 오브젝트에 타일 에셋 배치하기

05 타일맵을 생성할 때 Grid 오브젝트의 자식으로 배치되도록 Grid 오브젝트를 선택한 상태에서 [GameObject → 2D Object → Tilemap → Rectangular] 메뉴로 낭떠러지 타일을 배치하는 사각형 타일맵을 생성하고 이름을 'Holes'로 변경합니다. 레이어 순서가 0인 Background 배경 타일 앞에 그려지도록 Tilemap Renderer 컴포넌트의 Order in Layer를 '2'로 설정합니다.



낭떠러지 타일을 배치하는 타일맵 오브젝트 생성하고 설정하기

06 낭떠러지 타일맵은 플레이어와 충돌해야 하므로 Tilemap Collider 2D와 Composite Collider 2D 컴포넌트를 추가합니다. Tilemap Collider 2D 컴포넌트의 Composite Operation을 [Merge]로, Rigidbody 2D 컴포넌트의 Body Type을 [Static]으로 설정합니다.



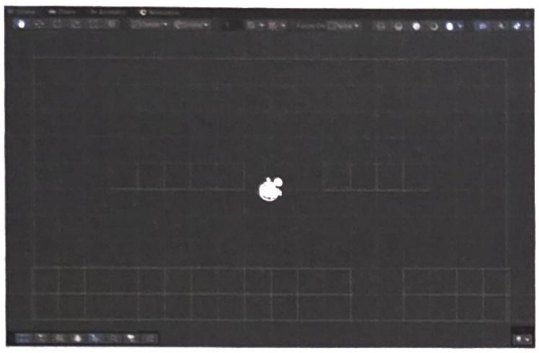
Holes 타일맵 오브젝트 충돌 처리 설정하기

알아 두면 좋아요! Composite Collider 2D 컴포넌트란?

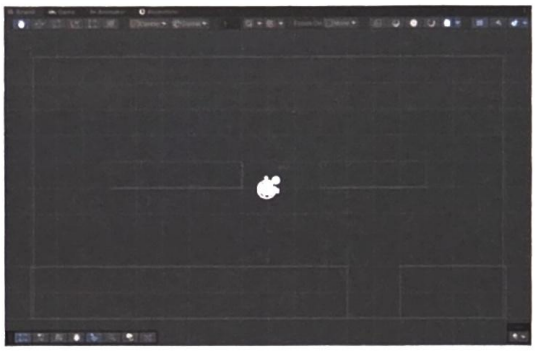
Composite Collider 2D 컴포넌트는 단일로 사용하지 않고 여러 오브젝트의 충돌 범위를 하나의 충돌 범위로 묶어서 관리합니다. 하나로 묶으므로 두 충돌 범위 사이에 생길 수 있는 균열이 생기지 않도록 하고 연산량을 줄이므로 최적화에 도움됩니다.

Composite Collider 2D 컴포넌트를 추가하면 Rigidbody 2D 컴포넌트가 함께 추가됩니다. Composite Collider 2D 컴포넌트로 충돌 범위를 하나로 묶으려면 기존 충돌 컴포넌트(OO Collider 2D)의 Composite Operation 옵션을 [Merge]로 설정해야 합니다.

▶ 계속



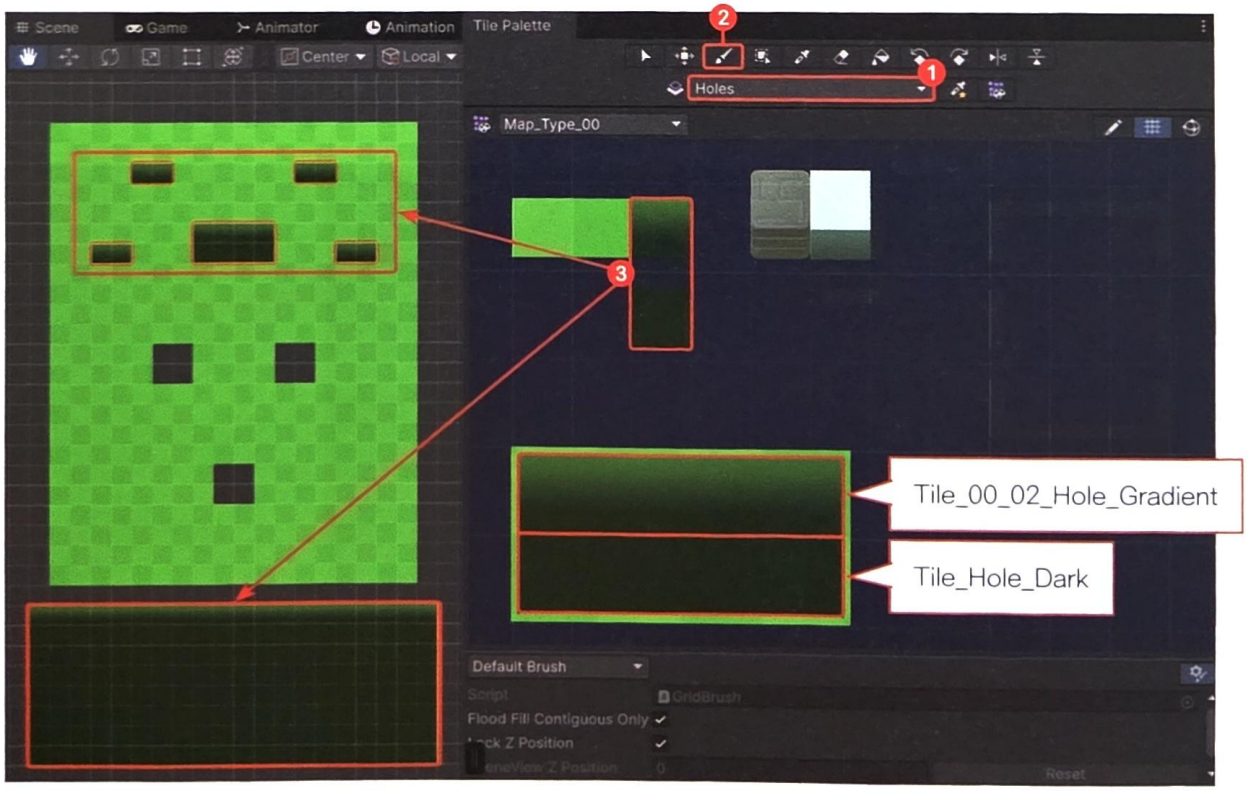
Composite Collider 2D를 적용하기 전



Composite Collider 2D를 적용한 후

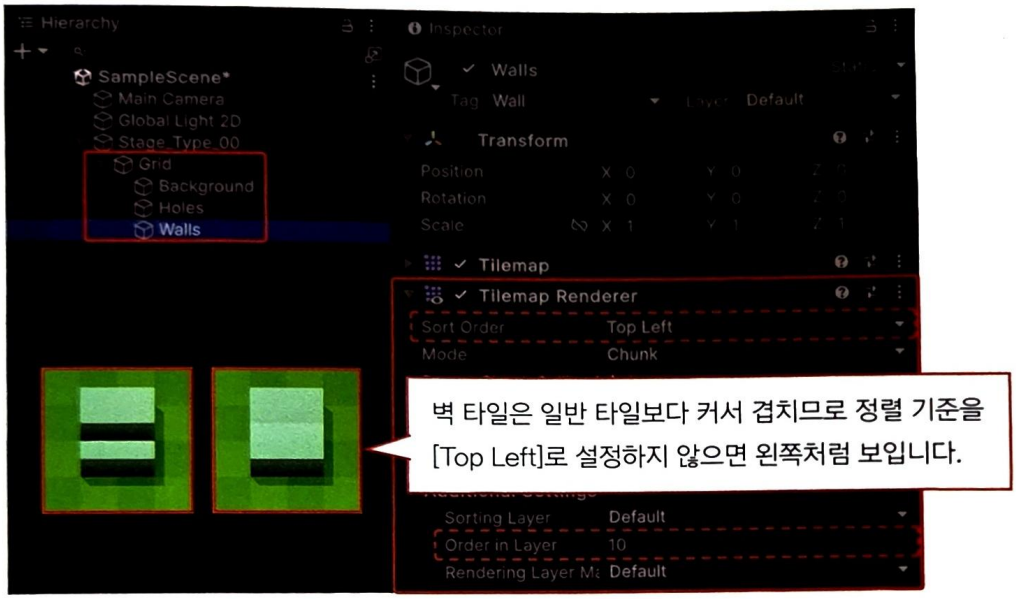
Rigidbody 2D 컴포넌트의 Body Type 옵션은 3가지입니다. 중력과 힘의 영향을 받는 [Dynamic], 중력과 힘의 영향이 없어 자원을 더 적게 사용하는 [Kinematic], 고정 오브젝트에 사용하며 자원을 가장 적게 사용하는 [Static] 등이 있습니다. 예를 들어, Holes, Walls, SideWalls, Shadows 오브젝트는 고정 오브젝트이므로 Body Type 옵션을 [Static]으로 설정합니다.

07 ❶ 타일 팔레트의 활성 타일맵을 [Holes]로 선택하고 ❷ 〈Paint with active brush〉 버튼 을 클릭해서 ❸ Tile_00_02_Hole_Gradient와 Tile_Hole_Dark 타일 에셋을 배치합니다.



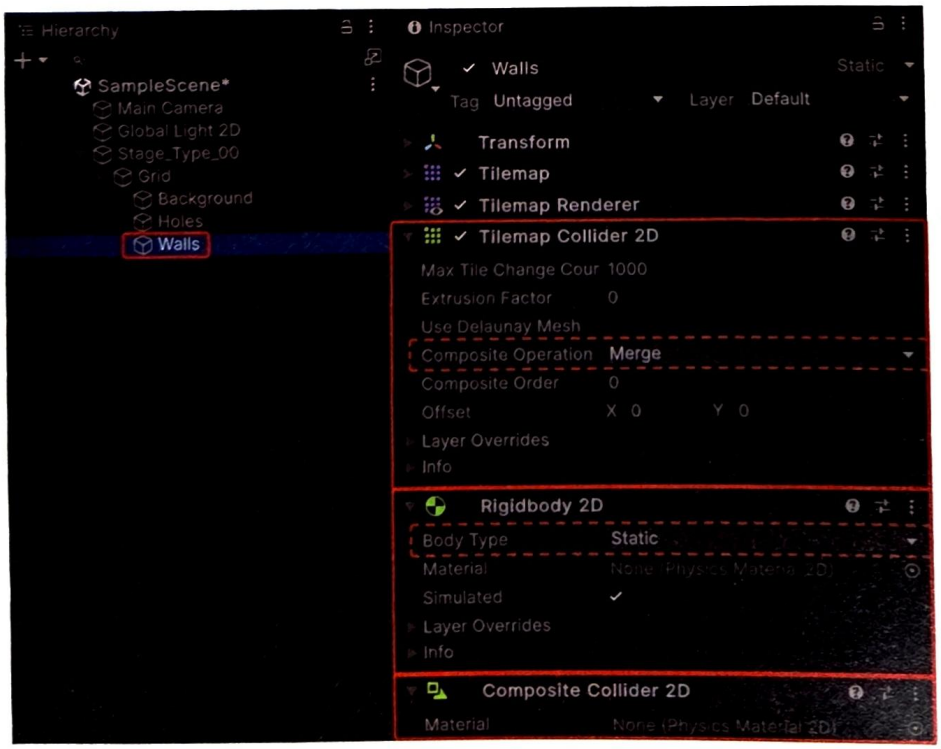
Holes 타일맵 오브젝트에 타일 에셋 배치하기

08 Grid 오브젝트를 선택한 상태에서 벽 타일을 배치하는 사각형 타일맵을 [GameObject → 2D Object → Tilemap → Rectangular] 메뉴로 생성하고 이름을 'Walls'로 변경합니다. 벽 타일은 일반 타일보다 크기가 커서 Y축에 나란히 배치했을 때 Y값이 더 큰 타일이 앞에 있는 듯이 보입니다. 그러므로 정렬 기준인 Sort Order를 [Top Left]로, Order in Layer를 '10'으로 설정합니다.



벽 타일을 배치하는 타일맵 오브젝트 생성하고 설정하기

09 벽 타일맵은 플레이어와 충돌해야 하므로 Tilemap Collider 2D와 Composite Collider 2D 컴포넌트를 추가합니다. Tilemap Collider 2D 컴포넌트의 Composite Operation을 [Merge]로, Rigidbody 2D 컴포넌트의 Body Type을 [Static]으로 설정합니다.

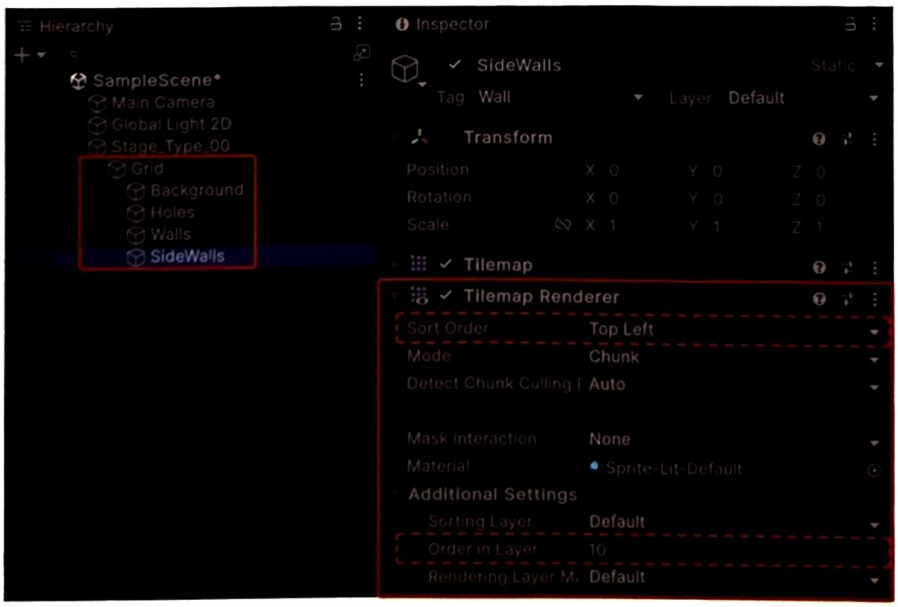


Walls 타일맵 오브젝트 충돌 처리 설정하기

10 타일 팔레트의 현재 활성 타일맵을 [Walls]로 선택하고 〈Paint with active brush〉 버튼 을 클릭해서 Tile_Box_00 타일 에셋을 배치합니다.

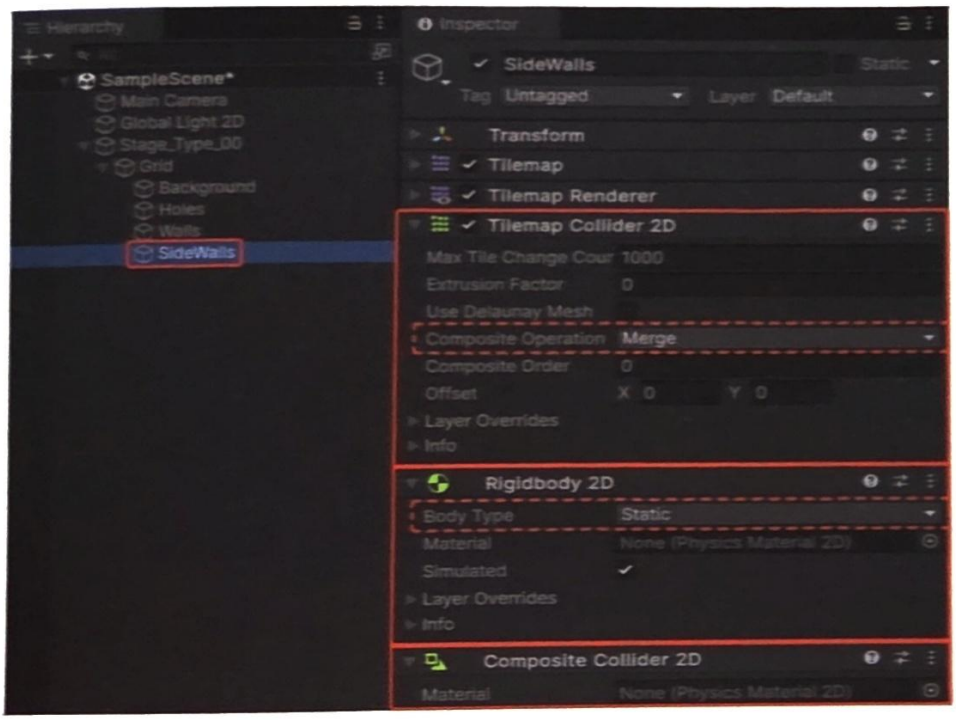
Walls 타일맵 오브젝트에 타일 에셋 배치하기

11 Grid 오브젝트를 선택한 상태에서 외곽 벽 타일을 배치하는 사각형 타일맵을 [Game Object → 2D Object → Tilemap → Rectangular] 메뉴로 생성하고 이름을 'SideWalls'로 변경합니다. 외곽 벽 타일도 마찬가지로 정렬 기준인 Sort Order를 [Top Left]로, Order in Layer를 '10'으로 설정합니다.



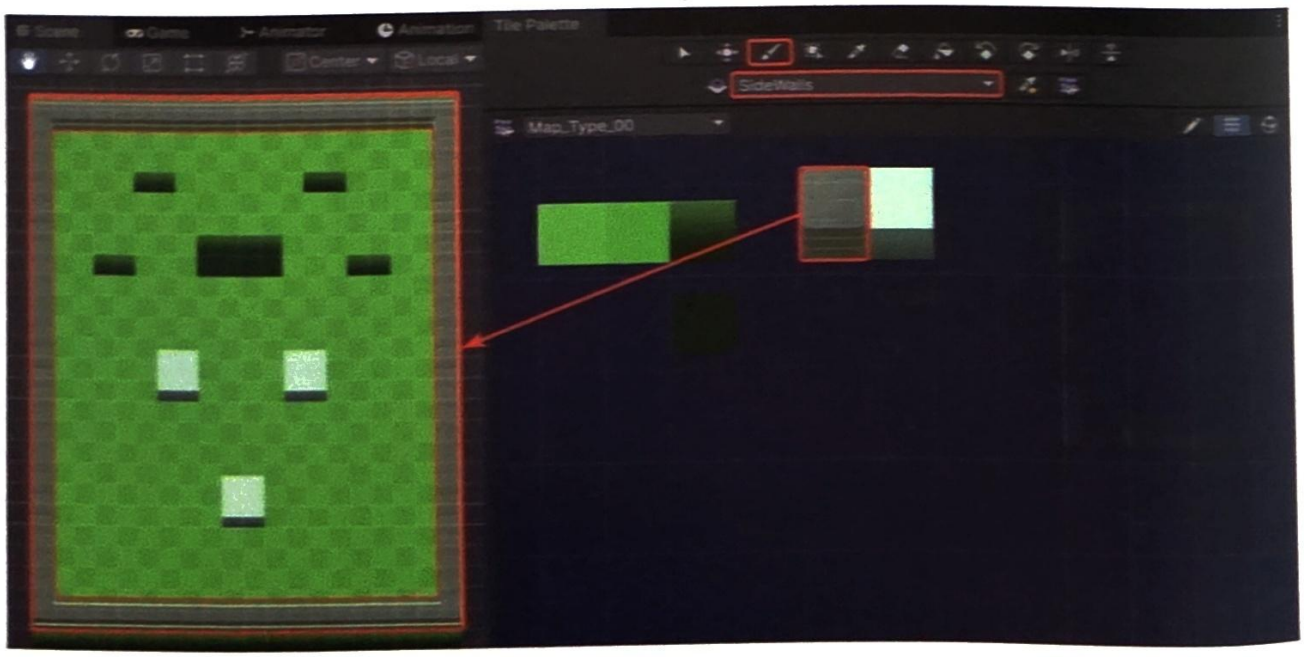
외곽 벽 타일을 배치하는 타일맵 오브젝트 생성하고 설정하기

12 외곽 벽 타일맵도 플레이어와 충돌해야 하므로 Tilemap Collider 2D와 Composite Collider 2D 컴포넌트를 추가합니다. Tilemap Collider 2D 컴포넌트의 Composite Operation을 [Merge]로, Rigidbody 2D 컴포넌트의 Body Type을 [Static]으로 설정합니다.



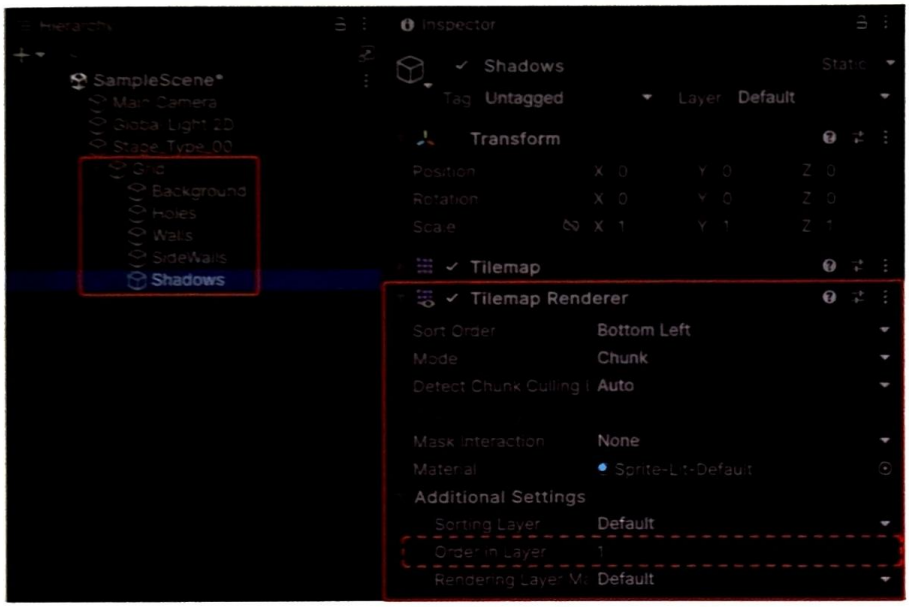
SideWalls 타일맵 오브젝트 충돌 처리 설정하기

13 타일 팔레트의 활성 타일맵을 [SideWalls]로 선택하고 〈Paint with active brush〉 버튼 을 클릭해서 Tile_Wall_00 타일 에셋을 배치합니다.



SideWalls 타일맵 오브젝트에 타일 에셋 배치하기

14 Grid 오브젝트를 선택한 상태에서 그림자 타일을 배치하는 사각형 타일맵을 [Game Object → 2D Object → Tilemap → Rectangular] 메뉴로 생성하고 이름을 'Shadows'로 변경합니다. 그림자 타일맵은 Order in Layer를 '1'로 설정해서 배경 타일보다 앞에 보이고 낭떠러지와 벽 타일보다 뒤에 보이도록 합니다.

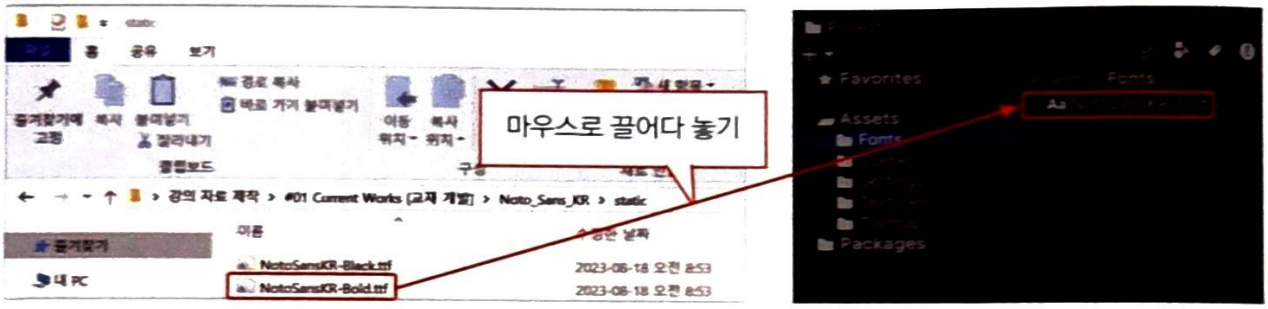


그림자 타일을 배치하는 타일맵 오브젝트 생성하고 설정하기

15 타일 팔레트의 활성 타일맵을 [Shadows]로 선택하고 〈Paint with active brush〉 버튼 을 클릭해 외곽 벽 안쪽으로 그림자가 만들어질 수 있도록 Shadow_* 타일 에셋을 배치합니다.

Shadows 타일맵 오브젝트에 타일 에셋 배치하기

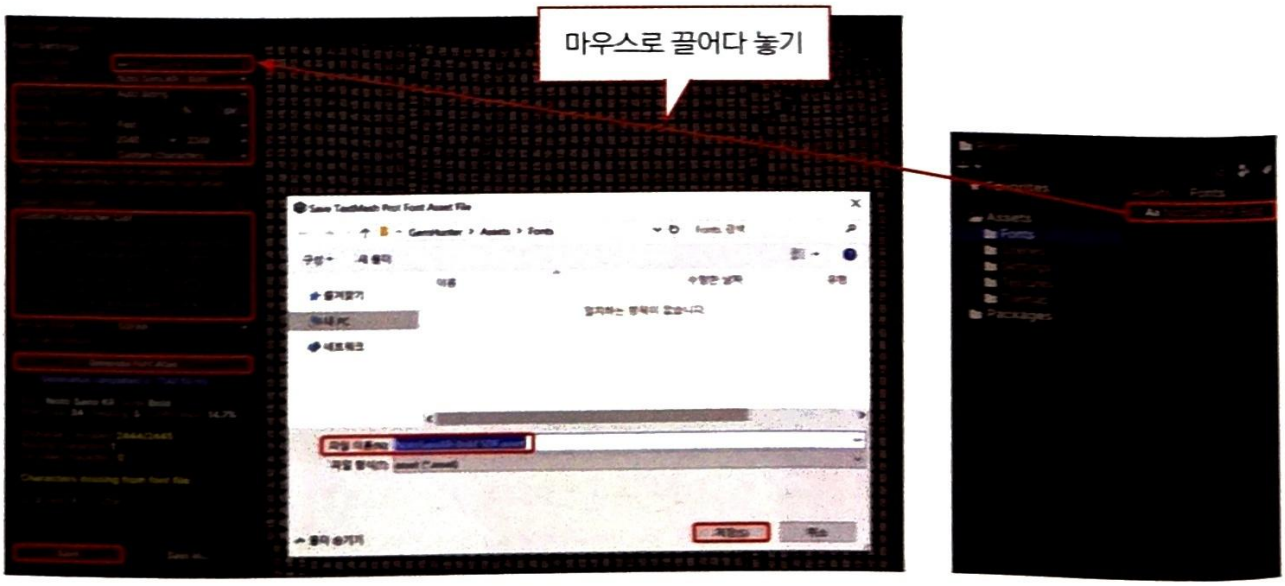
16 게임에서는 플레이어와 적 정보처럼 다양한 텍스트를 출력하는데, 이때 사용할 폰트는 별도로 설정합니다. 이 책에서는 구글에서 무료로 제공하는 Noto Sans 폰트를 사용합니다. 구글에서 'Noto Sans 한글'로 검색해 폰트 파일을 내려받고 압축을 푼 뒤 static 폴더의 NotoSansKR-Bold.ttf를 Project View의 Assets 〉 Fonts 폴더에 마우스로 끌어다 놓습니다.



게임에 사용할 NotoSansKR-Bold.ttf 파일 불러오기

17 ttf 파일은 유니티에서 바로 사용할 수 없으므로 폰트 에셋Font Asset을 생성해야 합니다. [Window → TextMeshPro → Font Asset Creator] 메뉴로 Font Asset Creator View를 열고 Source Font에 NotoSansKR-Bold.ttf 파일을 마우스로 끌어다 놓아 등록합니다. Sampling Point Size는 [Auto Sizing], Padding은 '5px', Atlas Resolution은 [2048, 2048]로 설정하고 Character Set를 [Custom Characters]로 변경한 후 Custom Character List에 'TextMeshPro용 커스텀 한글 2350자.txt' 파일 내용을 복사해 붙여 넣습니다. 〈Generate Font Atlas〉 버튼을 클릭해 폰트 아틀라스를 생성하고 〈Save〉 버튼을 클릭해 폰트 에셋을 프로젝트에 저장합니다.

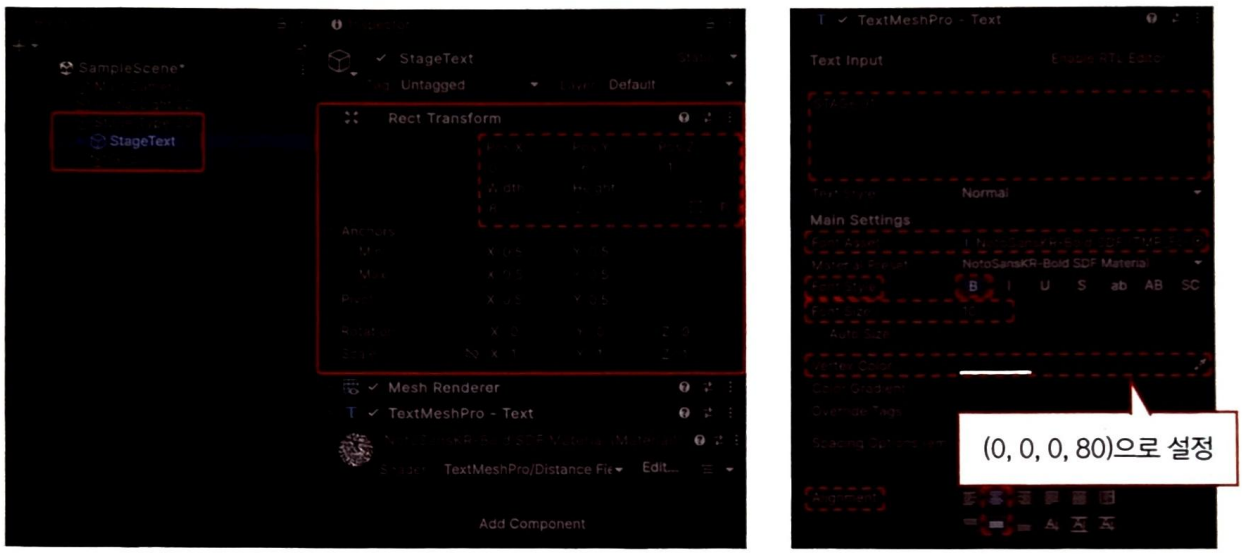
● 폰트 아틀라스(Font Atlas)란 여러 개의 글자를 하나의 큰 텍스처 이미지로 모은 것입니다.



Font Asset Creator View에서 폰트 에셋 생성하기

18 맵 바닥에 스테이지 번호를 출력하고자 [GameObject → UI → Text - TextMeshPro] 메뉴로 Text UI를 생성하고 이름을 'StageText'로 변경한 후 Stage_Type_00 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 X, Y, Z는 (0, -6, -1)로, Width와 Height는 (8, 2)로 설정합니다. 아직 스테이지를 클리어하고 넘어가는 기능이 없으므로 TextMeshPro - Text 컴포넌트의 Text Input Box에 'STAGE 01'을 입력하고 Font Asset은 앞서 만든 NotoSansKR-Bold SDF로 설정합니다. Font Style은 굵게를 뜻하는 [B]로, Font Size는 '10'으로, Vertex Color는 (0, 0, 0, 80)으로 설정합니다. Alignment는 Center | Middle로 설정합니다.

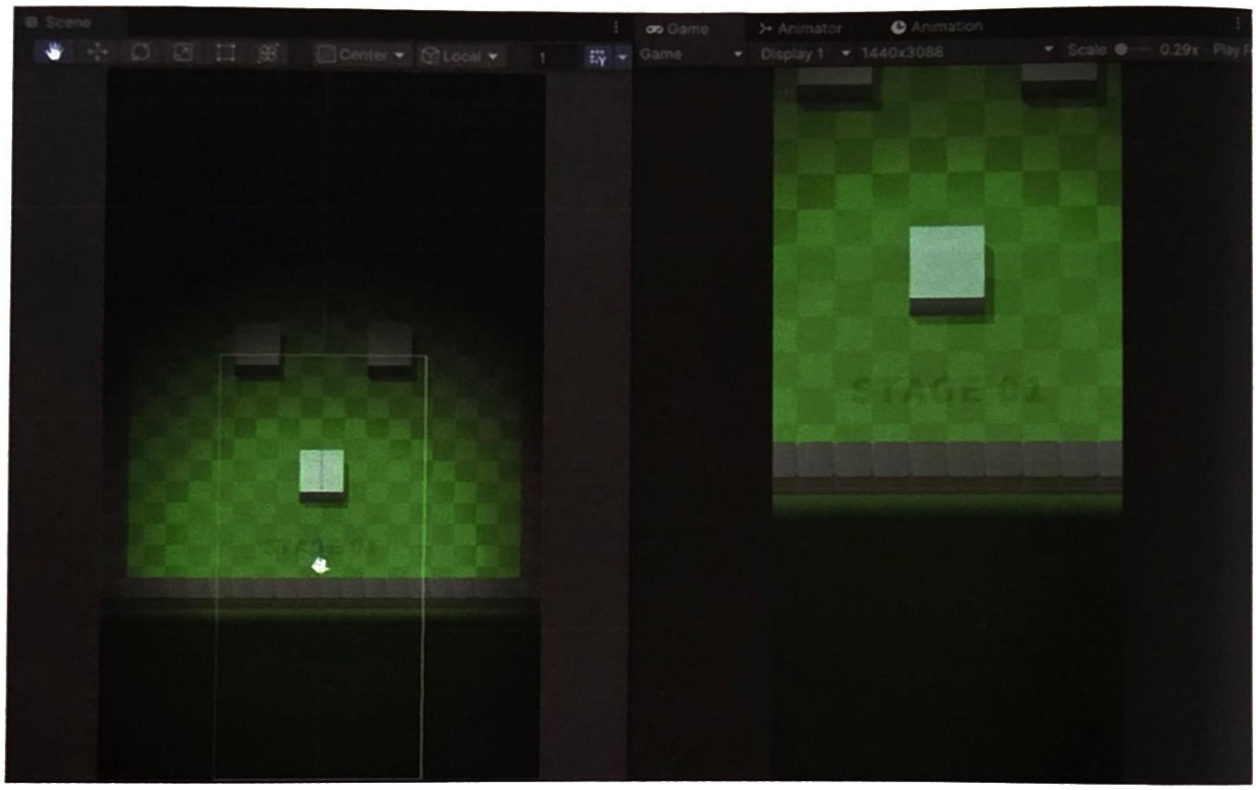


스테이지 번호를 출력하는 Text UI 생성하고 설정하기

19 Project View에서 Assets 〉 Prefabs 〉 Stages 폴더를 만들고 완성한 타일맵 오브젝트 Stage_Type_00을 이곳으로 끌어서 놓아 프리팹을 생성합니다.

Stage_Type_00 오브젝트의 프리팹 생성하기

20 지금까지 배경, 낭떠러지, 벽, 외곽 벽, 그림자 타일맵을 이용해서 만든 Stage_Type_00은 Scene View 또는 Game View에서 확인할 수 있습니다. 맵 정보를 Stage_Type_00 프리팹으로 저장했으므로 프리팹을 복제한 후 낭떠러지와 벽의 위치, 맵의 크기 등을 변경해서 다양한 형태의 맵을 추가로 만들 수 있습니다.



Stage_Type_00 결과 화면

03-2

플레이어 오브젝트 만들고 물리 기반 이동 구현하기

이 절에서는 플레이어 오브젝트를 생성하고 이동을 제어하는 방법을 배웁니다. 이때 플레이어가 제자리에서 대기할 때와 이동할 때 각각 다른 애니메이션을 재생합니다. 좌우 방향에 따라 오브젝트를 반전하고 플레이어가 이동할 때마다 발끝에서 효과를 재생하면서 카메라와 조명 오브젝트가 플레이어를 추적하도록 만들어 봅니다.
마지막으로, 플레이어가 벽 앞뒤에 있을 때를 구분하고자 y축을 기준으로 플레이어 Y 좌표가 벽 타일보다 낮으면 플레이어가 앞에 보이고 높으면 뒤에 보이도록 설정합니다.

Do it! 실습 플레이어 오브젝트 만들기

플레이어를 화면에 출력하는 오브젝트, 다양한 효과를 재생하는 오브젝트, 게임 월드와 충돌하는 오브젝트 등 다양한 오브젝트를 묶어 하나의 플레이어 오브젝트로 관리합니다. 그래서 플레이어뿐만 아니라 다른 오브젝트도 빈 오브젝트를 부모 오브젝트로 만들어 사용합니다. 이제 플레이어 오브젝트를 생성해 봅시다.

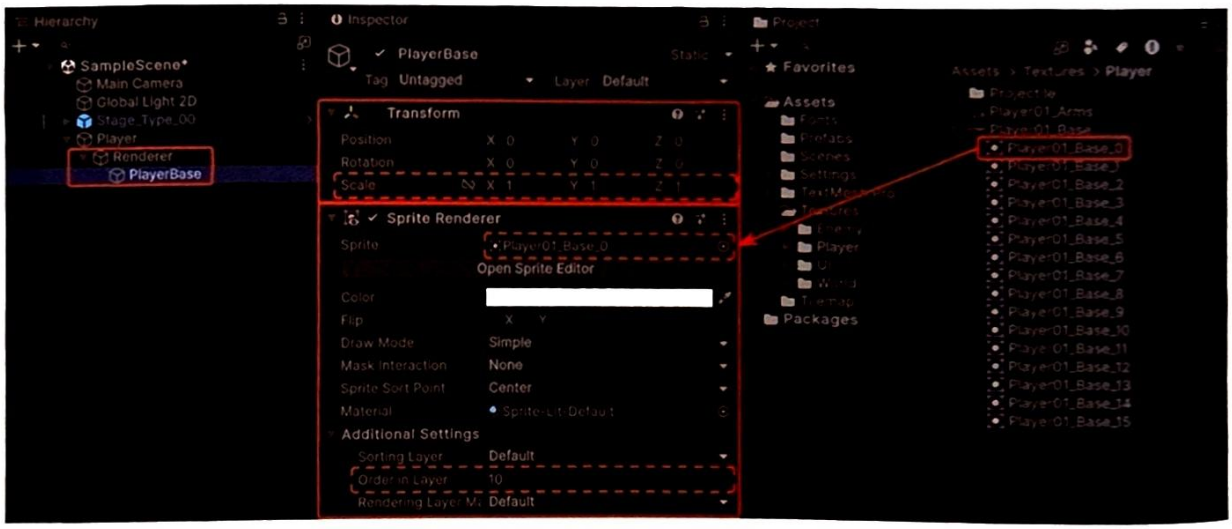
01 [GameObject → Create Empty] 메뉴로 플레이어 캐릭터 오브젝트를 관리하는 빈 오브젝트를 생성하고 이름을 'Player'로 변경한 후 Transform 컴포넌트의 Position을 (0, -7, 0)으로 설정합니다.

Player 오브젝트 생성하고 설정하기

02 같은 방법으로 화면에 플레이어를 출력하는 오브젝트를 관리할 빈 오브젝트를 생성하고 이름을 'Renderer'로 변경한 후 Player 오브젝트의 자식으로 배치합니다. 위치가 바뀌었다면 Position을 (0, 0, 0)으로 초기화하고 타일보다 좀 더 커 보이도록 Scale을 (1.5, 1.5, 1.5)로 설정합니다.

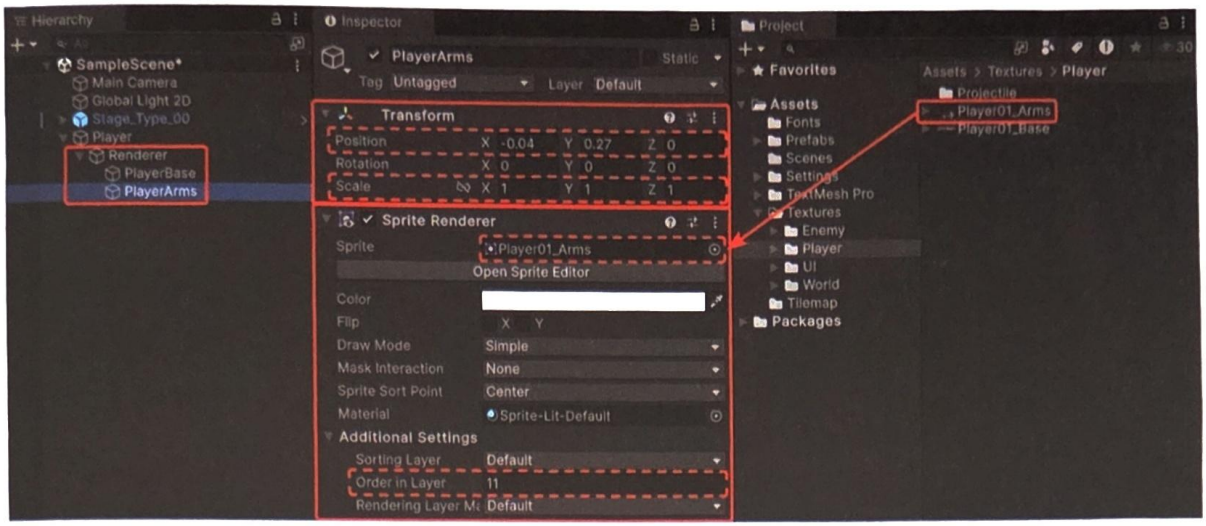
Renderer 오브젝트 생성하고 설정하기

03 [GameObject → 2D Object → Sprites → Square] 메뉴로 플레이어 캐릭터를 출력하는 2D Square 오브젝트를 생성하고 이름을 'PlayerBase'로 변경한 후 Renderer 오브젝트의 자식으로 배치합니다. Sprite Renderer 컴포넌트의 Sprite를 Player01_Base_0으로 설정하고 Order in Layer를 '10'으로 설정합니다.



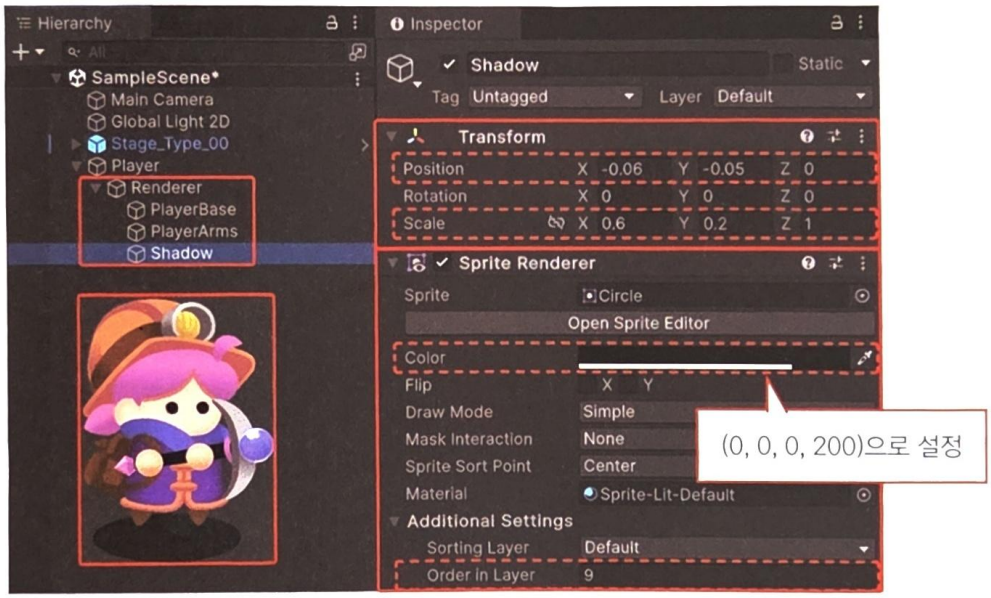
PlayerBase 오브젝트 생성하고 설정하기

04 [GameObject → 2D Object → Sprites → Square] 메뉴로 플레이어 캐릭터의 무기를 출력하는 2D Square 오브젝트를 생성하고 이름을 'PlayerArms'로 변경한 후 Renderer 오브젝트의 자식으로 배치합니다. PlayerArms 오브젝트 Transform 컴포넌트의 Position을 (-0.04, 0.27, 0)으로, Scale을 (1, 1, 1)로 설정합니다. Sprite Renderer 컴포넌트의 Sprite를 Player01_Arms로 설정하고 Order in Layer를 '11'로 설정합니다.



PlayerArms 오브젝트 생성하고 설정하기

05 [GameObject → 2D Object → Sprites → Circle] 메뉴로 플레이어 캐릭터의 그림자를 출력하는 2D Circle 오브젝트를 생성하고 이름을 'Shadow'로 변경한 후 Renderer 오브젝트의 자식으로 배치합니다. Transform 컴포넌트의 Position을 (-0.06, -0.05, 0)으로, Scale을 (0.6, 0.2, 1)로 설정합니다. Sprite Renderer 컴포넌트의 Color를 (0, 0, 0, 200)으로, Order in Layer를 '9'로 설정합니다.



Shadow 오브젝트 생성하고 설정하기

Do it! 실습 플레이어 움직이기

이제 플레이어가 움직이도록 만들어 볼까요? 게임을 완성한 후에 모바일 빌드를 설정하므로 지금은 키보드의 방향키나 W, A, S, D 키로 플레이어 이동을 제어합니다.

01 먼저 Assets 〉 Scripts 〉 Common 폴더를 만든 다음, 이곳에서 마우스 오른쪽 버튼을 클릭하고 [Create → MonoBehaviour Script] 메뉴로 스크립트 파일을 만듭니다. 그리고 이름을 'MovementRigidbody2D'로 바꿉니다. 이 파일은 Rigidbody 2D 컴포넌트로 오브젝트 이동을 제어합니다.

Assets/Scripts/Common/MovementRigidbody2D.cs

코드 03-01 Rigidbody2D를 이용한 오브젝트 이동 제어

```
01: using UnityEngine;
02:
03: [RequireComponent(typeof(Rigidbody2D))]
04: public class MovementRigidbody2D : MonoBehaviour
05: {
06:     [SerializeField]
07:     private float moveSpeed;
08:     private Rigidbody2D rigid2D;
09:
10:     private void Awake()
11:     {
12:         rigid2D = GetComponent<Rigidbody2D>();
13:     }
14:
15:     public void MoveTo(Vector3 direction)
16:     {
17:         rigid2D.linearVelocity = direction * moveSpeed;
18:     }
19: }
```

03줄 클래스 정의 앞에 [RequireComponent()] 어트리뷰트를 선언하면 해당 클래스를 게임오브젝트에 컴포넌트로 등록할 때 어트리뷰트에 선언한 컴포넌트도 함께 등록됩니다. 즉, 현재 클래스 내부에서 Rigidbody 2D 컴포넌트를 이용하므로 Movement Rigidbody 2D 컴포넌트를 등록할 때 함께 등록하도록 설정합니다. 외부 클래스에서 호출하는 MoveTo()

메서드는 매개변수로 받은 방향 direction에 속도 moveSpeed를 곱해 오브젝트 속력 rigid2D.linearVelocity로 설정합니다.

어트리뷰트(Attribute)는 클래스, 메서드, 변수 등에 메타데이터를 추가해 컴파일러나 유니티 에디터가 해당 코드를 특정 방식으로 처리하도록 합니다.

02 다음으로 Assets 〉 Scripts 〉 Player 폴더에 플레이어의 입력을 받아 플레이어 오브젝트의 이동과 애니메이션 재생 등을 제어하는 PlayerController.cs 스크립트를 생성합니다.

Assets/Scripts/Player/PlayerController.cs

코드 03-02 플레이어 오브젝트 제어

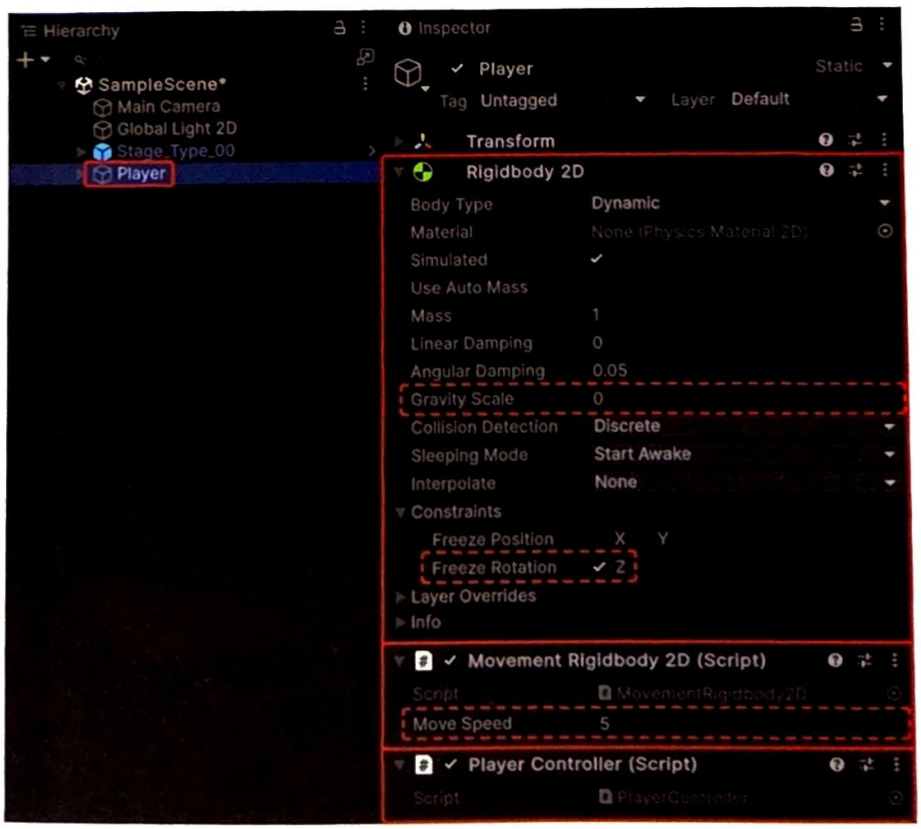
```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerController : MonoBehaviour
{
    private MovementRigidbody2D movement2D;
    private Vector2 moveInput = Vector2.zero;

    private void Awake()
    {
        movement2D = GetComponent<MovementRigidbody2D>();
    }

    private void Update()
    {
        // 플레이어 이동
        movement2D.MoveTo(moveInput);
    }

    public void OnMove(InputAction.CallbackContext context)
    {
        if ( context.performed || context.canceled )
        {
            moveInput = context.ReadValue<Vector2>();
        }
    }
}
```

20~26줄 키보드의 방향키 또는 W, A, S, D 키, 게임 패드의 왼쪽 조이스틱, 모바일 터치처럼 기기별로 키를 미리 설정하고 이 키를 눌렀을 때 **OnMove()** 메서드를 호출합니다. 키를 누르거나(performed) 뗄 때(canceled) 입력한 키 정보를 읽어 **moveInput** 변수에 저장합니다.

InputAction.CallbackContext 동작

속성	의미	사용 예
started	입력 시작됨	Hold 입력 체크, 애니메이션 트리거 등
performed	입력 동작이 수행됨	공격, 점프와 같은 단발성 동작
canceled	입력 취소됨(키를 뗄 때)	이동 중지 등

03 Player 오브젝트에 Player Controller와 Movement Rigidbody 2D 컴포넌트를 추가합니다. Movement Rigidbody 2D 컴포넌트의 **[RequireComponent()]** 어트리뷰트 덕분에 Rigidbody 2D 컴포넌트도 함께 추가됩니다. Rigidbody 2D 컴포넌트의 Gravity Scale을 '0'으로 설정하고 Constraints 섹션의 Freeze Rotation Z에 체크하여 true로 설정합니다. 그리고 Movement Rigidbody 2D 컴포넌트의 Move Speed를 '5'로 설정합니다.

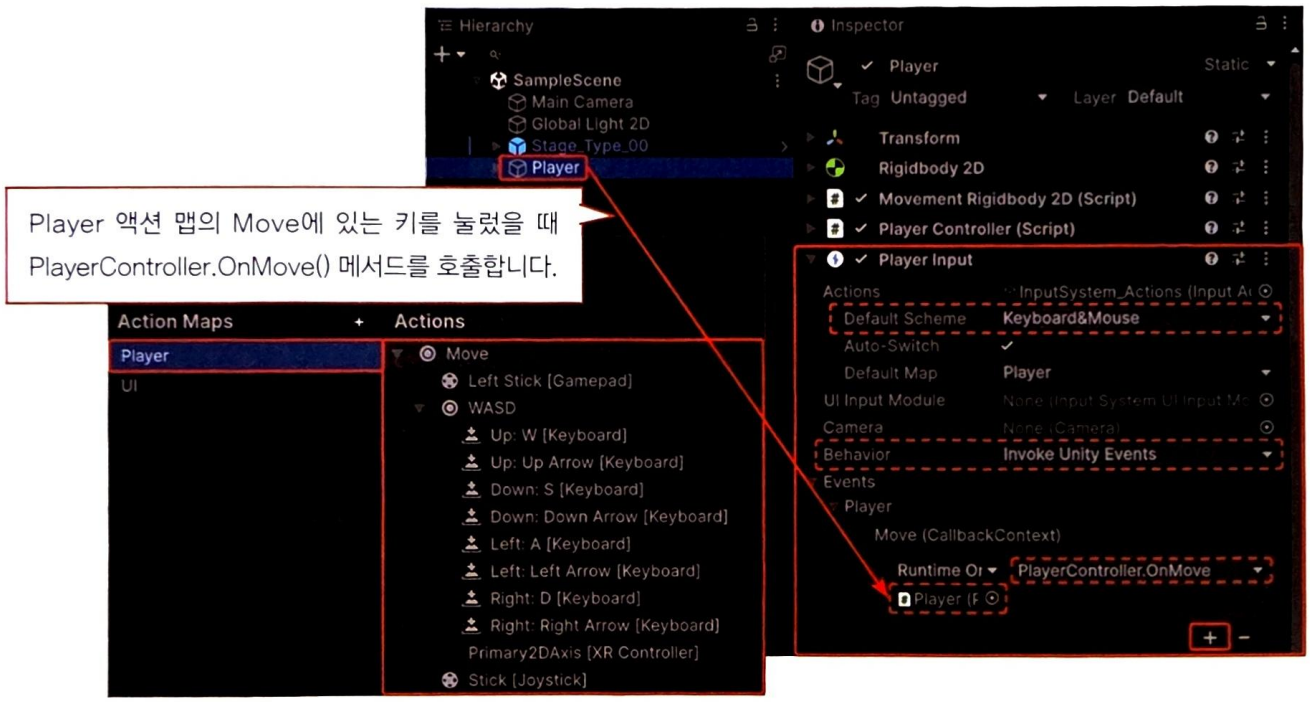


Player 오브젝트에 컴포넌트 추가하고 설정하기

04 Player 오브젝트에 Player Input 컴포넌트를 추가합니다. Player Input 컴포넌트는 InputSystem_Actions 파일에 등록한 [Action Maps → Actions]의 키를 눌렀을 때 호출할 메서드를 등록하고 호출하는 역할을 합니다.

Player Input 컴포넌트의 Default Scheme을 [Keyboard&Mouse]로, Behavior를 [Invoke Unity Events]로 설정합니다. Events 〉 Player 섹션의 Move() 이벤트에 Player 오브젝트를 추가하고 Player Controller 컴포넌트의 OnMove() 메서드를 등록합니다.

즉, InputSystem_Actions.inputactions 파일 [Action Maps → Player]의 [Actions → Move]에 있는 W, A, S, D 키 또는 방향키(키보드 기준)를 눌렀을 때 누른 키 정보를 매개변수로 `PlayerController.OnMove()` 메서드를 호출합니다.



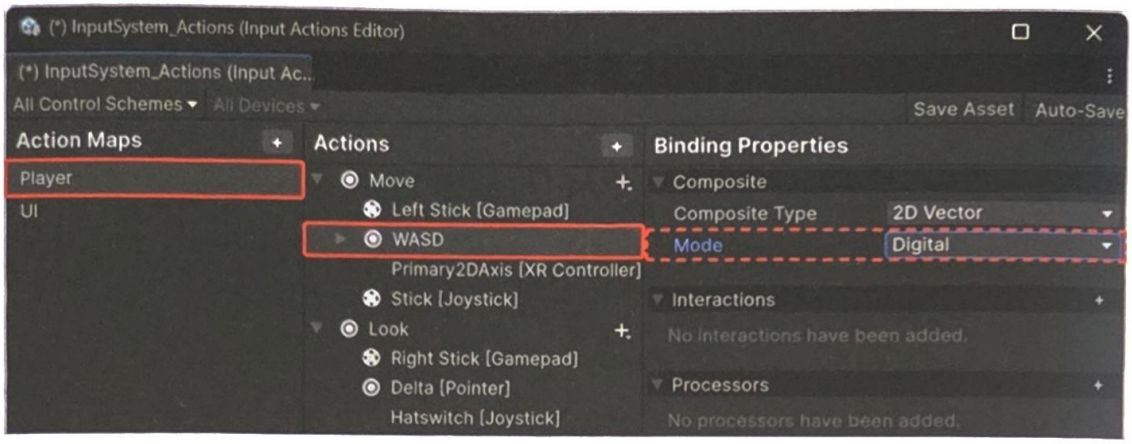
Player 오브젝트 Player Input 컴포넌트에 OnMove() 이벤트 등록하기

알아 두면 좋아요! InputAction을 이용한 키 입력에서는 Move 이동값을 정규화합니다

InputAction을 이용한 키 입력에서 [Actions → Move → WASD]는 정규화Normalized가 적용되므로 방향키 1개만 눌렀을 때는 (1, 0), (0, 1)과 같은 값을 반환하고 대각선으로 이동하고자 방향키 2개를 동시에 눌렀을 때는 이동 거리를 1로 정규화한 (0.71, 0.71)과 같은 값을 반환합니다.

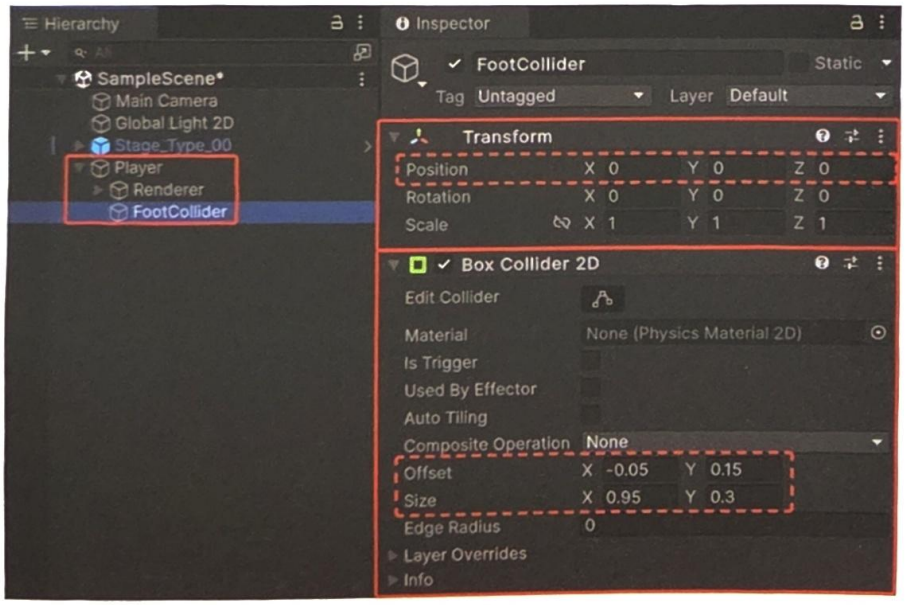
방향키 2개를 동시에 눌렀을 때 정규화를 사용하지 않고 (1, 1)과 같은 값을 반환하려면 WASD Action을 선택하고 Binding Properties의 Mode를 [Digital]로 변경합니다.

▶ 계속



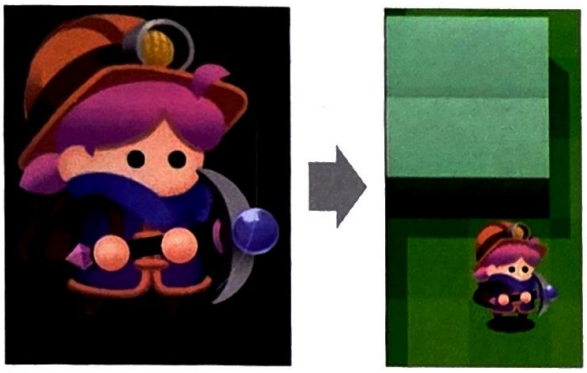
Input Actions 편집기

05 타일맵을 배치한 벽과 플레이어의 충돌을 처리하고자 [GameObject → Create Empty] 메뉴로 빈 오브젝트를 생성하고 이름을 'FootCollider'로 변경한 후 Player 오브젝트의 자식으로 배치합니다. Transform 컴포넌트의 Position을 (0, 0, 0)으로 초기화하고 Box Collider 2D 컴포넌트를 추가한 후 Offset은 (-0.05, 0.15)로, Size는 (0.95, 0.3)으로 설정합니다.

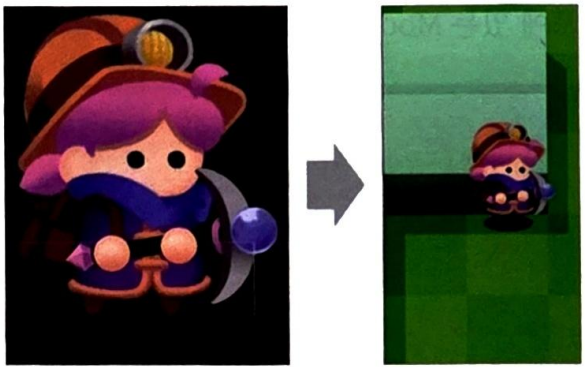


플레이어 충돌을 처리하고자 FootCollider 오브젝트 생성하고 설정하기

06 여기서는 플레이어 충돌을 발 주변에만 설정했습니다. 플레이어 전체에 적용하면 벽 옆에 선 듯한 입체감을 줄 수 없으므로 2D 탑다운 게임을 제작할 때는 이 책처럼 발 부분에만 충돌 범위를 설정하는 방식이 일반적입니다.



플레이어 이미지 전체를 충돌 처리했을 때



플레이어 이미지 발 부분만 충돌 처리했을 때

Do it! 실습 플레이어와 벽 타일의 Y 좌표 기준으로 렌더링 순서 설정하기

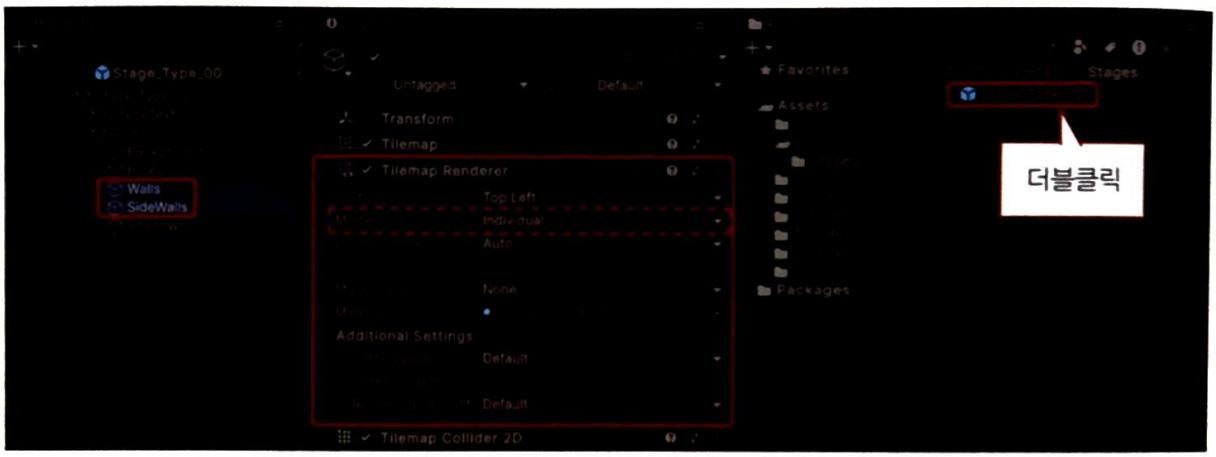
현재 플레이어와 벽 타일맵 오브젝트의 Order in Layer가 모두 10이므로 플레이어는 벽 타일보다 항상 뒤에 보입니다. 탑다운 게임에서 입체적으로 보이려면 플레이어가 벽 아래에 서 있을 때는 앞에 보이고 벽 위에 서 있을 때는 뒤에 보이도록 해야 합니다.

코드를 이용해 Y값에 따라 Order in Layer를 실시간으로 변경할 수도 있지만, 유니티의 Universal 2D에서는 코드 작성 없이 조건에 따라 Order in Layer가 같은 오브젝트의 렌더링 순서를 설정할 수 있습니다.

01 Renderer2D 에셋의 Transparency Sort Mode는 [Custom Axis]로, Transparency Sort Axis는 (0, 1, 0)으로 설정합니다.

Renderer2D 에셋의 Transparency Sort Mode 옵션 설정하기

02 이렇게 설정하더라도 타일맵 오브젝트는 오브젝트에 배치한 타일을 하나의 그룹으로 관리하므로 정상적으로 정렬되지 않습니다. Stage_Type_00 프리팹을 더블클릭해서 활성화하고 벽 타일을 출력하는 Walls와 SideWalls 타일맵 오브젝트의 Tilemap Renderer 컴포넌트에 있는 Mode를 [Individual]로 설정합니다.



Walls, SideWalls 오브젝트에서 Tilemap Renderer 컴포넌트의 Mode 설정하기

03 플레이 버튼 ▶ 을 눌러 게임을 실행하면 이전과 다르게 플레이어 Y 위치가 벽 아래에 있을 때는 플레이어가 앞에 보이고 벽 위에 있을 때는 뒤에 보입니다.

Y 좌표 기준으로 렌더 순서를 설정하기 전

Y 좌표 기준으로 렌더 순서를 설정한 후

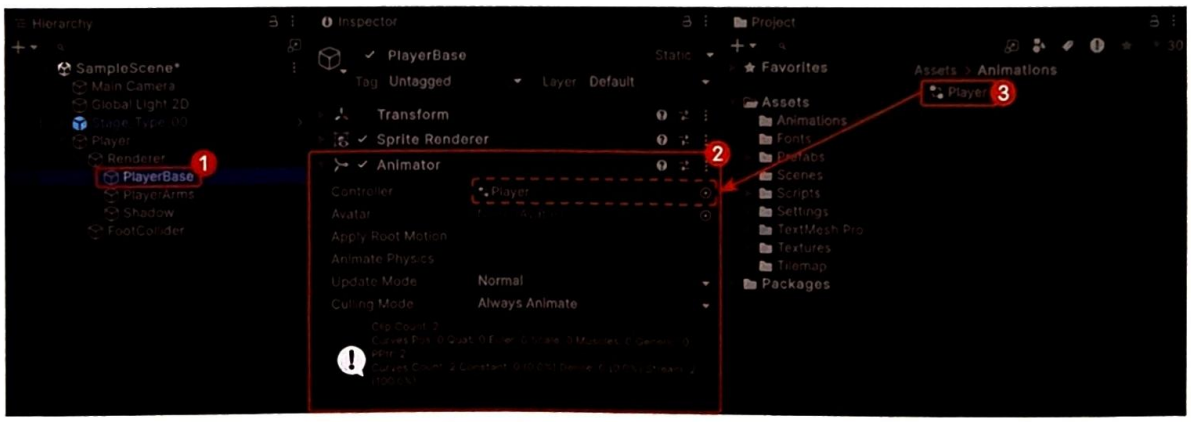
Do it! 실습 플레이어 애니메이션 만들기

플레이어 오브젝트가 제자리에 가만히 있거나 이동할 때 애니메이션을 재생하고 플레이어가 이동할 때 발 위치에 먼지가 생성되는 것처럼 보이도록 플레이어 이동 효과를 만듭니다. 이와 함께 플레이어가 이동할 때는 항상 화면 가운데에 보이도록 카메라와 조명이 쫓아와야 합니다.

01 먼저 Assets 〉 Animations 폴더를 생성합니다. 그리고 이 폴더에서 Project View의 ＋▾ 버튼을 클릭해 [Animation → Animator Controller] 메뉴로 플레이어의 애니메이션을 제어하는 애니메이터 컨트롤러를 만들고 이름을 'Player'로 변경합니다.

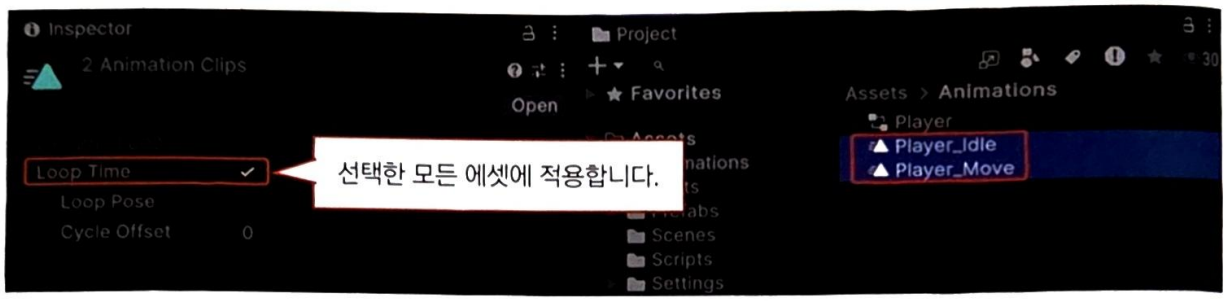
애니메이터 컨트롤러 만들기

02 플레이어 캐릭터를 화면에 출력하는 ❶ PlayerBase 오브젝트에 ❷ Animator 컴포넌트를 추가하고 ❸ Controller 변수에 Player 컨트롤러를 등록합니다.



플레이어의 애니메이션 재생을 제어하고자 Animator 컴포넌트 추가하고 설정하기

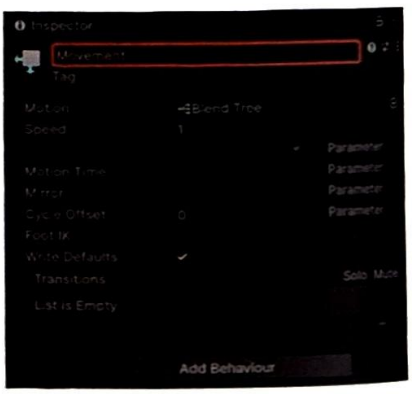
03 Project View의 ＋▾ 버튼을 클릭해 [Animation → Animation Clip] 메뉴로 플레이어 대기, 이동 애니메이션을 Assets 〉 Animations 폴더 안에 생성하고 이름을 'Player_Idle', 'Player_Move'로 변경한 후 Loop Time을 true로 설정합니다.



애니메이션 클립 에셋 생성하고 반복 설정하기

04 PlayerBase 오브젝트를 선택하고 [Window → Animation → Animator] 메뉴로 Animator View를 엽니다. 빈 곳을 마우스 오른쪽 버튼으로 클릭하고 [Create State → From New Blend Tree] 메뉴로 블렌드 트리를 생성합니다. 생성한 블렌드 트리를 선택하고 Inspector View에서 이름을 'Movement'로 설정합니다.

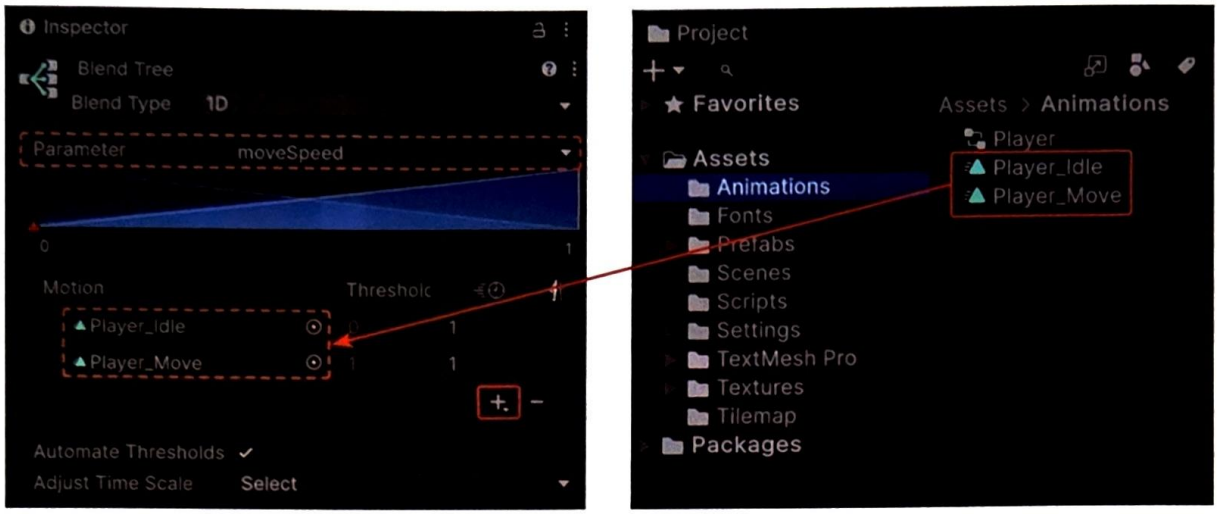
블렌드 트리와 함께 생성되는 Parameters의 이름을 'moveSpeed'로 변경합니다. Movement를 더블클릭해서 트리 안으로 이동합니다.



플레이어의 대기, 이동 애니메이션 재생을 제어하는 Movement 블렌드 트리 생성하기

05 Blend Tree를 선택하고 Inspector View에서 + 버튼을 클릭해 [Add Motion Field] 메뉴로 2개의 모션을 생성하고 Player_Idle과 Player_Move 애니메이션을 순서대로 등록합니다.

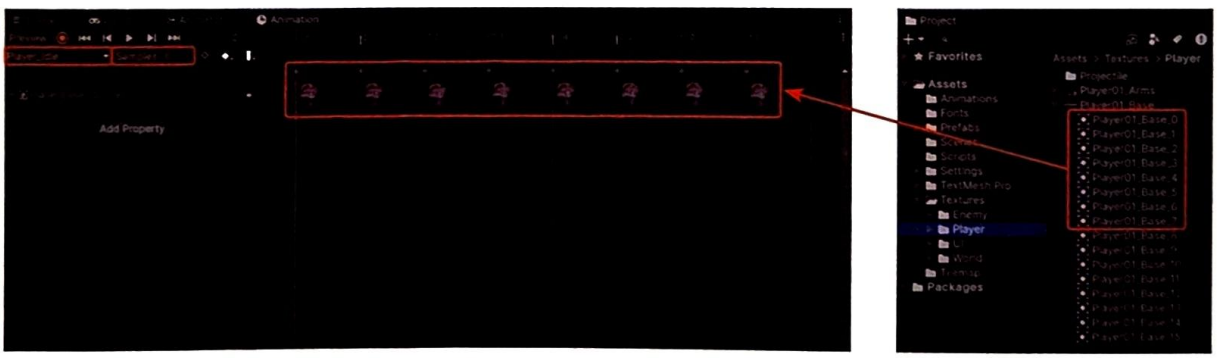
Blend Tree 선택하기



Inspector View에서 Player_Idle과 Player_Move 모션 등록하기

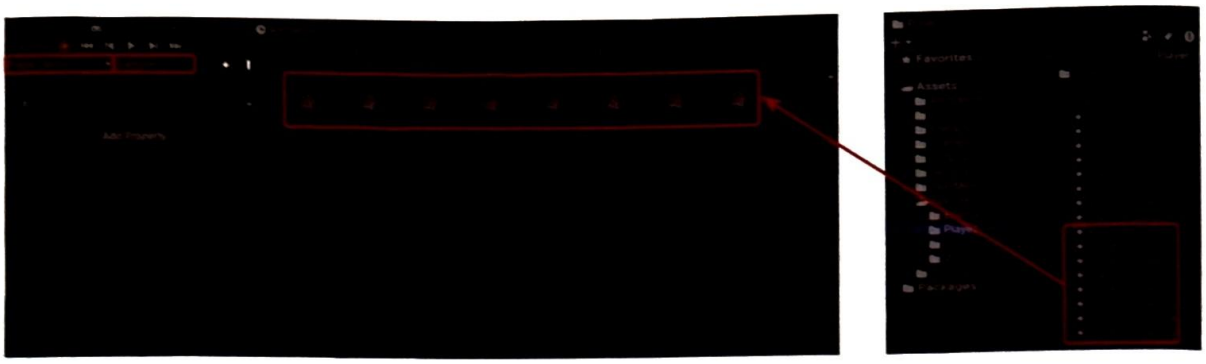
06 플레이어 대기, 이동 애니메이션을 설정하고자 PlayerBase 오브젝트를 선택하고 [Window → Animation → Animation] 메뉴 또는 단축키 Ctrl+6으로 Animation View를 엽니다. 애니메이션 클립의 드롭다운 메뉴에서 [Player_Idle]을 선택하고 Samples를 '6'으로 설정한 후 Player01_Base_0부터 Player01_Base_7까지의 이미지를 드래그해서 대기 동작 애니메이션에 등록합니다.

Samples 입력란이 보이지 않으면 타임라인 오른쪽에 있는 점 3개 추가 옵션 메뉴를 클릭하고 [Show Sample Rate] 메뉴를 선택하세요.



플레이어 대기 애니메이션인 Player_Idle 설정하기

07 애니메이션 클립의 드롭다운 메뉴에서 [Player_Move]를 선택하고 Samples를 '6'으로 설정한 후 Player01_Base_8부터 Player01_Base_15까지의 이미지를 드래그해서 이동 동작 애니메이션에 등록합니다.



플레이어 이동 애니메이션인 Player_Move 설정하기

08 Assets 〉 Scripts 〉 Player 폴더에 플레이어의 애니메이션 재생을 제어하는 PlayerRenderer.cs 스크립트를 생성합니다.

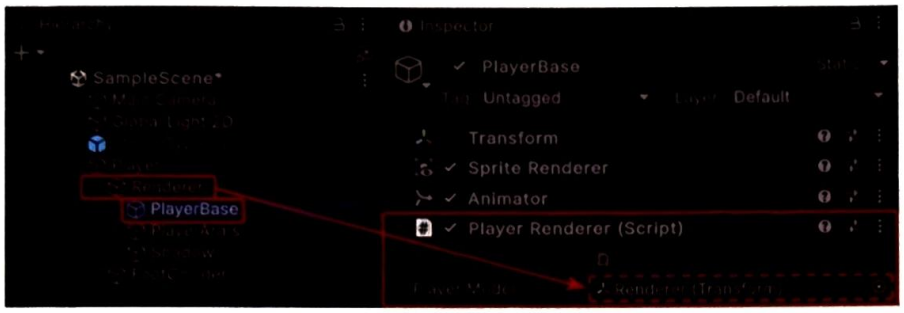
Assets/Scripts/Player/PlayerRenderer.cs

코드 03-03 플레이어 좌우 반전과 애니메이션 재생 제어

```
01: using UnityEngine;
02:
03: public class PlayerRenderer : MonoBehaviour
04: {
05:     [SerializeField]
06:     private Transform playerModel;  // 좌우 반전을 위한 플레이어 Transform
07:     private Animator animator;
08:
09:     private void Awake()
10:     {
11:         animator = GetComponent<Animator>();
12:     }
13:
14:     public void OnMovement(float speed)
15:     {
16:         animator.SetFloat("moveSpeed", speed);
17:     }
18:
19:     // SpriteRenderer 컴포넌트의 Flip을 이용해 이미지를 반전하면
20:     // 화면에 출력하는 이미지 자체만 반전하므로
21:     // 플레이어의 전방 특정 위치에서 발사체를 생성할 때처럼
22:     // 방향을 전환해야 할 때는 Transform.Scale.x를 -1, 1과 같이 설정
23:     public void SpriteFlipX(float x)
24:     {
25:         Vector3 currentScale = playerModel.localScale;
26:         currentScale.x = x < 0 ? -1.5f : 1.5f;
```

```
27:         playerModel.localScale = currentScale;
28:     }
29: }
```

09 작성이 끝난 PlayerRenderer.cs 스크립트를 PlayerBase 오브젝트에 컴포넌트로 추가하고 Player Model 변수에 Renderer 오브젝트를 등록합니다.



PlayerBase 오브젝트에 Player Renderer 컴포넌트 추가하고 설정하기

10 이제 플레이어의 이동 여부에 따라 각각 대기와 이동 애니메이션을 재생하고 좌우 이동 방향에 따라 이미지를 반전하도록 PlayerController.cs 스크립트를 수정합니다.

● 편집기에서 저장하고 유니티 에디터를 활성화하면 스크립트를 반영합니다.

Assets/Scripts/Player/PlayerController.cs

코드 03-04 플레이어 애니메이션 변경과 좌우 이미지 반전 제어

```
    (... 생략 ...)
04: public class PlayerController : MonoBehaviour
05: {
06:     private MovementRigidbody2D movement2D;
07:     private PlayerRenderer playerRenderer;          ← 추가
08:     private Vector2 moveInput = Vector2.zero;
09:
10:     private void Awake()
11:     {
12:         movement2D = GetComponent<MovementRigidbody2D>();
13:         playerRenderer = GetComponentInChildren<PlayerRenderer>();   ← 추가
14:     }
15:
16:     private void Update()
17:     {
18:         // 플레이어 이동 여부 검사                  ← 추가
```

```
19:         bool isMoved = moveInput.x != 0 || moveInput.y != 0;
20:         // 플레이어 좌우 반전
21:         if ( moveInput.x != 0 ) playerRenderer.SpriteFlipX(moveInput.x);
22:         // 플레이어 애니메이션 재생
23:         playerRenderer.OnMovement(isMoved ? 1 : 0);
24:         // 플레이어 이동
25:         movement2D.MoveTo(moveInput);
26:     }
27:
        (... 생략 ...)
35: }
```

18~19줄 방향키를 눌렀는지 검사하고 방향키를 누르면 isMoved는 true로, 누르지 않으면 false로 설정합니다.

20~21줄 x축 방향키를 누르고 있을 때 moveInput.x가 음수이면 플레이어는 왼쪽을 바라보고 양수이면 오른쪽을 바라보도록 설정합니다.

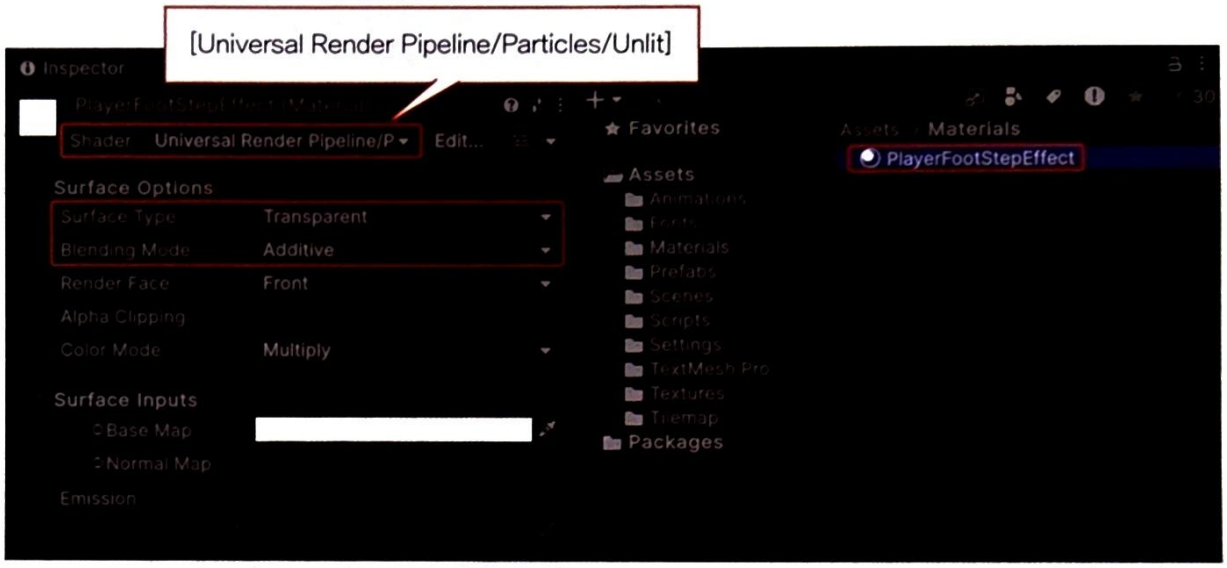
22~23줄 isMoved가 true이면 OnMovement() 메서드의 매개변수로 1을 전달해 이동 애니메이션을 재생하고 false이면 매개변수로 0을 전달해 대기 애니메이션을 재생합니다.

11 플레이 버튼 ▶을 눌러 게임을 실행하면 플레이어가 제자리에 가만히 있을 때는 대기 애니메이션(Player01_Base_0~Player01_Base_7 이미지)을 재생하고 이동할 때는 이동 애니메이션(Player01_Base_8~Player01_Base_15 이미지)을 재생합니다. 또한 왼쪽으로 이동할 때는 플레이어 캐릭터가 왼쪽을 바라보고 오른쪽으로 이동할 때는 오른쪽을 바라봅니다.

플레이어 대기 애니메이션(Player_Idle, 위)과 이동 애니메이션(Player_Move, 아래)

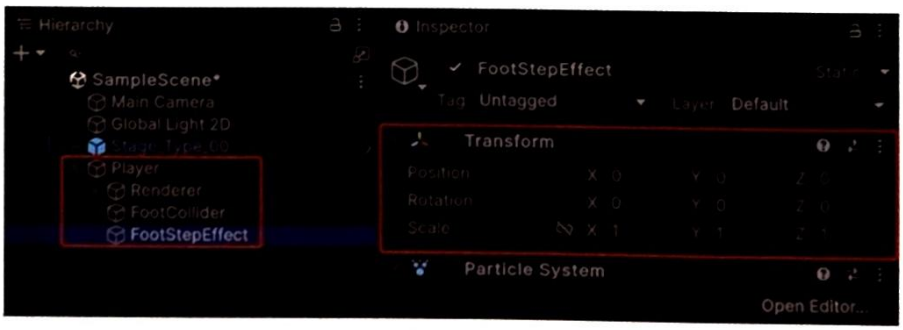
Do it! 실습 플레이어 움직임에 효과 추가하기

01 Assets 〉 Materials 폴더를 만들고 나서 Project View의 + ▾ 버튼을 클릭해 [Material] 메뉴로 플레이어 이동 효과에 적용할 재질을 이곳에 생성하고 이름을 'PlayerFootStep Effect'로 변경합니다. 재질의 Shader는 [Universal Render Pipeline/Particles/Unlit]으로, Surface Type은 [Transparent]로, Blending Mode는 [Additive]로 설정합니다.



PlayerFootStepEffect 재질 생성하고 설정하기

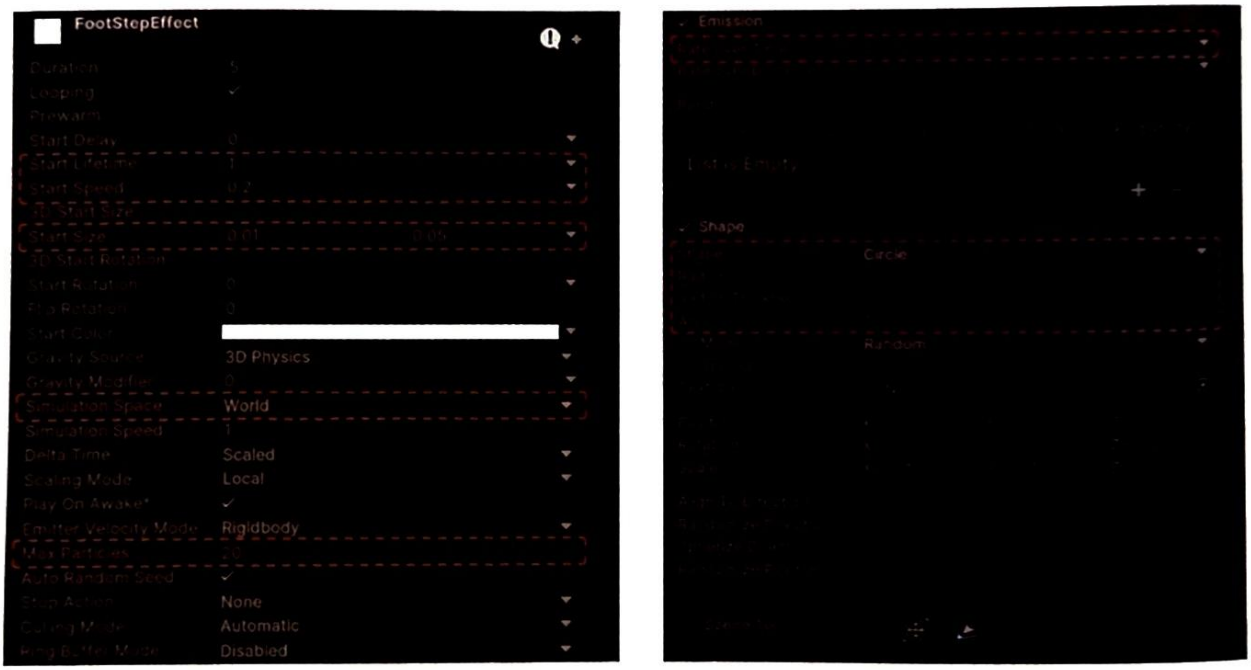
02 [GameObject → Effects → Particle System] 메뉴로 플레이어 이동 효과를 생성하고 이름을 'FootStepEffect'로 변경한 후 Player 오브젝트의 자식으로 배치합니다. Transform 컴포넌트의 Position과 Rotation을 (0, 0, 0)으로 초기화합니다.



FootStepEffect 오브젝트 생성하고 설정하기

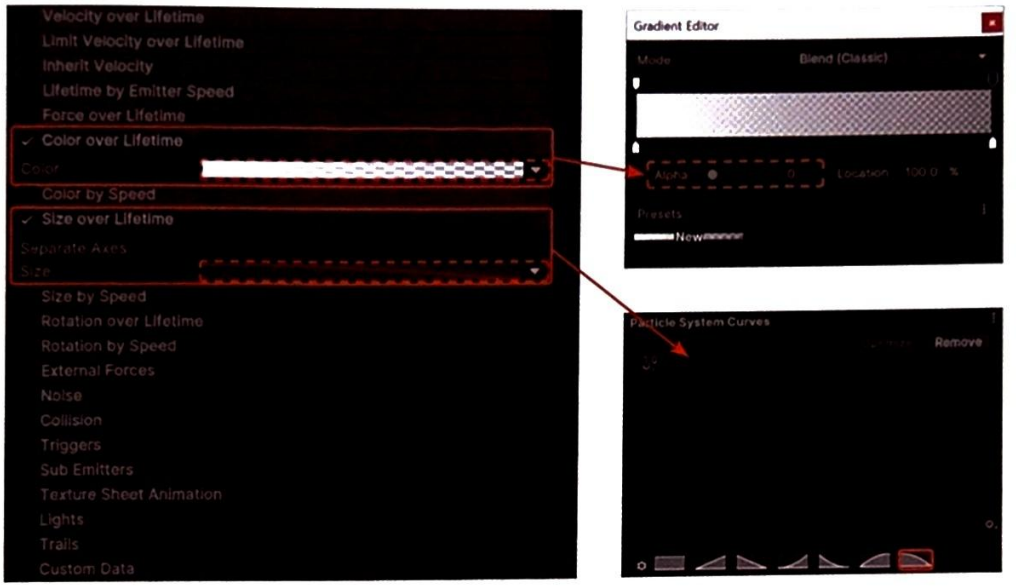
03 FootStepEffect 오브젝트의 Particle System 컴포넌트를 설정해 먼지 효과를 제작합니다. 먼저 Main 모듈에서 Start Lifetime은 '1'로, Start Speed는 '0.2'로 설정합니다. Start Size는 2개의 값을 설정하고자 Start Size의 드롭다운 버튼 ▼을 클릭해서 Random Between Two Constants를 선택하고 (0.01, 0.05)를 입력합니다. Simulation Space를 [World]로 설정해서 이미 생성된 효과의 위치가 부모인 Player 오브젝트를 쫓아오지 않도록 하고, Max Particles를 '20'으로 설정합니다.

Emission 모듈에서는 Rate over Time을 '20'으로 설정합니다. Shape 모듈에서는 Shape를 [Circle]로, Radius를 '0.0001'로, Arc를 '180'으로 설정합니다.



FootStepEffect 오브젝트의 Particle System 컴포넌트(Main 모듈, Emission 모듈, Shape 모듈)

04 시간 경과에 따라 생성한 파티클의 색상을 설정할 수 있는 Color over Lifetime 모듈을 활성화하고 Color를 선택합니다. Gradient Editor 창에서 100% 위치에 있는 Alpha를 '0'으로 설정합니다. 그리고 시간 경과에 따라 생성한 파티클의 크기를 설정할 수 있는 Size over Lifetime 모듈을 활성화하고 Size 그래프를 선택해 시간이 갈수록 작아지도록 곡선 그래프를 선택합니다.



Color over Lifetime, Size over Lifetime 모듈 설정하기

05 Renderer 모듈에서 Material을 PlayerFootStepEffect 재질로, Order in Layer를 '9'로 설정해서 플레이어보다 뒤에 그려지고 배경, 낭떠러지, 그림자 타일보다 앞에 그려지도록 합니다.

Renderer 모듈 설정하기

06 플레이어가 대기하거나 이동하는 상태에 따라 FootStepEffect의 파티클 개수를 설정하고자 PlayerRenderer.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerRenderer.cs

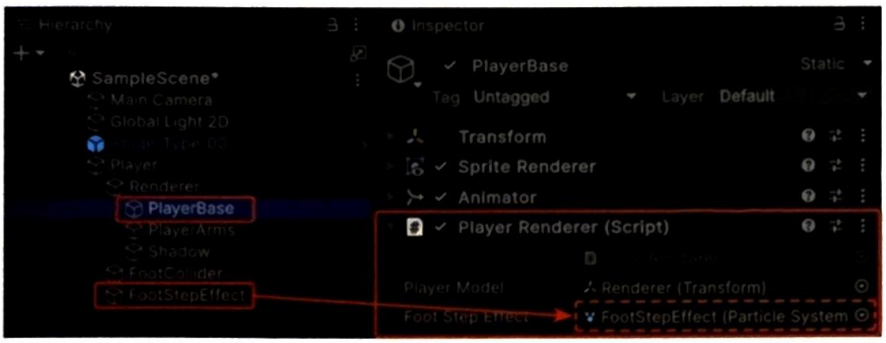
코드 03-05 플레이어 좌우 반전과 애니메이션 재생 제어

```
01: using UnityEngine;
02:
03: public class PlayerRenderer : MonoBehaviour
04: {
05:     [SerializeField]
06:     private Transform playerModel;
07:     [SerializeField]                                    추가
08:     private ParticleSystem footStepEffect;
09:     private ParticleSystem.EmissionModule footEmission;
10:     private Animator animator;
11:
12:     private void Awake()
13:     {
14:         footEmission = footStepEffect.emission;         추가
15:         animator = GetComponent<Animator>();
16:     }
        (... 생략 ...)
22:
23:     public void OnFootStepEffect(bool isMoved)          추가
24:     {
25:         footEmission.rateOverTime = isMoved == true ? 20 : 0;
26:     }
27:
28:     // SpriteRenderer 컴포넌트의 Flip으로 이미지를 반전하면
        (... 생략 ...)
38: }
```

23~26줄 매개변수로 플레이어의 이동 여부인 isMoved값을 받아서 플레이어가 이동 중이면 rateOverTime을 20으로 설정해 먼지 효과를 재생하고 제자리에 서 있으면 0으로 설정해 효과를 재생하지 않습니다.

다른 모듈과 마찬가지로 ParticleSystem.emission도 구조체(자료형)이므로 직접 수정하면 변경값이 적용되지 않습니다. 그 대신 변수를 따로 만들어 복사본의 값을 수정하면 유니티 안에서 원본 모듈을 간접적으로 수정하는 메커니즘을 제공합니다.

07 PlayerBase 오브젝트의 Player Renderer 컴포넌트에 있는 Foot Step Effect 변수에 FootStepEffect 오브젝트를 등록합니다.



PlayerBase 오브젝트의 Player Renderer 컴포넌트 변수 설정하기

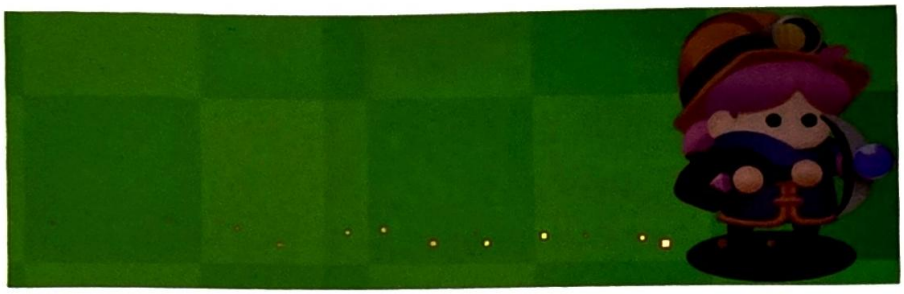
08 플레이어의 대기와 이동에 따라 이동 효과를 제어하는 `OnFootStepEffect()` 메서드를 호출하고자 PlayerController.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerController.cs

코드 03-06 이동 효과를 제어하는 OnFootStepEffect() 메서드 호출

```
    (... 생략 ...)
04: public class PlayerController : MonoBehaviour
05: {
        (... 생략 ...)
16:     private void Update()
17:     {
18:         // 플레이어 이동 여부 검사
19:         bool isMoved = moveInput.x != 0 || moveInput.y != 0;
20:         // 플레이어 좌우 반전
21:         if ( moveInput.x != 0 ) playerRenderer.SpriteFlipX(moveInput.x);
22:         // 플레이어 애니메이션 재생
23:         playerRenderer.OnMovement(isMoved ? 1 : 0);
24:         // 먼지 효과 재생                                    추가
25:         playerRenderer.OnFootStepEffect(isMoved);
26:         // 플레이어 이동
27:         movement2D.MoveTo(moveInput);
28:     }
        (... 생략 ...)
37: }
```

09 플레이 버튼 ▶을 눌러 게임을 실행하면 플레이어가 이동할 때 이동 효과가 재생됩니다.



FootStepEffect 결과 화면

Do it! 실습 플레이어를 추적하는 카메라와 조명 설정하기

01 대상을 추적하는 행위는 카메라 외에 다른 오브젝트도 할 수 있으므로 Assets 〉 Scripts 〉 Common 폴더에 target 오브젝트를 추적하는 FollowTarget.cs 스크립트를 생성합니다.

Assets/Scripts/Common/FollowTarget.cs

코드 03-07 목표(target) 위치를 추적하는 오브젝트

```
01: using UnityEngine;
02:
03: public class FollowTarget : MonoBehaviour
04: {
05:     [SerializeField]
06:     private Transform target;
07:     [SerializeField]
08:     private bool x, y, z;
09:
10:     private void Update()
11:     {
12:         if ( target == null ) return;
13:
14:         // 활성화 축은 target 위치, 비활성화 축은 자기 자신 위치로 설정
15:         transform.position = new Vector3(
16:             (x ? target.position.x : transform.position.x),
17:             (y ? target.position.y : transform.position.y),
18:             (z ? target.position.z : transform.position.z));
19:     }
20: }
```

10~19줄 오브젝트가 추적할 대상은 플레이어, 적, 카메라, 조명 등 다양한 월드 오브젝트가 될 수 있습니다. 그러므로 추적할 대상인 target을 설정하고 추적할 축인 x, y, z를 true 또는 false로 설정해서 원하는 축만 추적하도록 할 수 있습니다. Update() 메서드에서 x, y, z가 true이면 목표 위치인 target.position을, false이면 자기 위치인 transform.position을 사용합니다.

02 모든 스테이지가 같은 모양이라면 상관없지만, 스테이지마다 구조와 크기가 다르면 스테이지에 따라 이동할 수 있는 범위, 플레이어, 카메라의 시작 지점 등의 데이터를 저장해야 합니다.

Assets 〉 Scripts 〉 Database 폴더에 스테이지 데이터를 별도로 저장할 StageData.cs 스크립트를 생성합니다.

Assets/Scripts/Database/StageData.cs

코드 03-08 스테이지 데이터를 저장

```
01: using UnityEngine;
02:
03: [CreateAssetMenu]
04: public class StageData : ScriptableObject
05: {
06:     [SerializeField]
07:     private Vector2 cameraLimitMin;
08:     [SerializeField]
09:     private Vector2 cameraLimitMax;
10:
11:     [SerializeField]
12:     private Vector3 cameraStartPoint;
13:     [SerializeField]
14:     private Vector2 playerStartPoint;
15:
16:     public Vector2 CameraLimitMin => cameraLimitMin;
17:     public Vector2 CameraLimitMax => cameraLimitMax;
18:     public Vector3 CameraStartPoint => cameraStartPoint;
19:     public Vector2 PlayerStartPoint => playerStartPoint;
20: }
```

유니티에서 제공하는 ScriptableObject 클래스를 상속받고 [CreateAssetMenu] 어트리뷰트를 사용하면 Project View에서 생성 메뉴를 이용해 해당 스크립트 내용에 해당하는 에셋 파일을 생성할 수 있습니다.

03 Assets 〉 Database 폴더를 만든 후, Project View의 ＋▾ 버튼을 클릭해서 [Stage Data] 메뉴로 방금 작성한 StageData형의 에셋 파일을 이곳에 생성하고 이름을 'Stage_Type_00'으로 변경합니다. Camera Limit Min은 (-5.34, -7), Camera Limit Max는 (5.34, 6.5), Camera Start Point는 (0, -7, -10), Player Start Point는 (0, -7)로 설정합니다.

StageData형 에셋 파일 생성하고 설정하기

04 플레이어를 추적하는 카메라가 스테이지의 한계 범위를 넘지 않도록 Assets 〉 Scripts 〉 Camera 폴더에 카메라 위치를 제어하는 CameraController.cs 스크립트를 생성합니다.

Assets/Scripts/Camera/CameraController.cs

코드 03-09 카메라 오브젝트 위치 제어

```
01: using UnityEngine;
02:
03: public class CameraController : MonoBehaviour
04: {
05:     [SerializeField]
06:     private StageData stageData;
07:
08:     private void LateUpdate()
09:     {
10:         Vector3 position = transform.position;
```

```
11:
12:         position.x = Mathf.Clamp(position.x, stageData.CameraLimitMin.x,
                stageData.CameraLimitMax.x);
13:         position.y = Mathf.Clamp(position.y, stageData.CameraLimitMin.y,
                stageData.CameraLimitMax.y);
14:
15:         transform.position = position;
16:     }
17: }
```

12~13줄 Mathf.Clamp() 메서드를 호출해 카메라의 x, y 위치가 CameraLimitMin값보다 작아지거나 CameraLimitMax값보다 커지지 않도록 설정합니다.

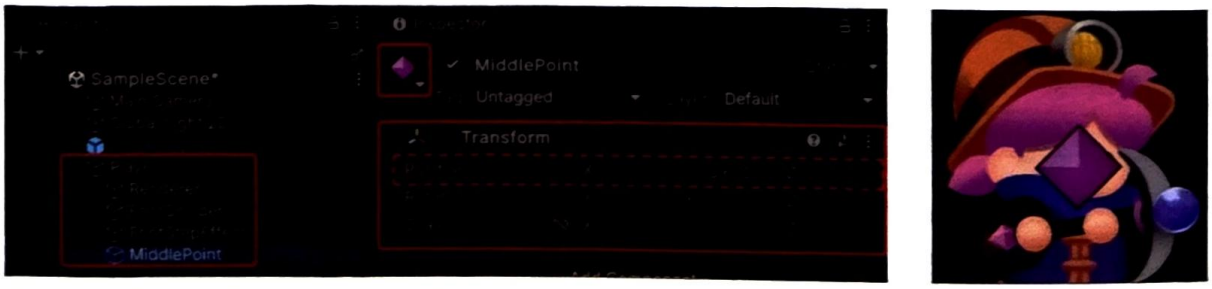
알아 두면 좋아요! 코드에서 사용한 유니티 메서드

```
float result = Mathf.Clamp(float value, float min, float max);
```

value값이 min보다 작으면 min, max보다 크면 max, min과 max 사이의 값이면 value값을 반환해 min과 max로 설정한 범위를 넘지 않도록 합니다.

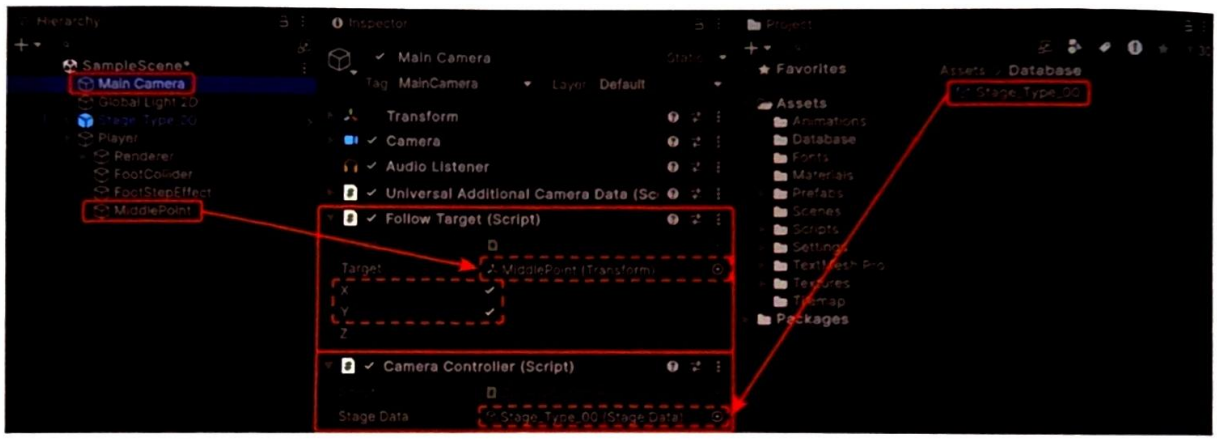
05 현재 Player 게임오브젝트의 위치는 화면에 출력되는 플레이어의 발 위치에 있습니다. 카메라가 Player 오브젝트를 추적하도록 설정하면 발을 쫓아다니므로 플레이어의 중심 위치를 나타내는 빈 오브젝트를 별도로 생성해서 사용합니다.

[GameObject → Create Empty] 메뉴로 빈 오브젝트를 생성하고 이름을 'MiddlePoint'로 변경한 후 Player 오브젝트의 자식으로 배치합니다. MiddlePoint 오브젝트 Transform 컴포넌트의 Position을 (0, 0.675, 0)으로 설정하고 Scene View에서 위치를 확인할 수 있도록 〈Select Icon〉 버튼을 눌러 아이콘을 설정합니다.



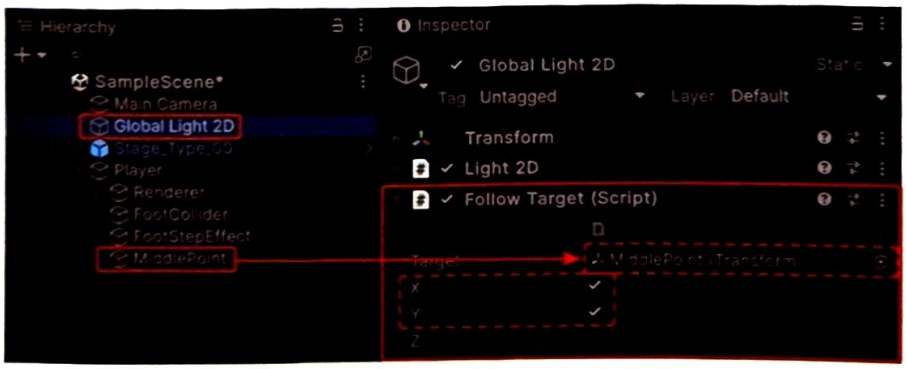
플레이어의 중심 위치를 나타내는 빈 오브젝트 생성하고 설정하기

06 Main Camera 오브젝트에 Follow Target과 Camera Controller 컴포넌트를 추가합니다. 그리고 Follow Target 컴포넌트의 Target 변수에 MiddlePoint 오브젝트를 등록합니다. 카메라의 x, y축이 플레이어를 추적하도록 X, Y를 true로 설정합니다. 마지막으로 Camera Controller 컴포넌트의 Stage Data 변수에 Stage_Type_00 에셋을 등록합니다.



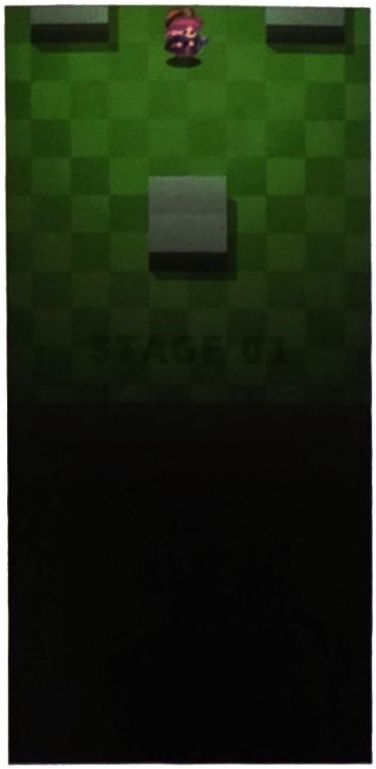
Main Camera 오브젝트에 컴포넌트 추가하고 설정하기

07 플레이어와 조명의 거리가 멀어지면 화면이 어두워져서 플레이를 할 수 없습니다. 조명은 항상 플레이어를 쫓아다니도록 Global Light 2D 오브젝트에 Follow Target 컴포넌트를 추가하고 Target 변수에 MiddlePoint 오브젝트를 등록합니다. 카메라와 마찬가지로 X, Y 변수를 true로 설정합니다.

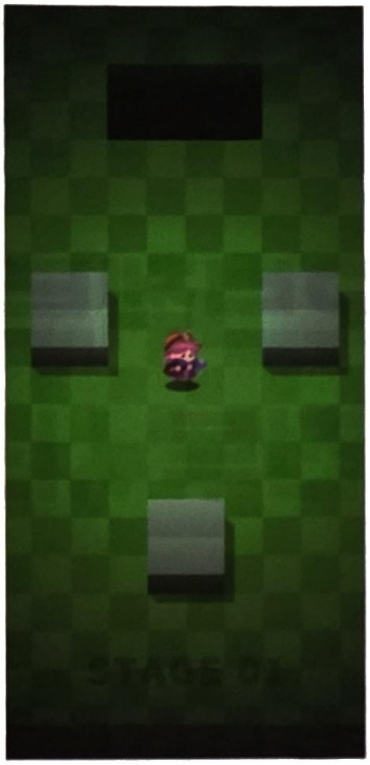


조명 오브젝트 Global Light 2D에 Follow Target 컴포넌트 추가하고 설정하기

08 플레이 버튼 ▶을 눌러 게임을 실행하면 Follow Target 컴포넌트를 적용하기 전에는 카메라가 움직이지 않으므로 플레이어를 움직였을 때 화면 밖으로 벗어나지만, Follow Target 컴포넌트를 적용한 다음에는 카메라가 플레이어를 추적해 항상 화면 가운데에 보이도록 합니다. 또한 월드 바깥은 출력하지 않도록 카메라의 한계 범위를 설정해서 플레이어가 월드의 외곽으로 이동하더라도 카메라가 더는 바깥쪽으로 이동하지 않고 멈춥니다.



Follow Target 컴포넌트 적용 전



Follow Target 컴포넌트 적용 후

04장

적 오브젝트와 기본 전투 시스템

이번 장에서는 적 오브젝트를 생성하고 적이 플레이어를 공격하도록 합니다. 플레이어와 적의 레벨, 체력, 공격력과 같은 기본 능력치를 구현하고 체력 UI를 게이지 형태로 머리 위에 출력합니다. 이때 플레이어는 자신과 거리가 가장 가까운 적을 공격 목표로 설정합니다. 이와 함께 플레이어의 공격 목표 위치에 표식을 출력하고 플레이어와 무기가 목표 방향으로 회전하도록 설정합니다.

학습 목표

1. 스탯 클래스를 설계하고 UI(Slider)와 연동하여 캐릭터 상태를 시각화할 수 있다.
2. 공격 쿨타임을 적용한 적 공격을 구현할 수 있다.
3. 거리 연산을 활용하여 최적의 공격 대상을 자동으로 선정할 수 있다.

04-1 적 오브젝트 만들고 행동 제어하기
04-2 기본 스탯 설정과 체력 출력하기
04-3 적 기본 공격 구현하기
04-4 플레이어 공격 목표 설정하기

04-1 적 오브젝트 만들고 행동 제어하기

이 절에서는 적 오브젝트를 생성하고 제자리에서 대기할 때 애니메이션을 재생합니다. 그리고 타일맵에서 적을 배치할 수 있는 임의의 위치에 원하는 숫자만큼 적을 자동으로 생성하도록 설정해 봅시다.

적 캐릭터 만들기부터 자동 생성까지를 크게 세 부분으로 나누어 구현합니다.

Do it! 실습 적 캐릭터 만들기

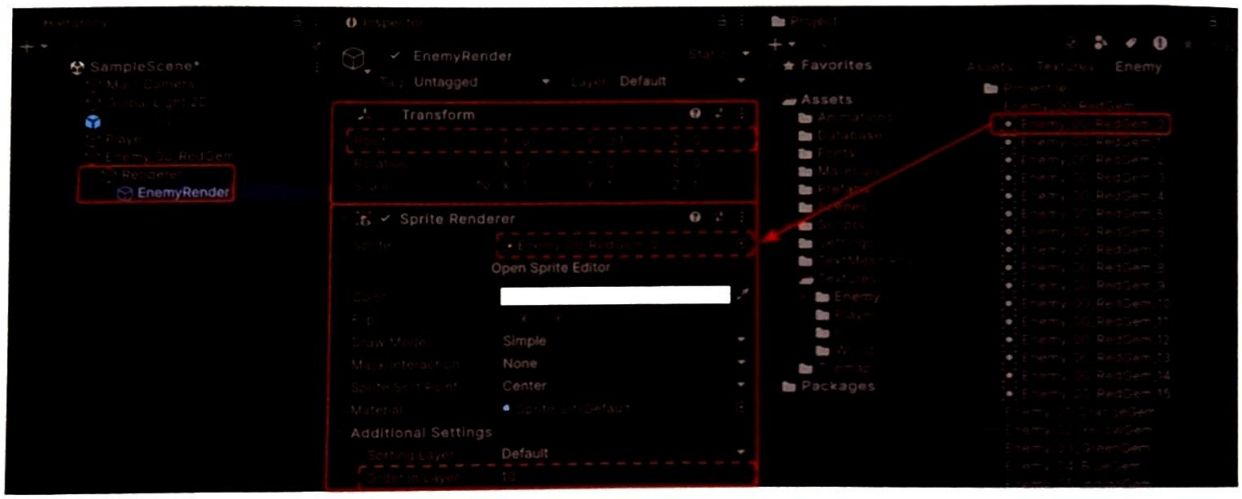
01 플레이어를 만들 때와 마찬가지로 [GameObject → Create Empty] 메뉴로 적 오브젝트를 관리하는 빈 오브젝트를 생성하고 이름을 'Enemy_00_RedGem'으로 변경합니다.

Enemy_00_RedGem 오브젝트 생성하고 설정하기

02 같은 방법으로 화면에 적을 출력하는 오브젝트를 관리하는 빈 오브젝트를 생성하고 이름을 'Renderer'로 변경한 후 Enemy_00_RedGem 오브젝트의 자식으로 배치합니다.

Renderer 오브젝트 생성하고 설정하기

03 [GameObject → 2D Object → Sprites → Square] 메뉴로 적 캐릭터를 출력하는 2D Square 오브젝트를 생성하고 이름을 'EnemyRender'로 변경한 후 Renderer 오브젝트의 자식으로 배치합니다. 적 캐릭터가 공중에 살짝 떠 보이도록 Transform 컴포넌트의 Position을 (0, 0.1, 0)으로 설정합니다. Sprite Renderer 컴포넌트의 Sprite를 Enemy_00_RedGem_0으로, Order in Layer를 '10'으로 설정합니다.



EnemyRender 오브젝트 생성하고 설정하기

04 [GameObject → 2D Object → Sprites → Circle] 메뉴로 적 캐릭터의 그림자를 출력하는 2D Circle 오브젝트를 생성하고 이름을 'Shadow'로 변경한 후 Renderer 오브젝트의 자식으로 배치합니다. Transform 컴포넌트의 Scale을 (0.5, 0.2, 1)로 설정합니다. Sprite Renderer 컴포넌트의 Color를 (0, 0, 0, 200)으로, Order in Layer를 '9'로 설정합니다.

● Scene View에서 개별 오브젝트를 확인합니다.

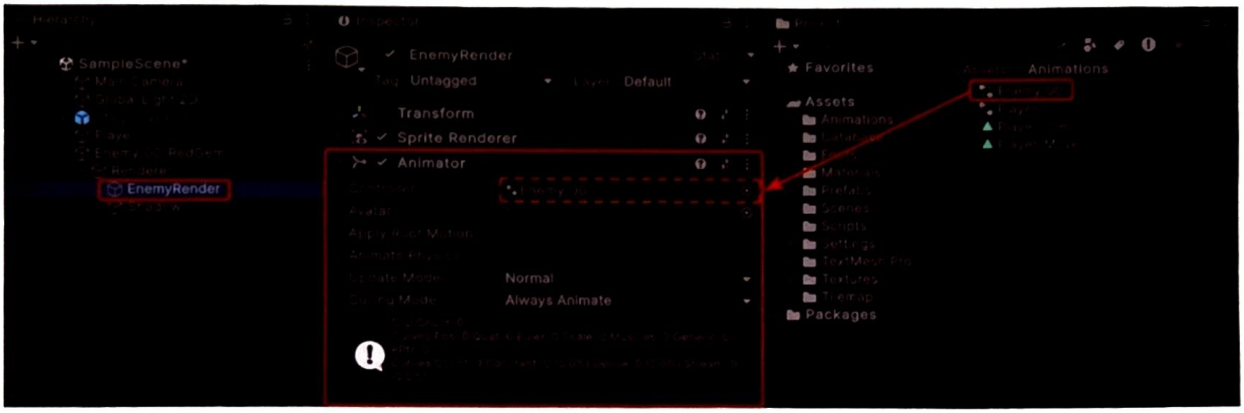
Shadow 오브젝트 생성하고 설정하기

Do it! 실습 적 캐릭터 대기 애니메이션 만들기

이번에는 적 오브젝트가 제자리에 가만히 있을 때 재생할 애니메이션을 만듭니다.

01 Project View의 + ▾ 버튼을 클릭해 [Animation → Animator Controller] 메뉴로 적 캐릭터의 애니메이션을 제어하는 애니메이터 컨트롤러를 Assets 〉 Animations 폴더 안에 생성하고 이름을 'Enemy_00'으로 변경합니다.

적 캐릭터를 화면에 출력하는 EnemyRender 오브젝트에 Animator 컴포넌트를 추가하고 Controller 변수에 Enemy_00 에셋을 등록합니다.

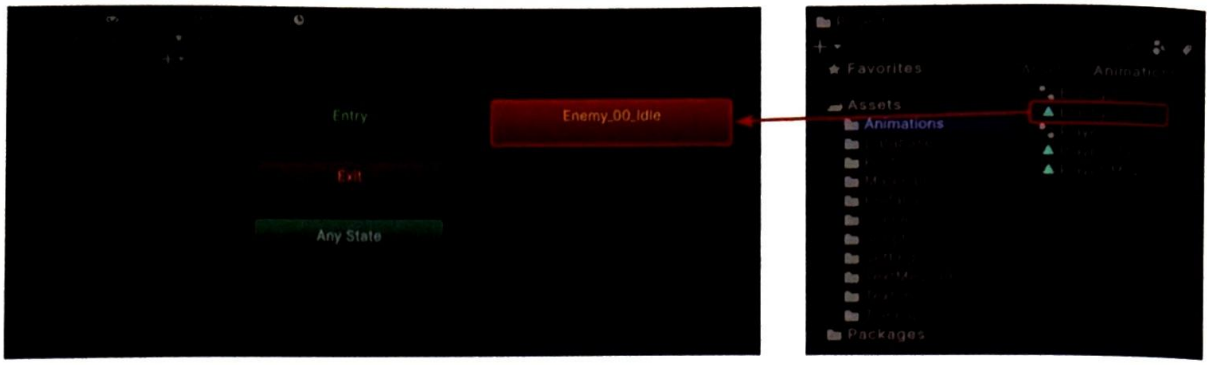


적의 애니메이션 재생을 제어하는 Animator 컴포넌트 추가하고 설정하기

02 Project View의 + ▾ 버튼을 클릭해 [Animation → Animation Clip] 메뉴로 적의 대기 애니메이션을 Assets 〉 Animations 폴더 내부에 생성하고 이름을 'Enemy_00_Idle'로 변경한 후 Loop Time을 true로 설정합니다.

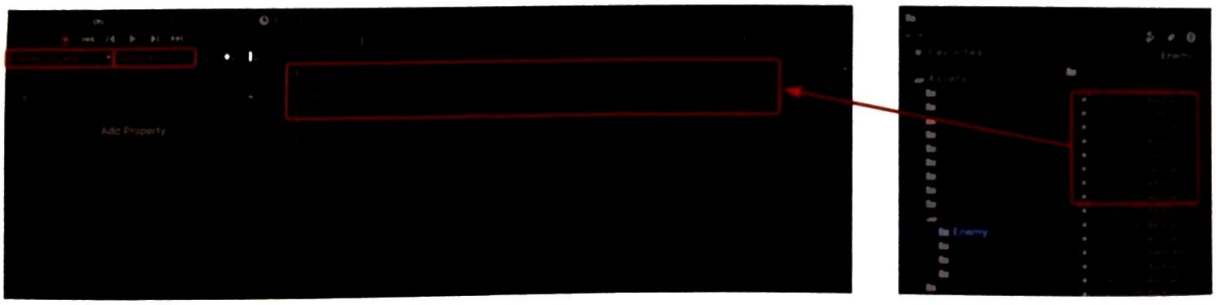
적 대기 애니메이션 클립 에셋 생성하고 반복 설정하기

03 Enemy_00_RedGem 오브젝트의 자식인 EnemyRender 오브젝트를 선택하고 Animator View로 가서 Enemy_00_Idle 에셋 애니메이션을 드래그해서 등록합니다.



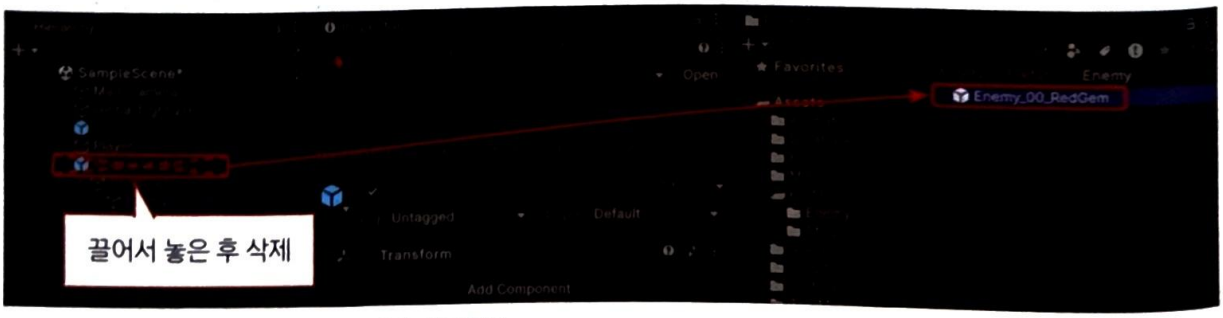
적 대기 애니메이션 Enemy_00_Idle 에셋 등록하기

04 적의 대기 애니메이션 설정을 위해 Enemy_00_RedGem 오브젝트의 자식인 Enemy Render 오브젝트를 선택하고 Animation View로 이동합니다. Samples를 '6'으로 설정하고 Enemy_00_RedGem_0부터 Enemy_00_RedGem_7까지의 이미지를 드래그해서 등록합니다.



적 대기 애니메이션 Enemy_00_Idle 에셋 설정하기

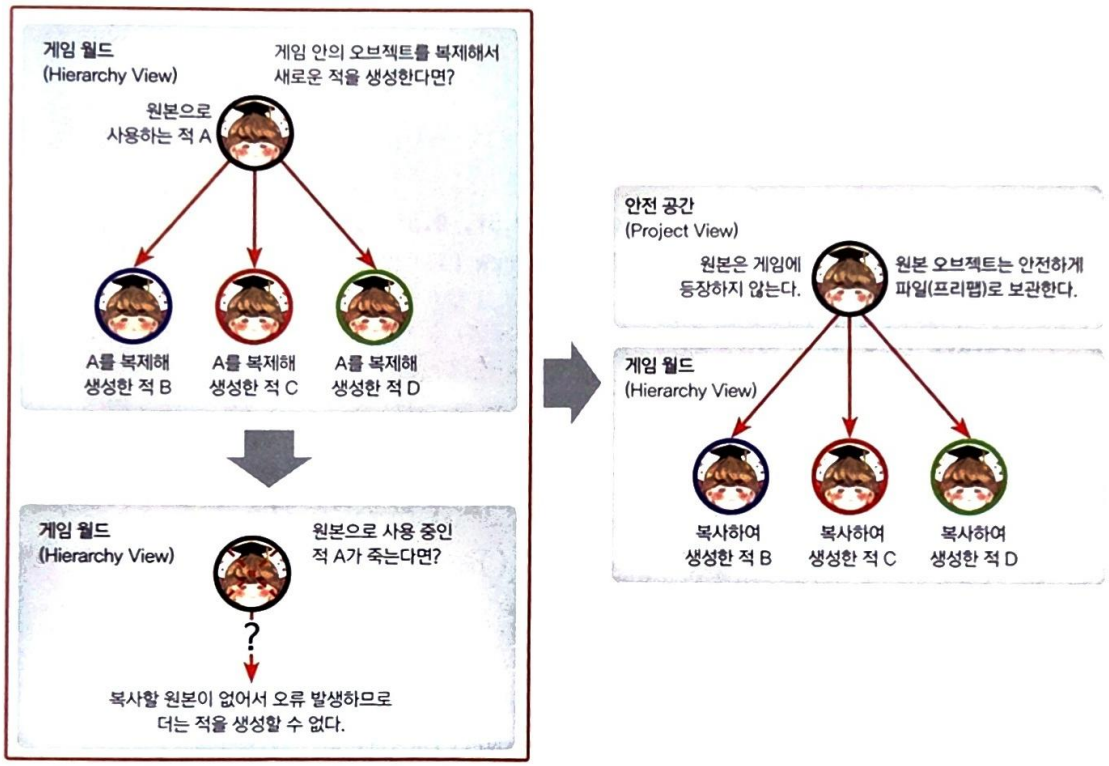
05 적 오브젝트(Enemy_00_RedGem)를 Project View의 Assets 〉 Prefabs 〉 Enemy 폴더로 끌어서 놓아 프리팹을 생성하고 Hierarchy View에 있는 Enemy_00_RedGem 오브젝트는 삭제합니다.



Enemy_00_RedGem 오브젝트 프리팹 생성하기

알아 두면 좋아요! 프리팹이란?

프리팹Prefab이란 씬 내부에 데이터로 존재하는 게임 오브젝트를 Project View로 드래그해서 파일을 생성하는 것입니다. 일정한 주기마다 적을 생성할 때 생성하려는 적 원본을 게임 안 오브젝트로 사용했을 때는 이 오브젝트가 사망하면 더는 적을 생성할 수 없습니다. 그래서 안전 공간인 Project View에 적 원본 프리팹 파일을 생성하고, 이 프리팹을 복제해서 게임에 등장하는 적을 생성합니다. 또한 A 프로젝트에서 만든 게임 오브젝트를 B 프로젝트에서 사용할 때 파일로 옮겨 사용할 수 있도록 프리팹을 사용하기도 합니다.



프리팹을 사용하는 이유

Do it! 실습 적 캐릭터 자동 생성하기

이 게임에서는 등장할 숫자와 속성 등을 설정해 각 스테이지에 적을 자동으로 생성합니다. 이 기능은 스테이지 시스템을 다루는 12-1절에서 구현하기로 하고 여기서는 임시로 무작위 위치에 원하는 숫자만큼 적을 생성하도록 합니다.

01 먼저 타일 위에 적 10기를 무작위로 생성하는 EnemySpawner.cs 스크립트를 Assets 〉 Scripts 〉 Enemy 폴더에 생성합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

코드 04-01 적 자동 생성 스크립트

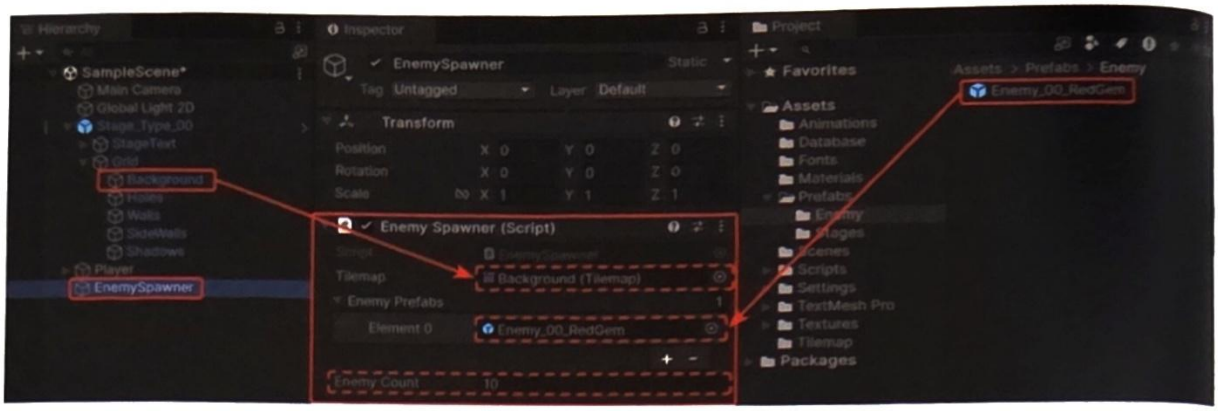
```
01: using System.Collections.Generic;
02: using UnityEngine;
03: using UnityEngine.Tilemaps;
04:
05: public class EnemySpawner : MonoBehaviour
06: {
07:     [SerializeField]
08:     private Tilemap tilemap;
09:     [SerializeField]
10:     private GameObject[] enemyPrefabs;
11:     [SerializeField]
12:     private int enemyCount = 10;
13:
14:     private Vector3 offset = new Vector3(0.5f, 0.5f, 0);
15:     private List<Vector3> possibleTiles = new List<Vector3>();
16:
17:     private void Awake()
18:     {
19:         // Tilemap의 Bounds 재설정(맵을 수정할 때 Bounds가 변경되지 않는 문제 해결)
20:         tilemap.CompressBounds();
21:         // 타일맵의 모든 타일을 대상으로 적을 배치할 수 있는 타일 계산
22:         CalculatePossibleTiles();
23:
24:         // 임의의 타일에 적 10기 생성
25:         for ( int i = 0; i < enemyCount; ++ i )
26:         {
27:             int type = Random.Range(0, enemyPrefabs.Length);
28:             int index = Random.Range(0, possibleTiles.Count);
29:
30:             Instantiate(enemyPrefabs[type], possibleTiles[index],
                    Quaternion.identity, transform);
31:         }
32:     }
33:
34:     private void CalculatePossibleTiles()
35:     {
36:         BoundsInt bounds = tilemap.cellBounds;
37:         TileBase[] allTiles = tilemap.GetTilesBlock(bounds);
38:
39:         // 외곽 벽에 붙은 타일은 제외하고자
40:         // x, y의 시작값은 1, 끝값은 bounds.size.x - 1, bounds.size.y - 1
41:         for ( int y = 1; y < bounds.size.y - 1; ++ y )
```

```
42:         {
43:             for ( int x = 1; x < bounds.size.x - 1; ++ x )
44:             {
45:                 TileBase tile = allTiles[y * bounds.size.x + x];
46:
47:                 if ( tile != null )
48:                 {
49:                     Vector3Int localPosition = bounds.position +
                            new Vector3Int(x, y);
50:                     Vector3 position = tilemap.CellToWorld(localPosition) + offset;
51:                     position.z = 0;
52:
53:                     possibleTiles.Add(position);
54:                 }
55:             }
56:         }
57:     }
58: }
```

24~31줄 적을 배치할 수 있는 타일 중에서 임의의 타일 possibleTiles[index]에 적을 enemyCount개만큼 생성합니다.

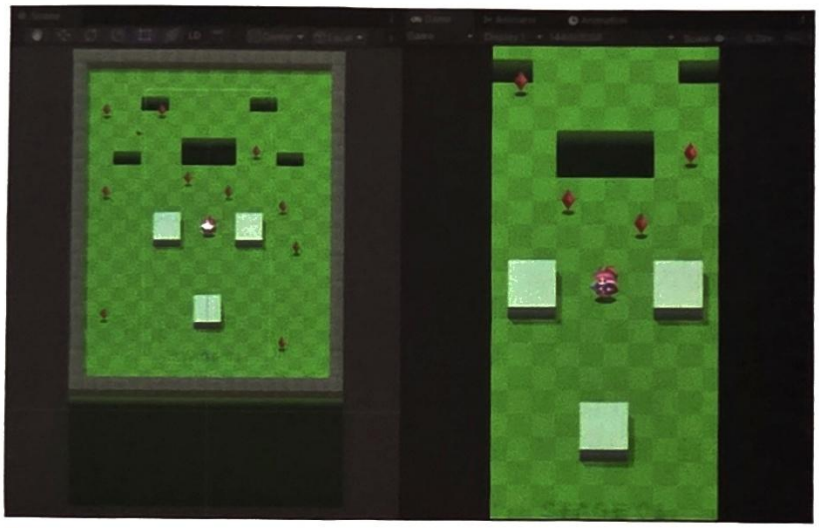
34~57줄 CalculatePossibleTiles() 메서드는 적을 배치할 수 있는 타일을 찾아 possibleTiles 리스트에 저장합니다. 45줄처럼 tilemap 변수에 등록한 타일맵의 0번 위치부터 차례대로 tile 변수에 저장하고 바닥 타일이 그려졌으면(tile != null) 해당 타일 위치인 position을 possibleTiles 리스트에 추가합니다.

02 [GameObject → Create Empty] 메뉴로 빈 오브젝트를 생성하고 이름을 'Enemy Spawner'로 변경합니다. EnemySpawner 오브젝트에 Enemy Spawner 컴포넌트를 추가하고 Tilemap 변수에 Background 오브젝트를, Enemy Prefabs 배열 변수에 Enemy_00_RedGem 프리팹을 등록합니다.



EnemySpawner 오브젝트에 Enemy Spawner 컴포넌트 추가하고 설정하기

03 플레이 버튼 ▶을 눌러 게임을 실행하면 바닥으로 사용하는 Background 타일맵 오브젝트 임의의 위치에 Enemy Count 변수에 설정한 숫자만큼 적 오브젝트를 생성합니다.



적 오브젝트 자동 생성 결과 화면

04-2

기본 스탯 설정과 체력 출력하기

이 절에서는 플레이어와 적의 레벨, 체력, 공격력과 같은 기본 능력치를 구현하고 플레이어의 머리 위에 체력 UI를 게이지 형태로 출력해 봅니다.

플레이어와 적의 기본 능력치 구현하기

플레이어와 적이 사용하는 기본 스탯은 다음 표처럼 레벨, 경험치, 기본 공격력, 공격 쿨타임, 크리티컬 확률, 크리티컬 공격력 계수, 체력, 회피 등의 정보로 이루어집니다.

● 쿨타임이란 캐릭터가 특정 스킬이나 아이템을 사용한 후 다시 사용하기까지 기다려야 하는 시간을 뜻합니다.

플레이어와 적 기본 스탯

스탯	변수 이름	기본값	최솟값	최댓값
레벨	Level	1	1	99
경험치	Exp	0	0	99999999
기본 공격력	Damage	25	1	9999
공격 쿨타임	Cooldown Time	2.5	0.5	2.5
크리티컬 확률	Critical Chance	0.05	0	1.0 (1.0 = 100%)
크리티컬 공격력 계수	Critical Multiplier	2	1	4
(현재, 최대) 체력	HP	100	0	9999
회피	Evasion	0.05	0	1.0 (1.0 = 100%)

Do it! 실습 플레이어와 적의 기본 스탯 만들기

01 Assets 〉 Scripts 〉 Entity 폴더에 플레이어와 적이 사용하는 스탯 구조체를 정의하는 EntityStats.cs 스크립트를 생성합니다. 앞서 표로 정리한 플레이어와 적의 기본 스탯을 바탕으로 하면 됩니다.

Assets/Scripts/Entity/EntityStats.cs

코드 04-02 플레이어와 적 기본 스탯

```
01: using UnityEngine;
02:
03: [System.Serializable]
04: public struct EntityStats
05: {
06:     [Header("Level, Exp")]
07:     public int level; // 레벨
08:     public long exp;  // 경험치
09:
10:     [Header("Attack")]
11:     public float damage;             // 공격력
12:     public float cooldownTime;       // 기본 공격 쿨타임
13:     public float criticalChance;     // 크리티컬 확률
14:     public float criticalMultiplier; // 크리티컬 공격력
15:
16:     [Header("Defense")]
17:     public float currentHP; // 현재 체력
18:     public float maxHP;     // 최대 체력
19:     public float evasion;   // 회피율
20: }
```

02 같은 폴더에 기본 스탯, 최대 체력, 피격 등 플레이어와 적의 공통 정보를 모아 게임에 있는 모든 캐릭터가 상속받는 기반 클래스인 EntityBase.cs 스크립트를 생성합니다.

Assets/Scripts/Entity/EntityBase.cs

코드 04-03 모든 캐릭터가 상속받는 기반 클래스

```
01: using UnityEngine;
02:
03: public class EntityBase : MonoBehaviour
04: {
05:     [SerializeField]
06:     protected EntityStats stats;
07:
08:     public EntityStats Stats => stats;
09:     public bool IsDead => stats.currentHP <= 0;
10:
11:     protected virtual void Setup()
12:     {
```

```
13:         stats.currentHP = stats.maxHP;
14:     }
15:
16:     public void TakeDamage(float damage)
17:     {
18:         if ( IsDead ) return;
19:
20:         stats.currentHP = stats.currentHP - damage > 0 ?
                stats.currentHP - damage : 0;
21:
22:         if ( stats.currentHP <= 0 )
23:         {
24:             // 사망 처리
25:         }
26:     }
27: }
```

16~26줄 모든 캐릭터는 피격을 당해 체력이 감소하고 체력이 0이 되면 사라지도록 **TakeDamage()** 메서드를 정의합니다.

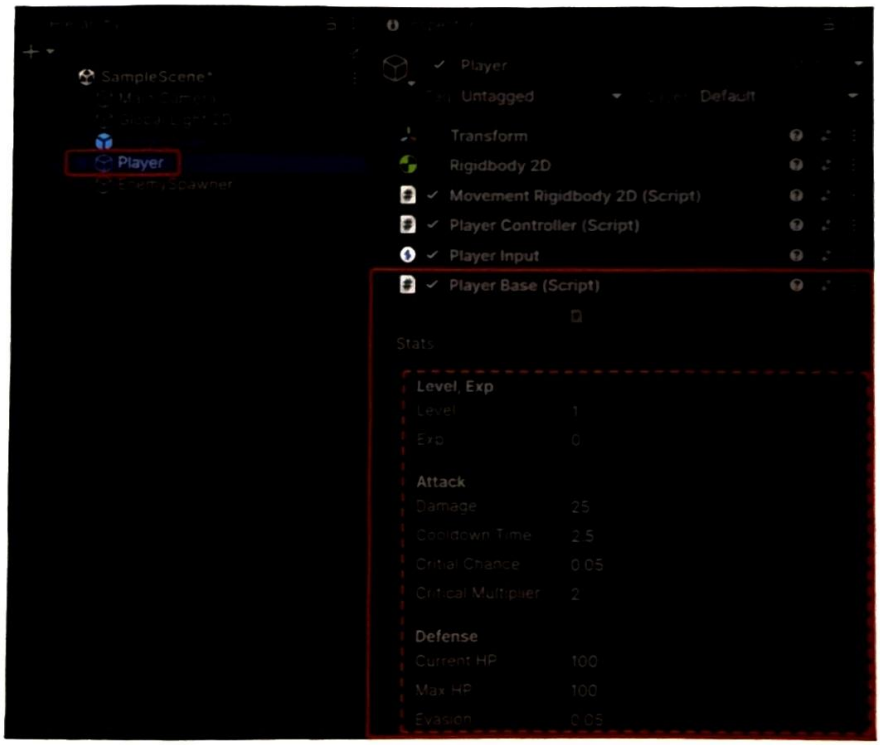
03 Assets 〉 Scripts 〉 Player 폴더에 **EntityBase** 클래스를 상속받아 플레이어 캐릭터를 제어하는 PlayerBase.cs 스크립트를 생성합니다.

Assets/Scripts/Player/PlayerBase.cs

코드 04-04 플레이어 캐릭터 제어

```
01: using UnityEngine;
02:
03: public class PlayerBase : EntityBase
04: {
05:     // 현재 플레이어가 이동 중인지
06:     // 스킬 사용 등 여러 곳에서 필요하므로 public 속성으로 정의
07:     public bool IsMoved { get; set; } = false;
08:
09:     private void Awake()
10:     {
11:         base.Setup();
12:     }
13: }
```

04 Player 오브젝트에 Player Base 컴포넌트를 추가합니다. 플레이어와 적 기본 스탯 표처럼 Stats 구조체의 Level은 '1', Damage는 '25', Cooldown Time은 '2.5', Critical Chance는 '0.05', Critical Multiplier는 '2', Current HP는 '100', Max HP는 '100', Evasion은 '0.05'로 설정합니다.



Player 오브젝트에 Player Base 컴포넌트 추가하고 설정하기

05 지역 변수 `isMoved` 대신 `PlayerBase`에 정의한 `IsMoved` 속성을 사용하고자 PlayerController.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerController.cs

코드 04-05 플레이어 캐릭터 제어

```
    (... 생략 ...)
04: public class PlayerController : MonoBehaviour
05: {
06:     private MovementRigidbody2D movement2D;
07:     private PlayerRenderer playerRenderer;
08:     private PlayerBase playerBase;
09:     private Vector2 moveInput = Vector2.zero;
10:
11:     private void Awake()
12:     {
```

추가

```
13:         movement2D = GetComponent<MovementRigidbody2D>();
14:         playerRenderer = GetComponentInChildren<PlayerRenderer>();
15:         playerBase = GetComponent<PlayerBase>();
16:     }
17:
18:     private void Update()
19:     {
20:         // 플레이어 이동 여부 검사
21:         playerBase.IsMoved = moveInput.x != 0 || moveInput.y != 0;
22:         // 플레이어 좌우 반전
23:         if ( moveInput.x != 0 ) playerRenderer.SpriteFlipX(moveInput.x);
24:         // 플레이어 애니메이션 재생
25:         playerRenderer.OnMovement(playerBase.IsMoved ? 1 : 0);
26:         // 먼지 효과 재생
27:         playerRenderer.OnFootStepEffect(playerBase.IsMoved);
28:         // 플레이어 이동
29:         movement2D.MoveTo(moveInput);
30:     }
        (... 생략 ...)
39: }
```

추가

수정

수정

수정

20~21줄 플레이어 이동 여부에 따라 애니메이션 재생, 스킬 사용 등을 제어하므로 지역 변수 isMoved가 아닌 PlayerBase 클래스에 있는 IsMoved 속성 정보를 불러와 이동 여부를 판단합니다.

06 Assets 〉 Scripts 〉 Enemy 폴더에 적 캐릭터를 제어하는 EnemyBase.cs 스크립트를 생성합니다.

Assets/Scripts/Enemy/EnemyBase.cs

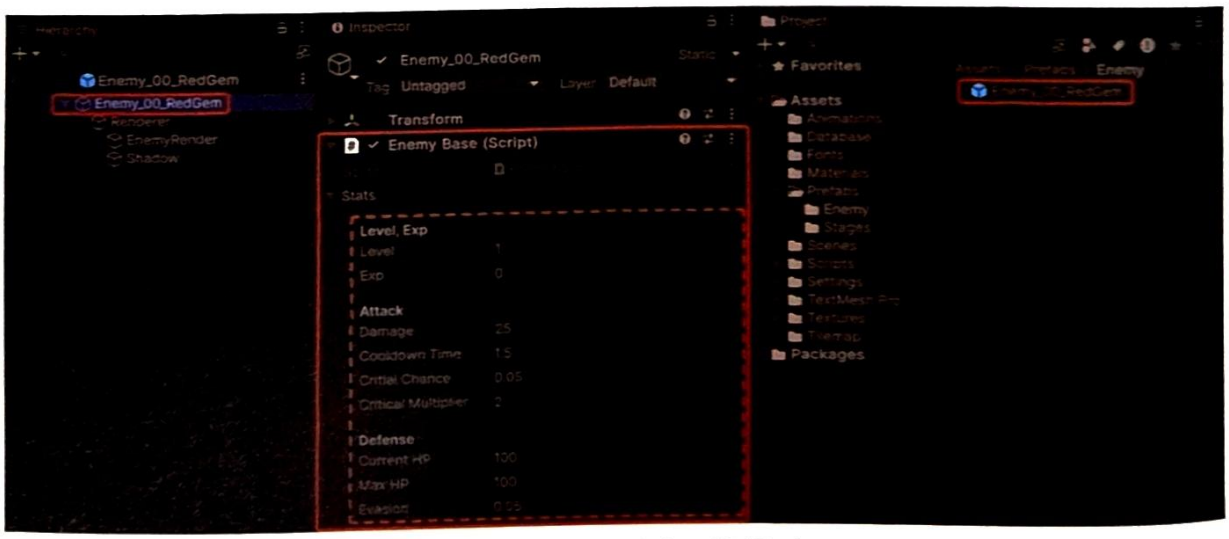
코드 04-06 적 캐릭터 제어

```
01: using UnityEngine;
02:
03: public class EnemyBase : EntityBase
04: {
05:     private void Awake()
06:     {
07:         Setup();
08:     }
```

```
09:
10:     protected override void Setup()
11:     {
12:         stats.maxHP = 100 + 50 * (stats.level - 1); // 기본(100)+추가(50*(레벨-1))
13:
14:         base.Setup();
15:     }
16: }
```

12줄 적의 최대 체력은 선형 수식을 이용해 레벨이 증가할 때마다 50씩 늘어나도록 설정합니다.

07 Enemy_00_RedGem 프리팹에 Enemy Base 컴포넌트를 추가합니다. Stats 구조체의 Level은 '1', Damage는 '25', Cooldown Time은 '1.5', Critical Chance는 '0.05', Critical Multiplier는 '2', Current HP는 '100', Max HP는 '100', Evasion은 '0.05'로 설정합니다.

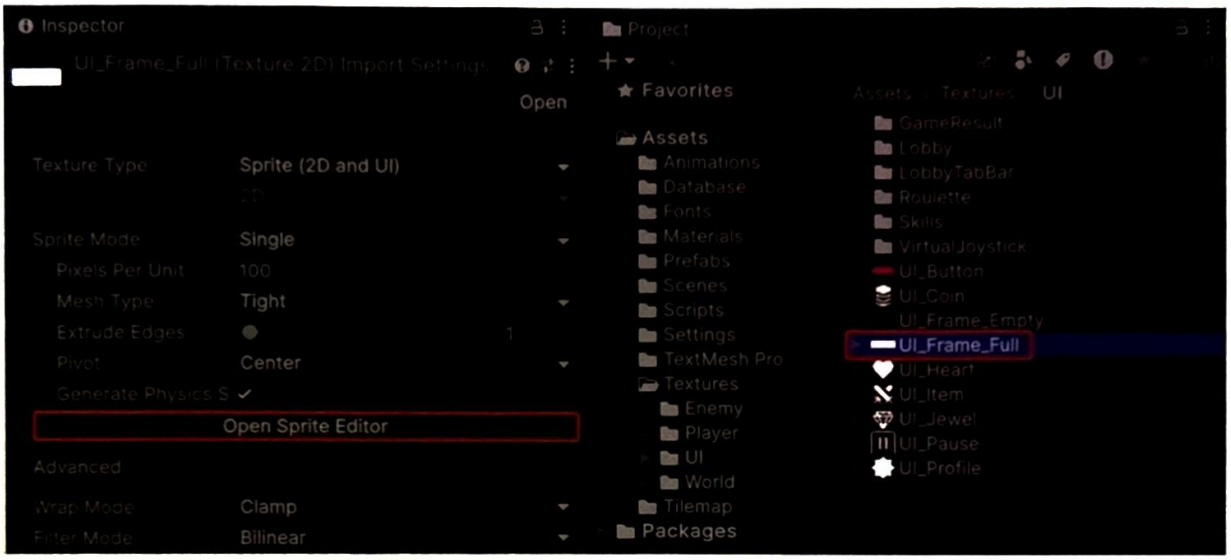


Enemy_00_RedGem 프리팹에 Enemy Base 컴포넌트 추가하고 설정하기

Do it! 실습 플레이어를 따라다니는 체력 UI 추가하기

이번에는 플레이어 머리 위에 표시할 체력 게이지를 만듭니다. 단계가 많으므로 하나씩 천천히 확인하며 진행하세요.

01 이미지를 확대 또는 축소해서 출력할 때 외곽으로 지정한 부분은 크기가 변하지 않도록 설정합니다. Assets 〉 Textures 〉 UI 〉 UI_Frame_Full.png 이미지를 선택하고 Inspector View에서 〈Open Sprite Editor〉 버튼을 클릭해서 Sprite Editor View를 엽니다.

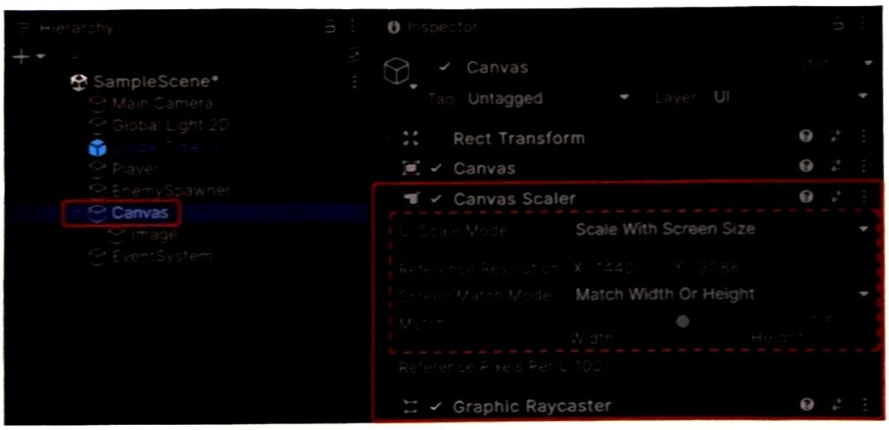


UI_Frame_Full.png 이미지의 스프라이트 에디터 창 열기

02 Sprite Editor View에서 ❶ 이미지 Border를 (15, 15, 15, 15)로 설정하고 ❷ 〈Apply〉 버튼을 클릭합니다.

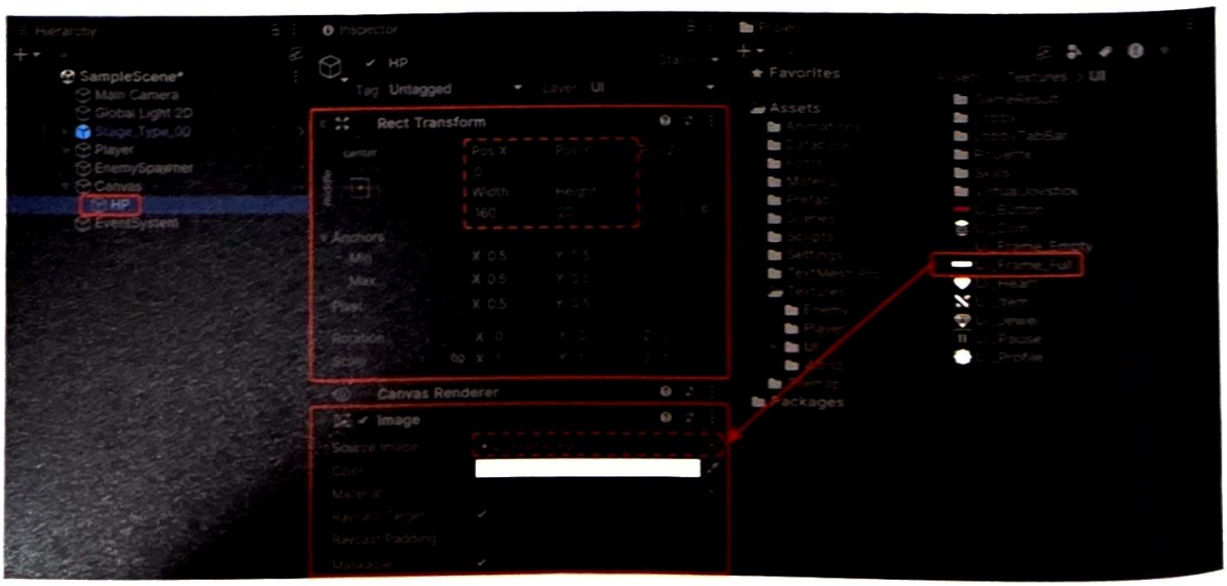
UI_Frame_Full.png 이미지 Border 설정하기

03 플레이어의 체력 정보를 게이지 형태로 출력하고 플레이어의 머리 위치를 쫓아다니도록 설정합니다. [GameObject → UI → Image] 메뉴로 체력을 출력하는 Image 오브젝트를 생성합니다. UI 오브젝트와 함께 생성된 Canvas 오브젝트를 선택하고 Canvas Scaler 컴포넌트의 UI Scale Mode를 [Scale With Screen Size]로 설정합니다. Reference Resolution을 (1440, 3088)로, Screen Match Mode를 [Match Width Or Height]로, Match를 '0.5'로 설정합니다.



Canvas 오브젝트의 Canvas Scaler 컴포넌트 변수 설정하기

04 앞서 생성한 Image 오브젝트 이름을 'HP'로 변경합니다. 그리고 HP 오브젝트 Rect Transform 컴포넌트의 Rect를 (0, 0, 160, 20)으로 설정하고 Image 컴포넌트의 Source Image에 UI_Frame_Full.png 이미지를 등록합니다.

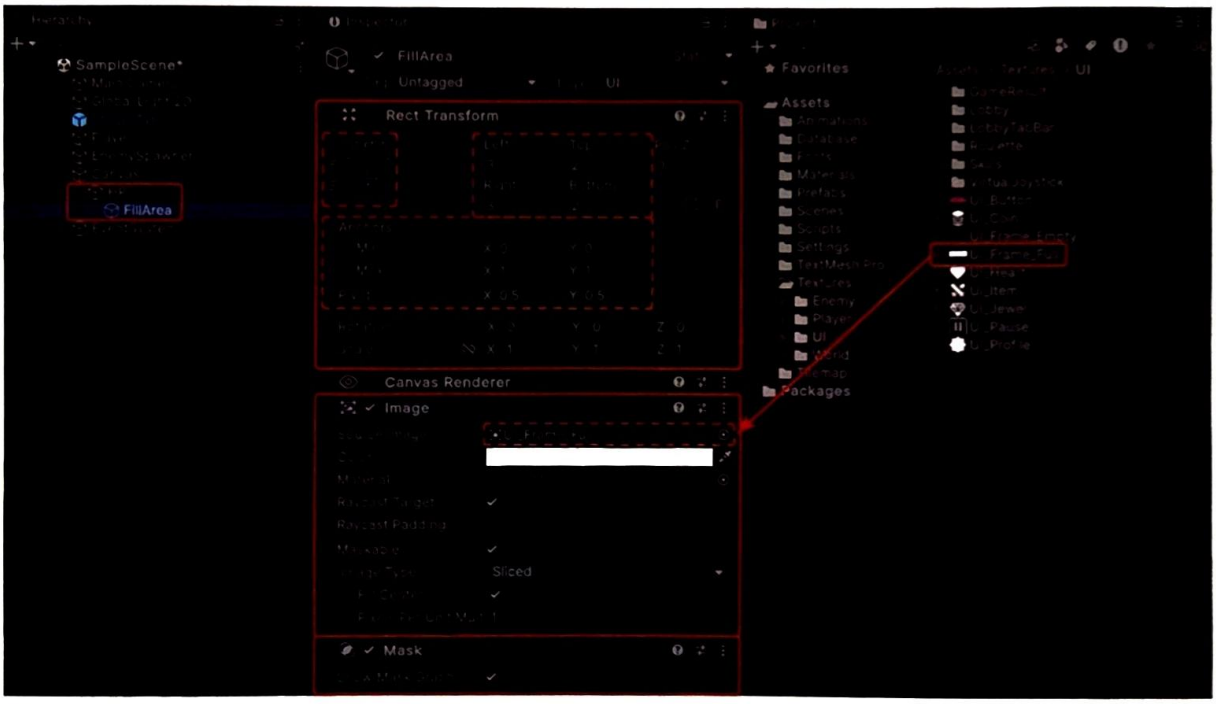


체력 출력을 제어하는 HP 오브젝트 설정하기

05 [GameObject → UI → Image] 메뉴로 체력 게이지 출력 범위를 제어하는 Image 오브젝트를 생성하고 이름을 'FillArea'로 변경한 후 HP 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets을 stretch | stretch로 설정하고 Rect를 (3, 2, 3, 2)로 설정합니다. 마지막으로 Image 컴포넌트의 Source Image에 UI_Frame_Full.png 이미지를 등록하고 Mask 컴포넌트를 추가합니다.

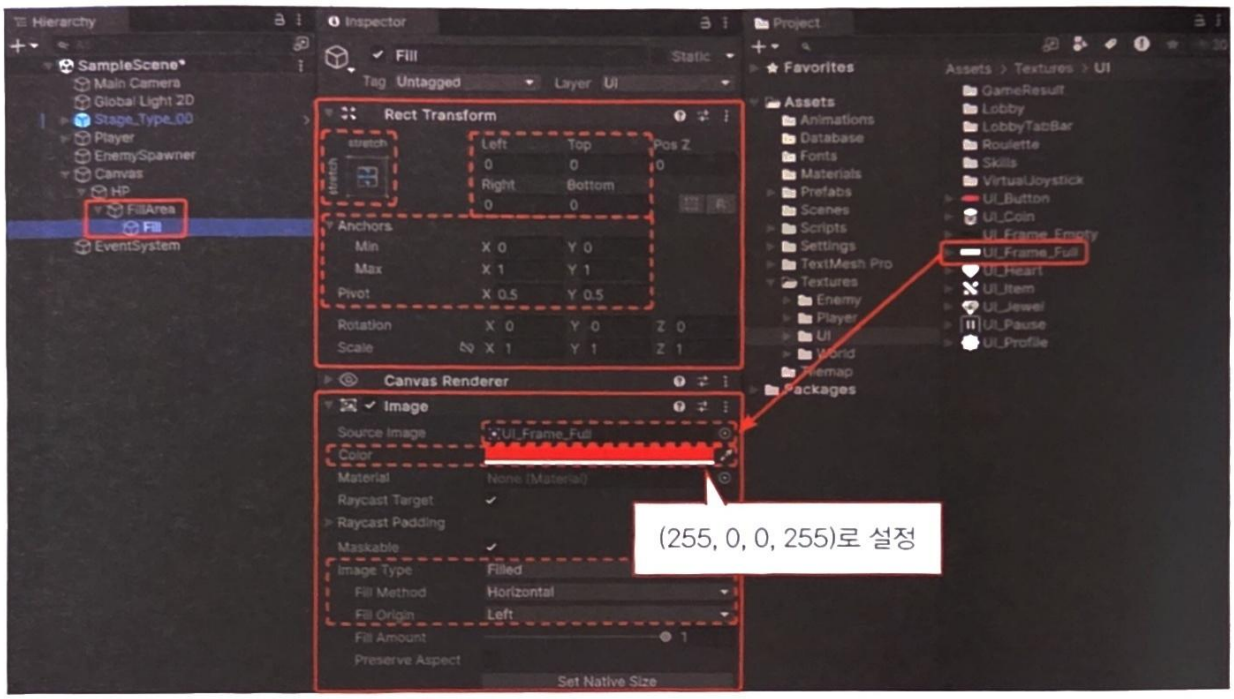
● 모든 UI의 Rect Transform 컴포넌트에서 Anchor Presets를 이용해 Anchors를 설정하며, Shift 키와 Alt 키를 누른 채 클릭하여 Pivot과 좌표를 해당 Anchors에 맞게 설정한 후 좌표를 별도로 설정합니다.



체력 게이지 출력 범위를 제어하는 Image UI 생성하고 설정하기

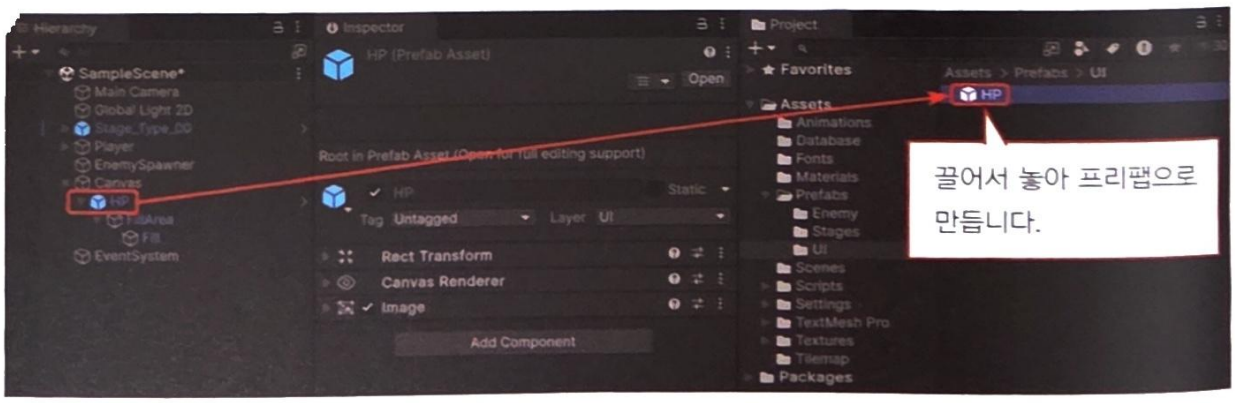
06 [GameObject → UI → Image] 메뉴로 체력 게이지를 출력하는 Image 오브젝트를 생성하고 이름을 'Fill'로 변경한 후 FillArea 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets을 stretch | stretch로 설정하고 Rect를 (0, 0, 0, 0)으로 설정합니다. Image 컴포넌트의 Source Image에 UI_Frame_Full.png 이미지를 등록하고 색을 (255, 0, 0, 255)로 설정합니다. 마지막으로 Image Type은 [Filled]로, Fill Method는 [Horizontal]로, Fill Origin은 [Left]로 설정합니다.



체력 게이지를 출력하는 Image UI 생성하고 설정하기

07 Prefabs 폴더 아래에 UI 폴더를 만들고 Project View의 Assets 〉 Prefabs 〉 UI 폴더로 HP 오브젝트를 끌어서 놓아 프리팹을 생성합니다.



HP 오브젝트로 프리팹 생성하기

08 플레이어와 적 오브젝트는 월드 좌표를 기준으로 위치를 정하지만, 체력을 출력하는 UI는 화면상의 좌표를 기준으로 위치를 정합니다. Assets 〉 Scripts 〉 Common 폴더에 플레이어와 적의 월드 좌표를 화면상의 좌표로 변환하고 이를 월드 오브젝트를 추적하는 UI 위치로 사용하는 FollowTargetUI.cs 스크립트를 생성합니다.

Assets/Scripts/Common/FollowTargetUI.cs

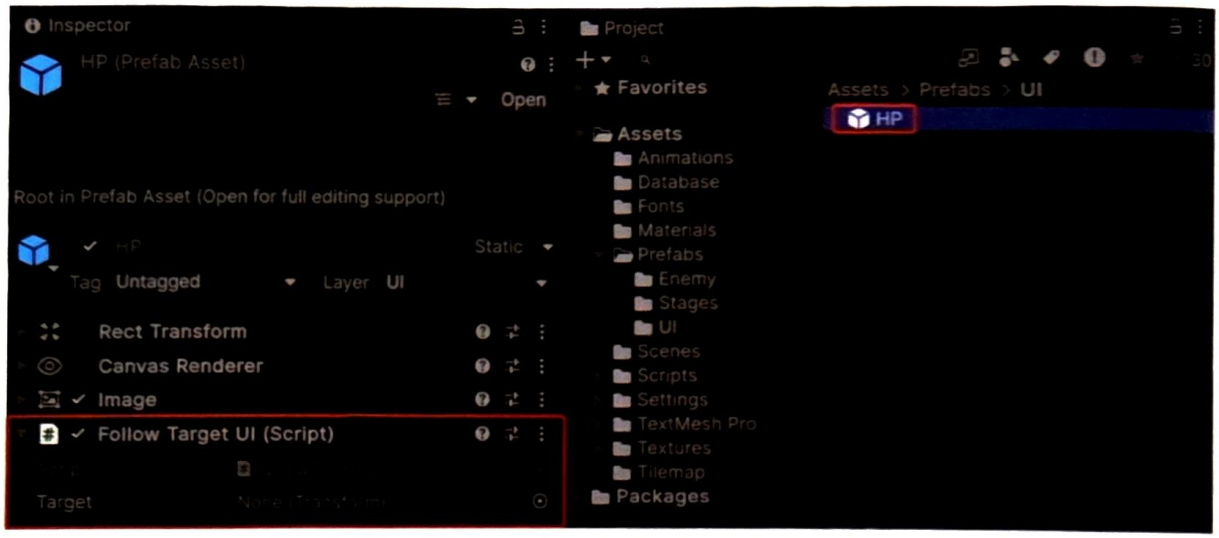
코드 04-07 월드 좌표를 화면 좌표로 변환해 오브젝트 추적

```
01: using UnityEngine;
02:
03: public class FollowTargetUI : MonoBehaviour
04: {
05:     [SerializeField]
06:     private Transform target;
07:     private RectTransform rectTransform;
08:     private Camera mainCamera;
09:
10:     private void Awake()
11:     {
12:         rectTransform = GetComponent<RectTransform>();
13:         mainCamera = Camera.main;
14:     }
15:
16:     public void Setup(Transform target)
17:     {
18:         this.target = target;
19:     }
20:
21:     private void FixedUpdate()
22:     {
23:         if ( target == null )
24:         {
25:             Destroy(gameObject);
26:             return;
27:         }
28:
29:         rectTransform.position = mainCamera.WorldToScreenPoint(target.position);
30:     }
31: }
```

29줄 목표는 Transform 컴포넌트를 사용하는 월드 좌표, 목표를 추적하는 UI는 Rect Transform 컴포넌트를 사용하는 화면 좌표이므로 `WorldToScreenPoint()` 메서드로 목표의 월드 좌표를 현재 화면 좌표로 전환해 UI 위치인 `rectTransform.position`으로 설정합니다.

Canvas 컴포넌트의 Render Mode가 [Screen Space - Camera]일 때는 `RectTransformUtility`를 사용하는 것이 훨씬 안전하고 정확합니다.

09 Assets 〉 Prefabs 〉 UI 폴더에 있는 HP 프리팹을 선택하고 Follow Target UI 컴포넌트를 추가합니다.

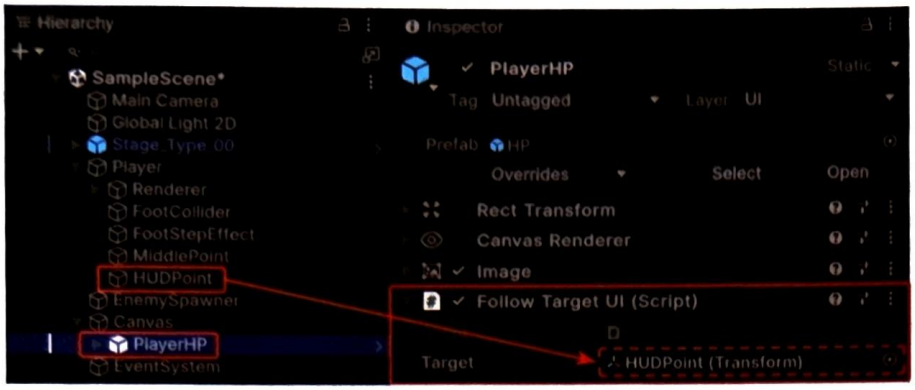


HP 프리팹에 Follow Target UI 컴포넌트 추가하기

10 [GameObject → Create Empty] 메뉴로 플레이어의 체력 UI를 출력할 위치 정보가 있는 빈 오브젝트를 생성하고 이름을 'HUDPoint'로 변경한 후 Player 오브젝트의 자식으로 배치합니다. Scene View에서 위치를 확인할 수 있도록 Inspector View 왼쪽 위 〈Select Icon〉 버튼 을 클릭해서 아이콘을 설정하고 오브젝트 Transform 컴포넌트의 Position을 (0, 1.6, 0)으로 설정합니다.

플레이어의 체력을 출력하는 위치 오브젝트 생성하고 설정하기

11 HP 오브젝트의 이름을 'PlayerHP'로 변경하고 Follow Target UI 컴포넌트의 Target 변수에 Player 오브젝트의 자식으로 있는 HUDPoint 오브젝트를 등록합니다.



PlayerHP 오브젝트의 Follow Target UI 컴포넌트 변수 설정하기

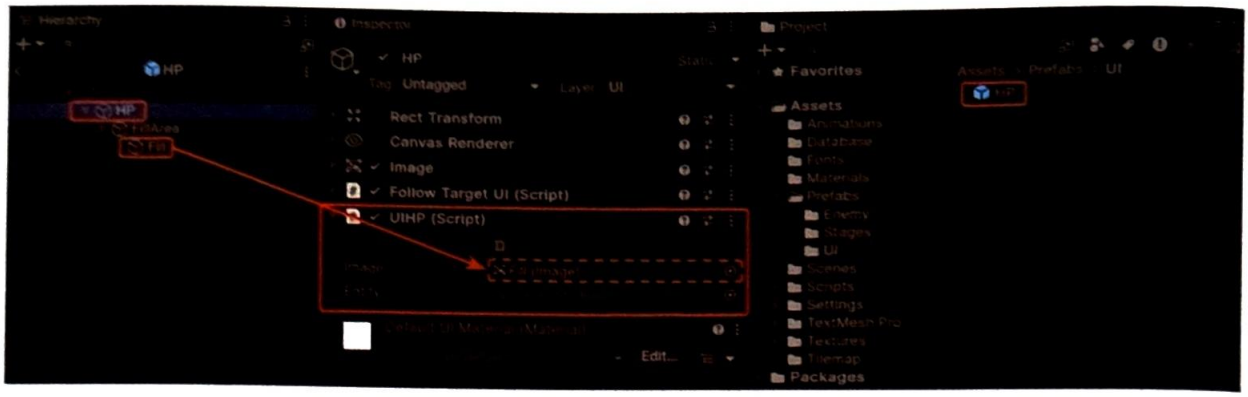
12 Assets 〉 Scripts 〉 UI 폴더에 플레이어와 적의 체력 정보를 UI에 출력하는 UIHP.cs 스크립트를 생성합니다.

Assets/Scripts/UI/UIHP.cs

코드 04-08 플레이어와 적의 체력 정보 출력

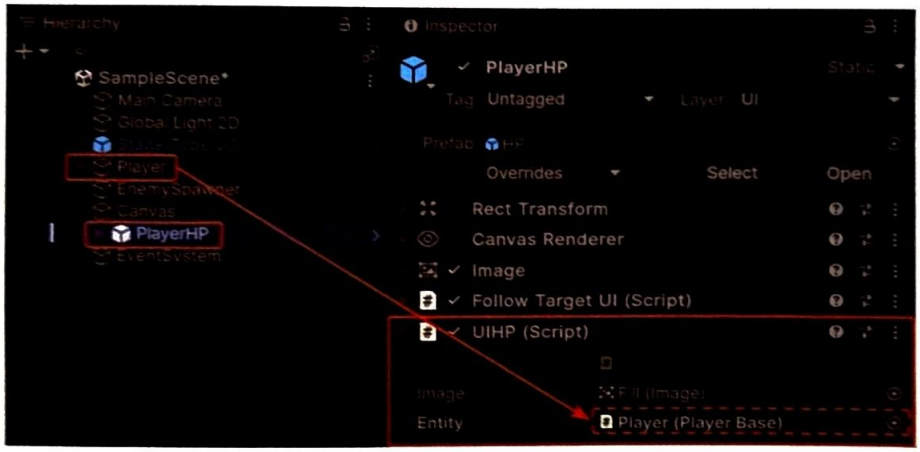
```
01: using UnityEngine;
02: using UnityEngine.UI;
03:
04: public class UIHP : MonoBehaviour
05: {
06:     [SerializeField]
07:     private Image image;
08:     [SerializeField]
09:     private EntityBase entity;
10:
11:     public void Setup(EntityBase entity)
12:     {
13:         this.entity = entity;
14:     }
15:
16:     private void Update()
17:     {
18:         image.fillAmount = entity.Stats.currentHP / entity.Stats.maxHP;
19:     }
20: }
```

13 HP 프리팹에 UIHP 컴포넌트를 추가하고 Image 변수에 Fill 오브젝트를 등록합니다.



HP 프리팹에 UIHP 컴포넌트 추가하고 설정하기

14 프리팹에서 나와 PlayerHP 오브젝트의 UIHP 컴포넌트에 있는 Entity 변수에 Player 오브젝트를 등록합니다.



PlayerHP 오브젝트의 UIHP 컴포넌트 변수 설정하기

15 적이 생성될 때 적 체력 UI를 생성하고 적의 체력을 출력하도록 설정하고자 EnemyBase.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemyBase.cs

코드 04-09 적 캐릭터 제어

```
01: using UnityEngine;
02:
03: public class EnemyBase : EntityBase
04: {
05:     [SerializeField]
06:     private Transform hudPoint;
```

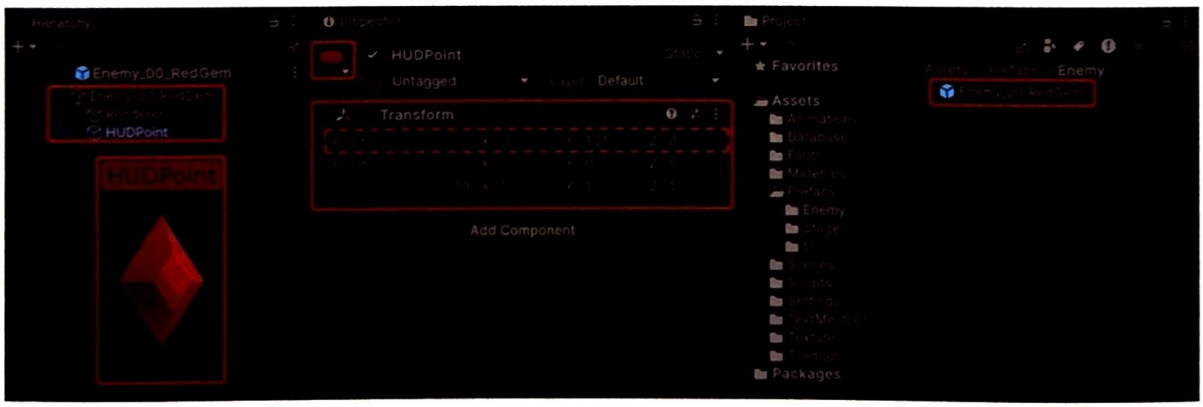
추가

```
07:     [SerializeField]
08:     private GameObject uiPrefab;
        (... 생략 ...)
22:     public void Initialize(Transform parent)
23:     {
24:         GameObject clone = Instantiate(uiPrefab, parent);
25:         clone.transform.localScale = Vector3.one;
26:         clone.GetComponent<FollowTargetUI>().Setup(hudPoint);
27:         clone.GetComponentInChildren<UIHP>().Setup(this);
28:     }
29: }
```

추가

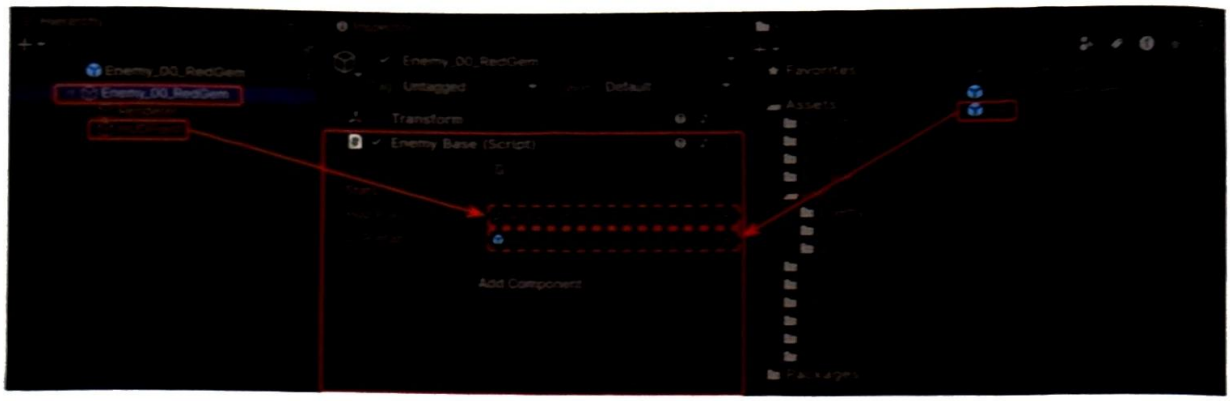
22~28줄 Initialize() 메서드는 적의 체력을 출력하는 UI를 생성합니다. FollowTargetUI.Setup() 메서드를 호출해 UI가 추적할 대상을 hudPoint로 설정하고 UIHP.Setup() 메서드를 호출해 현재 적(this)의 체력을 UI에 출력하도록 설정합니다.

16 Enemy_00_RedGem 프리팹을 더블클릭해서 활성화합니다. [GameObject → Create Empty] 메뉴로 적 체력 UI를 출력할 위치 정보가 있는 빈 오브젝트를 생성하고 이름을 'HUDPoint'로 변경합니다. Scene View에서 위치를 확인할 수 있도록 Select Icon 버튼 을 클릭해서 아이콘을 설정하고 Transform 컴포넌트의 Position을 (0, 1.25, 0)으로 설정합니다.



적의 체력을 출력하는 위치 오브젝트 생성하고 설정하기

17 Enemy_00_RedGem 프리팹에 Enemy Base 컴포넌트를 추가하고 Hud Point 변수에는 HUDPoint 오브젝트를, UI Prefab 변수에는 HP 프리팹을 등록합니다.



Enemy_00_RedGem 프리팹의 Enemy Base 컴포넌트 변수 설정하기

18 적을 생성할 때 EnemyBase.Initialize() 메서드를 호출하고자 EnemySpawner.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

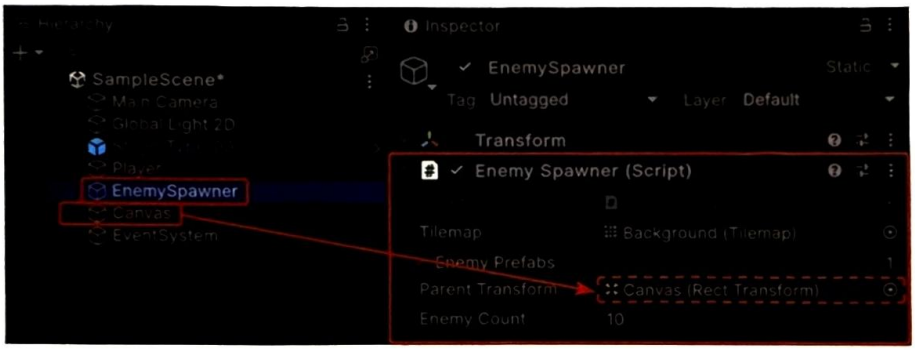
코드 04-10 적 자동 생성 스크립트

```
    (... 생략 ...)
05: public class EnemySpawner : MonoBehaviour
06: {
07:     [SerializeField]
08:     private Tilemap tilemap;
09:     [SerializeField]
10:     private GameObject[] enemyPrefabs;
11:     [SerializeField]                                    추가
12:     private Transform parentTransform;
    (... 생략 ...)
19:     private void Awake()
20:     {
21:         // Tilemap의 Bounds 재설정(맵을 수정할 때 Bounds가 변경되지 않는 문제 해결)
22:         tilemap.CompressBounds();
23:         // 타일맵의 모든 타일을 대상으로 적을 배치할 수 있는 타일 계산
24:         CalculatePossibleTiles();
25:
26:         // 임의의 타일에 적 10개 생성
27:         for ( int i = 0; i < enemyCount; ++ i )
28:         {
29:             int type = Random.Range(0, enemyPrefabs.Length);
30:             int index = Random.Range(0, possibleTiles.Count);
31:
32:             GameObject clone = Instantiate(enemyPrefabs[type], possibleTiles[index],
                    Quaternion.identity, transform);            수정
```

```
33:             clone.GetComponent<EnemyBase>().Initialize(parentTransform);
34:         }
35:     }
        (... 생략 ...)
61: }
```

추가

19 적과 함께 생성하는 적 체력 UI를 Canvas 오브젝트의 자식으로 배치하도록 Enemy Spawner 오브젝트 Enemy Spawner 컴포넌트의 Parent Transform 변수에 Canvas 오브젝트를 등록합니다.



EnemySpawner 오브젝트 Enemy Spawner 컴포넌트의 Parent Transform 변수 설정하기

20 플레이 버튼 ▶을 눌러 게임을 실행하면 PlayerHP 오브젝트가 플레이어를 쫓아다니며 체력을 출력하고, 적을 생성할 때 HP 프리팹을 생성해 적 오브젝트를 쫓아다니며 체력을 출력합니다.

플레이어와 적의 체력을 출력하는 UI를 생성한 결과 화면

04-3

적 기본 공격 구현하기

이 절에서 적은 쿨타임이 될 때마다 공격 발사체를 생성하고 발사체는 생성 당시 플레이어 위치를 기준으로 직선 이동하도록 합니다. 적이 생성한 발사체는 벽과 충돌하면 그대로 소멸하고 플레이어와 충돌하면 플레이어 체력을 적 공격력만큼 감소시키고 소멸하도록 구현해 봅시다.

Do it! 실습 적 발사체 만들기

01 플레이어와 적이 생성하는 발사체는 생성 직후에 원래 크기의 20%로 출력하고 이동하면서 서서히 100% 크기로 커지도록 설정합니다. 이를 위해 Assets 〉 Scripts 〉 Common 폴더에 크기 변화 효과를 재생하는 ScaleEffect.cs 스크립트를 생성합니다.

Assets/Scripts/Common/ScaleEffect.cs

코드 04-11 크기 변화 효과

```
01: using System.Collections;
02: using UnityEngine;
03: using UnityEngine.Events;
04:
05: public class ScaleEffect : MonoBehaviour
06: {
07:     [SerializeField]
08:     private float playTime = 0.1f;
09:
10:     public void Play(Vector3 start, Vector3 end, UnityAction action=null)
11:     {
12:         StartCoroutine(ScaleAnimation(start, end, action));
13:     }
14:
15:     private IEnumerator ScaleAnimation(Vector3 start, Vector3 end,
            UnityAction action)
16:     {
```

```
17:         float percent = 0;
18:
19:         while ( percent < 1 )
20:         {
21:             percent += Time.deltaTime / playTime;
22:
23:             transform.localScale = Vector3.Lerp(start, end, percent);
24:
25:             yield return null;
26:         }
27:
28:         action?.Invoke();
29:     }
30: }
```

10~13줄 외부에서 호출하는 Play() 메서드는 오브젝트의 크기를 start부터 end까지 점점 커지도록 하고 애니메이션이 종료될 때 호출하고 싶은 메서드가 있으면 action에 등록합니다.

15~29줄 percent의 초깃값은 0.0이고 1.0이 될 때까지 while 반복문을 호출하므로 playTime 시간 동안 오브젝트 크기인 transform.localScale이 start부터 end까지 변합니다. 반복문이 끝나면 action에 메서드가 있는지 검사하고 있으면 해당 메서드를 호출합니다.

02 Assets 〉 Scripts 〉 Enemy 폴더에 적 발사체를 제어하는 EnemyProjectile.cs 스크립트를 생성합니다.

Assets/Scripts/Enemy/EnemyProjectile.cs

코드 04-12 적 발사체 제어

```
01: using UnityEngine;
02:
03: [RequireComponent(typeof(MovementRigidbody2D))]
04: public class EnemyProjectile : MonoBehaviour
05: {
06:     private MovementRigidbody2D movementRigidbody2D;
07:     private ScaleEffect scaleEffect;
08:     private float damage;
09:
10:     public void Setup(Vector3 target, float damage)
11:     {
```

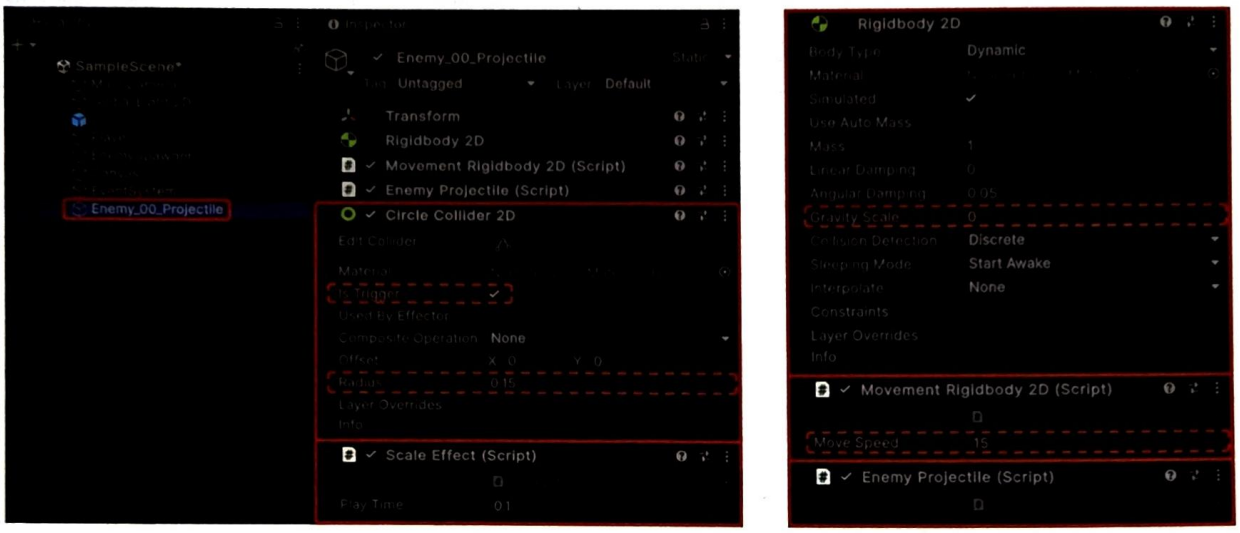
```
12:         movementRigidbody2D = GetComponent<MovementRigidbody2D>();
13:         scaleEffect = GetComponent<ScaleEffect>();
14:         this.damage = damage;
15:
16:         // 발사체 크기를 20%에서 100%로 확대
17:         scaleEffect.Play(transform.localScale * 0.2f, transform.localScale);
18:         // 발사체 이동 방향 설정
19:         movementRigidbody2D.MoveTo((target - transform.position).normalized);
20:     }
21:
22:     private void OnTriggerEnter2D(Collider2D collision)
23:     {
24:         if ( collision.CompareTag("Wall") )
25:         {
26:             Destroy(gameObject);
27:         }
28:         else if ( collision.CompareTag("Player") &&
29:             collision.TryGetComponent<EntityBase>(out var entity) )
30:         {
31:             entity.TakeDamage(damage);
32:             Destroy(gameObject);
33:         }
34:     }
35: }
```

03줄 적 발사체는 생성한 후 MovementRigidbody2D를 이용해 이동하므로 EnemyProjectile 컴포넌트를 추가할 때 함께 추가되도록 [RequireComponent()] 어트리뷰트를 선언합니다.

10~20줄 적 발사체가 생성되면 발사체 크기를 20%에서 100%로 커지도록 ScaleEffect.Play() 메서드를 호출하고 발사체를 목표 방향으로 설정합니다. 이동 방향을 Setup() 메서드에서 1회만 설정하므로 발사체는 목표의 움직임과 관계없이 직선으로 이동합니다.

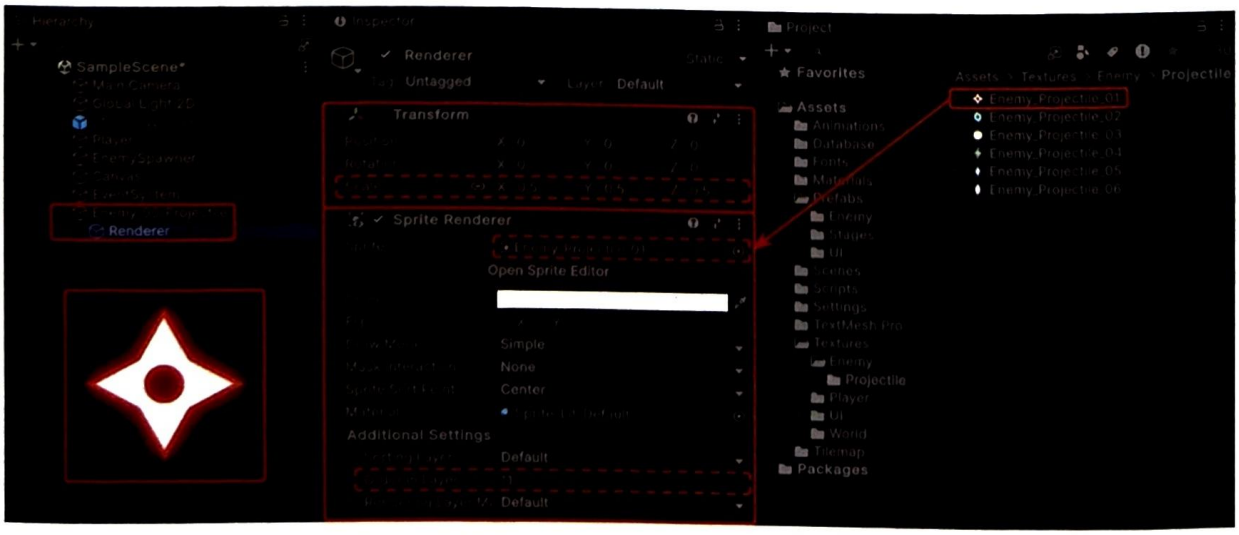
22~34줄 적 발사체가 다른 오브젝트와 충돌했을 때 충돌한 오브젝트의 태그가 "Wall"이면 발사체를 삭제합니다. 충돌한 오브젝트의 태그가 "Player"이고 EntityBase 컴포넌트가 있으면 플레이어(entity)의 체력을 damage만큼 줄이고 발사체를 삭제합니다.

03 [GameObject → Create Empty] 메뉴로 발사체 오브젝트를 제어하는 빈 오브젝트를 생성하고 이름을 'Enemy_00_Projectile'로 변경합니다. Enemy_00_Projectile 오브젝트에 Enemy Projectile, Circle Collider 2D, Scale Effect 컴포넌트를 추가합니다. Rigidbody 2D 컴포넌트의 Gravity Scale은 '0'으로, Movement Rigidbody 2D 컴포넌트의 Move Speed는 '15'로 설정합니다. 마지막으로 Circle Collider 2D 컴포넌트의 Is Trigger에 체크하여 true로, Radius는 '0.15'로 설정합니다.



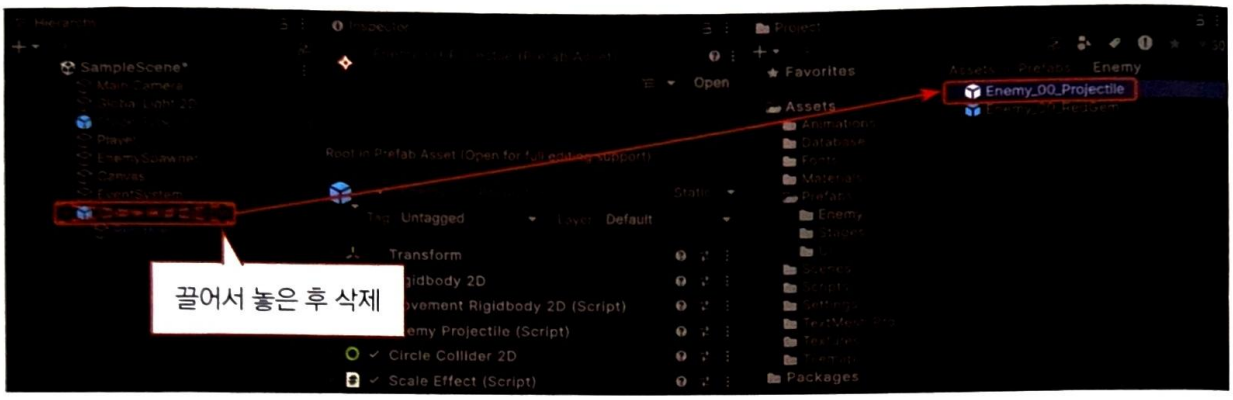
발사체 오브젝트를 제어하는 오브젝트 생성하고 설정하기

04 [GameObject → 2D Object → Sprites → Square] 메뉴로 발사체를 출력하는 2D Square 오브젝트를 생성하고 이름을 'Renderer'로 변경한 후 Enemy_00_Projectile 오브젝트의 자식으로 배치합니다. Renderer 오브젝트 Transform 컴포넌트의 Scale을 (0.5, 0.5, 0.5)로, Sprite Renderer 컴포넌트의 Sprite는 Enemy_Projectile_01.png 이미지로, Order in Layer는 '11'로 설정합니다.



발사체를 출력하는 오브젝트 생성하고 설정하기

05 Enemy_00_Projectile 오브젝트를 Project View의 Assets 〉 Prefabs 〉 Enemy 폴더로 드래그해서 프리팹으로 생성하고 Hierarchy View에 있는 Enemy_00_Projectile 오브젝트는 삭제합니다.



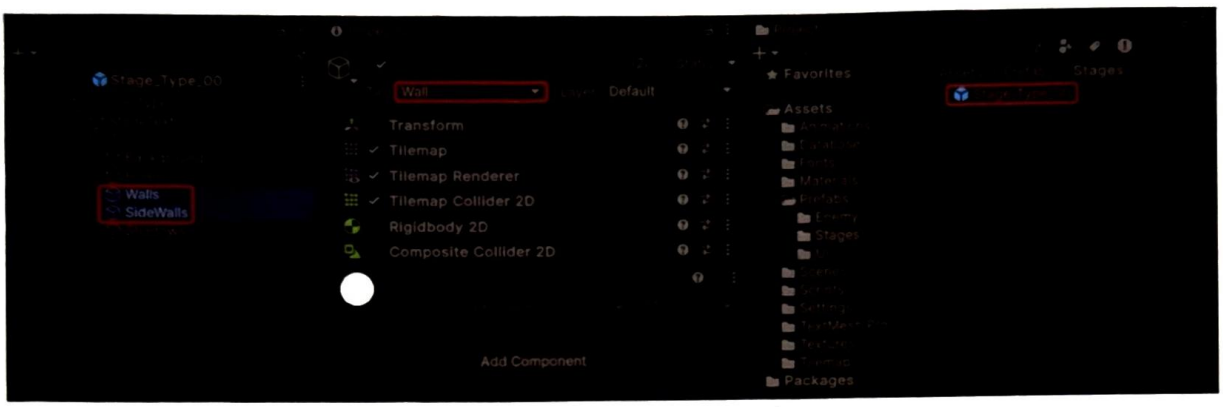
Enemy_00_Projectile 오브젝트 프리팹 생성하기

Do it! 실습 플레이어와 벽 충돌 설정하기

01 적이 생성하는 발사체가 플레이어나 벽과 충돌했을 때를 처리하고자 Player 오브젝트의 태그를 [Player]로 설정하고 Capsule Collider 2D 컴포넌트를 추가합니다. Capsule Collider 2D 컴포넌트의 Is Trigger를 true로 설정하고 Offset을 (-0.07, 0.65)로, Size를 (1.1, 1.5)로 설정합니다.

Player 오브젝트에 Capsule Collider 2D 컴포넌트 추가하고 설정하기

02 아무 오브젝트나 선택하고 Tag 드롭다운의 [Add Tag...] 메뉴로 Tags & Layers 옵션을 활성화합니다. Tags 리스트의 + 버튼을 누르고 'Wall' 태그를 생성합니다. Stage_Type_00 프리팹의 자식인 Walls와 SideWalls 오브젝트 태그를 모두 [Wall]로 설정합니다.



Wall 태그 생성하고 설정하기

Do it! 실습 플레이어 위치로 직선 공격하기

01 Entity의 공격 대상 정보를 저장하는 Target 속성을 정의하고자 EntityBase.cs 스크립트를 수정합니다.

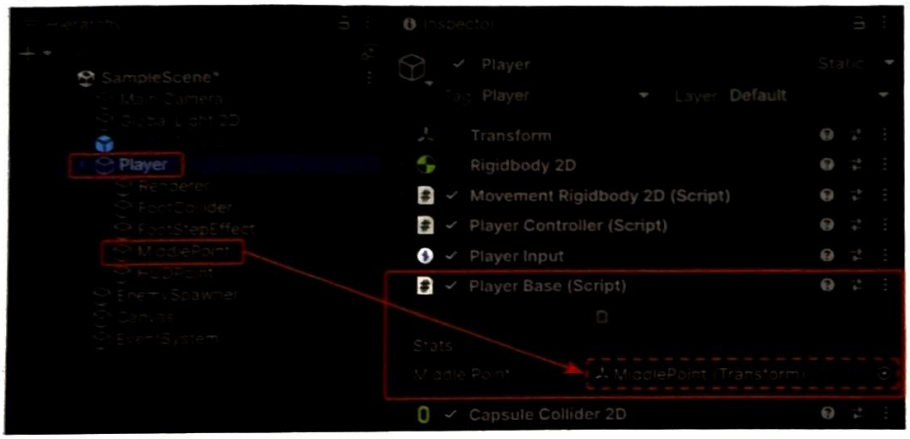
Assets/Scripts/Entity/EntityBase.cs

코드 04-13 모든 캐릭터가 상속받는 기반 클래스

```
01: using UnityEngine;
02:
03: public class EntityBase : MonoBehaviour
04: {
05:     [SerializeField]
06:     protected EntityStats stats;
07:     [SerializeField]                                    추가
08:     private Transform middlePoint;
09:
10:     public EntityStats Stats => stats;
11:     public bool IsDead => stats.currentHP <= 0;
12:     public Vector3 MiddlePoint => middlePoint != null   추가
            ? middlePoint.position : Vector3.zero;
13:     public EntityBase Target { get; set; }
14:
        (... 생략 ...)
31: }
```

07~08줄, 12줄 Player, Enemy 오브젝트 위치는 화면에 출력되는 캐릭터 이미지의 발 위치로 설정되므로 공격, 추적과 같은 행동을 할 때 이 발 위치를 기준으로 작동합니다. 몸통과 같은 오브젝트의 중심 위치를 기준으로 공격할 수 있도록 별도의 빈 오브젝트를 만들어 각 오브젝트의 중심 위치를 설정하고 **middlePoint** 변수에 등록해서 사용합니다.

02 Player 오브젝트의 Player Base 컴포넌트에서 Middle Point 변수에 MiddlePoint 오브젝트를 등록합니다.



Player 오브젝트의 Player Base 컴포넌트 변수 설정하기

03 Assets 〉 Scripts 〉 Enemy 폴더에 적의 행동을 제어하는 EnemyFSM.cs 스크립트를 생성합니다.

Assets/Scripts/Enemy/EnemyFSM.cs

코드 04-14 적 행동 제어

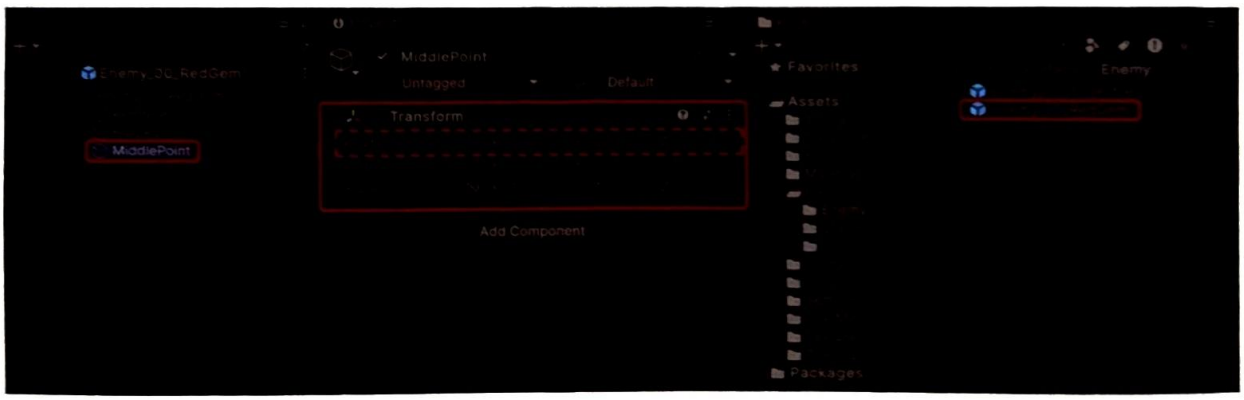
```
01: using System.Collections;
02: using UnityEngine;
03:
04: public enum EnemyState { None = -1, Attack, }
05:
06: public class EnemyFSM : MonoBehaviour
07: {
08:     [SerializeField]
09:     private GameObject projectilePrefab;
10:     [SerializeField]
11:     private Transform projectileSpawnPoint;
12:
13:     private EnemyBase owner;
```

```
14:     private EnemyState enemyState;
15:
16:     private void Awake()
17:     {
18:         owner = GetComponent<EnemyBase>();
19:
20:         ChangeState(EnemyState.Attack);
21:     }
22:
23:     public void Setup(EntityBase target)
24:     {
25:         owner.Target = target;
26:     }
27:
28:     public void ChangeState(EnemyState newState)
29:     {
30:         // Tip. 열거형 변수.ToString()은 열거형으로 정의한 변수 이름을 문자열로 반환한다.
31:         // ex) enemyState가 현재 EnemyState.Idle이면 "Idle" 문자열 반환
32:         // 이를 이용해 열거형 이름과 코루틴 이름을 일치시켜
33:         // 열거형 변수에 따라 코루틴 함수 재생을 제어할 수 있다.
34:
35:         // 이전에 재생 중이던 상태 종료
36:         StopCoroutine(enemyState.ToString());
37:         // 상태 변경
38:         enemyState = newState;
39:         // 새로운 상태 재생
40:         StartCoroutine(enemyState.ToString());
41:     }
42:
43:     private IEnumerator Attack()
44:     {
45:         var wait = new WaitForSeconds(owner.Stats.cooldownTime);
46:
47:         while ( true )
48:         {
49:             yield return wait;
50:
51:             Vector3 target = owner.Target.MiddlePoint;
52:             GameObject clone = Instantiate(projectilePrefab);
53:             clone.transform.position = projectileSpawnPoint.position;
54:             clone.GetComponent<EnemyProjectile>().Setup(target, owner.Stats.damage);
55:         }
56:     }
57: }
```

16~21줄 Awake() 메서드에서 현재 상태를 공격(EnemyState.Attack)으로 설정해 Attack() 코루틴 메서드를 호출합니다.

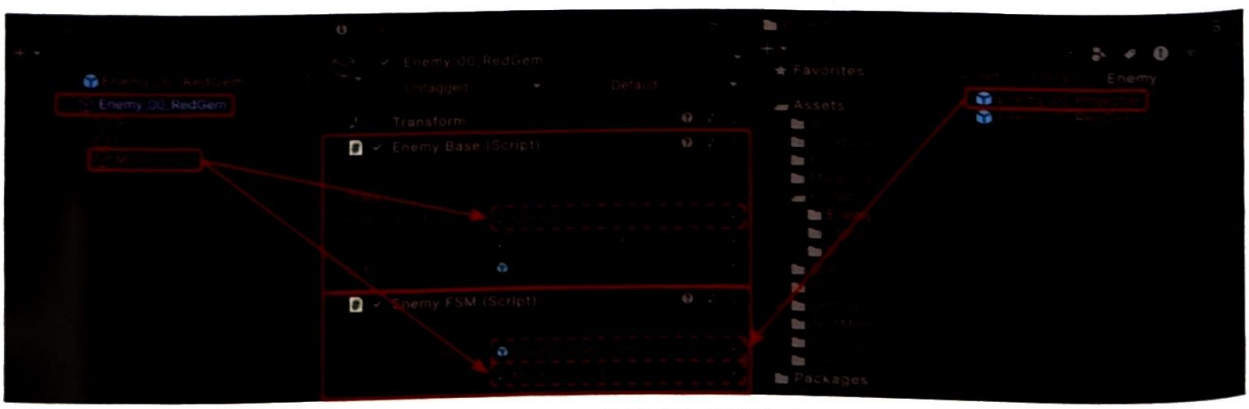
43~56줄 Attack() 코루틴 메서드에서 적의 쿨타임 시간인 owner.Stats.cooldownTime이 될 때마다 발사체 projectilePrefab을 생성하고 플레이어의 현재 중심 위치인 owner.Target.MiddlePoint 방향으로 날아가도록 EnemyProjectile.Setup() 메서드를 호출합니다.

04 Enemy_00_RedGem 프리팹을 더블클릭해서 활성화하고 [GameObject → Create Empty] 메뉴로 적 발사체 생성 위치(적 중심 위치)로 사용할 빈 오브젝트를 생성한 후 이름을 'MiddlePoint'로 변경합니다. MiddlePoint 오브젝트 Transform 컴포넌트의 Position을 (0.08, 0.55, 0)으로 설정합니다.



발사체 생성 위치(적 중심 위치) 오브젝트 생성하고 설정하기

05 Enemy_00_RedGem 프리팹에 Enemy FSM 컴포넌트를 추가합니다. Enemy Base 컴포넌트의 Middle Point 변수에 MiddlePoint 오브젝트를 등록합니다. 그리고 Enemy FSM 컴포넌트의 Projectile Prefab 변수에 Enemy_00_Projectile 프리팹을, Projectile Spawn Point 변수에 MiddlePoint 오브젝트를 등록합니다.



Enemy_00_RedGem 프리팹에 Enemy FSM 컴포넌트 추가하고 설정하기

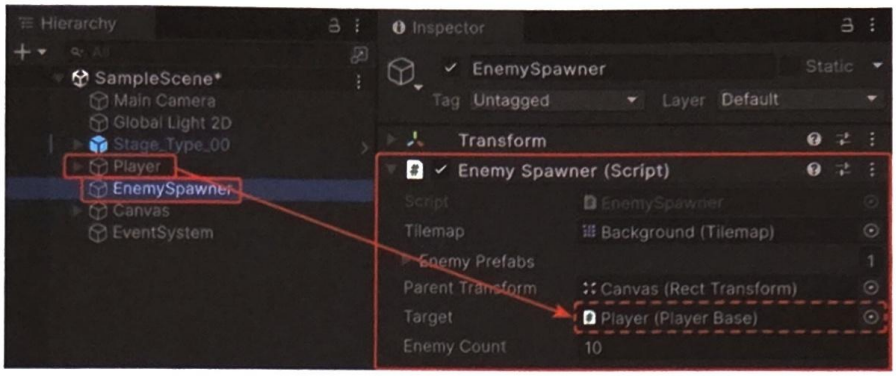
06 적을 생성할 때 **EnemyFSM.Setup()** 메서드를 호출해 플레이어 정보를 전달하고자 Enemy Spawner.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

코드 04-15 적 자동 생성 스크립트

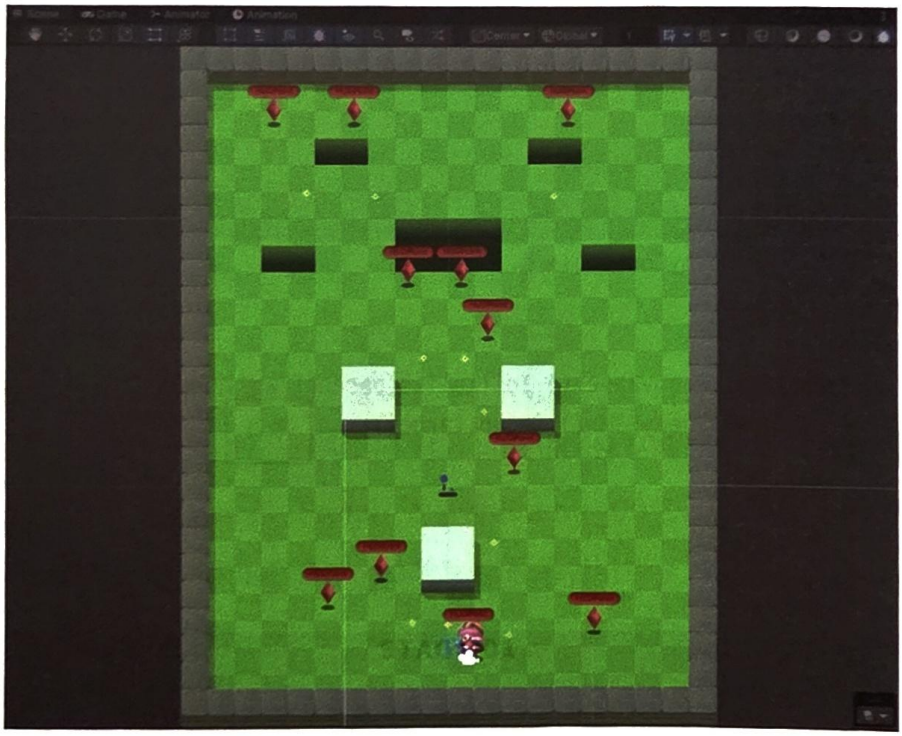
```
    (... 생략 ...)
05: public class EnemySpawner : MonoBehaviour
06: {
07:     [SerializeField]
08:     private Tilemap tilemap;
09:     [SerializeField]
10:     private GameObject[] enemyPrefabs;
11:     [SerializeField]
12:     private Transform parentTransform;
13:     [SerializeField]                                    추가
14:     private EntityBase target;
        (... 생략 ...)
20:
21:     private void Awake()
22:     {
            (... 생략 ...)
27:
28:         // 임의의 타일에 적 10개 생성
29:         for ( int i = 0; i < enemyCount; ++ i )
30:         {
                (... 생략 ...)
35:             clone.GetComponent<EnemyBase>().Initialize(parentTransform);
36:             clone.GetComponent<EnemyFSM>().Setup(target);    추가
37:         }
38:     }
        (... 생략 ...)
64: }
```

07 EnemySpawner 오브젝트 Enemy Spawner 컴포넌트의 Target 변수에 Player 오브젝트를 등록합니다.



EnemySpawner 오브젝트의 Enemy Spawner 컴포넌트 변수 설정하기

08 플레이 버튼 ▶ 을 눌러 게임을 실행하면 적 오브젝트가 1.5초마다 플레이어 오브젝트의 현재 위치인 `MiddlePoint`로 발사체를 발사합니다.



적 기본 공격을 구현한 화면

04-4

플레이어 공격 목표 설정하기

이 절에서는 플레이어가 자신과 가장 가까운 적을 공격 목표로 설정하도록 합니다. 플레이어의 공격 목표 위치에 표식을 출력하고 플레이어와 무기가 목표 방향으로 회전하도록 설정합니다.

Do it! 실습 공격 목표 설정하기

현재 게임에 있는 적 정보를 알아야 그중 가장 가까운 적 정보를 플레이어가 알 수 있을 겁니다.

01 적을 생성할 때 적 정보를 Enemies 리스트 속성에 저장하고자 EnemySpawner.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

코드 04-16 적 자동 생성 스크립트

```
    (... 생략 ...)
05: public class EnemySpawner : MonoBehaviour
06: {
        (... 생략 ...)
19:     private List<Vector3> possibleTiles = new List<Vector3>();
20:
21:     public static List<EntityBase> Enemies { get; private set; } =
            new List<EntityBase>();
22:
23:     private void Awake()
24:     {
            (... 생략 ...))
30:         // 임의의 타일에 적 10개 생성
31:         for ( int i = 0; i < enemyCount; ++ i )
32:         {
                (... 생략 ...)
38:             clone.GetComponent<EnemyFSM>().Setup(target);
```

추가

```
39:
40:             Enemies.Add(clone.GetComponent<EntityBase>());
41:         }
42:     }
        (... 생략 ...)
68: }
```

추가

21줄 플레이어가 공격할 목표를 설정하고 월드에 남은 적 숫자를 파악하도록 월드에 있는 모든 적의 정보를 저장할 Enemies 리스트 속성을 정의하여 하나만 만들고 외부에서 쉽게 접근할 수 있도록 정적(static)으로 설정합니다.

40줄 적을 생성할 때마다 방금 생성한 적(clone)의 EntityBase 정보를 Enemies 리스트에 저장합니다.

02 외부에서 목표 정보를 설정할 때 호출하는 SetTarget() 메서드를 정의하고자 FollowTarget.cs 스크립트를 수정합니다.

Assets/Scripts/Common/FollowTarget.cs

코드 04-17 목표(target) 위치를 추적하는 오브젝트

```
01: using UnityEngine;
02:
03: public class FollowTarget : MonoBehaviour
04: {
05:     [SerializeField]
06:     private Transform target;
07:     [SerializeField]
08:     private bool x, y, z;
09:
10:     public void SetTarget(Transform target)
11:     {
12:         this.target = target;
13:     }
14:
        (... 생략 ...)
25: }
```

추가

03 플레이어와 가장 가까운 적을 공격 목표로 설정하도록 PlayerBase.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerBase.cs

코드 04-18 플레이어 캐릭터 제어

```
01: using UnityEngine;
02:
03: public class PlayerBase : EntityBase
04: {
05:     [SerializeField]                                    추가
06:     private FollowTarget targetMark;
07:
08:     // 현재 플레이어가 이동 중인지
09:     // 스킬 사용 등 여러 곳에서 필요하므로 public 속성으로 정의
10:     public bool IsMoved { get; set; } = false;
        (... 생략 ...)
17:     private void Update()                               추가
18:     {
19:         if ( Target == null ) targetMark.gameObject.SetActive(false);
20:
21:         SearchTarget();
22:     }
23:
24:     private void SearchTarget()
25:     {
26:         float closestDistSqr = Mathf.Infinity;
27:
28:         foreach ( var entity in EnemySpawner.Enemies )
29:         {
30:             // 가장 가까운 대상을 찾으므로 sqrMagnitude 사용
31:             float distance = (entity.transform.position -
                    transform.position).sqrMagnitude;
32:             if ( distance < closestDistSqr )
33:             {
34:                 closestDistSqr = distance;
35:                 Target = entity.GetComponent<EntityBase>();
36:             }
37:         }
38:
39:         if ( Target != null )
40:         {
41:             targetMark.SetTarget(Target.transform);
42:             targetMark.transform.position = Target.transform.position;
```

```
43:                 targetMark.gameObject.SetActive(true);
44:             }
45:         }
46: }
```

24~45줄 SearchTarget() 메서드는 플레이어와 가장 가까운 적을 목표로 설정하므로 계산을 위한 거리 closestDistSqr은 가장 큰 값인 Mathf.Infinity로 설정합니다. foreach 반복문을 호출해 현재 월드에 있는 모든 적 EnemySpawner.Enemies를 차례대로 탐색합니다. 적 위치 entity.transform.position에서 플레이어 위치인 transform.position을 뺀 값을 벡터 크기 제곱(sqrMagnitude)해서 distance에 저장합니다.

정확한 거리가 아니라 누가 더 가까운지를 조사하므로 연산 속도가 더 빠른 sqrMagnitude 속성을 사용합니다.

32~36줄 적을 대상으로 계산한 distance가 기존 거리보다 작으면 closestDistSqr에 저장하고 방금 거리를 계산한 적(entity)의 EntityBase 정보를 Target에 저장합니다. 모든 적을 대상으로 해당 검사를 했을 때 플레이어와 가장 가까운 적이 Target이 됩니다.

39~44줄 Target이 null이 아니면 TargetMark 오브젝트가 Target을 추적하도록 SetTarget() 메서드를 호출합니다. TargetMark 오브젝트의 위치를 Target의 위치로 설정하고 월드에 출력하도록 활성화합니다.

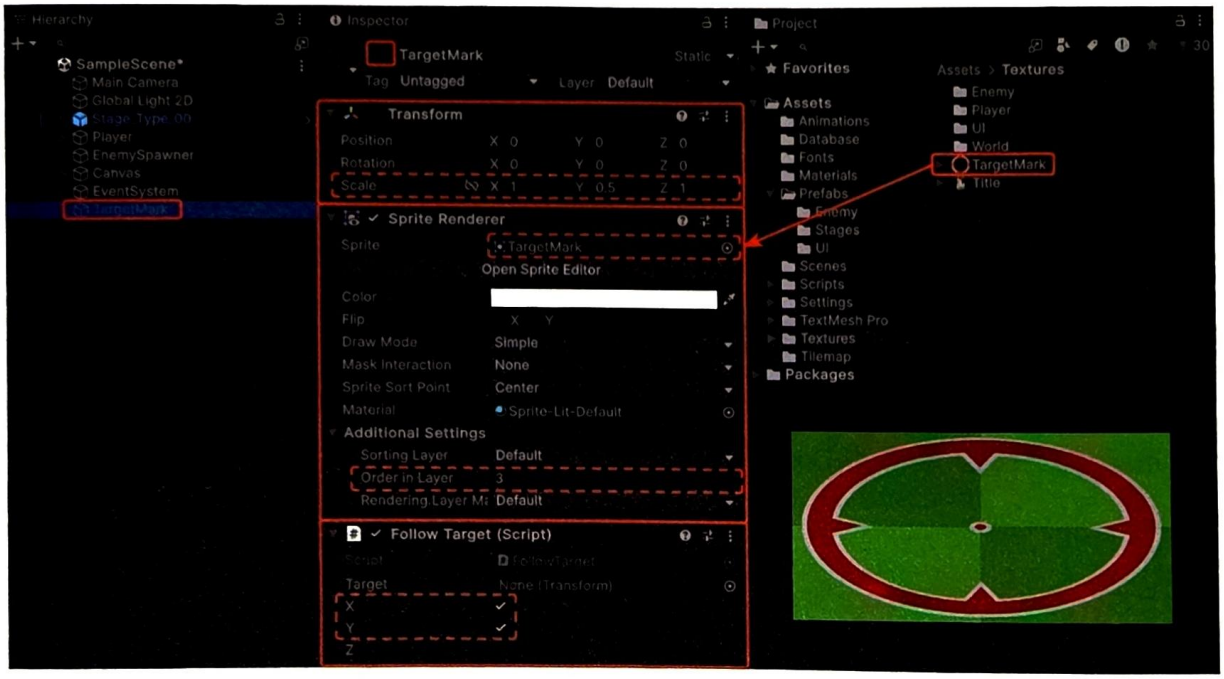
알아 두면 좋아요! 거리를 비교하는 magnitude 속성과 sqrMagnitude 속성의 차이는?

유니티에서는 두 오브젝트 사이의 실제 거리를 계산하는 magnitude 속성과 성능을 최적화하고자 제곱근을 계산하지 않는 sqrMagnitude 속성을 제공합니다.

```
Vector3 a = new Vector3(3, 4, 0);
float mag = a.magnitude;        // sqrt(3^2 + 4^2) = 5
float sqrMag = a.sqrMagnitude; // 3^2 + 4^2 = 25
```

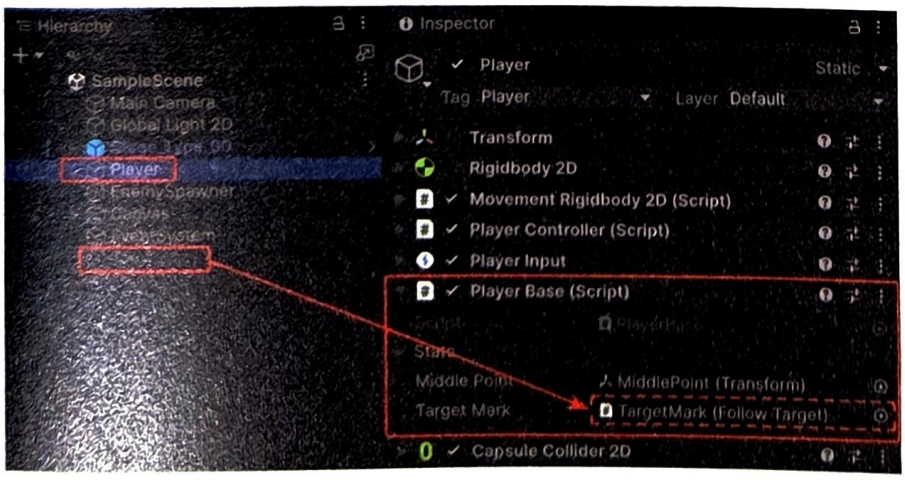
04 [GameObject → 2D Object → Sprites → Square] 메뉴로 플레이어의 목표 위치에 출력할 2D Square 오브젝트를 생성하고 이름을 'TargetMark'로 변경합니다. TargetMark 오브젝트 Transform 컴포넌트의 Scale을 (1, 0.5, 1)로 설정합니다.

Sprite Renderer 컴포넌트의 Sprite에 TargetMark.png 이미지를 등록하고 Order in Layer를 '3'으로 설정합니다. Follow Target 컴포넌트를 추가하고 X, Y를 각각 true로 설정합니다. 플레이어의 목표가 있을 때만 TargetMark 오브젝트가 활성화되므로 일단 비활성화합니다.



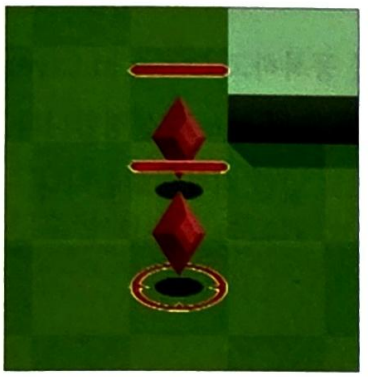
플레이어의 목표 위치에 출력하는 TargetMark 오브젝트 생성하고 설정하기

05 Player 오브젝트 Player Base 컴포넌트의 Target Mark 변수에 TargetMark 오브젝트를 등록합니다.



Player 오브젝트 Player Base 컴포넌트의 변수 설정하기

06 플레이 버튼 ▶을 눌러 게임을 실행하면 현재 플레이어가 목표(Target)로 설정한 적 위치에 TargetMark 오브젝트를 출력합니다.



공격 목표를 표시한 결과 화면

Do it! 실습 목표 방향으로 플레이어 회전하기

01 먼저 Assets 〉 Scripts 〉 Common 폴더에 게임에서 범용으로 사용하는 기능을 정의하는 Utils.cs 스크립트를 생성합니다.

Assets/Scripts/Common/Utils.cs

코드 04-19 게임에 범용으로 사용하는 기능

```
01: using UnityEngine;
02:
03: public static class Utils
04: {
05:     /// <summary>
06:     /// 회전값이 Vector3.zero라면 방향을 보는 오브젝트 기준이고
07:     /// 방향이 다르면 weight에 각도를 더한다.
08:     /// 반시계 방향 기준 +, 시계 방향 기준 -
09:     /// </summary>
10:     public static Quaternion RotateToTarget(Vector2 owner, Vector2 target,
            float weight=0)
11:     {
12:         // 원점에서 거리, 수평축에서 각도를 이용해 위치를 구하는 극좌표계 이용
13:         // 각도 = arctan(y/x)
14:         // x, y 변윗값 구하기
15:         float dx = target.x - owner.x;
16:         float dy = target.y - owner.y;
17:
18:         // x, y 변윗값을 바탕으로 각도 구하기
```

```
19:         // 각도가 radian 단위이므로 Mathf.Rad2Deg를 곱해 각도 단위를 구함
20:         float degree = Mathf.Atan2(dy, dx) * Mathf.Rad2Deg;
21:
22:         return Quaternion.Euler(0, 0, degree-weight);
23:     }
24: }
```

02 목표 방향으로 회전하는 LookRotation() 메서드를 정의하고자 PlayerRenderer.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerRenderer.cs

코드 04-20 플레이어 좌우 반전과 애니메이션 재생 제어

```
01: using UnityEngine;
02:
03: public class PlayerRenderer : MonoBehaviour
04: {
05:     [SerializeField]
06:     private Transform playerModel;
07:     [SerializeField]                                    추가
08:     private Transform playerArmsModel;
        (... 생략 ...)
41:     public void LookRotation(PlayerBase playerBase)     추가
42:     {
43:         if ( playerBase.IsMoved == true )
44:         {
45:             playerArmsModel.rotation = Quaternion.identity;
46:         }
47:         else
48:         {
49:             if ( playerBase.Target == null ) return;
50:
51:             Vector3 target = playerBase.Target.MiddlePoint;
52:             // 목표물이 플레이어 왼쪽이라면 -1, 오른쪽이라면 1
53:             float flip = target.x - transform.position.x < 0 ? -1 : 1;
54:             // 플레이어 좌우 반전
55:             SpriteFlipX(flip);
56:             // 플레이어 무기 회전
57:             // 왼쪽을 볼 때는 부모 오브젝트에 의해 회전이 적용되어
                   무기 방향이 틀어지므로 180만큼 가중치를 줌
58:             playerArmsModel.rotation = Utils.RotateToTarget(
                    playerArmsModel.position, target, (1-flip)*90);
```

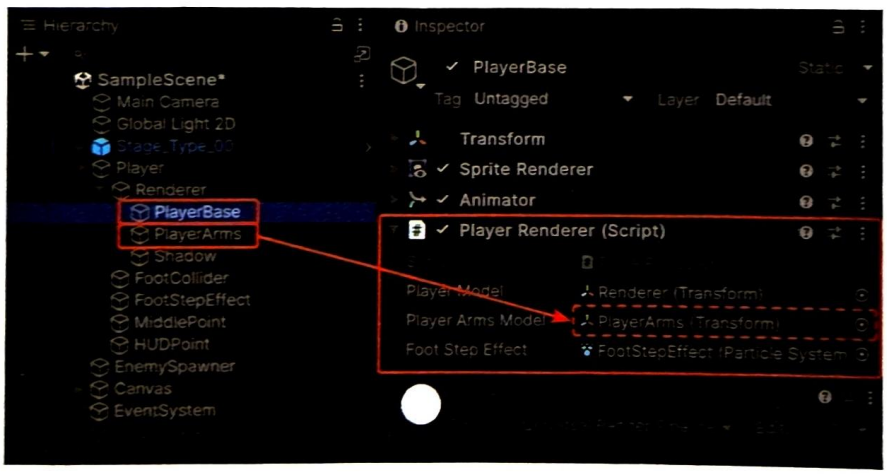
```
59:         }
60:     }
61: }
```

43~46줄 플레이어의 몸체와 무기 회전을 제어하는 LookRotation() 메서드 내부에서 플레이어가 이동 중이면(playerBase.IsMoved == true) 무기 회전을 (0, 0, 0)으로 설정합니다.

52~55줄 플레이어가 제자리에 멈춰 있으면(playerBase.IsMoved == false) 현재 플레이어의 x 위치를 기준으로 적이 왼쪽에 있으면 플레이어는 왼쪽을 바라보고 오른쪽에 있으면 오른쪽을 바라봅니다.

56~58줄 플레이어 무기(playerArmsModel.position)를 목표(target) 방향으로 회전합니다.

03 PlayerBase 오브젝트 Player Renderer 컴포넌트의 Player Arms Model 변수에 PlayerArms 오브젝트를 등록합니다.



PlayerBase 오브젝트의 Player Renderer 컴포넌트 변수 설정하기

04 Enemy_00_RedGem 프리팹의 충돌을 처리하고자 Capsule Collider 2D 컴포넌트를 추가하고 IsTrigger를 true로, Offset은 (0, 0.55)로, Size는 (0.6, 0.9)로 설정합니다.

Enemy_00_RedGem 프리팹에 Capsule Collider 2D 컴포넌트 추가하고 설정하기

05 `PlayerRenderer.LookRotation()` 메서드를 호출하도록 PlayerController.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerController.cs

코드 04-21 플레이어 캐릭터 제어

```
    (... 생략 ...)
04: public class PlayerController : MonoBehaviour
05: {
        (... 생략 ...)
18:     private void Update()
19:     {
            (... 생략 ...)
28:         // 플레이어 이동
29:         movement.MoveTo(moveInput);
30:         // 목표 방향으로 플레이어/무기 회전
31:         playerRenderer.LookRotation(playerBase);
32:     }
        (... 생략 ...)
41: }
```

추가

06 플레이 버튼 ▶을 눌러 게임을 실행하면 현재 월드에 있는 적 오브젝트 중에 플레이어와 가장 가까운 적을 목표로 설정해 TargetMark 오브젝트를 목표 위치에 출력하고, 플레이어 캐릭터와 무기는 목표를 바라보도록 회전합니다.

플레이어 공격 목표를 설정한 결과 화면

05장

플레이어 능력치와 기본 스킬 시스템 만들기

이번 장에서는 아이템이나 스킬에 따라 증가하는 보너스값과 기본값으로 공격력, 체력 등 능력치를 구분해서 관리하는 모듈식 스탯 클래스를 구현합니다. 스탯 정보를 바탕으로 플레이어 기본 공격을 구현하고, 플레이어 공격력 증가와 쿨타임 시간 감소 등 버프 스킬을 만듭니다.

학습 목표

1. 확장 가능한 모듈식 스탯 시스템을 설계하고 이벤트 기반 구조(delegate, event)를 구현할 수 있다.
2. ScriptableObject를 활용하여 스킬 데이터를 정의하고 게임 로직에 적용할 수 있다.

05-1 유연하게 데이터를 관리하는 스탯 클래스 만들기
05-2 기본 공격 스킬 만들기
05-3 버프 스킬 만들기

05-1

유연하게 데이터를 관리하는 스탯 클래스 만들기

이 절에서는 아이템이나 스킬에 따라 증가하는 보너스값과 기본값으로 공격력이나 체력 같은 능력치를 구분해서 관리하는 스탯 클래스를 구현합니다.

스탯 클래스 정의하기

스탯Stat에는 기본으로 설정하는 값과 스킬, 아이템 등을 통해 추가로 얻는 보너스값이 있습니다. 해당 스탯을 사용할 때는 기본값과 보너스값을 더한 값을 사용하지만, 스탯 정보를 UI에 출력하는 것처럼 때에 따라 기본값과 보너스값을 따로 사용하기도 합니다. 그러므로 기본값과 보너스값은 구분해야 하고 최솟값과 최댓값 등을 관리해야 합니다.

또한 값이 바뀌거나 최솟값이나 최댓값에 근접했을 때 유연하게 대응할 수 있도록 호출할 메서드를 등록하는 데 **델리게이트**Delegate와 **이벤트**Event를 사용합니다.

현재 `EntityStats` 구조체에서는 아이템이나 스킬 등으로 추가 스탯을 적용할 때 다음처럼 스탯 변수에 보너스를 추가하는 형식으로 작성해야 하고 새로운 버프나 스탯을 추가할 때마다 코드를 추가해야 합니다.

```
DamageBuffSkill은 Stats.damage += bonusDamage;
HPBuffSkill은 Stats.maxHP += bonusHP;
```

하지만 나중에 수정할 스탯 클래스를 사용하여 다음처럼 작성하면 새로운 버프 스킬이나 새로운 스탯을 추가하더라도 코드를 변경하지 않아도 됩니다.

```
Stats.GetStat(stat).BonusValue += bonusValue;
```

Do it! 실습 스탯 클래스 만들기

01 Assets 〉 Scripts 〉 Entity 폴더에 공격력이나 체력처럼 하나의 스탯을 나타내는 Stat 클래스를 정의하고자 Stat.cs 스크립트를 생성합니다.

Assets/Scripts/Entity/Stat.cs

코드 05-01 스탯 클래스 정의

```
using UnityEngine;

[System.Serializable]
public class Stat
{
    public delegate void ValueChangedHandler(Stat stat, float prev, float current);
    public event ValueChangedHandler OnValueChanged;
    public event ValueChangedHandler OnValueMax;
    public event ValueChangedHandler OnValueMin;

    [SerializeField]
    private StatType statType;  // 공격력, 체력 등 스탯 속성
    [SerializeField]
    private float maxValue;     // 최댓값
    [SerializeField]
    private float minValue;     // 최솟값
    [SerializeField]
    private float defaultValue; // 기본값
    [SerializeField]
    private float bonusValue;   // 아이템, 스킬 등으로 증가하는 추갓값

    public StatType StatType => statType;
    public float Value => Mathf.Clamp(defaultValue + bonusValue,
        minValue, maxValue);

    public float DefaultValue
    {
        get => defaultValue;
        set
        {
            float prev = Value;
            defaultValue = Mathf.Clamp(value, minValue, maxValue);
            TryInvokeValueChangedEvent(prev, Value);
        }
    }

```

```
36:     public float BonusValue
37:     {
38:         get => bonusValue;
39:         set => bonusValue = value;
40:     }
41:
42:     private void TryInvokeValueChangedEvent(float prev, float current)
43:     {
44:         if ( !Mathf.Approximately(prev, current) )
45:         {
46:             OnValueChanged?.Invoke(this, prev, current);
47:
48:             if ( Mathf.Approximately(current, maxValue) )
                    OnValueMax?.Invoke(this, prev, maxValue);
49:             else if ( Mathf.Approximately(current, minValue) )
                    OnValueMin?.Invoke(this, prev, minValue);
50:         }
51:     }
52: }
53:
54: public enum StatType { Damage = 0, CooldownTime, CriticalChance,
        CriticalMultiplier, HP, Evasion, }
```

03줄 Stat 클래스는 공격력이나 체력을 하나의 스탯 데이터로 활용하고 Inspector View에서 스탯 속성, 최솟값, 최댓값 등을 설정할 수 있도록 [System.Serializable] 어트리뷰트를 선언해 직렬화합니다.

06줄 이 스탯의 값(defaultValue)이 바뀌거나 최솟값이나 최댓값에 도달했을 때 외부 메서드를 호출하도록 매개변수가 3개인 델리게이트를 정의합니다.

07~09줄 값이 변경될 때(OnValueChanged), 최댓값에 도달했을 때(OnValueMax), 최솟값에 도달했을 때(OnValueMin) 호출할 이벤트를 선언합니다.

23줄 Value 속성은 스탯의 최종값으로, 기본값(defaultValue)과 추갓값(bonusValue)을 합한 값입니다.

25~34줄 DefaultValue 속성은 기본값을 사용하거나 수정할 때 호출하며 DefaultValue의 set를 호출할 때는 TryInvokeValueChangedEvent() 메서드를 호출해 각 이벤트에 등록된 메서드를 자동으로 호출합니다.

42~51줄 TryInvokeValueChangedEvent() 메서드는 최종값(Value)의 변화가 있을 때 OnValueChanged.Invoke() 메서드를 호출해 OnValueChanged 이벤트에 등록된 메서드를 호출합니다. 최종값이 최솟값이나 최댓값과 같을 때 OnValueMin, OnValueMax 이벤트에 등록된 메서드를 호출합니다.

54줄 게임에서 사용하는 모든 스탯은 StatType 열거형에 상수로 선언하므로 현재 선언된 스탯뿐만 아니라 속성 공격력, 속성 저항 등 새로운 스탯을 추가하고 싶을 때 열거형에 선언하고 Inspector View에서 해당 스탯 정보를 설정합니다.

02 플레이어의 스탯 변수를 Stat으로 설정하고자 EntityStats.cs 스크립트를 수정합니다.

Assets/Scripts/Entity/EntityStats.cs

코드 05-02 플레이어의 스탯 변수 선언

```
01: using System.Linq;                          ← 추가
02: using UnityEngine;
03:
04: [System.Serializable]
05: public struct EntityStats
06: {
07:     [Header("Level, Exp")]
08:     public int level; // 레벨
09:     public long exp;  // 경험치
10:
11:     [Header("Current HP")]                  ← 추가
12:     [SerializeField]
13:     private Stat currentHP;
14:
15:     [Header("Stats")]
16:     [SerializeField]
17:     private Stat[] stats;
18:
19:     public readonly Stat CurrentHP => currentHP;
20:     public readonly Stat GetStat(Stat stat) =>
            stats.FirstOrDefault(s => s.StatType == stat.StatType);
21:     public readonly Stat GetStat(StatType statType) =>
            stats.FirstOrDefault(s => s.StatType == statType);
22: }
```

11~13줄 stats 배열에서 StatType으로 스탯 정보를 구분하고 현재 체력과 최대 체력은 동일하게 StatType.HP를 사용하므로 현재 체력은 별도의 변수로 선언합니다.

15~17줄 그 외에 공격력, 쿨타임 등의 스탯 정보는 stats 배열에 추가해서 사용합니다.

20~21줄 외부에서 원하는 스탯 정보에 접근할 때는 GetStat() 메서드를 호출하고 stats 배열에서 FirstOrDefault() 메서드를 호출해 매개변수 Stat 또는 StatType과 동일한 값의 스탯 정보를 반환합니다.

03 기존에 작성된 코드의 스탯 정보를 Stat으로 변경하고자 EntityBase.cs 스크립트를 수정합니다.

Assets/Scripts/Entity/EntityBase.cs

코드 05-03 기존의 스탯 정보를 Stat으로 변경

```
01: using UnityEngine;
02:
03: public class EntityBase : MonoBehaviour
04: {
05:     [SerializeField]
06:     private EntityStats stats;                                    수정
07:     [SerializeField]
08:     private Transform middlePoint;
09:
10:     public EntityStats Stats => stats;
11:     public bool IsDead => Stats.CurrentHP != null &&
            Mathf.Approximately(Stats.CurrentHP.DefaultValue, 0f);    수정
12:     public Vector3 MiddlePoint => middlePoint != null ? middlePoint.position :
            Vector3.zero;
13:     public EntityBase Target { get; set; }
14:
15:     protected virtual void Setup()
16:     {
17:         Stats.CurrentHP.DefaultValue = Stats.GetStat(StatType.HP).Value;    수정
18:     }
19:
20:     public void TakeDamage(float damage)
21:     {
22:         if ( IsDead ) return;
23:
24:         Stats.CurrentHP.DefaultValue -= damage;                   수정
25:
```

```
26:         if ( Mathf.Approximately(Stats.CurrentHP.DefaultValue, 0f) )
            (... 생략 ...)
30:     }
31: }
```

수정

11줄 현재 체력 Stats.CurrentHP가 null이 아니고 CurrentHP의 기본값(DefaultValue)이 0에 가까우면 사망한 것으로 판단해 true를 반환합니다.

17줄 현재 체력(Stats.CurrentHP.DefaultValue)은 최대 체력(Stats.GetStat(StatType.HP).Value)과 같게 설정합니다.

04 적의 기본 체력인 100은 Inspector View에서 설정하고 레벨에 따라 추가로 증가하는 체력 정보를 연산하고자 EnemyBase.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemyBase.cs

코드 05-04 Stats.GetStat() 메서드를 이용해 스탯 정보 설정

```
01: using UnityEngine;
02:
03: public class EnemyBase : EntityBase
04: {
        (... 생략 ...)
15:     protected override void Setup()
16:     {
17:         // 기본 체력은 DefaultValue에 할당하므로 추가 체력(BonusValue)만 설정
18:         Stats.GetStat(StatType.HP).BonusValue = 50 * (Stats.level - 1);
19:
20:         base.Setup();
21:     }
        (... 생략 ...)
30: }
```

수정

05 Stat을 이용해 적의 쿨타임과 공격력을 적용하고자 EnemyFSM.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemyFSM.cs

코드 05-05 Stat을 이용한 적의 쿨타임, 공격력 적용

```
    (... 생략 ...)
04: public enum EnemyState { None = -1, Attack, }
05:
06: public class EnemyFSM : MonoBehaviour
07: {
        (... 생략 ...)
43:     private IEnumerator Attack()
44:     {
45:         var wait = new WaitForSeconds(
                owner.Stats.GetStat(StatType.CooldownTime).Value);   ← 수정
46:
47:         while ( true )
48:         {
49:             yield return wait;
50:
51:             Vector3 target = owner.Target.MiddlePoint;
52:             GameObject clone = Instantiate(projectilePrefab);
53:             clone.transform.position = projectileSpawnPoint.position;
54:             clone.GetComponent<EnemyProjectile>().Setup(target,
                    owner.Stats.GetStat(StatType.Damage).Value);   ← 수정
55:         }
56:     }
57: }
```

06 현재 체력 정보가 변경될 때만 체력 UI 정보를 갱신하도록 UIHP.cs 스크립트를 수정합니다.

Assets/Scripts/UI/UIHP.cs

코드 05-06 체력 정보가 변경될 때만 체력 UI 정보 갱신

```
    (... 생략 ...)
04: public class UIHP : MonoBehaviour
05: {
06:     [SerializeField]
07:     private Image image;
08:     [SerializeField]
09:     private EntityBase entity;
10:
11:     private void Awake()   ← 추가
12:     {
```

```
13:         if ( entity != null ) entity.Stats.CurrentHP.OnValueChanged += UpdateHP;
14:     }
15:
16:     public void Setup(EntityBase entity)
17:     {
18:         this.entity = entity;
19:         this.entity.Stats.CurrentHP.OnValueChanged += UpdateHP;
20:     }
21:
22:     private void UpdateHP(Stat stat, float prev, float current)
23:     {
24:         image.fillAmount = entity.Stats.CurrentHP.Value /
                entity.Stats.GetStat(StatType.HP).Value;
25:     }
26: }
```

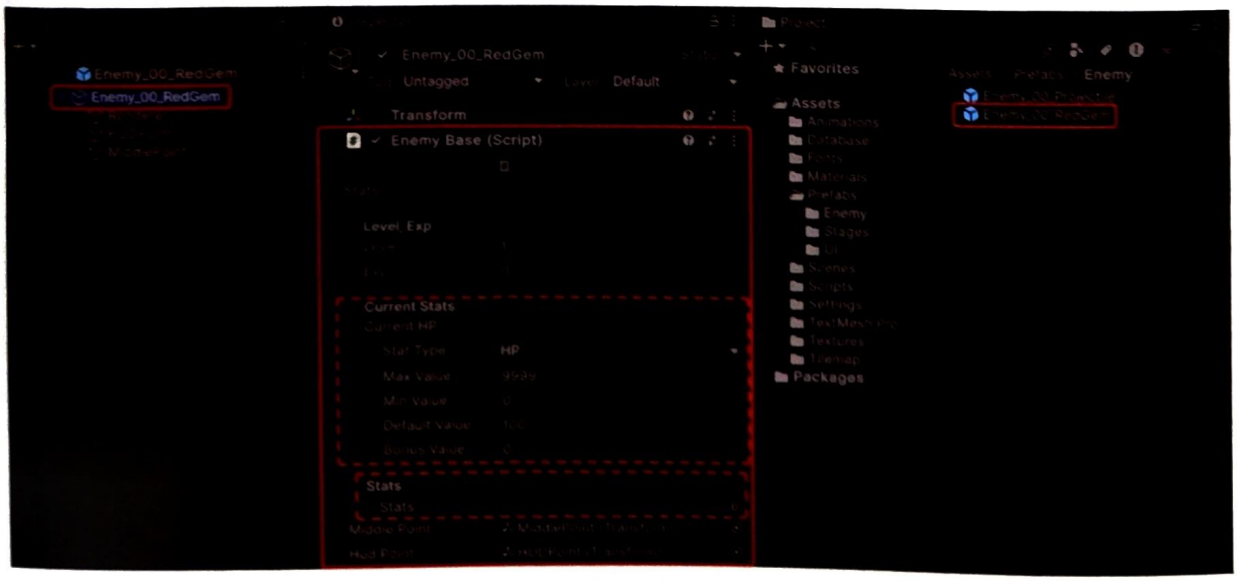
추가

수정

수정

13줄, 19줄 현재 체력값에 변화가 있을 때마다 UpdateHP() 메서드를 호출하도록 entity.Stats.CurrentHP.OnValueChanged에 UpdateHP() 메서드를 등록합니다. 플레이어의 체력 UI는 게임을 시작하기 전에 Hierarchy View에 미리 생성했으므로 Awake() 메서드에서 처리하고 적 체력 UI는 적이 생길 때 생성하므로 Setup() 메서드에서 처리합니다.

07 Enemy_00_RedGem 프리팹의 Enemy Base 컴포넌트에서 Current HP의 변숫값을 설정합니다. Stat Type은 [HP]로, Max Value는 '9999'로, Default Value는 '100'으로 설정합니다.

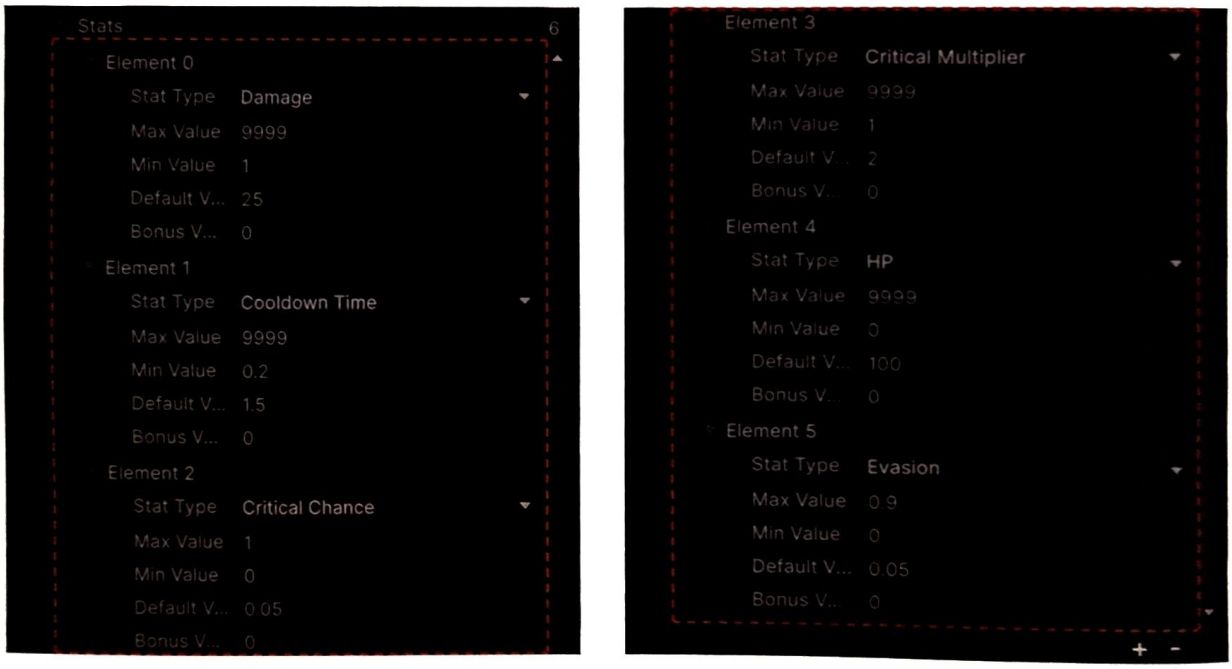


Enemy_00_RedGem 프리팹의 Enemy Base 컴포넌트에서 현재 체력 변수 Current HP 설정하기

08 Stats 배열 크기를 '6'으로 설정한 후, 다음 표를 참고해서 각 요소에 해당하는 값을 입력합니다.

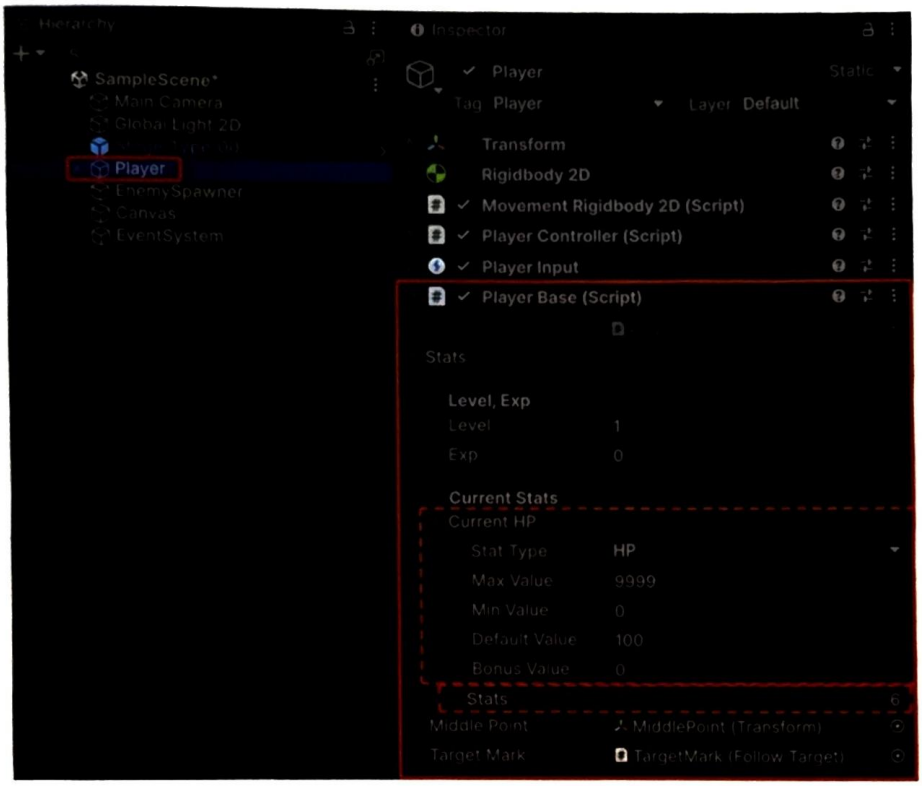
Enemy_00_RedGem의 Stats 배열 요소 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	1	25	0
1	Cooldown Time	9999	0.2	1.5	0
2	Critical Chance	1	0	0.05	0
3	Critical Multiplier	9999	1	2	0
4	HP	9999	0	100	0
5	Evasion	0.9	0	0.05	0



Enemy_00_RedGem 프리팹의 Enemy Base 컴포넌트에서 스탯 배열 변수 Stats 설정하기

09 Player 오브젝트의 Player Base 컴포넌트에서 Current HP 변숫값을 설정합니다. Stat Type은 [HP]로, Max Value는 '9999'로, Default Value는 '100'으로 설정합니다.

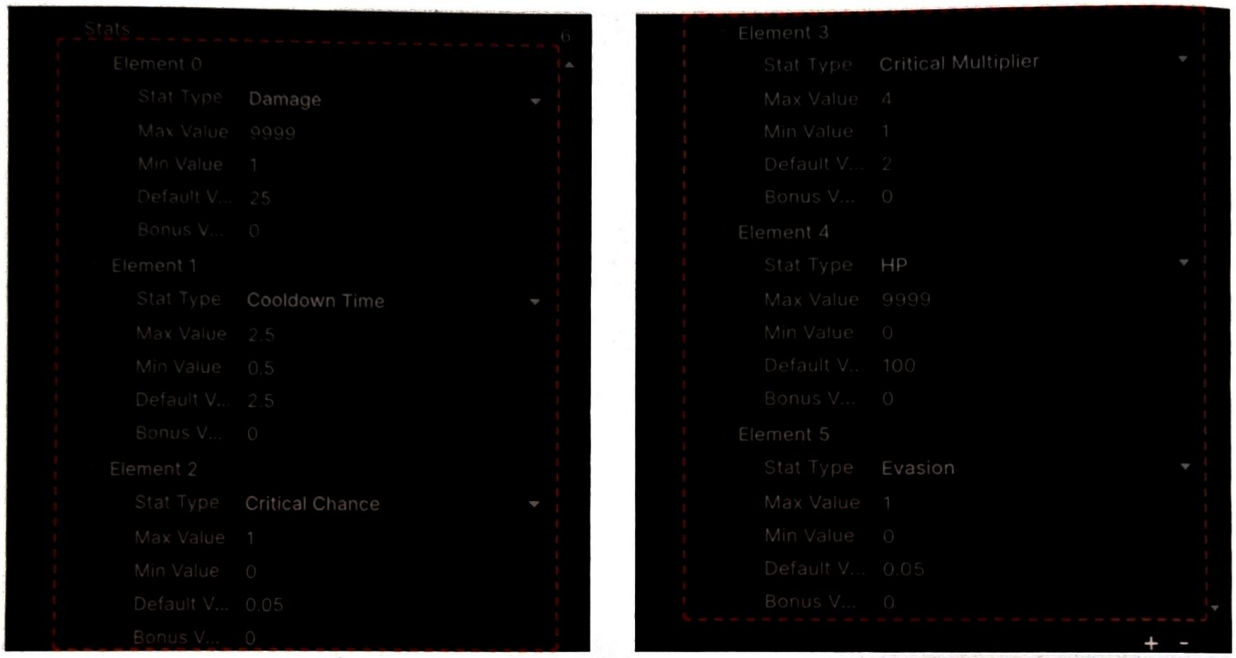


Player 오브젝트의 Player Base 컴포넌트에서 현재 체력 변수 Current HP 설정하기

10 Stats 배열의 크기를 '6'으로 설정한 후, 다음 표를 참고하여 각 요소에 해당하는 값을 입력합니다.

Player의 Stats 배열 요소 설정값

Element	Stat Type	Max Value	Min Value	Default Value	Bonus Value
0	Damage	9999	1	25	0
1	Cooldown Time	2.5	0.5	2.5	0
2	Critical Chance	1	0	0.05	0
3	Critical Multiplier	4	1	2	0
4	HP	9999	0	100	0
5	Evasion	1	0	0.05	0



Player 오브젝트의 Player Base 컴포넌트에서 스탯 배열 변수 Stats 설정하기

05-2

기본 공격 스킬 만들기

플레이어가 제자리에 멈춰 있고 월드에 적이 있으면 플레이어의 쿨타임이 될 때마다 기본 공격 발사체를 생성합니다. 발사체는 생성 당시 적의 위치를 기준으로 직선 이동하고 벽과 충돌하면 그대로 소멸합니다. 적과 충돌하면 발사체는 피격 효과와 데미지를 표시하고 적의 체력을 플레이어의 공격력만큼 감소시키고 나서 사라집니다.

Do it! 실습 기본 공격 발사체 만들기

01 Assets 〉 Scripts 〉 Skill 〉 Projectile 폴더를 만들고 이곳에 기본 공격 발사체를 제어하는 ProjectileGad.cs 스크립트를 생성합니다.

Assets/Scripts/Skill/Projectile/ProjectileGad.cs

코드 05-07 기본 공격 발사체 제어

```
01: using UnityEngine;
02:
03: public class ProjectileGad : MonoBehaviour
04: {
05:     private MovementRigidbody2D movementRigidbody2D;
06:     private ScaleEffect scaleEffect;
07:     private EntityBase target;
08:     private float damage;
09:
10:     public void Setup(EntityBase target, float damage)
11:     {
12:         movementRigidbody2D = GetComponent<MovementRigidbody2D>();
13:         scaleEffect = GetComponent<ScaleEffect>();
14:         this.target = target;
15:         this.damage = damage;
16:
17:         // 발사체 크기를 35%에서 100%로 확대
18:         scaleEffect.Play(transform.localScale * 0.35f, transform.localScale);
```

```
19:         // 발사체를 목표 방향으로 회전
20:         transform.rotation = Utils.RotateToTarget(transform.position,
                target.MiddlePoint, 90);
21:         // 발사체 이동 방향 설정
22:         movementRigidbody2D.MoveTo((target.MiddlePoint -
                transform.position).normalized);
23:     }
24:
25:     private void OnTriggerEnter2D(Collider2D collision)
26:     {
27:         if ( collision.CompareTag("Wall") )
28:         {
29:             Destroy(gameObject);
30:         }
31:         else if ( collision.CompareTag("Enemy") &&
32:             collision.TryGetComponent<EntityBase>(out var entity) )
33:         {
34:             if ( entity != target ) return;
35:
36:             entity.TakeDamage(damage);
37:             Destroy(gameObject);
38:         }
39:     }
40: }
```

34줄 방금 충돌한 오브젝트가 target과 같은지 검사하는 코드가 없으면 target에 날아가는 도중에 다른 적과 충돌해 발사체가 사라질 수도 있고, 적이 뭉쳐 있을 때 충돌한 모든 적을 공격할 수도 있습니다.

02 아무 오브젝트나 선택하고 Tag 드롭다운 메뉴에서 [Add Tag...]를 선택합니다. ➕▾버튼으로 Tags 리스트에 요소를 추가하고 'Enemy' 태그를 생성합니다. 그런 다음, Enemy_00_RedGem 프리팹의 태그를 방금 추가한 [Enemy]로 설정합니다.